



2ND EDITION

Salesforce Anti-Patterns

Build resilient Salesforce solutions using expert advice
to create robust and efficient programs

LARS MALMQVIST



Salesforce Anti-Patterns

Copyright © 2025 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager : Aaron Tanna

Publishing Product Manager : Kushal Dave

Senior Editor : Nisha Cleetus

Technical Editor : Vidhisha Patidar

Copy Editor : Safis Editing

Associate Project Manager : Manisha Singh

Proofreader : Nisha Cleetus

Indexer : Pratik Shirodkar

Production Designer : Ponraj Dhandapani

Business Development Executive : Saloni Garg

First published: November 2022

Second edition: January 2025

Production reference: 1131224

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83546-784-8

www.packtpub.com

*To Damiana, Ada, and Pino. To my mother,
Inger Beijder, and the memory of my father, Finn
Malmqvist.*

– *Lars Malmqvist*

Contributors

About the author

Lars Malmqvist has spent the past 16 years working as an architect and CTO within the Salesforce ecosystem. He has worked on over 40 Salesforce implementations, ranging from simple out-of-the-box scenarios to advanced, bespoke, multi-cloud solutions for large global brands. He is a 33-time certified Salesforce CTA with degrees in anthropology, mathematics, and technology management, an MBA from the University of Cambridge, and a Ph.D. in computer science from the University of York. Currently, he is a partner at Implement Consulting Group.

*I would like to thank my wife, Damiana, for her undying support, and my children, Ada and Pino,
for their constant inspiration.*

About the reviewers

Ashvin Bhatt is an enterprise architect, technical writer, trainer, and speaker with over 11 years of experience with the Salesforce platform working on various clouds. He has trained over 600 people on Salesforce through various initiatives. He is a certified Application Architect and holds a wealth of experience with various Salesforce products. Ashvin is the co-organizer of the Salesforce Architect Summit and has been a speaker at various events, such as Dreamforce and TDX. He has worked in a variety of domains, such as high tech, advertising, manufacturing, healthcare, insurance, and pharma, to name a few. He believes in sharing his knowledge through community-led initiatives to help foster learning and innovation.

Rafael Gutiérrez Castillo is a Salesforce Architect with multi-cloud implementation experience. With more than 10 years of working experience, he started his career in the education industry, developing leadership programs for university students.

He has worked in different positions within the Salesforce ecosystem, as a business consultant, tester, solution consultant, and solution architect.

Working as an associate manager at Accenture, he designs solutions that bridge the gap between business processes and technology and advises companies on their digitalization and transformation journeys. Being the proud owner of 24 Salesforce certificates, he is passionate about automation processes, solution design, process management, innovation, and leadership.

Table of Contents

[Preface](#)

Part 1: Technical Anti-Patterns

1

Why Do We Need to Understand Salesforce Anti-Patterns?

Understanding anti-patterns

From pattern to anti-pattern

Where do anti-patterns come from?

An anti-pattern template

How can anti-patterns deepen your architectural skills?

How great architects learn from mistakes

An example – the Hero anti-pattern

Hero (development life cycle and deployment planning)

Analyzing the example

Summary

2

How to Keep Your System Architecture Clean

Muddling up the systems landscape

Stovepipe

Architecture as Requirements

A Big Ball of Mud

Too much coupling

Intense coupling

Org complications

Ungoverned Org Proliferation

Knowing the takeaways

Summary

3

How Not to Get Confused about Security

Getting confused about shared responsibility

Shared Belongs to Salesforce

Mistaking other things for security

Compliant Is Secure

Declarative Is Always Safe

Sharing isn't always caring

Spaghetti Sharing Model

Knowing the takeaways

Summary

4

What Not to Do about Data

Mistaking Salesforce for a regular database

Salesforce as a Relational Database

Forgetting about governance

Disconnected Entities

Unplanned Growth

Synchronizing excessively

Unconstrained Data Synchronization

Too much AI is not always a good thing

Let's Add AI to It

AI Architecture Is Like Normal Architecture

Knowing the takeaways

Summary

Part 2: Solution Anti-Patterns

5

Unpicking Solution Architecture Troubles

Picking the wrong thing

Ignoring the Ecosystem

License-Based Solutioning

Applying the thing wrong

Assumption-Driven Customization

Golden Hammer

Engineering overflow

Automation Bonanza

Overweight Component

Making a mess of code

God Class

Error Hiding

Knowing the takeaways

Summary

6

Keeping Integration Straight

Messing up the integration landscape

Middleware in name only (MINO)

Service proliferation syndrome

Interfacing indecently

Fat interface

Chatty integration

Getting the patterns wrong

Integration pattern monomania

Event hell

Example

Problem

Proposed solution

Results

Better solutions

AI's impact on integrations

Knowing the takeaways

Summary

Part 3: Process and Communication Anti-Patterns

7

Keeping the Development Life Cycle from Going Off Track

Misaligning the process

Big-Bang Release

Project Pieism

Infrequent Check-In

Unpacking the pipeline

Using Packages to Create Silos

Testing without testing

Dummy Unit Tests

The challenge of DevOps integration

Knowing the takeaways

Summary

8

Communication without Mistranslation

Communicating too much

Cognitive overload

Being unclear in several ways

Ambiguous solution

Groundhog Day

Drawing diagrams with excess creativity

Non-standard documentation

Mismanaging change

Ignoring stakeholder concerns

Example

The problem

Proposed solution

Results

Better solutions

Underestimating the impact of change

Example

The problem

Proposed solution

Result

Better solutions

Knowing the takeaways

Summary

Conclusion

Summarizing the key messages

Explaining how to use anti-patterns in practice

Using anti-patterns constructively

Going further with anti-patterns

Diving deeper into the anti-pattern literature

Extending your knowledge of Salesforce architecture

Extending your general architecture knowledge

Summary

Index

Other Books You May Enjoy

Preface

Salesforce is the world's leading enterprise systems platform. This is a position it has reached over the last decade, with its scope having grown from its roots in sales and customer service to encompass a wide range of business domains.

However, as Salesforce has grown, so have the complexities of the technological environments that contain it. And with that growing complexity, a number of common mistakes have emerged that often end up derailing Salesforce projects in a number of interesting ways.

These mistakes are what we will learn to call anti-patterns and investigate in a structured manner in this book. There are quite a few books on Salesforce architecture already available on the market today, but they all approach the subject from a normative view, grounded in good practice.

In this book, we flip that perspective around. We look at bad practices that commonly occur in Salesforce projects, and that, in fact, can seem like a good idea at the time you make the decision.

By doing that, we see what happens when things don't go to plan – when you don't make the right call, and your solution suffers as a consequence. That gives a great background for us to review key architectural concepts and good practices as they apply across a range of scenarios.

In this book, you will get information structured into the seven domains of the **Certified Technical Architect (CTA)** exam roadmap. That way, while all the information is based on real-world examples, you will also be able to directly apply it in your journey to becoming a CTA, should that be a path you are on or are considering.

In this second edition, we have included many new anti-patterns as well as updates to the existing content. In particular, we have included sections and references about the increasing use of artificial intelligence and DevOps in the Salesforce ecosystem.

Who this book is for

This book is for anyone who has a position of responsibility in a Salesforce implementation and wants to see it avoid the common mistakes that plague many projects. While the primary audience is architects or aspiring architects, it will also be of value for project managers, business analysts, functional consultants, and executives overseeing Salesforce projects.

What this book covers

[Chapter 1](#) , *Why Do We Need to Understand Salesforce Anti-Patterns?* , starts by explaining what anti-patterns are by going through a number of different definitions and ways of looking at them. You will then learn how they can help you deepen your understanding of architecture by understanding how these common mistakes are made and how they can seem like a good idea at the time. We then finish the chapter by understanding the book and the ways in which you can approach the content.

[Chapter 2](#) , *How to Keep Your System Architecture Clean* , guides you through common anti-patterns relating to the systems landscape, the mix of systems you use in your solution, and how they can be combined. We will start by looking at anti-patterns at the highest level that describe problems around the composition and scoping of systems. Then we will look at anti-patterns that create highly coupled and brittle system landscapes, after which we will look at anti-patterns around selecting the right systems to include. The chapter will end with a review of the key takeaways for real life and the CTA Review Board.

[Chapter 3](#) , *How Not to Get Confused about Security* , includes anti-patterns related to securing your Salesforce org. It starts by reviewing a key anti-pattern around the shared security model that is integral to Salesforce. Then we will look at anti-patterns that can occur when you mistake other things for security. We will continue to look at how not to make a mess of your sharing model and once again finish up with a review of the takeaways.

[Chapter 4](#) , *What Not to Do About Data* , contains anti-patterns related to data architecture, modeling, and governance. We will start by looking at what goes wrong when you don't design with Salesforce's object model in mind. Then we will look at some of the common governance disasters that can happen around data. From there, we will look intently at data synchronization and the ways in which it can go wrong, and finish up with takeaways for life and the CTA Review Board.

[Chapter 5](#) , *Unpicking Solution Architecture Troubles* , covers anti-patterns related to your solution architecture. We will start by looking at anti-patterns related to your choice of solutions. Then we will look at things that can go wrong when you do functional design. Thirdly, we will look at some particular anti-patterns that affect customizations, first at the conceptual level and then at the code level. We will end the chapter by summarizing the key takeaways.

[Chapter 6](#) , *Keeping Integration Straight* , looks at anti-patterns around your integrations with other systems. The first part of the chapter looks at anti-patterns around the integration landscape and integration patterns. The second part looks at what can go wrong in the design and use of interfaces,

and the third zooms in on problems with particular integration styles such as event-based architectures and batch transfers. As always, we end the chapter by distilling the key takeaways.

[Chapter 7](#), *Keeping the Development Life Cycle from Going off Track*, looks at anti-patterns related to areas such as development process, governance, and DevOps. We will start by tackling a few significant process-level anti-patterns and then move on to a couple that deal with DevOps and packaging. Finally, we will tackle a common and very unfortunate anti-pattern related to testing. At the end of the chapter, we will summarize the key takeaways for real life and the CTA Review Board.

[Chapter 8](#), *Communication without Mistranslation*, deals with anti-patterns centered around the way you communicate architecture to different audiences. The first part relates to anti-patterns that relate to information control. Then we look at a few anti-patterns that relate to the clarity of your communication. Thirdly, we look at an anti-pattern specifically about the way you create architectural artifacts. As we've been doing all along, we will end the chapter by summarizing our key takeaways.

[Chapter 9](#), *Conclusion*, gives a broad overview of the content of the book, including the overarching takeaways from the preceding chapters. It then explains how you can use the anti-pattern method and perspective in your own work to improve your architecture skills. Finally, it gives you pointers on how you can progress further in this direction.

To get the most out of this book

This book is intended for anyone who has a stake in making Salesforce projects successful. While some of the technical chapters will make assumptions about general technical knowledge and experience, the only real precondition is a general knowledge of and interest in the Salesforce platform.

Conventions used

There are a number of text conventions used throughout this book.

Code in text : Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: “Pranit, one of the Salesforce architects, makes a proposal to simplify the current system architecture by creating a `CentralDispatch` Apex class that will take the role of dispatching calls to other code or flow.”

TIPS OR IMPORTANT NOTES

Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback : If you have questions about any aspect of this book, email us at customercare@packtpub.com and mention the book title in the subject of your message.

Errata : Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit www.packtpub.com/support/errata and fill in the form.

Piracy : If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author : If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit authors.packtpub.com.

Share Your Thoughts

Once you've read *Salesforce Anti-Patterns* , we'd love to hear your thoughts! [Please click here to go straight to the Amazon review page for this book](#) and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/978-1-83546-784-8>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1: Technical Anti-Patterns

This part will teach you how to identify and mitigate anti-patterns in the technical domains of system architecture, data architecture, and security architecture.

This part has the following chapters:

- [Chapter 1](#), *Why Do We Need to Understand Salesforce Anti-Patterns?*
- [Chapter 2](#), *How to Keep Your System Architecture Clean*
- [Chapter 3](#), *How Not to Get Confused about Security*
- [Chapter 4](#), *What Not to Do About Data*

1

Why Do We Need to Understand Salesforce Anti-Patterns?

Salesforce anti-patterns tell us about the kinds of systematic mistakes that make their way into many Salesforce projects – mistakes that can even seem like a good idea at the time of implementation but end up having serious drawbacks in the long run. We'll learn about the most common types of mistakes and how to think about them throughout this book. However, to do so, we must understand what anti-patterns are and how understanding them can help us architect and design better systems.

In consequence, this chapter will start by explaining what anti-patterns are by going through several different definitions and ways of looking at them. Then, you'll learn how they can help you deepen your understanding of architecture by knowing how these common mistakes are made and how they can seem like a good idea at the time. We'll finish this chapter by explaining what will be covered in this book and how you can approach the content provided.

In this second edition, we've expanded our exploration of Salesforce anti-patterns to include new patterns and insights that address the latest trends and developments in the Salesforce ecosystem. These additions are based on feedback from readers of the first edition and our ongoing research into the evolving landscape of Salesforce architecture. Notably, we've included new patterns related to AI and DevOps practices, such as “*AI architecture is like normal architecture*” and “*Let's add AI to it*,” as well as updates to existing patterns to reflect the current best practices.

In this chapter, we're going to cover the following main topics:

- Understanding anti-patterns
- Where do anti-patterns come from?
- How can anti-patterns deepen your architectural skills?
- Learning about anti-patterns from an example

Understanding anti-patterns

Most developers and architects will be familiar with the concept of a pattern – a good solution to a recurring problem within an architectural domain described in a formalized and reusable way. Some classic examples are as follows:

- **Singleton** : A software design pattern that limits the number of instances of a given type to 1
- **Fire-and-forget** : An asynchronous integration pattern that sends off a message from a computational context and proceeds without waiting for a response

- **Model-View-Controller (MVC)** : An architectural pattern that divides an application into three tiers with specifically defined responsibilities:
 - I. First, a model maintains the state of the application and is responsible for any changes to data.
 - II. Second, a view shows a particular representation of that model to an end user via some interface.
 - III. Third, a controller implements the business logic that responds to events in the user interface or changes in the model and does the necessary mediation between the view and the model.

This pattern is shown in the following diagram:

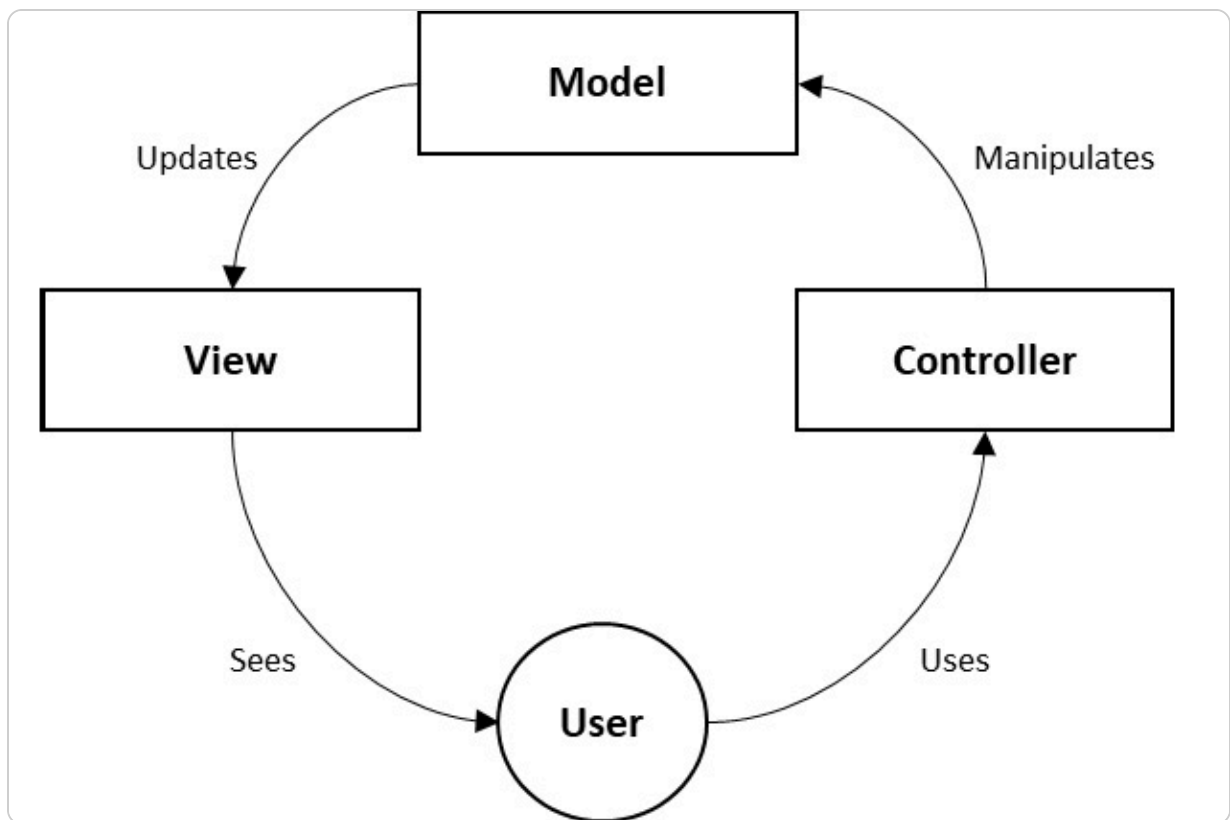


Figure 1.1 – MVC pattern diagram

Patterns such as these have been defined at many levels of abstraction and for many different platforms.

REFERENCES

Take a look at the following resources to get a good introduction to the various patterns that you can apply from a Salesforce perspective:

The integration patterns guide lists all the main patterns you should use when designing Salesforce integrations:

[https://developer.salesforce.com/docs/atlas.en-](https://developer.salesforce.com/docs/atlas.en-us.integration_patterns_and_practices.meta/integration_patterns_and_practices/integ_pat_intro_overview.htm)

[us.integration_patterns_and_practices.meta/integration_patterns_and_practices/integ_pat_intro_overview.htm](https://developer.salesforce.com/docs/atlas.en-us.integration_patterns_and_practices.meta/integration_patterns_and_practices/integ_pat_intro_overview.htm). In a Salesforce world, this may be the most commonly referenced set of patterns as they're ubiquitous for integration design.

The book *Apex Design Patterns* from Packt, by Anshul Verma and Jitendra Zaa, provides patterns at the software design level and the concrete code level for the Apex language.

The Salesforce Architects site, while new, contains a range of patterns across domains, from code-level specifics to reference architectures and solution kits to good patterns for selecting governance: <https://architect.salesforce.com/design/#design-patterns>.

The point is that we have lots of good patterns to choose from on the Salesforce platform, many of which are provided by Salesforce themselves, though others are provided by the wider community. Many patterns that apply to other platforms are also relevant to us and we learn much by studying them.

But this is a book about anti-patterns, not patterns. So, why am I starting with a discussion about patterns? It turns out that the two are nearly inseparable and originate in the same tradition. So, understanding anti-patterns begins with understanding what a pattern is. One common form of anti-pattern is a *design pattern* that has been misapplied. We'll explore this in the next section.

From pattern to anti-pattern

The design pattern movement in software architecture originates from the work of Christopher Alexander, an architect whose job was to design buildings rather than systems. In his work *The Timeless Way of Building*, he introduced a pattern template for physical architecture that consisted of a name, a problem statement, and a solution in a standardized format.

The rhetorical structure provided by the Alexandrian template was rapidly adopted by architects of a different kind – the ones that build software. They recognized the power of a standardized way to describe problem-solution sets for communicating good practice. With the publication of the classic Gang of Four book, *Design Patterns: Elements of Reusable Object-Oriented Software*, the use of patterns became mainstream within software development and remains so to this day.

The research on patterns inspired an incipient community of practitioners and researchers in software engineering to think about failure modes of software systems analogously to how design patterns were being used. This happened over an extended period, and it isn't possible to point to anyone in the anti-patterns movement who can be seen as the genuinely foundational figure.

However, many research papers on this topic start with the definition given by Andrew Koenig in *Journal of Object-Oriented Programming* in 1995. This definition says that an anti-pattern is very similar to a pattern and can be confused for one. However, using it doesn't lead to a solution; instead, it has negative consequences.

This definition captures much of the essence and can be combined with the following thoughts from Jim Coplien, another early pioneer. He thought that good patterns in and of themselves weren't

sufficient to define a successful system. You also have to be able to show that anti-patterns are absent.

So, in a nutshell, an anti-pattern is a pattern that occurs in unsuccessful software systems or projects that can look superficially like a good solution but, in practice, gets you into trouble. The following are some common anti-patterns that have been around for ages and are still relevant:

- **Stovepipe** : A system or module that's effectively impossible to change because of how its interfaces are designed. See the following diagram for an illustration:

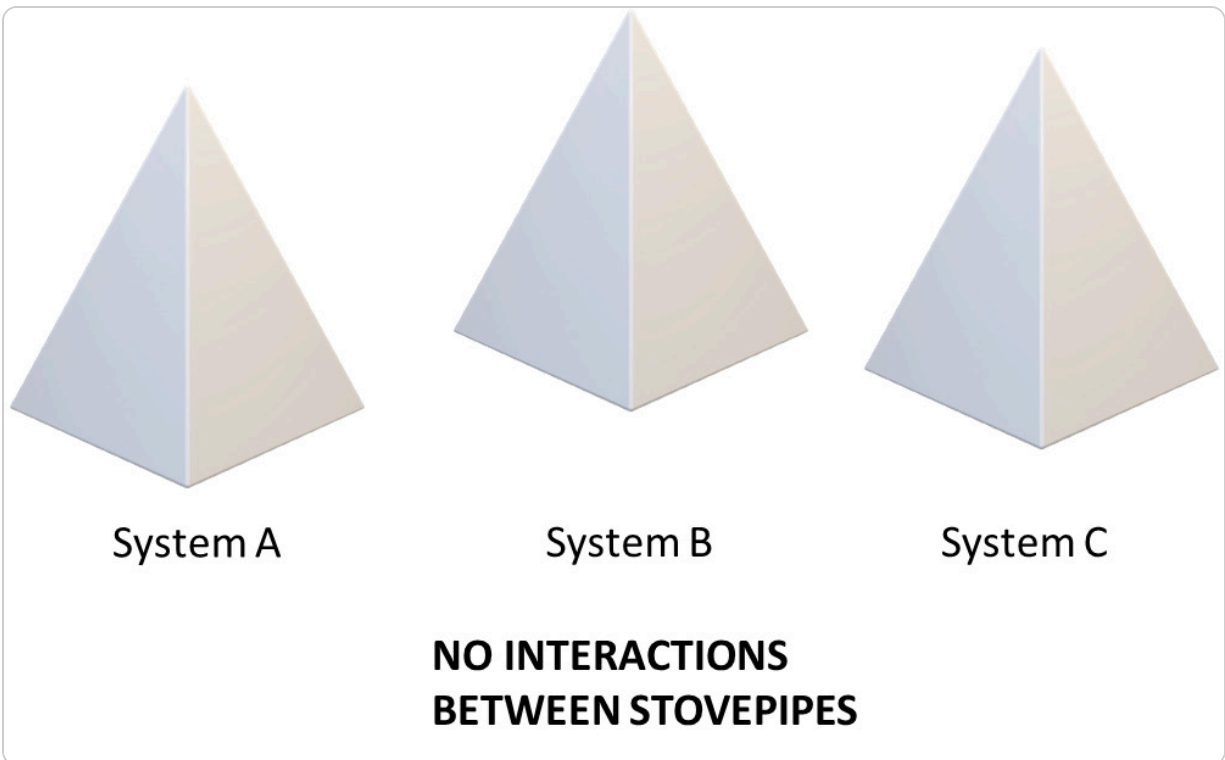


Figure 1.2 – The Stovepipe anti-pattern

- **Blob** : A design where a single class effectively encapsulates all the business logic and functionality, leading to major maintenance headaches.
- **Intensive coupling** : A design that uses an excessively large number of methods from another class or module in implementing a feature, leading to a deep dependency that's hard to understand or change.

We'll dig into all these anti-patterns from a Salesforce perspective in later chapters, underscoring the unfortunate fact that Salesforce, while a great platform, is still a software system that's prone to the kind of mistakes that have plagued software systems for decades. If this weren't the case, then there'd be no need for this book – or Salesforce architects, for that matter.

In addition to having a foundational understanding of anti-patterns, it's crucial to recognize how these patterns align with the latest trends in the Salesforce ecosystem. With the growing interest in AI and DevOps, new anti-patterns have emerged that specifically address challenges in these areas. For

instance, the “ *Let’s add AI to it* ” pattern warns against the temptation to incorporate AI into solutions without a clear, justified need, leading to unnecessary complexity and maintenance challenges.

DevOps practices have also introduced new anti-patterns, such as “ *Infrequent check-in* ,” which underscores the risks of not integrating code changes frequently enough, leading to integration hell and deployment issues. By understanding these new trends and their associated anti-patterns, architects can better navigate the complexities of modern Salesforce implementations.

Having discussed the historical origins of anti-patterns, let’s discuss how they arise in the real world.

Where do anti-patterns come from?

Anti-patterns tend to arise again and again because the circumstances that drive technology projects into trouble are at least somewhat predictable. Equally predictable are the responses of people put into those situations. Since people in tough spots frequently make the same bad decisions, we get systematic patterns to study.

Generally speaking, the most important thing to remember about anti-patterns is that they seem like a good idea at the time. They aren’t simply bad practices that someone should know better than to apply. You can make a reasoned argument that they should lead to good or at least acceptable outcomes when applied.

Sometimes, the decision to apply an anti-pattern is down to inexperience, while other times, it’s down to desperation. But as often as not, it’s down to experienced professionals convincing themselves that in this case, doing what they’re doing is the right call or that this situation is different.

Anti-patterns often arise from predictable circumstances in technology projects, but recent technological trends have introduced new contexts in which these patterns manifest. For instance, the rapid adoption of **artificial intelligence (AI)** and machine learning in Salesforce projects has led to the emergence of new anti-patterns, such as “ *AI architecture is like normal architecture* .” These patterns often arise from a lack of understanding of the unique requirements and challenges associated with integrating AI into Salesforce solutions.

Similarly, the shift toward DevOps practices has highlighted new anti-patterns related to continuous integration and deployment. The “ *Infrequent check-in* ” pattern, for example, arises when teams fail to integrate code changes frequently, leading to significant integration challenges and deployment delays.

We’ll try to reflect this diversity of origins in the examples we’ll provide throughout this book.

However, to do this, we need to explain how we’ll present the examples in a consistent way to enable learning.

An anti-pattern template

One of the key characteristics of both patterns and anti-patterns is that they're written using a recognizable template. Many templates have been suggested over the years, with some more elaborate than others.

The template we'll use in this book contains the bare bones that recur in nearly all existing anti-pattern templates. However, we won't include a great number of optional elements. Most additional elements that are included in other templates serve to ease categorization or cross-referenceability. Those are highly desirable elements to have when creating a searchable catalog of patterns or anti-patterns but are less useful in a printed book. Therefore, we'll omit them and include the following elements instead:

- **Name** : An easy-to-remember moniker for the pattern that serves to identify it uniquely and help with discourse among architects.
- **Type** : In this book, we'll categorize our anti-patterns by the domain from the **Certified Technical Architect (CTA)** review board examination that they're relevant to. This is both to help people on the CTA journey, but also because this is a familiar typology for Salesforce architects.
- **Example** : We'll introduce each anti-pattern by giving an example of how it might occur. The examples will be fictional but grounded in real-world events. This will frame the anti-pattern and give you an immediate understanding of the issues involved before we delve deep into the nuts and bolts.
- **Problem** : This section describes the problem(s) that the anti-pattern purports to solve more formally. These are the real issues that the anti-pattern is meant to be a solution for, although using it in practice will turn out to be a bad idea.
- **Proposed solution** : How the anti-pattern claims to solve the problem described in the previous section and how that can be tempting to believe given certain circumstances.
- **Results** : In the results section, we'll outline what results you can expect from applying the anti-pattern, along with its main variations. We'll explain why this is a bad solution, although it might look good at the outset.
- **Better solutions** : The final section in the template will tell you what to do instead when you're faced with the problem that's the basis for the anti-pattern. Not all problems have easy solutions, but you can generally do better than applying an anti-pattern.

Now that we understand what anti-patterns are and how they're going to be structured in this book, we'll move on and explain how you, as an architect, can improve your skills by using them to learn.

How can anti-patterns deepen your architectural skills?

While a theoretical understanding of what patterns and anti-patterns are and how they may generally be helpful is great, you're no doubt looking for more from this book than just a theoretical discussion.

In this section, we'll go through the value of learning from mistakes, both your own and other people's, and show you how we're going to use anti-patterns to sharpen your architectural chops in the context of a concrete example.

How great architects learn from mistakes

We work in an industry where failure is a normal occurrence. The Standish Group CHAOS report, which is the most commonly cited source on these matters, in 2020 estimated that 66% of all technology projects globally ended in partial or complete failure.

This is a little bit better than when I started my career more than 20 years ago. Then, the figure stood in the low 70s. However, while things have improved, they've only improved a little bit, despite agile development, cloud computing, AI, and great software platforms such as Salesforce.

This is disheartening, but it only proves the point of one of my heroes, Fred Brooks, that “ *The complexity of software is an essential property, not an accidental one,* ” which unfortunately means that we'll never find a silver bullet to solve all problems in software architecture and design. Instead, we're faced with the hard work of learning how to manage this complexity reasonably. To do so is the principal job of an architect.

It stands to reason that in an area with high failure rates and irreducible complexity, we need to have good rules and guidelines to keep us on the right path. That is what patterns and best practices are for. They're indispensable, but they're not enough.

To become great at our profession and to be able to design software that bucks the industry trend, we need to not just learn from our failures but also from the vast repository of failed projects and bad practices we see all around us.

That usually isn't hard to do. Often, when the architect is brought into a project, it's because there's already an issue to fix. Using these occurrences as learning opportunities and analyzing them with that view in mind can be greatly rewarding.

However, there's a step further to go in this direction, which is what anti-patterns offer. They encompass how things frequently go wrong in a way that allows both post-hoc learning but also real-time intervention.

While projects can go wrong, they don't do so randomly. There are systematic patterns that repeat time and again. Learning how things go wrong in systematic ways can give you an entirely new set of responses in your toolbox that you can deploy to help make your project one of the 34% that don't fail.

Understanding anti-patterns not only helps you avoid common pitfalls but also enhances your ability to navigate the complexities of modern Salesforce architectures, including AI and DevOps practices. These insights are particularly valuable for those preparing for the CTA review board examination as they demonstrate a deep understanding of the challenges and best practices in modern Salesforce architecture.

We'll start that learning journey with an explanatory example.

An example – the Hero anti-pattern

There's no better way to start learning than using an example. We won't have the chance to cover many general management-level anti-patterns in this book, so I'll use one of the classics in this genre to show you how the template works and how to read it to get the most out of it.

First, we'll present the anti-pattern and then provide an analysis of what we can learn from it.

Hero (development life cycle and deployment planning)

Tom is the project manager for a large greenfield Salesforce implementation in the manufacturing industry working with Sales and Service Cloud for 2,000 users. The project is meant to be quite simple – a basic MVP to get the platform off the ground – and that's how it's been scoped and staffed. The project is meant to go into full production after 6 months, with 4 months of implementation followed by testing, training, rollout, and hypercare.

The first 3 months of implementation fly by and everything on the project management dashboard stays green. The team makes steady progress and relations with the external consultancy that's helping provide specialist resources remain good.

However, when Tom delivers the first round of business reviews, things start to change quickly. It turns out that a lot of detailed requirements have been missed from the initial scope and that many edge cases aren't covered by the current configuration. The feedback from the business is that they won't be able to go live with the system unless a large list of additional functionality is included.

Tom goes to the steering committee to ask for a bigger budget and a scheduled increase to accommodate the business demands. The steering committee grants the request for an increased budget but tells him that the schedule is immovable. He must find a way to get it done within the current timeline.

Tom replans the project from the ground up. He can make it all fit by compressing the testing and cutover plan if he adds some new resources from the external partner and asks for some overtime from his existing team. He sends out the new plan to the team, along with a rousing email calling on everyone to rise to the challenge.

Over the next month, the project slips again, and Tom's new plan is looking less and less likely to succeed. It's not that anything big goes wrong, but lots of little things just aren't completed on time or

need to be reworked because of misunderstandings. In particular, the new consultants he's brought in from the external partner seem to make a lot of basic mistakes.

Tom calls his boss, the senior director for IT, to tell him about the situation and ask for help in getting an extension to the schedule. She tells him that the schedule has been committed to the board of directors of the company and that heads will roll if it isn't met. She says that this is the time for the team to pull out all the stops and get it done.

Tom goes back to his team to relay the news and once again calls for everyone to give it everything they've got to get things over the line. Unfortunately, most people are already working as hard as their situations allow. In addition, relations with the external partner have soured and they're not willing to put in additional hours without additional funding, which Tom doesn't have in the budget.

There are some bright spots, however. In particular, two young members of the technical staff, Kayleigh and Negash, prove willing to go above and beyond to get things done. Over the final month of delivery, they work 24/7 on the project, with Tom cheering them on:



Figure 1.3 – The dangerous feeling you might have when engaging in the Hero anti-pattern

Between the two of them, they manage to clear away enough of the new features and change requests during the final stretch that Tom becomes confident that he'll be able to meet enough of the requests for the project launch to not be a disaster. There will be exceptions, but he can find a way of managing those later. So long as the impending go-live goes well, the project can still succeed.

However, **user acceptance testing (UAT)** throws a spanner in the works as major quality issues are discovered. The steering committee holds a crisis meeting that ends up concluding that the go-live will have to be postponed for a week. The team will have to work flat out during this period to fix the issues.

While everyone pitches in, the responsibility falls disproportionately on Kayleigh and Negash, who are both starting to show the strain of their continuous effort. Tom encourages them at every chance and singles them out for public praise. He also promises them a cash bonus and extra holidays when the project is done.

The day for retesting arrives and while many issues have been fixed satisfactorily, there are quite a few remaining issues, including a good number that had previously been fixed and are now recurring.

The steering committee holds another crisis meeting and they decide to go ahead with the launch, despite the issues. These issues will need to be fixed during the hypercare period, but they can be tolerated for a short amount of time.

The next few weeks of Tom's, Kayleigh's, and Negash's lives happen in a blur of constant motion. They're pulled from escalation to escalation as issues occur, are fixed, and reoccur. Kayleigh and Negash start buckling under the pressure, but with no alternative resources knowing the configuration, they're effectively forced to carry on.

Eventually, the issues settle down. The important bugs are fixed, the business puts manual workarounds in place for the things that were missed, and life starts to go back to normal. Tom calls the team for a victory celebration, but it's a muted affair.

After taking their extra holidays, Kayleigh and Negash both accept offers from big consulting companies, leaving the company with no one to support large chunks of functionalities on their newly implemented platform.

Problem

The Hero anti-pattern generally purports to fix an urgent delivery problem that has occurred either in a project context, as in our example, or during normal operations. When it occurs in normal operational mode, this is often in a context where firefighting issues with the system have become a run-of-the-mill occurrence.

Usually, the problem occurs in a context characterized by some of the following factors:

- There are limited resources to carry out the work needed to fix the urgent problem and there are good reasons why new resources can't be brought in at this time.
- The project has a tight schedule that's perceived to be set in stone or the issue is live, critical, and affecting important business users adversely in ways that cause a lot of noise.

- There's knowledge about the problem concentrated in a small number of heads – that is, a few people, such as Kayleigh and Negash, who volunteered to take on the role, or frequently a lead developer who's the only one with the technical knowledge to fix the issue at the given time.
- The situation is considered somehow special: either this is a new project and there isn't a precedent, or the issue is considered so unique that you can't plan for it.
- The crisis element is often quite visible in situations that foster the Hero anti-pattern. Sometimes, important parts of the company's future success or even survival are brought into play.

These factors can all make the problem seem more important to fix in a limited time scale and make the Hero anti-pattern seem attractive.

Proposed solution

The Hero anti-pattern proposes to solve the problem described in the preceding section by allowing an individual or a small group to take too much responsibility for resolving it effectively by working as much as is required, even at some cost to themselves, to get things done.

This can be attractive both to management and to the people involved for a variety of reasons:

- The effort does tend to produce some results in the short term, giving a sense of momentum and success.
- Everyone, or at least nearly everyone, wants to be a hero and be singled out for praise and rewards. To some people, that's worth the inconvenience of the additional effort.
- It's always possible to imagine that the current situation is somehow unique and not reflective of a deeper problem with process or culture within the organization, thereby justifying what's done as exceptional.
- Even if we acknowledge that there are underlying issues, often, these can be put out of mind as something to be dealt with later. Of course, in organizations that rely on the Hero anti-pattern, later never comes.

There are several common variants of the Hero anti-pattern that are worth mentioning:

- Superman, a variant where someone, usually a senior technical person, is glorified and held up as the only person who can fix serious issues with a given system. Often, this myth becomes self-perpetuating.
- Rookies, the variant shown in the preceding example, where junior team members take on extra responsibilities to step up to the challenge that's being presented to them.
- No Time for Knowledge Transfer, a situation where heroics are required by a seemingly never-ending time crunch that would make it possible for the hero or heroes to transfer required knowledge to others.

While this anti-pattern is seductive, and many of us have fallen prey to it several times throughout our careers, it almost invariably has negative long-term consequences, something we'll explore next.

Results

While the Hero anti-pattern tends to give good short-term results, which is a major source of its enduring appeal, there's a long list of negative results that tend to accumulate over time in organizations that rely on this anti-pattern to get things done.

Some of the most common negative results are as follows:

- The creation of a single point of failure that increases risks to an organization substantially, should the hero fall under the proverbial bus, and gives the hero a lot of leverage in negotiations with the organization.
- Over time, the hero will start to feel pressure, just like Kayleigh and Negash did in our example, but they'll have very limited options to change the situation. This situation is highly conducive to burnout, which brings with it all the problems of the first point, as well as the risk of the hero making serious errors due to the strain.
- Heroes don't scale. This means that the organization won't be able to deploy projects at a bandwidth that's wider than what the hero can accommodate. This can be seriously limiting to new initiatives in some cases.
- Heroes aren't replicable. You can't replicate the hero or their special powers easily, so you have limited options for creating a predictable and repeatable process.
- Heroes can accumulate serious technical debt. This may often go unmanaged because they must do things quickly, under pressure, and without real supervision. This can lead to major maintenance issues in the long term.
- There's low transparency into the process by which heroes get things done, leading to a lack of predictability and manageability.
- Heroes don't have time to explain how things were implemented, so there's often poor or entirely missing documentation.
- The rest of the team may feel disempowered, overlooked, and demotivated as all the attention goes to the heroes, with little opportunity for others to make contributions in a non-heroic way.

You don't necessarily see all these negative outcomes in all instances of this anti-pattern, and unfortunately, this list isn't exhaustive either. But hopefully, this is enough to make you think twice about applying this anti-pattern and look at better options, something we'll explore next.

Better solutions

The fundamental problem with the Hero anti-pattern is that you're relying on specific individuals with often hidden knowledge working hard – usually too hard – to get things done rather than on repeatable, transparent, and manageable processes that will allow you to continue to deliver, even as the context and the people involved change.

So, the primary way to get away from the Hero anti-pattern is to work on your processes and spread skills and knowledge systematically across the team. In our example, there were potential issues with scope management, the initial discovery work, governance and its understanding of the real issues on the ground, and the way the project had been structured to go live with a big bang rather than in small increments.

How specific interventions will provide the most leverage will vary a lot between organizations, but some good aspects to look at are as follows:

- Moving toward a DevOps culture with smaller incremental releases that have lower risk profiles
- Having multi-functional teams with frequent direct collaboration and peer review to spread knowledge around
- Encouraging and rewarding leads and specialists more for mentoring and bringing up juniors rather than for putting out the latest fire

- Incorporating better risk management and governance in projects so that you can have the right contingencies in place when things go wrong, as they inevitably will
- Challenging the cultural norms that put primacy on delivering big dramatic wins against the odds, rather than on making steady, undramatic, but repeatable progress regularly
- Emphasize roles and processes, not individuals when planning, building, and operating systems, especially when communicating with the wider stakeholder community
- Make the costs of the Hero anti-pattern visible by capturing the technical debt, the risk, and the missed opportunity to be able to replicate efforts that the organization loses by relying on this pattern
- Ensure that detailed requirements and edge cases are planned for when beginning the project to reduce the probability that you'll need a hero

In truth, it isn't always possible to completely avoid the Hero anti-pattern. Sometimes, things are on fire and there's only one person around who can fix it. What we need to recognize is that this isn't a good situation but an anti-pattern that we need to work hard to fix so that it doesn't recur. The more you do this, the less you'll have to rely on heroes and the fewer fires you'll have to put out.

Having covered our first example of a real anti-pattern, let's analyze it a little more deeply to see how we can maximize our learning from it.

Analyzing the example

The Hero anti-pattern is a classic and most seasoned IT professionals will have encountered it during their careers. However, interesting as it is, in this book, we'll also be looking to pull out the larger patterns so that we can learn from our examples to hone our architectural skills.

Throughout this book, we'll do this by having a section toward the end of each chapter that extracts key learning points for you to take on your future architecture journey. We'll do this while implementing a mind-to-real-life practice, but we'll also list learning points specifically for those who are on track toward the CTA exam.

Considering the Hero anti-pattern, the following are a few learning points you might extract for real-life practice:

- When you're faced with a crisis that calls for extraordinary effort on the part of some or all of the team, take the time to step back and consider the process failures that led to this situation. Capture this for future use.
- Relying on a small number of extremely skilled individuals can be dangerous in the long run, even if it's useful right now.
- The pressure you might feel toward going above and beyond may reflect a culture that doesn't have its priorities right from a technical point of view. You may want to challenge that if possible.
- Go out of your way to empower and bring up junior staff to avoid being in a position where you have to be the hero.
- Be diligent about advocating for good governance both at the project and technical levels, as well as capturing and remedying the technical debt that accumulates from "special" situations.

Looking at the CTA review board, you can note the following lessons:

- Be careful about suggesting big-bang delivery approaches. They can be the right choice, but frequently, they can lead to the issues highlighted in this example. Prefer using agile, incremental approaches unless there's a specific reason not to.
- Ensure that you include the necessary governance functions, including a **project management office (PMO)**, steering committee, design authority, and maybe a change advisory board. In this example, much could have been avoided if the latter two had been in place.
- Be explicit about risks and risk management. Include risks upfront and be ready to talk about how to manage and mitigate them.

While we'll be able to pull out many learning points directly, we encourage you to go further with this method and see how much more you can get out of them. Learning from anti-patterns is a continuous and very rewarding activity for aspiring architects.

In addition to the traditional lessons learned from the Hero anti-pattern, it's important to consider how this pattern intersects with modern trends such as AI and DevOps. For instance, in a DevOps environment, the reliance on a single "hero" to manage deployments can lead to significant bottlenecks and integration issues, highlighting the importance of practices such as continuous integration and frequent check-ins.

Similarly, in AI projects, the Hero anti-pattern can manifest when a single data scientist or AI specialist is relied upon to deliver complex AI solutions without adequate support or knowledge transfer. This can lead to burnout and a lack of scalability, underscoring the need for collaborative, multi-functional teams.

With that, we have a foundational understanding of what anti-patterns are and how they can help us achieve greater mastery of Salesforce architecture. All that's left to do is summarize our progress before we dive into the deep end by looking at anti-patterns in the system architecture domain.

Summary

Phew – that was a lot of background! Thank you for sticking with it. You're now in a position to move forward with this book since you understand the key facts about what anti-patterns are and how you can learn from them.

Here are a few things to remember as you go into the next chapters:

- It's important not to simply think about anti-patterns as bad practices. Bad practice is obvious, but anti-patterns are nefarious. They masquerade as good solutions and can be persuasive in certain situations, even to experienced professionals. That's also what makes them so fun to study.
- While we'll be presenting a lot of individual anti-patterns in this book, it's important to draw out the parallels and meta-patterns that can be gleaned by comparing and contrasting them. We'll do some of that work for you in the analysis sections that can be found throughout this book. You should also try to do this activity for yourself.

- While we'll be focusing on a Salesforce context, many of the patterns you'll learn about apply to other platforms as well. Some are even classics that will be known by a great number of grizzled architects.

As we move forward with this book, remember that anti-patterns aren't just bad practices but deceptive solutions that can seem appealing in certain situations. In this second edition, we've expanded our exploration to include new anti-patterns related to AI and DevOps that reflect the latest trends and challenges in the Salesforce ecosystem. By understanding these patterns and their implications, you can enhance your architectural skills and better navigate the complexities of modern Salesforce projects, whether in real life or in preparation for the CTA review board examination.

With that out of the way, we're now ready to look at our first architectural domain in detail: systems architecture.

2

How to Keep Your System Architecture Clean

This chapter will start by exploring how you can muddle up your system landscape by not retaining a disciplined approach to implementation and governance. We'll examine the classic Stovepipe anti-pattern, where systems are built in isolation with little regard for standards or common abstractions, leading to a fragmented and hard-to-maintain architecture. We'll also look at the Big Ball of Mud anti-pattern, characterized by a lack of structure and organization, resulting in an unmaintainable system.

Next, we'll introduce the Architecture as Requirements anti-pattern, a common pitfall where architectural decisions are treated as mere requirements without considering the broader system context and long-term implications. We'll delve into the consequences of this approach, such as increased technical debt, scalability issues, and limited adaptability.

We'll then discuss how excessive coupling between systems can make them fragile and difficult to modify, emphasizing the importance of defining clear boundaries and favoring looser integration patterns. We'll also touch on the challenges of org proliferation in Salesforce environments and the need for a well-defined org strategy to prevent sprawl and ensure consistency.

Throughout the chapter, we'll highlight the critical role of architectural governance, technical standards, and best practices in avoiding these anti-patterns and creating robust, maintainable Salesforce systems. We'll provide insights on how to navigate organizational silos, secure stakeholder buy-in, and balance individual requirements with long-term architectural sustainability.

In today's rapidly evolving technological landscape, **artificial intelligence (AI)** and automation play increasingly critical roles in Salesforce architectures. As we explore various anti-patterns, we'll also consider how AI-driven features and automated processes can either exacerbate these issues or, when properly implemented, help mitigate them. Understanding the interplay between traditional architectural concerns and emerging AI capabilities is crucial for creating robust, future-proof Salesforce solutions.

By the end of this chapter, you'll have a deep understanding of key architectural pitfalls to avoid and strategies to employ when designing and implementing Salesforce solutions. You'll be equipped with the knowledge and skills needed to create systems that are scalable, adaptable, and aligned with your organization's evolving needs.

In this chapter, we're going to cover the following main topics:

- How the Stovepipe anti-pattern can leave us with systems that can only work in isolation and what you can do to avoid building stovepipes

- What a Big Ball of Mud is, why a system can deteriorate into a state that fits this description, and how you can prevent it from happening to your system
- Exploring the Architecture as Requirements anti-pattern and its implications for Salesforce implementations
- How intense coupling makes systems fragile and potentially unusable, and how to draw clearer system boundaries that avoid this issue
- The challenges of ungoverned org proliferation in Salesforce environments and the importance of a well-defined org strategy
- The critical role of architectural governance, standards, and best practices in creating maintainable and evolvable Salesforce systems

After completing this chapter, you will have a solid grasp of common architectural anti-patterns that can afflict Salesforce implementations and strategies to mitigate them. You'll be prepared to lead the charge in establishing a strong architectural foundation for your organization's Salesforce initiatives, ensuring they deliver lasting value and support your business objectives.

Muddling up the systems landscape

The easiest way to muddle up your systems landscape, as we will see, is to go ahead with implementation in a local, unstructured, and badly governed way. While there are many attractions to small local projects, they very easily deteriorate into anti-patterns that have serious negative consequences for your overall enterprise architecture. We will start by looking at the classic Stovepipe anti-pattern, which is a common outgrowth of such projects, and then look at its organizational cousin, *Stovepipe Enterprise*.

Stovepipe

A stovepipe is a system that is built with no attention to standards or common abstractions, leading to a system that may work but is hard to maintain, extend, or interoperate with.

An example

John is the CRM manager at DreamCo, a provider of bespoke travel accessories. The company has decided to invest in a small implementation of Salesforce Sales Cloud, replacing an old Siebel system that's been in operation for more than a decade. While initially fearful, John has come to be quite excited about the project as it promises to fix a number of thorny issues that they've been unable to address with the old CRM.

DreamCo hires a small local Salesforce consultancy to do the implementation. Initially, it is a great success; the sales teams love the new tool, and many of the features requested by John are readily implemented. However, after a while, progress seems to slow, and the price of changes goes up. DreamCo's CIO makes the assessment that their business requirements are now too complex for the small local consultancy to manage and hires a leading Salesforce Summit partner to take over.

At the same time, DreamCo's head of customer service decides to commission a third Salesforce partner to implement Salesforce Service Cloud for the company's call center. John has been learning a lot about Salesforce during the initial period and is starting to get worried that there are no common conventions, standards, methodologies, or tools across the different partners or implementations. The sales department and the customer service department also seem to use very different data models to represent fundamentally the same things.

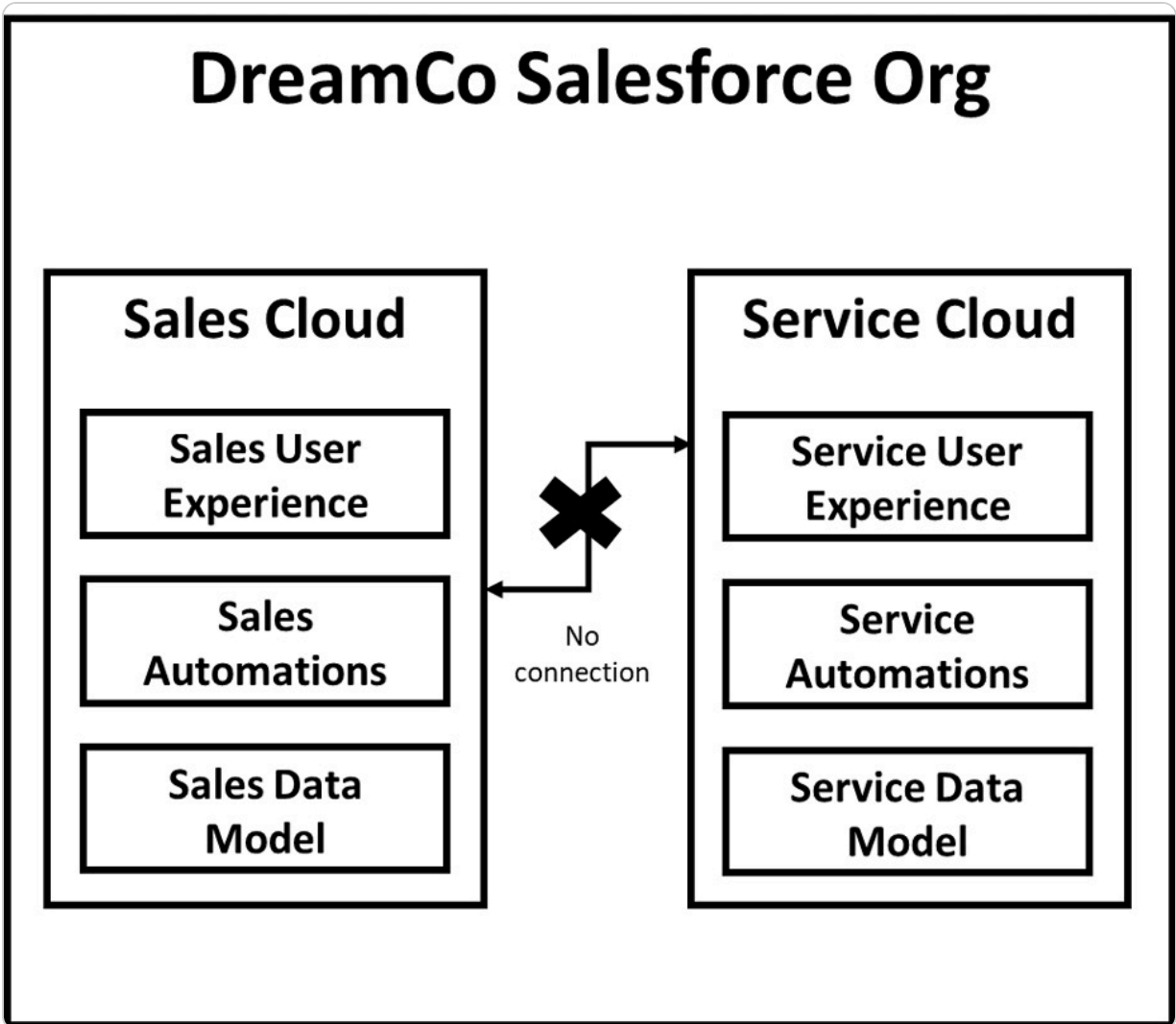


Figure 2.1 – DreamCo’s org after the initial implementations

John raises these points to the CIO and asks for a budget to hire an architect and to do some refactoring on the existing systems to bring them more in line. He is told that while his idea is good, it isn’t affordable right now, but he should put in a budget request for next year’s budget.

At this time, DreamCo purchases a smaller company that sells travel accessories via vending machines in airports. They reportedly have a very strong Salesforce implementation, much more advanced than DreamCo’s, including sales, service, and field service modules. John is given the task of finding a way to consolidate the two Salesforce orgs into one without requiring the two organizations to fundamentally change their processes.

John engages with the Summit partner to explore options for the consolidation. They come back with a proposal that meets the basic requirement but at a cost that is much higher than expected. The partner explains that the excessive technical complexity in the DreamCo org makes it very difficult to

integrate with the new org. Furthermore, they advise against going down the consolidation route and instead advise DreamCo to move everyone to the org from the company that has been recently acquired and change the processes to make this work without changing the technology.

After some internal discussion, DreamCo decides to go ahead with the consolidation anyway.

Problems start appearing almost right away:

- First, it proves very difficult for the different vendors that need to be involved to collaborate effectively, leading to a situation where the lead partner is effectively required to reverse-engineer a number of features in order to understand them.
- Second, the implementation progress is very slow, and John realistically can see no way for the project to complete on time.
- Third, the error rate is very high, and there are many recurring errors on each test cycle. All of this is leading toward a project that will be significantly over budget, behind schedule, and below expectation on quality.

John starts digging into the detail, and from the various technical people involved in the project, he learns a number of disturbing facts. First, basic things such as naming conventions are completely different, not only between the DreamCo and the NewCo org but also within the DreamCo org; at least three completely different sets of conventions exist. In addition, features are implemented using completely different methodologies and toolsets, and sometimes the same feature is reimplemented in different ways in different parts of the system.

There are also a number of custom fields and objects that exist in subtly different duplicated variants in different parts of the system. Finally, a number of third-party tools have been used to provide functionality. However, these tools and their functions were never properly documented, and no one in the organization knows how to use them after the change of vendors. Adding this to the inherent complexity involved in finding a common language between the DreamCo and the NewCo orgs, most of the technical teams are starting to throw their hands up in despair.

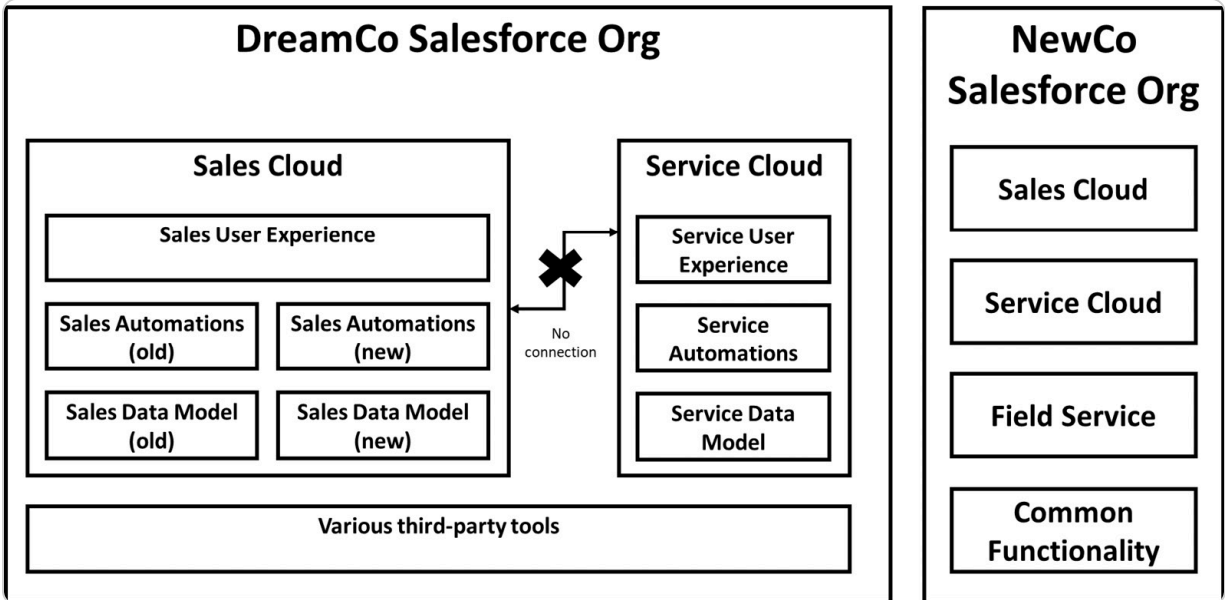


Figure 2.2 – The DreamCo consolidation scenario

John reports this back to senior management, and the situation escalates into a major crisis. Eventually, after much wrangling, DreamCo gives up on consolidating the two orgs and decides to leave them in place separately instead for the time being, with data consolidated elsewhere for reporting.

The problem

How do you implement an IT system in an organization where departments are siloed, power is distributed into individual fiefdoms, technical teams work independently of each other, and there is no strong central authority on architecture or technical standards? The answer, in general, is the *Stovepipe anti-pattern*. Given the prevalence of these organizational characteristics, Stovepipe is one of the most common anti-patterns you are likely to encounter as an architect.

Let's not disregard how hard it is to get around some of these issues. Changing organizational structures or cultures can require years of transformation, even with senior management support, and even then, many unfortunate practices may still linger.

So, when you are an ambitious manager with the need for a new IT system in the kind of organization just described, it is very tempting to simply go ahead within your own little kingdom and implement something, working with people that you personally trust, in a way that works for you, without much consideration for the rest of the business. Without strong corporate IT governance and enterprise architecture support, there is little most IT departments can do about this.

The Stovepipe anti-pattern, then, does provide a real answer to a real problem, as we will discuss in the next section. However, it is an answer that reliably leads to unfortunate results.

The proposed solution

Fundamentally, a stovepipe proposes to solve coordination problems inherent in many organizations by building a solution in splendid isolation, not taking into account any wider concerns or requirements, using whatever tooling and standards the implementing team prefers. That way, you can simply go ahead while short-circuiting corporate bureaucracy in the process.

This is an attractive solution in a number of ways:

- You do not have to engage in cross-silo collaboration or resolve thorny internal political issues
- Your technical teams often love it because they can use whatever tools they prefer to get the job done
- It tends to work very well in the short term, quickly delivering on requirements and generating local value, often at a very reasonable price point
- For organizations without strong central IT leadership, it can be the default way to operate, and it may simply become “the way things get done”

If we were living in a simpler world, where organizations could compartmentalize their operations between discrete groups that didn’t have to collaborate much to get things done, and consequently, systems didn’t have to support cross-cutting workflows or cross-system integrations, then a stovepipe would actually be a perfectly reasonable approach to software development.

However, in practice, most organizations do need most of their systems to work across a wide range of departments and use cases, which is when we start to get into trouble, as we’ll see in the next section.

The results

As previously pointed out, and as we saw in our example, a stovepipe often initially works well. For a point solution used by a single team, there isn’t any inherent downside to building in this local way. As we saw, issues start to accumulate when we have multiple players on the same underlying platform (for example, the sales and service organizations we mentioned), when we try to extend the system to use cases it wasn’t designed for, and when we need it to play well with other external systems.

On Salesforce, it is normal to have many different modules in simultaneous operation in the same org that leverage a common data model, user interface, and technical configuration. Unfortunately, this makes it particularly easy to end up with stovepipe-type problems if you do not implement these modules consistently.

Over time, the indicators of a Stovepipe anti-pattern tend to be the following:

- An increasing difficulty in understanding the code base, especially for new parties working with the system

- Inconsistent architectural design and implementation patterns in different areas of the system
- A diminishing user experience and user value due to the increasing inconsistency and inflexibility of the system
- Great difficulty whenever new requirements have to be implemented
- The system is hard to integrate with other systems in the landscape
- The system is hard to incorporate into global processes; for example, a consolidated DevOps pipeline
- Increasing error rates, both during development and in production
- A higher and higher cost of change due to all these factors

The fundamental reason a stovepipe tends to deteriorate in this way is due to a lack of common standards, practices, patterns, and tooling. Inconsistency increases the cognitive load on the technical team, making everything harder and more error-prone, as well as raising the learning curve across the board. The original team might have understood it, but over time, this understanding will be lost, and the result tends to be more inconsistent rather than a concerted effort being made to refactor it into a more comprehensible state.

You can profitably compare a team building a stovepipe to a well-functioning agile team to understand what goes wrong. An agile team may also work in relative isolation with empowered product owners calling the shots. However, they will work with standards, practices, patterns, and tools that are accepted and well understood in the organization, and they will work with stable interfaces for key integrations and common abstractions for crucial system components. That way, coordination happens implicitly via that common baseline and you do not end up with a stovepipe but, rather, a well-adjusted member of the system landscape.

Better solutions

The fact is that many organizations are quite siloed, and it therefore becomes hard to solve the business-level coordination problems that arise when building, implementing, or changing systems. As architects, dealing with these issues should be part of our bread-and-butter activities in order to avoid building stovepipe systems and ensure that our platforms, such as Salesforce, continue to deliver value over the long term.

While some of the more transformative things that can be done organizationally to break down silos and improve collaboration are often beyond our power, there are many things that we can reliably do to improve the situation:

- Define clear architectural patterns and practices that can be used between teams to create common abstractions
- Put in place conventions for how different things are to be implemented, both at the macro level (for example, when to use flows and when to use triggers) and micro level, such as a coding guideline
- Adopt standards for all technical areas, such as reporting and BI or data governance, in order to ensure consistency

- Have strong guidance for creating stable interfaces for cross-system integration, as well as integration patterns for different use cases
- Put in place a common methodology for implementation across Salesforce projects to ensure consistency
- Adopt consistent tooling for all elements of the development life cycle to enable cross-team and cross-system understanding
- Enforce these standards and practices vigorously

We are in a fortunate position as Salesforce architects in that much of this work can be taken from good practices already in existence, either established by Salesforce themselves or the wider Salesforce community. You can find many good resources on the Salesforce Architects website, and Salesforce has also recently introduced the **Salesforce Operating, Governance, and Architecture Framework (SOGAF)** to help us understand the best practices for Salesforce governance:

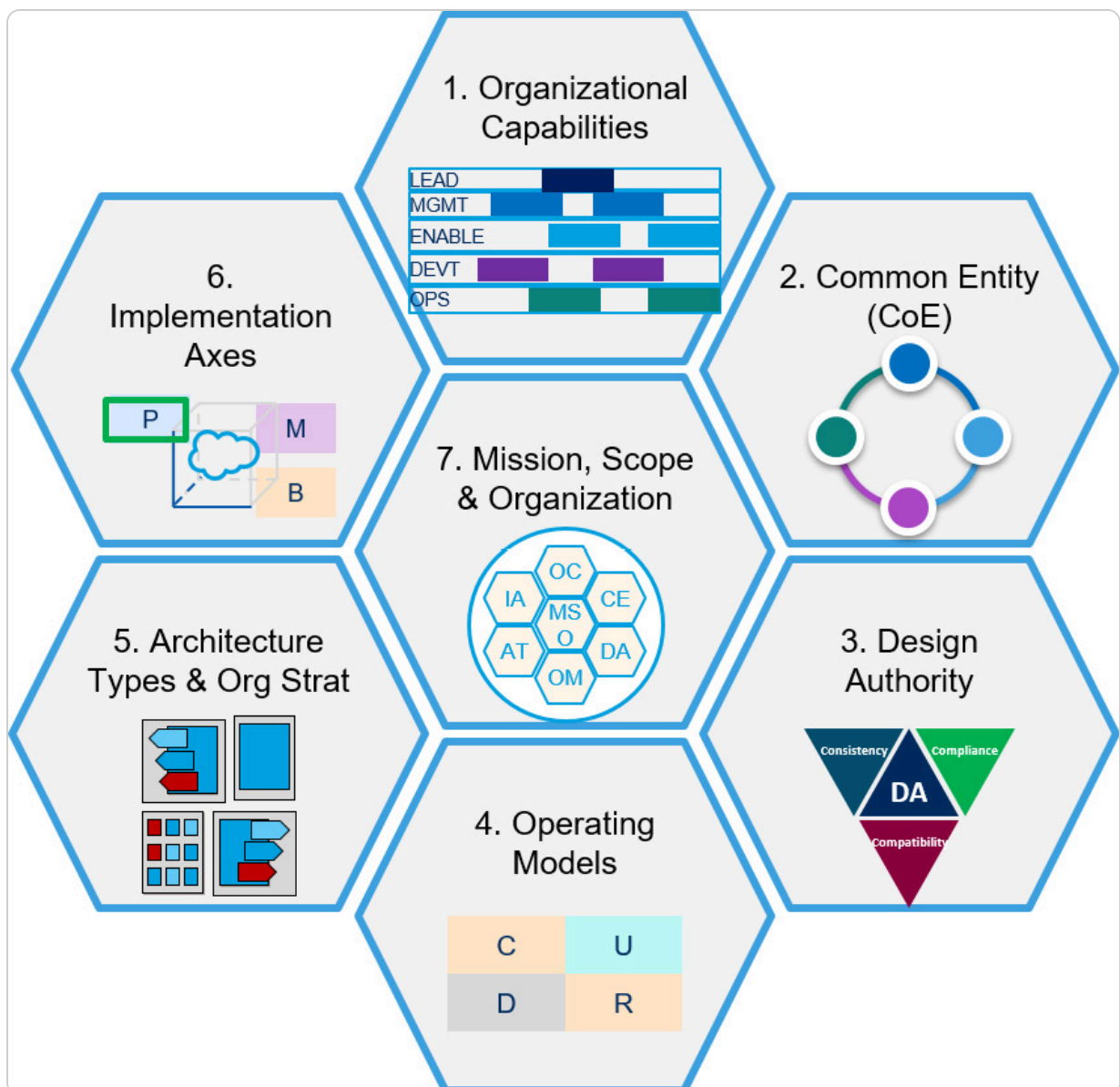


Figure 2.3 – SOGAF elements

However, we always have the responsibility to adopt, adapt, and implement it within our own organizations ourselves. This also includes adding business-specific standards and practices that are unique to our situation and finding ways of communicating the standards and practices in a way that resonates across our organization.

When implementing AI capabilities in Salesforce, be wary of creating AI stovepipes. Isolated AI models or predictive analytics tools that don't integrate well with the broader Salesforce ecosystem can lead to fragmented insights and inconsistent user experiences. Instead, aim for a cohesive AI strategy that aligns with your overall Salesforce architecture, ensuring AI-driven features complement and enhance existing processes across the platform.

It should be noted that sometimes standards should be broken. There are cases where you need different tools, methods, or patterns to get the job done, but you should have a standard to deviate from if you want to avoid ending up building stovepipes.

A note on Stovepipe Enterprise

The material we have discussed on this pattern so far has focused primarily on the system level. However, it will not have escaped a more perceptive reader that many of the issues that appear on the system level are related to organizational issues, particularly the tendency for some organizations to work in silos and be unproductively political.

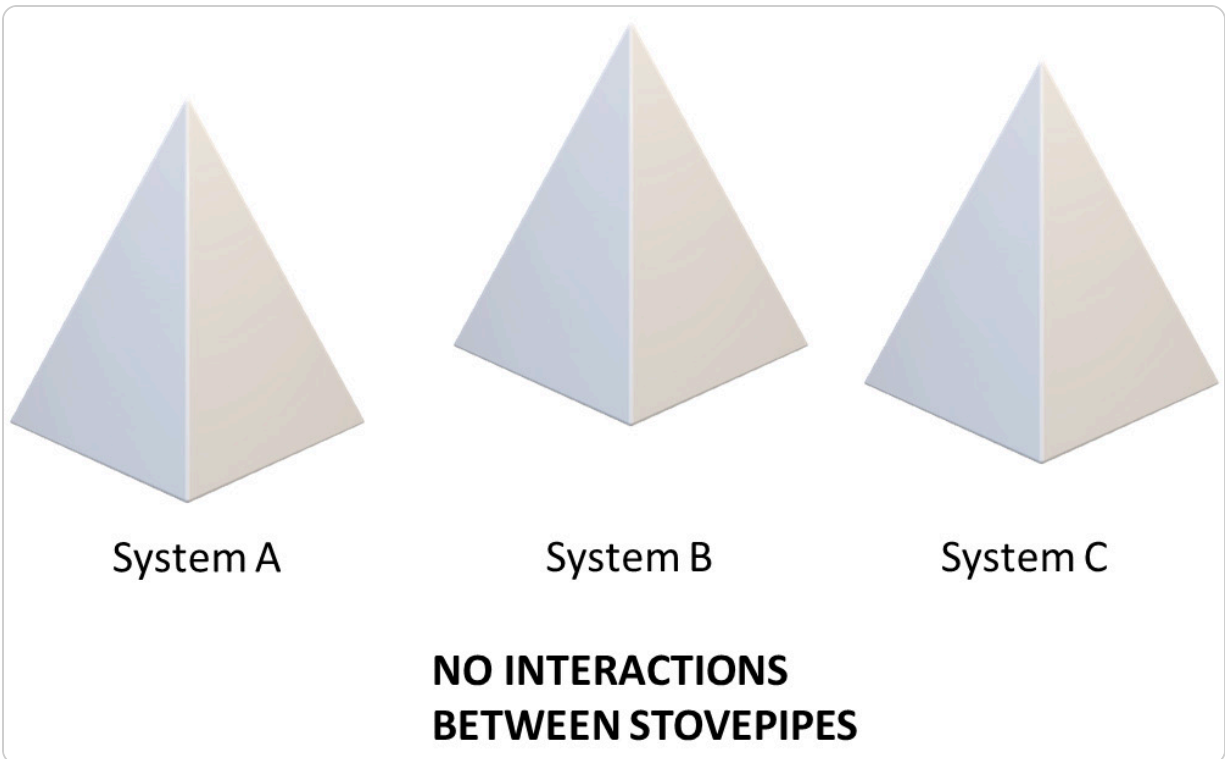


Figure 2.4 – A landscape of stovepipes making up a stovepipe enterprise

As a result, there's an organizational-level version of the Stovepipe anti-pattern called Stovepipe Enterprise, which focuses on this tendency of organizations to produce stovepipe systems due to their structural problems. While we can readily acknowledge this anti-pattern, as Salesforce architects, our remit does not usually extend to this level of problem, and we have therefore mostly focused on the system level in our discussion.

Architecture as Requirements

The Architecture as Requirements anti-pattern occurs when organizations treat architectural decisions as just another set of requirements to be gathered and implemented rather than as a fundamental framework that guides the system's design and evolution.

An example

Sarah Chen, the newly appointed Enterprise Architect at GlobeTech Industries, leaned back in her chair and sighed. She had been brought in to help modernize the company's aging CRM system, and the Salesforce implementation project seemed like the perfect opportunity to establish a robust, scalable architecture. With 15 years of experience under her belt, Sarah was known for her meticulous

planning and ability to see the big picture. However, as she stared at the project timeline pinned to her office wall, she couldn't shake the feeling that things were already veering off course.

The kick-off meeting for the Salesforce implementation had been a whirlwind of excitement and ambitious goals. Department heads from Sales, Marketing, and Customer Service had all presented their wish lists, each more elaborate than the last. Sarah had tried to steer the conversation toward establishing overarching architectural principles, but her suggestions were met with blank stares and impatient nods.

"We don't have time for all that high-level stuff," Jack, the Sales Director, had said dismissively. "We need to get this system up and running ASAP. Our competitors are eating our lunch with their fancy AI-powered sales forecasting."

Sarah had attempted to explain the importance of a solid architectural foundation, but her words seemed to fall on deaf ears. The project manager, Tom, was more concerned with ticking off requirements and meeting deadlines than considering the long-term implications of their decisions.

As the weeks progressed, Sarah found herself fighting an uphill battle. She had drafted a comprehensive architectural vision document, outlining key principles for data governance, integration patterns, and scalability considerations. But when she presented it to the steering committee, she was met with a mix of confusion and indifference.

"This all sounds great, Sarah," the CIO had said, glancing at his watch, "but can you distill this down into a list of requirements we can actually implement? We need concrete deliverables, not abstract concepts."

Frustrated but undeterred, Sarah had tried to work within the confines of the project's requirement-driven approach. She attended sprint planning meetings, hoping to inject architectural considerations into the development process. But more often than not, she found herself watching helplessly as the team made decisions that would have far-reaching consequences for the system's future flexibility and maintainability.

One particularly contentious issue arose around the implementation of an AI-powered lead scoring system. The sales team was adamant about having this feature, insisting it was crucial for their competitive edge. Sarah recognized the potential value but was concerned about how it would be integrated into the overall Salesforce architecture.

"We need to carefully consider how this AI system will interact with our existing data models," Sarah had argued during a project meeting. "We should also think about the long-term governance of the AI models and how we'll manage potential bias in the algorithms."

But her concerns were brushed aside. Jack from Sales had already been in talks with a third-party vendor, and they were eager to move forward.

“Look, Sarah,” Tom had said, trying to be diplomatic, “I understand your concerns, but we’ve got a tight deadline to meet. Let’s just get the system implemented, and we can worry about the architectural stuff later.”

Sarah had left the meeting feeling deflated. She knew that treating the AI implementation as just another requirement to be checked off would likely lead to problems down the line. But without buy-in from the rest of the team, her hands were tied.

As the project progressed, Sarah’s fears began to materialize. The hastily implemented lead scoring system was causing performance issues, bogging down the entire Salesforce instance during peak usage times. The data model had become a tangled mess, with redundant fields and inconsistent naming conventions making it difficult to generate accurate reports.

Integration with other systems was becoming a nightmare. The team had opted for quick, point-to-point integrations rather than following Sarah’s proposed API-first approach. As a result, data inconsistencies were cropping up across different departments, and any changes to one system had a ripple effect that was hard to predict or control.

User adoption was also suffering. The lack of a coherent architectural vision had resulted in a disjointed user experience, with different modules feeling like separate applications rather than parts of a unified whole. Sarah had lost count of the number of complaints she’d received about inconsistent navigation and confusing workflows.

6 months into the project, things came to a head. The CEO called an emergency meeting to address the mounting issues with the Salesforce implementation. Sarah sat in the conference room, surrounded by frustrated executives and a demoralized project team.

“I thought this new system was supposed to make our lives easier,” the CEO said, his voice tinged with disappointment. “Instead, it seems like we’ve just created a bigger mess. Can someone please explain how we got here?”

All eyes turned to Sarah. She took a deep breath, knowing that this was her chance to finally make her case for the importance of architecture.

“The root of our problems,” Sarah began, “is that we’ve been treating architecture as just another set of requirements rather than as the foundational framework it should be. We’ve been so focused on delivering individual features that we’ve lost sight of the bigger picture.”

She went on to explain how the lack of architectural governance had led to the current situation: a fragmented system that was difficult to maintain, scale, or adapt to changing business needs. She outlined how a proper architectural approach could have prevented many of the issues they were facing and could still help them course-correct.

To her surprise, she saw nods of understanding around the room. Even Jack, who had been one of the most vocal opponents of her architectural suggestions, looked sheepish.

“I guess I was so focused on getting those fancy AI features that I didn’t think about how it would all fit together,” he admitted.

The CEO leaned forward, his interest piqued. “So, what do you propose we do now, Sarah? Can we salvage this project?”

Sarah felt a glimmer of hope. She quickly outlined a plan to pause new feature development and focus on refactoring the existing implementation to align with proper architectural principles. She proposed establishing an Architecture Review Board to ensure future decisions were made with the overall system architecture in mind.

“It won’t be easy,” she cautioned, “and it might mean some short-term pain. But if we don’t address these architectural issues now, they’ll only become more expensive and difficult to fix down the line.”

To her relief, the room was receptive. The CEO gave his approval for Sarah’s plan, and she left the meeting feeling a mix of vindication and renewed determination.

Over the next few months, Sarah led the charge in restructuring the Salesforce implementation. It was challenging work, requiring long hours and difficult conversations. But gradually, she saw the system transform into something more cohesive and maintainable.

The AI-powered lead scoring system was re-implemented, this time with proper consideration for data flows and model governance. The team adopted a more standardized approach to integrations, creating a flexible API layer that could accommodate future changes more easily. Data models were refactored to eliminate redundancies and improve consistency.

As the architecture solidified, Sarah noticed a change in the team’s dynamics. Developers began to appreciate the clarity that came with a well-defined architectural framework. Project managers found it easier to plan and estimate work. Even the business stakeholders, who had initially been skeptical of the “architectural stuff,” started to see the benefits in terms of improved system performance and adaptability.

A year after that pivotal meeting, Sarah stood in front of the company’s annual all-hands meeting, presenting the success story of the Salesforce implementation. The system was now a point of pride for

GlobeTech, with high user adoption rates and a newfound ability to quickly roll out new features and adapt to changing market conditions.

“The lesson we learned,” Sarah said as she concluded her presentation, “is that architecture isn’t just a set of abstract principles or another box to tick off in our project plans. It’s the foundation that allows us to build systems that truly serve our business needs, both now and in the future.”

The problem

The core problem the Architecture as Requirements anti-pattern tries to address is the challenge of designing a system architecture that meets diverse and sometimes conflicting stakeholder requirements.

In complex projects with multiple stakeholders, each group often has its own priorities, technology preferences, and functional demands. Balancing these competing needs while ensuring overall system coherence and long-term viability is a significant architectural challenge.

Without a clear architectural vision and guiding principles, project teams may fall into the trap of treating architecture as just another set of requirements to be gathered and implemented piecemeal. They focus on fulfilling individual stakeholder demands without considering the bigger picture and the long-term implications of their design decisions.

This approach can lead to several issues:

- **Lack of system-level thinking** : By treating architecture as a collection of discrete requirements, teams lose sight of the overall system context and fail to consider critical aspects such as performance, scalability, maintainability, and interoperability
- **Short-term focus** : Prioritizing immediate stakeholder needs over long-term architectural sustainability can result in quick fixes and band-aid solutions that accumulate technical debt and hinder future growth
- **Inconsistent user experience** : When each module or feature is designed in isolation without a cohesive architectural vision, users may encounter inconsistent interfaces, navigation, and functionality across the system
- **Integration challenges** : Piecemeal architecture often leads to a proliferation of point-to-point integrations and custom interfaces, making the system brittle and difficult to maintain over time
- **Limited adaptability** : A fragmented architecture lacking clear abstractions and **separation of concerns** (SoC) makes it harder to accommodate changes and add new features as business requirements evolve

Overall, issues leading to Architecture as Requirements are common, and therefore, it occurs with some frequency.

The proposed solution

The flawed solution proposed by the Architecture as Requirements anti-pattern is to treat architectural decisions as just another set of requirements to be gathered and implemented incrementally, without a cohesive overarching vision.

Under this approach, project teams engage with stakeholders to collect their specific functional and technical requirements, often prioritizing them based on individual needs and preferences. They then attempt to design and build the system incrementally, addressing each requirement as it arises without much consideration for the broader architectural implications.

This solution may seem appealing initially, as it appears to be responsive to stakeholder needs and allows for quick progress on individual features. Stakeholders feel heard and see their specific demands being met, creating a sense of satisfaction and buy-in.

However, this approach fails to recognize that architecture is more than just a sum of its parts. It is a foundational framework that shapes the system's structure, behavior, and evolution. Treating architecture as a collection of isolated requirements ignores the critical role it plays in ensuring system-wide quality attributes, such as performance, scalability, security, and maintainability.

The proposed solution also tends to favor short-term gains over long-term sustainability. By prioritizing immediate stakeholder needs and rushing to implement features without a well-thought-out architectural plan, teams accrue technical debt and create a fragmented, inconsistent system that becomes increasingly difficult to maintain and extend over time.

The results

The consequences of falling into the Architecture as Requirements anti-pattern can be severe and far-reaching. Over time, the system becomes a complex web of tightly coupled components, custom integrations, and inconsistent design patterns, making it fragile, inflexible, and expensive to maintain.

Some common results include the following:

- **Technical debt accumulation** : As the system grows without a coherent architectural vision, quick fixes and ad hoc solutions pile up, leading to an accumulation of technical debt. This debt manifests as convoluted code, performance bottlenecks, and increased maintenance efforts.
- **Scalability and performance issues** : The lack of a well-designed architecture often leads to scalability and performance problems as the system struggles to handle increasing loads and data volumes. Inefficient resource utilization, excessive database queries, and poorly optimized code contribute to sluggish system performance.
- **Integration complexity** : Without clear architectural boundaries and standardized interfaces, integrations between different modules or external systems become a tangled mess of custom code and brittle connections. Changes in one part of the system can have unpredictable ripple effects, making integrations fragile and error-prone.
- **Limited reusability and maintainability** : The absence of a modular, loosely coupled architecture hinders code reusability and maintainability. Developers struggle to understand and navigate the code base, leading to increased development time and a higher risk of introducing bugs when making changes.
- **Resistance to change** : As the system becomes more complex and entangled, it becomes increasingly difficult to introduce new features or adapt to changing business requirements. The architecture's rigidity and lack of clear SoC make modifications time-consuming, costly, and prone to unintended side effects.

- **User frustration and workarounds** : Inconsistent user experiences, performance issues, and limited functionality can lead to user frustration and dissatisfaction. Users may resort to manual workarounds or alternative tools to circumvent the system's limitations, undermining the intended benefits and efficiency gains.
- **Increased costs and delayed time-to-market (TTM)** : The cumulative effect of technical debt, maintenance challenges, and development inefficiencies can significantly increase costs and delay the delivery of new features and enhancements. The organization may find itself investing more resources in firefighting and system overhauls rather than focusing on innovation and business value delivery.
- The allure of AI can sometimes lead organizations to treat AI capabilities as simple feature requirements rather than architectural considerations. This approach can result in haphazard AI implementations that fail to leverage Salesforce's native AI capabilities or integrate poorly with existing data models and business processes. To avoid this, approach AI initiatives with a holistic architectural mindset, considering data flows, model governance, and the long-term implications of AI-driven decision-making within your Salesforce ecosystem.

Ultimately, the Architecture as Requirements anti-pattern can lead to a system that fails to meet its intended goals, hinders organizational agility, and becomes a burden rather than an enabler for the business.

Better solutions

To avoid falling prey to the Architecture as Requirements anti-pattern and establish a solid foundation for a successful Salesforce implementation, consider the following better approaches:

- **Start with a clear architectural vision** : Begin by defining a clear architectural vision and guiding principles that align with the organization's goals, values, and long-term objectives. This vision should articulate key quality attributes and design principles that will shape the system's structure and behavior.
- **Engage architects early** : Involve experienced architects from the outset of the project to provide strategic guidance and ensure that architectural considerations are factored into requirements gathering and design decisions. Architects should collaborate closely with stakeholders to understand their needs while keeping the bigger picture in mind.
- **Establish architecture as a foundation** : Treat architecture as a key enabler and foundation for the system, not just another set of requirements. Define clear architectural patterns, principles, and guidelines upfront to inform design decisions throughout the project. Communicate the rationale behind these architectural choices to stakeholders and development teams.
- **Use reference architectures and best practices** : Leverage established reference architectures, design patterns, and best practices specific to Salesforce. These proven approaches can provide a solid starting point and help ensure that the system follows recommended architectural principles and guidelines.
- **Balance requirements and architecture** : Strike a balance between fulfilling individual requirements and maintaining a coherent, sustainable architecture. Be prepared to make trade-offs and prioritize requirements based on their alignment with the overall architectural goals. Continuously assess the impact of requirements on the system's architecture and make informed decisions.
- **Embrace modularity and loose coupling** : Design the system with modularity and loose coupling in mind. Break down complex requirements into smaller, self-contained components with well-defined interfaces. This approach promotes reusability, maintainability, and easier integration between different parts of the system.
- **Invest in architectural documentation** : Create and maintain clear, concise architectural documentation that communicates the system's structure, key components, and design decisions. This documentation serves as a reference for development teams, stakeholders, and future maintainers, ensuring a shared understanding of the system's architecture.

- **Continuously evaluate and refine the architecture** : Treat architecture as a living asset that evolves alongside the system. Regularly review and assess the architecture to identify areas for improvement, address emerging challenges, and align with changing business needs. Be proactive in refactoring and updating the architecture as necessary to keep it relevant and effective.
- **Foster collaboration and communication** : Encourage open communication and collaboration among architects, development teams, and stakeholders. Create forums for discussing architectural concerns, sharing knowledge, and making collective decisions. Foster a culture of continuous learning and improvement where team members can contribute ideas and insights to enhance the system's architecture.
- **Align architecture with organizational goals** : Ensure that the system's architecture aligns with the organization's broader goals and strategies. Consider factors such as scalability, performance, security, and future growth when making architectural decisions. Regularly communicate the value and benefits of a well-designed architecture to stakeholders and decision-makers.

By adopting these better solutions and embracing a proactive, holistic approach to architecture, organizations can lay the groundwork for a robust, scalable, and maintainable Salesforce implementation that delivers long-term value and supports the business's evolving needs.

A Big Ball of Mud

A Big Ball of Mud is a system that has lost all internal structure and coherence and can only be partially understood by its original creator(s) and not at all by anyone else.

An example

DreamCo has put adding more functionality to their Salesforce implementation on hold for a few months. During this time, John has been working on a plan to refactor key parts of the system to make it more maintainable. He's been consulting a lot of technical people from the different teams that have been involved since day one, and he finally thinks he has found a way forward.

However, shortly before John has a meeting to pitch his ideas to the CIO, things change rapidly on the ground. DreamCo has recently hired Ricky, an experienced architect from one of the big consultancies, to help figure out how to integrate the DreamCo and NewCo orgs. While he has generally been supportive of John's effort to get refactoring started, it clearly hasn't been at the top of his agenda.

Instead, he proposes to focus DreamCo's efforts on the Salesforce side into an integration project to combine the two orgs. He proposes to do much of the technical work himself, using a third-party tool that he has previously used on other projects. The integration will be point-to-point, but it will reliably ferry the data needed back and forth between the two environments and end the complex manual processes that the business has had to use to bridge the gap in the interim.

Given the pressure on the business, the CIO decides to go ahead with Ricky's plan, and he implements the solution over a 2-month period more or less single-handedly. John is told that his refactoring project will get the go-ahead after the integration is complete. However, once it's done, the CIO gives the go-ahead to a new set of feature upgrades on the existing platform for both the Sales and Service teams. John is told that the business can't wait for any new features and that instead, he will need to lead the development of a new logistics app that the business wants to build in a separate Salesforce org, with Ricky acting as the technical lead.

The new app is built quickly in the new environment but John remains concerned that it is being built with very little structure or discipline. The business seems to love what they're seeing, so John relents and lets things go ahead, supporting where he can. While the app was meant to be an experimental prototype, the business decides it wants to put it into production as is. The upgrade to the main org takes a long time to get done, but the day for the production deployment finally approaches.

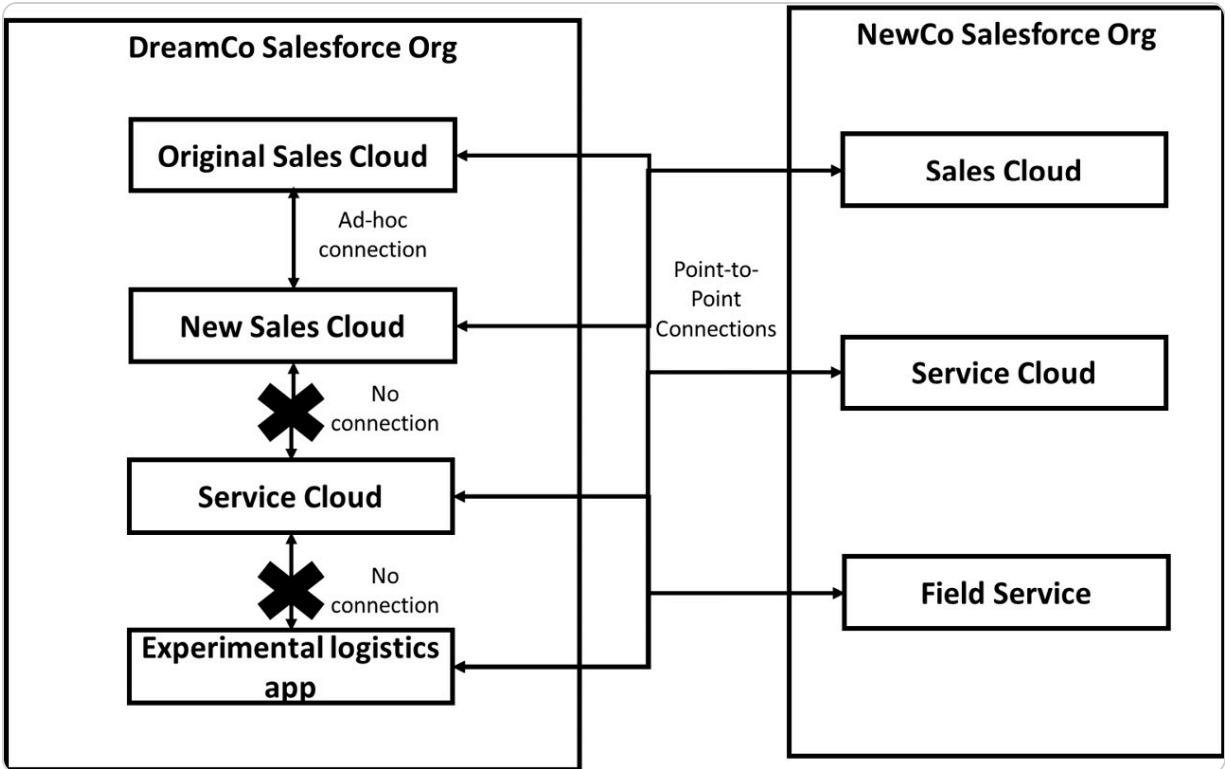


Figure 2.5 – The DreamCo Big Ball of Mud architecture

DreamCo will deploy the new upgrades as well as the new logistics app into the production org in a single deployment over a weekend. All hands are on deck, and there is a positive buzz around the release as many of these features, as well as the new app, are avidly anticipated. The last weeks are slightly marred by Ricky handing in his notice, as he has decided to take an offer from another consultancy. However, he promises to document and hand over everything before he leaves, so no one is overly concerned.

On the weekend it is going live, John is on call and avidly awaiting the results. The initial deployment runs into some issues and the full deployment doesn't actually make it into production before Sunday afternoon. The test team start their smoke test, and it is obvious that there are major problems with the release. Major functional areas don't seem to be working, the integration between the DreamCo and the NewCo org starts experiencing errors, and many test scripts that were working in the main org or the logistics org fail completely. Questions are raised about the quality of UAT testing, but the fact is that the release has failed.

The team works frantically overnight to get the system back to a reasonable state before the business starts using it in the morning. Communications go out stating that the release had to be pulled back. DreamCo's board calls the CIO for a consultation on what went wrong. However, no one can work it

out, and for both the logistics app and the DreamCo-to-NewCo integration, it seems to be impossible to fix any issues now that Ricky has gone, despite knowledge transfer sessions having taken place.

After a week of finger-pointing back and forth, the CIO is fired and a replacement comes in. He commissions a detailed technical audit of the current system from a consultancy that he personally trusts. After consultation with John and many other DreamCo stakeholders, they conclude that the architecture and code quality on many components is so weak and the structure so incomprehensible that the only option is to rebuild significant parts of the system.

The problem

A Big Ball of Mud is what happens when all internal structure within a system collapses and what you are left with is a big ball of undifferentiated mud that is impossible to understand or change. In contrast to a stovepipe, which can work quite well within its own limited sphere, a Big Ball of Mud is effectively a throwaway system. You might be able to keep it running with enough firefighting, but you'll never make it work well or significantly change it. It is a response to a system that has such great pressure to deliver functionality in several areas that going along with any idea that seems to deliver the functionality in question seems to be a good idea.

It happens usually either as a result of ignorance or desperation, as no one intentionally sets out to create a big mess. You do, however, frequently get Big Balls of Mud from systems that have been created due to one person or a couple of people working independently without supervision, clear documentation, or good decision-making processes, as with Ricky in our example.

You could say that a Big Ball of Mud shares the Nike approach to implementation, “*Just Do It.*” While this may be a good principle in athletics, it rarely works out well in enterprise IT.

The proposed solution

The solution proposed by a Big Ball of Mud, in general, is to put your trust in some team or person to just get something done with no regard for the consequences. It is often attractive in pressured situations because it gives you a seemingly good way out of your predicament without having to pay the costs of disciplined development or implementation.

Big Balls of Mud often evolve from systems such as stovepipes, when all discipline and control are lost. A stovepipe, if it is true to its own internal structure, can work well on its own for a long period of time, but once you give up on this internal discipline, it quickly devolves into a Big Ball of Mud.

Big Balls of Mud are also often created from experimental or prototype systems that are elevated to production status, as with the logistics app in our previous example. The results of a system reaching the Big Ball of Mud stage are usually catastrophic, as we'll discuss in the following section.

The results

A Big Ball of Mud is an end state for a system. Once the system reaches a state of total internal disorganization, the only way to keep it running is to have one or two people who still partially understand how it works continuously firefighting to keep it going. You won't be able to make changes that are anything other than trivial, nor will it be realistic to refactor the system into a reasonable state. It would be cheaper to rebuild the key modules, which is what tends to eventually happen with these kinds of systems.

As noted in the introduction, some people like to be heroes and also like the job security that comes from being indispensable, so the period during which a Big Ball of Mud can remain operational is surprisingly long in some cases.

The fundamental solution to the Big Ball of Mud scenario is technical discipline in various forms, which we'll discuss next.

Better solutions

In many ways, the better solution to the Big Ball of Mud anti-pattern is similar to those we listed for the Stovepipe anti-pattern. If you have good technical standards and practices in place and they are followed in practice, then you will not end up with a Big Ball of Mud.

More fundamentally, though, the Big Ball of Mud scenario reveals a profound lack of technical discipline and respect for craftsmanship within the executing organization. If your technical culture does not value good practice, architecture, well-structured code, and abstractions, but instead values cowboys just getting something up and running, no matter how quick and dirty, then you will be prone to end up with Big Balls of Mud.

To be sure, there are situations where quick and dirty is good enough. If you're writing code for an ephemeral marketing campaign, a research project, or a throwaway prototype, then you don't have to care so much about whether the system you create can evolve. However, for most of us, most of the time, this is not the case.

Too much coupling

Designing systems with low coupling and high coherence is one of the foundational aims of software architects of any stripe. Coupling, however, can be difficult to avoid, and in many cases, a level of coupling can be justified as a trade-off with other concerns. In this section, however, we will see how intense coupling can become an anti-pattern that seriously affects your system architecture.

Intense coupling

Intense coupling is an anti-pattern where one or more systems in the system landscape fail to maintain clear boundaries with other coupled systems, leading to a fragile and unmaintainable state.

An example

BigCo has a mid-size Sales Cloud implementation that they use for managing their opportunity pipeline. However, all financial processes, quoting, and order management are done in their main SAP system, which has served as the source of truth for the business for a number of years.

The sales department love their Salesforce system and would like to carry out the entire ordering and fulfillment process through Salesforce, leaving the SAP system to purely financial processes. The finance and supply chain teams think this idea is not only misguided but risks compromising the strong grip BigCo has had on its data for years.

After some internal political battles, the CFO and VP of Sales reach a compromise. The order and fulfillment process will stay in SAP, along with the financial processes, but quoting will move end-to-end to Salesforce using Salesforce CPQ. That way, the sales team will rarely need to go into SAP and can stay primarily in the Salesforce interface that they love, but the other teams still keep control of all master data. Anitha, a Salesforce architect, is tasked with making this compromise into reality.

She quickly discovers some major issues with this approach:

- First, all products and prices are kept directly in SAP, and there is no appetite for moving this to Salesforce; therefore, it will need to be accessed from SAP as part of the quoting process, and only stub data can be replicated to Salesforce.
- Second, a number of business logic checks happen for quotes during the business process to check that they fall within acceptable corporate parameters. Anitha looks into moving these checks to Salesforce as part of the implementation. While it is possible for many, some of these checks require access to data in SAP's financial module and can't be done on Salesforce. These will need to be accessed via an integration to SAP too.
- Third, a number of additional services need to be called as part of the quoting process to get current product availability and possible delivery dates for inclusion on the quote. These will have to be called via SAP, as they are already exposed via web services on this platform, and BigCo does not have the integration resources to make a direct connection to Salesforce or to connect via other middleware.

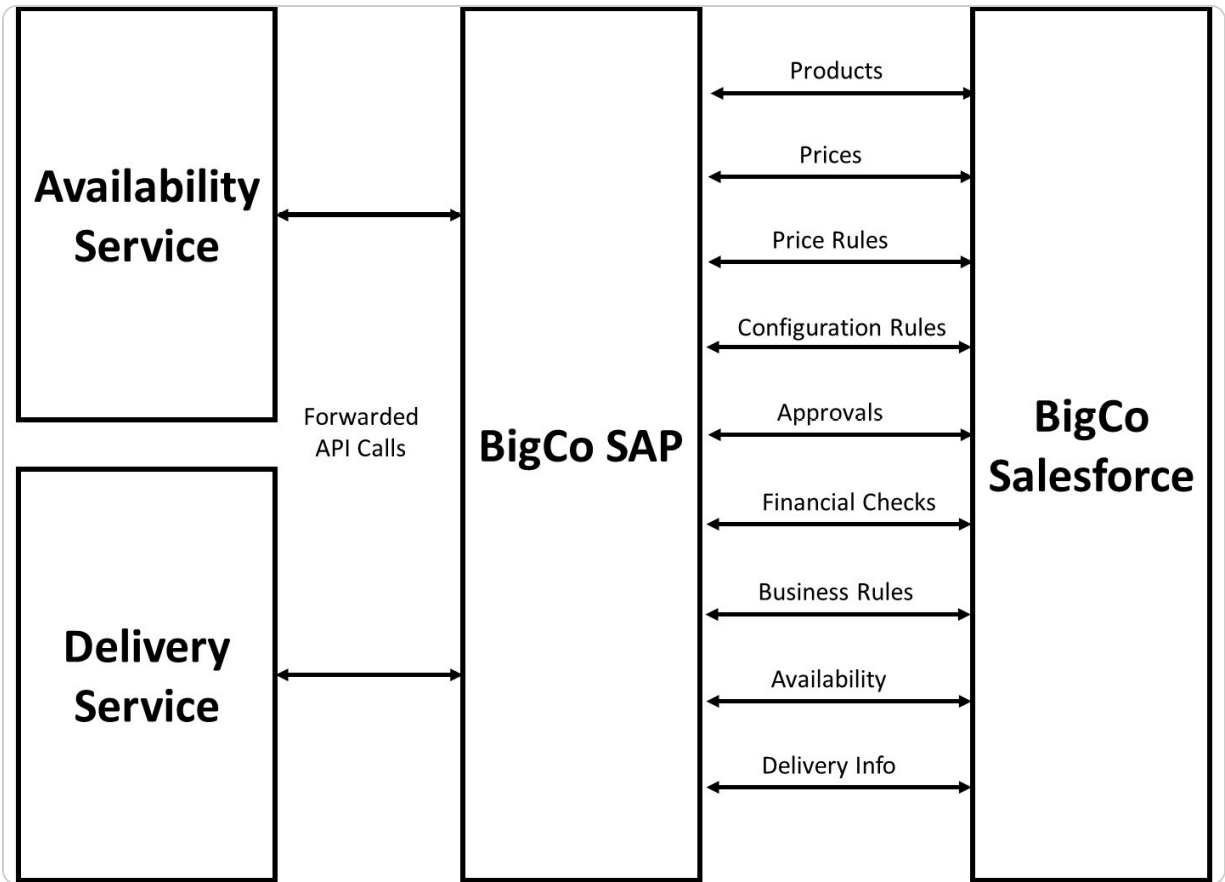


Figure 2.6 – The BigCo integration scenario

Anitha sounds the alarm and says that the degree to which Salesforce depends on SAP makes her doubt that the new system will perform to the high expectations of the sales team. She recommends looking into a different approach where there is a cleaner division between what is on SAP and what is on Salesforce. However, the Finance and Supply Chain teams rule out any process changes as part of the implementation, and therefore she has to go ahead despite her doubts.

The initial workshops with users go well, and the functional requirements of the sales team seem to be a good fit for what Salesforce has to offer. However, Anitha is still concerned that the final product will underperform due to the coupling to SAP.

During the UAT testing of the solution, some concerns are raised about the performance and responsiveness of the user interface, and there are also complaints that testing had to be interrupted several times due to maintenance or outages on SAP.

These concerns are not enough to hold back the go-live, however, as the team expects a boost in performance from moving to the production environment, and the basic functional aspects of the solution are acceptable.

Unfortunately, the problems are exacerbated and not alleviated by the move to production. Performance decreases significantly, and sales users frequently have to wait for minutes to see a response from SAP. In addition, there are numerous periods during the day when SAP runs large scheduled jobs that stop sales users from doing anything.

The team works frantically to try to improve the situation. However, after a couple of weeks, the team has to throw in the towel, and the sales team goes back to producing their quotes in SAP.

The problem

Intense coupling happens when the boundaries between systems or modules are blurred to such an extent that one depends entirely on the other for its operation. In general, as architects, we strive to reduce coupling while maintaining coherence. However, coupling – even tight coupling – can be a necessary trade-off with other requirements in some situations.

Intense coupling, however, is the most extreme version of this trade-off. At the system level, it happens when system boundaries are not respected, and you try to design a solution that incorporates and replicates significant parts of another system, using the other system as a synchronous backend.

The same pattern can repeat at the module level, where you have one module replicating all the functionality of another in a different context, calling intensely on functionality from the original module to do so.

Intense coupling is often done for good reason. There are usually real problems with the functionality of legacy systems or modules that are proving hard to address and that would seemingly be much simpler to solve in a different system or module context.

As we'll see in the following section, the proposal that intense coupling makes to address such issues can be very compelling, which is what makes it a classic anti-pattern.

The proposed solution

Intense coupling is a proposition that promises that you can have your cake and eat it too. By using APIs or SDKs, or using some other similar mechanism, you can get all the functionality and business logic that you are used to. Additionally, you can have it work within a new user experience that will address your changing requirements, and you won't have to go through the pain of introducing a new system or making substantial changes to your legacy ones. As with all cake-related propositions of this kind, you should be quite skeptical about the truth of these surface claims.

Salesforce is particularly prone to this anti-pattern because it has a user interface that many users love, and the business is used to things being fairly easy to implement on the platform. It is, therefore,

common for a request to be made for parts of other systems to be incorporated into the Salesforce platform.

This can be done successfully if you are careful about the boundaries, are open to making the necessary process changes, and have a strong integration capability in place. However, it is also quite easy to end up in an intensely coupled scenario.

The results

Intense coupling tends to result in an end-to-end user experience that is characterized by fragility, instability, and low performance. This happens for the fairly obvious reason that an intensely coupled system is, for all intents and purposes, a single-user-facing system encompassing two underlying technical systems. It is a distributed system that has not been built with the awareness that it is a distributed system.

The more you need to rely on the other system, the more frequently you need to call out and access its functionality. This means you have an issue whenever either system has an issue in any of the areas that are part of the solution.

You also get the brittleness that comes from any changes to relevant functionality on either side of the system being likely to result in required changes to your intensely coupled system. This means that you will likely see more errors and issues over time than you'd expect.

In addition, the cumulative wait for the functionality going back and forth between the two sides of the intensely coupled system leads to reduced performance, sometimes to the extent that the system becomes unusable.

On the module level, the consequences tend to be more about reduced maintainability, lower flexibility, and higher cost of change, but performance issues will also occasionally creep in.

Better solutions

While the problems that lead to intense coupling are often legitimate, there are many things you as an architect can do to avoid a situation resulting in an anti-pattern. The following list provides a starting point:

- Keep a keen eye on system and module boundaries, and ensure that if they shift, the cut is made at a maintainable point.
- When designing a cross-boundary process, be very clear on system responsibilities and the integration architecture that will support those processes.
- Adapt processes so that they work seamlessly with the revised system boundaries. Don't try to compensate with extra technical work if the processes don't fit.
- Avoid redundant functionality in multiple systems. Be clear on where an engagement with a given set of functionalities is done.

- If what you are building is, in fact, a distributed system, be mindful of that fact and use low-coupling integration patterns, such as event-based integration, to achieve your goals. Resist calls for large amounts of synchronous integration.
- As AI becomes more prevalent in Salesforce implementations, be cautious of creating intense coupling between AI models and core Salesforce functionality. While tight integration can provide powerful capabilities, it can also lead to brittle systems that are difficult to maintain or evolve. Strive for a modular approach to AI integration, using well-defined interfaces and data contracts to allow for flexibility and easier updates to AI models without disrupting core Salesforce operations.

Having now discussed a number of patterns that are applicable to Salesforce, as well as many other technologies, we will now move on to our first unique Salesforce anti-pattern, Ungoverned Org Proliferation.

Org complications

In the Salesforce world, you don't have to manage your underlying infrastructure, which frees you from a good number of potential temptations that can lead to anti-patterns. However, the way that you structure environments with different orgs in the Salesforce world is subject to a number of anti-patterns in its own right.

Ungoverned Org Proliferation

Ungoverned Org Proliferation is a Salesforce-specific anti-pattern due to a lack of defined org strategy, which leaves you with an ever-increasing number of unaligned orgs, eventually becoming architecturally unmanageable.

An example

Miranda has been hired as BigCo's new CRM Migration Manager. The company has had a business unit approach to IT and does not have any centralized CRM capabilities at this point. Instead, three units use Salesforce mainly for opportunity management, and there are at least 15 other systems in use from a variety of vendors, as well as in-house developed systems in play across departments and geographies.

Miranda has been tasked with consolidating this landscape, using Salesforce as the primary platform. There is a general aim to consolidate systems and standardize processes, but there are few central resources to drive this transformation.

Miranda plans a strategy to consolidate the three existing Salesforce orgs. She has previously been through org consolidation projects in Salesforce and doesn't think there is enough value to go through that pain in this case.

She will therefore create three variants of a core Salesforce implementation, dispersed across three geographies, and map the other CRM systems to whichever of these three is the best fit. She will implement reporting processes across these three orgs using the corporate data warehouse that already imports Salesforce data from two of the orgs.

However, as she starts to plan for migrations, she faces massive pushback from business users on her plans. While most teams are quite willing to move to Salesforce, users in France and China, two of BigCo's largest markets, demand their own unique and segregated environments.

According to the country leads, this is a legal requirement and not up for discussion. Miranda escalates the question to the Legal department but is unable to get an answer out of them quickly. She therefore has to accept the separate orgs for France and China.

BigCo's largest market is the UK, and the country lead there, once he understands that having a separate environment is a possibility, demands that they are also given a unique Salesforce org that can be customized specifically to their requirements. The UK operation is highly efficient but also unique, and there is no way it would work with the same processes as the rest of the business, the argument goes.

Once again, Miranda finds herself politically outgunned and has to assent to these demands. However, this leads to a new opening of the floodgates. Product development argue that their process is distinct

enough to merit a separate environment, and similar requests from other departments and geographical units follow.

At the end of the day, Miranda will be looking at a dozen different Salesforce orgs, with many having quite distinctive processes implemented. This is still preferable to the 15 totally different systems in play before Salesforce, but far from the original goals. At least, she finally has a way forward.

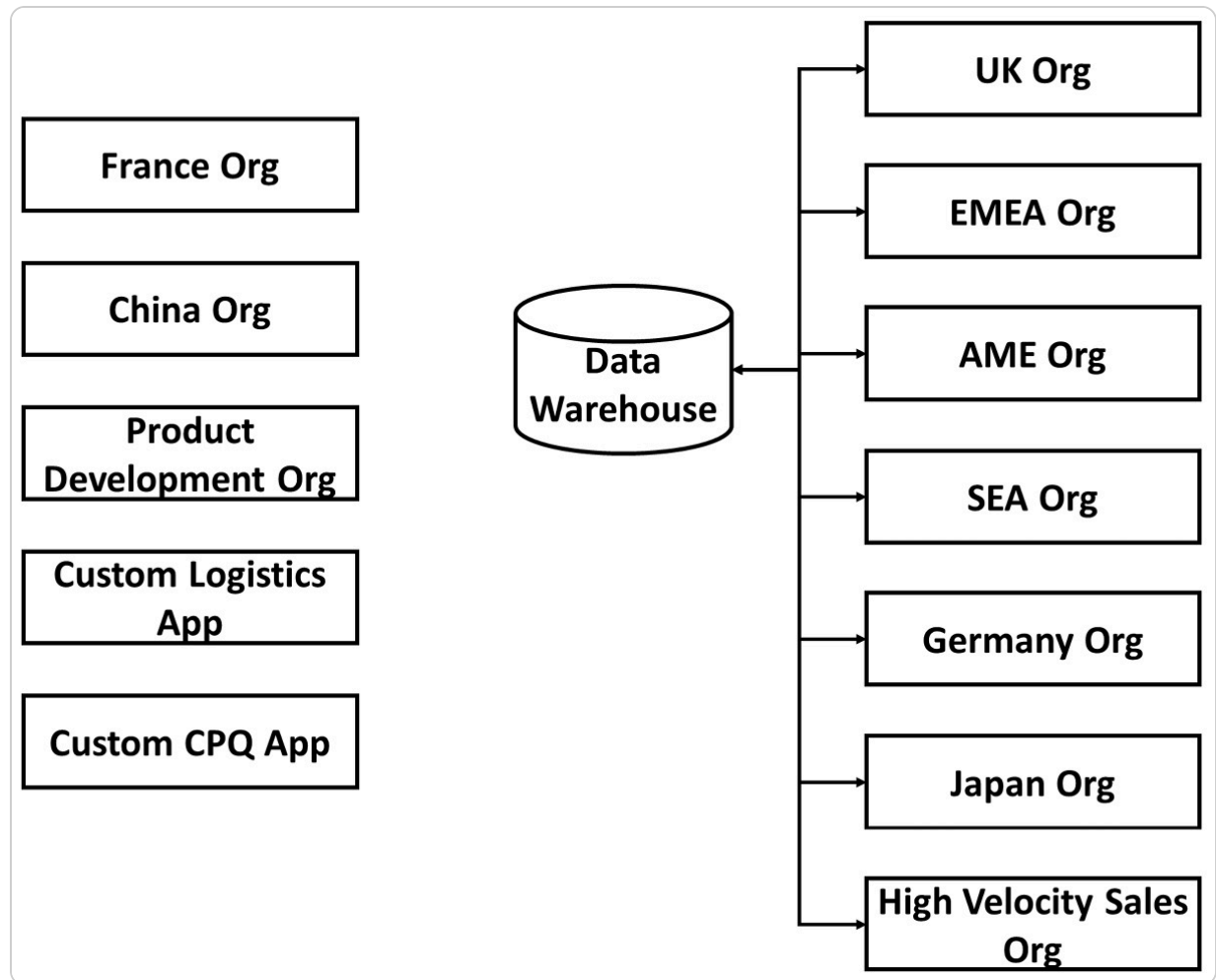


Figure 2.7 – The BigCo org overview

Now, a long-awaited report comes in from a leading management consultancy on how to optimize BigCo's global sales process. This promises major global efficiency in sales and sales operations by moving to a globally standardized process for all field sales. It is implicitly assumed that this global process will be implemented on Salesforce.

Miranda asks around to see whether anyone is willing to change their position on the implementation plans in light of the new objectives. The answer is a resounding no. She decides that enough is enough and hands in her notice the following day.

The problem

Ungoverned Org Proliferation happens in organizations that like Salesforce but don't like standardizing processes, user interfaces, or other similar requirements and don't have any strong architectural governance around their Salesforce platform.

Problems that lead to Ungoverned Org Proliferation tend to be found within the following scenarios:

- Different business units or geographies have different processes and can't agree to a compromise that can be implemented
- Different stakeholders have radically different views of how the user experience should be structured and can't agree on a reasonable compromise
- Different stakeholders have radically different views on the data model to be implemented and can't agree on a reasonable compromise
- There are certain critical features that are needed in some business units but can't be used in others – different integration backends, for example
- Different parts of the business are working with different Salesforce partners that are pushing them in different directions
- There are perceived or real security differences between different parts of the business that strongly impact the Salesforce implementation
- There are perceived or real legal compliance regime differences between different parts of the business that strongly impact the Salesforce implementation

NOTE

It's worth noting that in the old days, Salesforce actively encouraged organizations to try out the software in a number of different places, the so-called seed and grow strategy, which worked very well for many years. In large organizations with a long history with Salesforce, you can sometimes still see this legacy living on within rather unstructured org landscapes.

The proposed solution

The proposed solution to the problems leading to Ungoverned Org Proliferation is to give everyone who wants it their own Salesforce org and let them get on with it. This is attractive because it reduces coordination issues and alignment problems and allows easier resolution of thorny compliance issues. It also lets powerful stakeholders get things in their own way.

There are different variants and degrees of this anti-pattern, some of which are relatively benign or even beneficial. In our previous example, some of the asks are probably entirely legitimate and should have been part of a systematically evaluated org strategy. Product development processes, for instance, are often configured in a separate org from the one used for Sales and Service, and sometimes there are real compliance headaches that necessitate multiple orgs.

Let's be clear. There's nothing wrong with a multi-org strategy; it can both be the right architectural choice and work well in practice. However, with Ungoverned Org Proliferation, we are talking about a

multi-org operation without any strategy or architecture supporting it, and that tends to result in serious negative consequences, as we'll discuss in the following section.

The results

The result of Ungoverned Org Proliferation is a sprawl of orgs that cannot be controlled from the organization's center. How serious a problem that is will depend on the nature of the organizational culture and how much central control it requires.

In general, you will find some or all of these disadvantages present in an ungoverned org landscape:

- Difficulties in implementing processes that cut across business units or geographical divisions, as the Salesforce implementations vary by data model and components
- Difficulty aligning data for reporting, as the data model and the meaning of fields will be locally defined
- Complexity developing reports, even when data is brought together, due to the semantic gap between different uses of data
- Integrations often have to be replicated in several different places and can be hard to control
- Licensing can get expensive, as some users need multiple licenses
- Knowledge of the systems is heavily distributed, and getting an overview of any particular aspect can be hard to find, as technical teams also mostly work in a local context
- Instituting any kind of global governance can be near impossible due to a combination of the aforementioned factors
- As org consolidation is hard, once you find yourself in this situation, it can be difficult to undo

This list is, to a large extent, simply a more extreme version of the downsides of a multi-org strategy, which makes sense, of course. As we will cover in the following section, the key takeaway to avoid this anti-pattern is to clearly define your org strategy upfront, along with mitigations for the inevitable downsides.

Better solutions

It will come as no surprise at this point to anyone familiar with Salesforce architecture that the most important element for avoiding this anti-pattern is to have a clearly defined org strategy. This org strategy should clearly define when, if ever, it is permissible to spin up a new Salesforce org, as well as provide clear patterns to use for integration and data consolidation between orgs.

There are a number of good resources you can use to learn about org strategy. I recommend you start with SOGAE, which can be found on the Salesforce Architects site:

<https://architect.salesforce.com/well-architected/overview> . This is a comprehensive view from the horse's mouth, although it won't cover all the details that may be required.

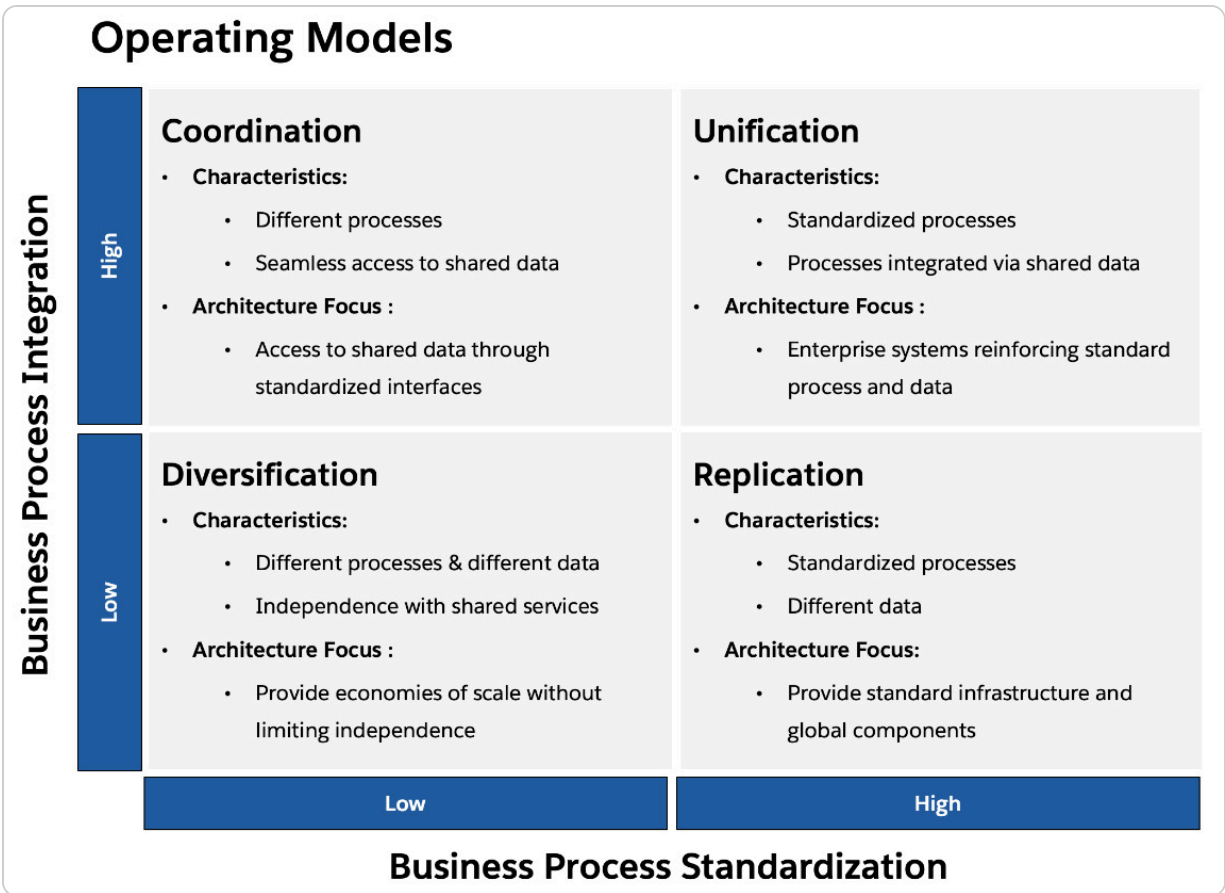


Figure 2.8 – SOGAF models

In addition, in what is becoming a refrain within this book, you should define a common framework for governance, ideally rooted in a **Center of Excellence (CoE)** that has a global oversight of all things Salesforce and that is followed by all orgs. This framework should also include common standards and practices and, ideally, a level of common tooling that must be used in every environment.

When developing your org strategy, consider the implications of AI and **machine learning (ML)** capabilities across different orgs. Inconsistent AI implementations or siloed predictive models can lead to conflicting insights and decision-making processes across the organization. Aim for a unified AI governance model that ensures consistent data quality, model performance, and ethical considerations across all Salesforce orgs, while still allowing for necessary customizations to meet specific business unit needs.

The opposite of this pattern

It is worth noting that this anti-pattern has an exact opposite, which I like to call a Procrustean Bed, in honor of the Greek mythological figure Procrustes.

This anti-pattern tries to fit all the requirements into a single org across geographies, business functions, and business units. While many organizations do well with a single global org, there are times when multiple orgs are needed to avoid creating a subpar user experience for key stakeholders, make genuinely diverse and business-critical processes work, or solve real legal issues.

That is to say, org strategy is a balancing act, and you can go too far in either direction, turning it into an anti-pattern. You should refer to the SOGAF models mentioned previously to find the right way forward in your scenario.

Knowing the takeaways

In this section, we will abstract a bit from the specific patterns and, instead, try to pull out the wider learning points that you can use in your day-to-day work as a Salesforce architect or when preparing for the CTA Review Board.

When architecting Salesforce solutions, you should be mindful of the following:

- Many organizations are internally siloed and political, and, in these kinds of organizations, the default mode of operation may be to develop stovepipes or Big Balls of Mud.
- To avoid this, you as a Salesforce architect need to push strongly for countermeasures.
- This will include pushing for strong architecture governance and coordination on Salesforce projects, including establishing relevant governance forums to coordinate activities and the right technical standards and practices to ensure everyone is building in a consistent way.
- You need to secure enough business support for these initiatives to make them enforceable when things get hectic; this will require careful cultivation and management of relevant stakeholders.
- Design your systems with common abstractions – for instance, by basing different modules on a common package of shared functionality. This will encourage technical consistency between projects.
- Having strong governance about common elements shared between different Salesforce projects, such as Hero objects that are used by nearly any Salesforce solution, is necessary.
- These must be used consistently and have clear ownership, or you will end up with a mess in your data model and, most likely, also in your automations.
- Don't have fuzzy boundaries between systems. Make it clear what processing needs to happen where, and if the boundaries have to change, make sure it is accompanied by process change to avoid intense-coupling scenarios.
- Favor simpler integration patterns where possible. Favor asynchronous integration patterns over synchronous ones where possible. This helps limit coupling.
- Have a clearly defined org strategy that defines when, if ever, it is appropriate to create a new production org.
- Ensure that relevant stakeholders are consulted before any new org is created. For instance, you can make it subject to approval by a Design Authority or an Architecture Governance board.
- Ensure that common abstractions, standards, practices, and tools are also used in any new orgs.

- Be mindful of the Architecture as Requirements anti-pattern and ensure that architecture is treated as a guiding framework, not just a set of requirements to be implemented.
- Continuously evaluate and refine the architecture to keep it aligned with evolving business needs and technology trends.
- Foster a culture of collaboration and knowledge sharing among architects, developers, and stakeholders to ensure architectural best practices are widely understood and followed.

In preparing for the CTA Review Board, you should be mindful of the following:

- Always include appropriate governance structures to ensure that you have a structured approach to implementation. These may include a CoE, an **Architecture Governance Forum** , and a **Project Management Office (PMO)**.
- It is also worth mentioning the importance of common patterns, practices, and tools as a way to coordinate implementation activities between teams and ensure maintainability.
- There is often risk associated with stakeholder management if there are multiple different departments or business units involved. As we have seen, bad stakeholder management can be a major contributing factor in selecting an anti-pattern.
- Design a clear system landscape with easy-to-understand roles between the different systems. Avoid having lots of overlap between systems when it comes to core functionality.
- Consider carefully how the different elements in your solution, both on- and off-platform, should interact in a way that doesn't introduce a stovepipe such as isolation for some elements. Your solution should present itself as a coherent whole.
- Avoid unnecessary coupling between systems, and avoid especially strong dependencies between systems where it isn't absolutely critical to the functional experience.
- Favor asynchronous integration patterns that reduce coupling if there isn't a direct need for synchronous communication.
- You should always have an org strategy, you should state it upfront, and you should be able to defend it from questioning.
- Be careful not to veer into having either too many orgs that will be extremely difficult to govern effectively or a single org in a scenario that has a strong impetus toward splitting, such as highly variable processes or definite legal and compliance requirements.
- In any case, be prepared to say how you will mitigate issues associated with your org strategy, as no scenario is ever clear-cut, and there will be trade-offs to consider.
- Highlight the importance of defining a clear architectural vision and principles early on to guide design decisions throughout the project.
- Discuss the role of architects in balancing individual requirements with overall architectural coherence and sustainability.
- Explain how embracing modularity, loose coupling, and clear abstractions can help create more adaptable and maintainable Salesforce systems.
- Emphasize the value of continuous architectural evaluation and refinement to keep the system aligned with changing business needs.
- Discuss strategies for fostering collaboration and knowledge sharing among team members to promote architectural best practices.

We have now covered the material for this chapter and are ready to proceed. First, however, we will summarize our learning.

Summary

In this chapter, we explored several anti-patterns that can arise when organizations fail to take a disciplined, architecture-driven approach to Salesforce implementations.

We started by examining the Stovepipe anti-pattern, where systems are built in isolation with little regard for standards or common abstractions, leading to a landscape of disconnected, hard-to-maintain applications. We then looked at the Big Ball of Mud anti-pattern, characterized by a lack of structure and organization, resulting in an unmaintainable system.

Next, we introduced the Architecture as Requirements anti-pattern, a common pitfall where architectural decisions are treated as mere requirements without considering the broader system context and long-term implications. We explored the consequences of this approach, such as increased technical debt, scalability issues, and limited adaptability.

We also discussed the dangers of intense coupling between systems and modules, highlighting the importance of clear boundaries and favoring looser integration patterns to maintain system stability and flexibility.

Finally, we touched on the challenges of org proliferation in Salesforce environments and the need for a well-defined org strategy and governance model to prevent sprawl and ensure consistency across implementations.

Throughout the chapter, we emphasized the critical role of architecture governance, technical standards, and best practices in avoiding these anti-patterns. We stressed the importance of securing business support, fostering collaboration, and striking a balance between individual requirements and long-term architectural sustainability.

By understanding these anti-patterns and following the guidance provided, Salesforce architects can create more robust, maintainable, and evolvable systems that deliver lasting value to their organizations. The key is to approach architecture proactively, think holistically, and continuously refine and adapt the system to meet changing business needs.

3

How Not to Get Confused about Security

This chapter covers anti-patterns related to securing your Salesforce org. We'll begin by examining a key anti-pattern around the shared security model that is integral to Salesforce. Then, we will look at anti-patterns that can occur when you mistake other things for security. We will continue by looking at how not to make a mess of your sharing model and once again finish up with a review of the key takeaways. As Salesforce continues to integrate AI technologies, new security considerations are emerging. This chapter will also touch on how AI impacts the security landscape within Salesforce.

In this chapter, we're going to cover the following main topics:

- The *Shared Belongs to Salesforce* anti-pattern and how people get the shared responsibility model wrong
- How the *Compliant Is Secure* anti-pattern lulls you into a false sense of security and what you can do about it
- The *Declarative Is Always Safe* anti-pattern and why you can't rely on declarative features to always be secure
- The *Spaghetti Sharing Model* anti-pattern – why it is deceptively easy to get into an overcomplicated sharing model on Salesforce and how you can prevent it from happening to your org

After completing this chapter, you will have a good idea of how to avoid common security anti-patterns on Salesforce and what steps you can take to implement better security practices on the platform.

Getting confused about shared responsibility

Almost all SaaS and PaaS platforms use a security model called shared responsibility. That is to say, the vendor takes responsibility for certain parts of the security architecture and other parts remain the responsibility of the customer. In this section, we will see how misunderstanding this model can lead to issues in Salesforce.

Shared Belongs to Salesforce

The Shared Belongs to Salesforce anti-pattern holds that security on the Salesforce platform is always the primary responsibility of Salesforce.

Example

SmallCo is a provider of specialty lenses for optical instruments. They sell globally and have recently adopted Salesforce Sales Cloud, Order Management, and B2B Commerce to drive electronic

commerce directly to the business instead of selling exclusively via distributors.

When they did their implementation, the Salesforce platform went through a security review with James, the dedicated security professional that SmallCo has on staff. He evaluated the platform as a whole and was satisfied that the Salesforce platform, including how Salesforce secures their data centers, platform, and web application, met all the standards that SmallCo would expect.

James evaluates the shared responsibility model that Salesforce uses for security and determines that for the initial project, there shouldn't be any need for additional security controls from SmallCo's side. Salesforce will ensure security adequately without additional work required by SmallCo.

After a period of successful running, SmallCo wants to extend the digital experience and online community adjacent to their web store and also bring in a much larger number of customers onto the web store itself. Mika, a Salesforce Product Owner at SmallCo, is given the task of running this extension project.

Because of the limited budget, Mika agrees with the implementation partner that they will extend a lot of functionality to anonymous users so that many tasks can be performed without a login. That way, SmallCo can opt for a relatively small login-based license SKU.

The system goes live without issue and, as hoped, user numbers go up by more than a factor of 10. Mika and the team celebrate and, after completing the last outstanding minor improvements, start gearing up for the next project.

Just then, Mika receives a security advisory from a Salesforce consultancy that helped with SmallCo's original Salesforce implementation. They point out that a number of new vulnerabilities have been found in Salesforce APIs used within digital experiences. These potentially allow serious exploits if there is a too-open model toward external users.

Mika is initially worried but remembers the conversation he had with James about Salesforce security prior to starting his new project. APIs sound like a core platform feature and therefore something Salesforce will take care of. The project is closed and doing anything new would require finding a new budget, which is difficult. He, therefore, decides to ignore the warning.

Three weeks later, an email arrives in an internal SmallCo inbox. It contains a data dump with all of SmallCo's proprietary product information and lists of customers and orders. The data dump comes with a ransom demand to be paid in cryptocurrency to a specified address. Otherwise, the data will be made public.

The situation quickly escalates and external consultants are brought in to help manage the situation. They quickly discover that some very permissive configurations have been made to help the process of

making functionality available to external users. These are likely the source of the breach and are also the cause of the security advisory Mika received from the other partner.

SmallCo is very disappointed that Salesforce would leave such a glaring security hole open and raise it with their account manager, who instead points the finger back at the configuration SmallCo has made. SmallCo finally decides that the only thing to do is to pay the ransom and plug the gap.

The external consultants facilitate both of these actions and the external configuration is locked down. SmallCo has to change their license package, but at this point, it has become a small price to pay to avoid a recurrence of the recent events.

On the suggestion of the external consultants, SmallCo sets up a process and some software to continuously monitor the security of their Salesforce environment. In addition, Salesforce-related personnel and James from security receive a number of education sessions on the Salesforce security model. Mika breathes a sigh of relief and gets ready for the next project.

Problem

The Shared Belongs to Salesforce anti-pattern is fundamentally about who is responsible for maintaining system and platform security. When you're dealing with cloud systems, there is always a level of shared responsibility in play. That means the system or platform provider will maintain some parts of the security and you are responsible for others.

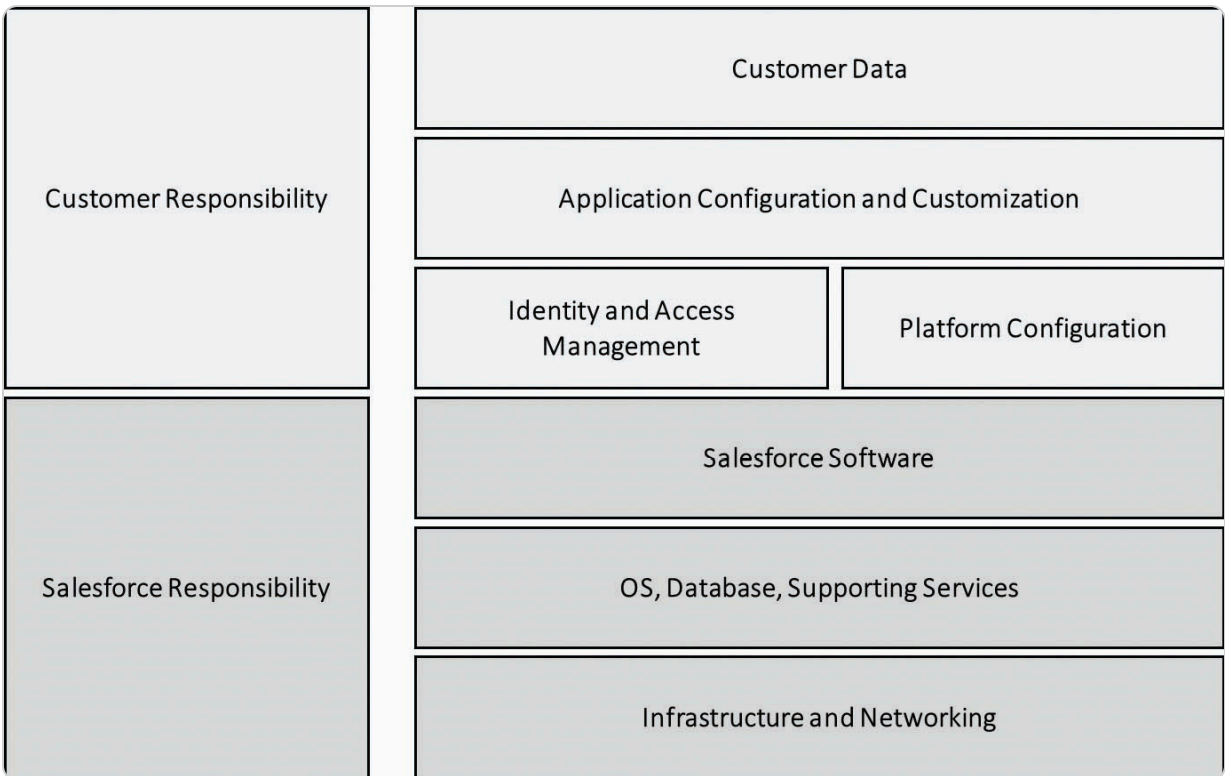


Figure 3.1 – Shared responsibility model

In Salesforce terms, Salesforce maintains infrastructure and platform security and helps you a lot with application-layer security. Salesforce Shield, including Platform Encryption, Event Monitoring, and Field Audit Trail, provides additional layers of security but requires proper configuration and management by the customer. However, when you configure or code something yourself, the responsibility for its security is ultimately on you. The now mandatory multi-factor authentication requirement shifts further security responsibility to customers, who must ensure proper implementation and user adoption. Finally, Einstein’s features introduce new considerations to the shared responsibility model, particularly around data usage, model training, and prediction explanations. While I’m discussing it in the context of Salesforce, it is relevant to all SaaS and PaaS providers, although the specific boundaries defining the shared responsibility will differ.

It is, however, very easy to get confused about this boundary and that leads to this anti-pattern. The problem that the Shared Belongs to Salesforce anti-pattern seeks to solve is the general security problem. That is to say, *“How can we keep our system secure?”* As we will see in the next section, it proposes to solve the problem by assuming that everything, or at least as much as possible, is the software provider’s problem, even when it isn’t.

Proposed solution

Shared Belongs to Salesforce proposes to solve the problem of system security by assuming that Salesforce as a world-leading software provider has already thought of all the issues and therefore will keep the platform secure. That way, you can safely ignore security as an aspect of your solution and move forward with the real work of getting configurations done.

It effectively means that you don't seriously engage with the shared responsibility model, although there are gradations of this. Most organizations, for example, will accept that they have a responsibility for the user permissions that they give the users who access the system.

However, this might extend to how you configure your sharing model; who you give access to APIs or custom-coded classes or who is allowed to run reports is much murkier. When it gets murky, the adopters of this anti-pattern make the simplifying choice that they don't have to bother with the complexities because Salesforce will already have thought about it. That, unfortunately, is not and cannot always be true.

This anti-pattern is attractive for a number of reasons:

- You don't have to engage seriously with complex security issues, which means you don't need to hire or develop specialist skills
- It reduces your internal effort when doing work on Salesforce
- It gives you a story to communicate to stakeholders that you are in good hands and don't have to worry about security
- It gives you someone to blame or hold responsible should things go wrong

While you can easily understand why people might be inclined to make their own lives easier by adopting this anti-pattern, the results are not generally optimal.

Results

Humans tend to seek to reduce cognitive load by making simplifications and taking shortcuts. Therefore, the Shared Belongs to Salesforce anti-pattern is a naturally occurring phenomenon and it takes active security awareness to counter it.

It's, therefore, not surprising that the first and most serious result of this anti-pattern is a reinforcement of low security awareness within the organization. If you aren't aware of security issues, it is just a matter of time before something happens. This anti-pattern sets you up for something bad to happen.

Other typical negative results include the following:

- Giving everyone from end users to senior management a false sense of security about their Salesforce system, leading to even more serious fallout when an incident occurs and a missing understanding that you actually have to do something, rather than Salesforce just fixing it for you.
- Not keeping up with the evolving security landscape on the Salesforce platform. This includes not just ignoring new security advisories, as in our example, but also not looking into new security updates or features that would improve your security stance.

- Ultimately, the anti-pattern leads to an increased organization risk profile that would be intolerable to most senior managers, had they fully understood it.

It's worth noting that you can run with this anti-pattern for years and years without discovering the negative consequences. It's only when a security incident that has substantial enough consequences to get senior management's attention occurs and is discovered that you really have to confront this anti-pattern.

It can, therefore, be a bit of a thankless task to point out that this anti-pattern is occurring and you may not get anywhere initially. There are, nonetheless, things that should be done, as we'll discuss in the next section.

Better solutions

The solutions to this anti-pattern are fairly obvious to anyone who has spent at least a bit of time on information security. First and foremost, this is about raising awareness about the security issues that are in play and getting the internal acceptance that something needs to be done about Salesforce security in an active way rather than assuming the vendor will take care of it.

Some of the main points include the following:

- Engaging actively with the shared responsibility model, getting a clear understanding of where the boundaries are, how to ensure your own parts are secure, and how to continuously remain vigilant about the parts that you aren't directly responsible for but that still directly affect you.
- Investing in the required skills to perform your parts of the security task both initially and on an ongoing basis. You can build this internally or use consultants; however you do it, the responsibility is on you to get it done.
- Leveraging modern DevOps practices with Salesforce DX or third-party tools to implement version control and CI/CD practices, enhancing security through consistent and auditable deployment processes.
- Assessing new risks whenever you extend your solution in significant ways. This does not just mean when you are adding new modules. Major extensions to existing functionality can also lead to new security threats being introduced.
- Shifting the company's mindset. Fundamentally, you need to shift the mindset of your organization to one where security is seen as fundamental to organizational resilience and growth. That may be beyond what most of us as architects can accomplish. However, pushing where we can in our sphere of influence is also helpful.
- Considering the principle of least privilege, which is crucial for preventing security vulnerabilities by ensuring that users and processes have only the minimum access rights necessary to perform their tasks. This approach limits the potential damage from compromised accounts or malicious insiders, reduces the attack surface, simplifies auditing and compliance, and helps contain the spread of malware. By avoiding overprivileged accounts and unnecessary permissions, organizations can significantly mitigate risks associated with data breaches, unauthorized access, and privilege escalation attacks, ultimately strengthening their overall security posture and reducing the likelihood of falling victim to common security anti-patterns.
- Continuously monitoring your orgs to evaluate Salesforce security recommendations and critical updates. Consider implementing Event Monitoring to track user activities, API usage, and other critical events. This helps in the early detection of potential security breaches or misuse.

We have now completed the discussion of the first anti-pattern in this chapter and will move on to another take on the general security problem, *Compliant Is Secure*.

Mistaking other things for security

Security is a complex and multi-faceted domain. This complexity can be so profound in many cases that there arises a strong temptation to simplify it to something more manageable. In this section, we will see two examples of how this can occur and what the negative consequences can be.

Compliant Is Secure

Compliant Is Secure holds that security for an organization can be upheld by strict adherence to one or more specified compliance regimes.

Example

Alexandra works as an architect for LifeCo, a major provider of medical devices for the continuous measurement of vital statistics. They sell devices across a number of global markets and as a consequence are subjected to a number of different regulatory regimes. It is fair to say that a big part of LifeCo's culture is focused on meeting these regulatory requirements and maintaining compliance.

When it comes to IT security, LifeCo has also adopted a compliance-centric approach combining the ISO 27001, HIPAA, and GxP protocols in a comprehensive set of checks and controls that are documented thoroughly at the beginning of each project and audited subsequently on a yearly basis. While there are processes to maintain compliance throughout the year, LifeCo focuses most of its attention on the initial documentation and audit events.

LifeCo has adopted Salesforce for field sales and care programs, using the Sales, Service, and Health Cloud components. As part of the implementation, Alexandra works closely with stakeholders across the business from IT, security, legal, and the data protection office.

They perform a thorough evaluation of all the controls Salesforce has in place at both the platform level and the level of each individual cloud. They spend considerable time with Salesforce security representatives to understand the details of the security implementation, pushing hard to get real answers to sensitive questions.

Additionally, they require comprehensive checklists and documentation from the implementation partner to ensure that all configuration work meets the necessary standards. This adds considerable overhead during both implementation and testing, but LifeCo is adamant that Salesforce meets the same standards that they apply to all internal systems.

The system documentation is finally signed off as compliant after a long period of adjustment to meet the detailed requirements. Alexandra keeps working in the Salesforce space but mainly focuses on preparing requirements for new releases and updating technical standards unrelated to security.

Then, after nine months, the unthinkable happens. The CEO receives a phone call from a popular news site that asks what their response is to the fact that thousands of the company's patients' personal information has been dumped online on the dark web. There is no doubt that the data comes from LifeCo as it contains the company's proprietary information about device usage.

The CEO gives a rushed response, but the damage is done. LifeCo suffers a major blow to its reputation and is battered in mainline news, social media, and the stock market. Alexandra takes a leading role in the investigation and cleanup after the incident.

The investigation is not entirely conclusive, but there is good circumstantial evidence pointing toward a disgruntled employee exporting a report file with personal information and uploading it online. There isn't enough evidence to bring any formal charges, however.

The root cause analysis concludes that the incident occurred because a special procedure to give users reporting rights to patient data had been used to extend the privileges to a team of 50 people, many quite junior and relatively untrained in IT security or compliance. This had been done with the blessing of several senior managers to work around a number of management reporting issues in the system. By exporting and manually processing the data in Excel, the team was able to compensate for certain functional shortcomings in the Salesforce implementation.

The problem is remediated, and an investigation is carried out to find other similar cases where a process might have been used to circumvent good practice. At audit time, Salesforce passes the audit with no serious reservations. However, in Alexandra's mind there still lingers a doubt that maybe they never really got to the bottom of the issue.

Problem

You might be tempted to think that the problem Compliant Is Secure seeks to resolve is the same as for Shared Belongs to Salesforce. That is to say, what the anti-pattern seeks to resolve is the general security problem of how to keep data and systems secure. That is, however, subtly wrong.

While our last anti-pattern sought to solve security in general, Compliant Is Secure sidesteps the problem of actually being secure with the practices required to demonstrate to certain external parties that you are secure. That is to say, the documentation and checklists become the point rather than the actual security practices they are meant to encourage.

This is similar to Shared Belongs to Salesforce in that we aim to reduce the complex, wicked problem of actually staying secure with the much simpler problem of ticking all the right boxes. The strategy of reducing cognitive load by simplifying to a set of well-understood, repeatable processes, as we'll see in the next section, is quite attractive to certain kinds of organizations.

Proposed solution

Compliant Is Secure proposes to substitute and thereby resolve the problem of keeping systems and data secure with the simpler problem of staying compliant with a set of clearly defined standards. That way, you can move forward in confidence that you have followed good practices and, therefore, are as secure as you are likely to get.

A good image to keep in mind for this anti-pattern is a checklist.

Let's be very clear, compliance is a great thing! Compliance with the standards that define good practice in your industry and in general is indeed crucial. This applies to information security, and it applies in general.

Compliance with standards can be an extremely good way to encourage good practice within your organization and embed it within its culture. What makes this scenario an anti-pattern is that compliance becomes the end goal, whereas the end goal should in fact be the awareness and improvement in practice that the compliance exercise is meant to encourage.

Put in another way, for information security, you should aim to use compliance as a tool to become more secure and more security aware, not as a way to get security out of the way because you've ticked all the boxes. It is, however, not difficult to understand why it is attractive to organizations to focus principally on compliance:

- Compliance turns the fuzzy and wicked problem of information security into a manageable and governable one. Managers tend to like problems that are easy to understand and clearly measurable. Compliance is easy to understand; security is not.
- The processes needed to establish and maintain compliance tend to fit well with existing processes and skills, especially in companies that operate in regulated industries such as life sciences or the defense sector.
- When things go wrong, you have clear and unambiguous documentation that you have followed the rules. That helps both in terms of explaining incidents externally and protecting relevant stakeholders internally.
- You have to stay compliant anyway. For most large organizations, a certain level of compliance is mandatory and embedded in law or industry standards. Given that you have to be compliant, there is a clear temptation to make that the end goal.

The results you get from falling into this anti-pattern, however, are frequently problematic, as we'll discuss in the next section.

Results

What you achieve with the Compliant Is Secure anti-pattern is, generally speaking, point-in-time compliance with a standard. You will be compliant at each audit point, and in between, people will follow the necessary processes that are required of them.

However, you have not instituted any deeper sense of the importance of security within the organization or given people the training and tools to engage meaningfully with it. That means that you will often find many workarounds like the one described in our example, where people do something that is technically compliant but completely circumvents the spirit of security that inspired the rules of the compliance standard.

You often see two patterns emerge in compliance-centric organizations:

- **Drift and remediation** : In this pattern, you have ongoing drift away from the standards that have been implemented, because people see compliance as a point-in-time exercise. This is corrected periodically to bring the organization back into compliance for an audit.
- **Structured circumvention** : In this pattern, you continuously follow the rules, but you have included within your compliance system certain ways of granting exceptions, which get exploited systematically to make the lives of workers within the organization easier in one way or another. This is the pattern we saw in the preceding example.

Fundamentally, with the Compliant Is Secure anti-pattern, your organization is missing the “why” of security. They are doing certain things to check the boxes, but do not engage with them beyond that exercise. That points toward the key interventions that will help with this pattern, which we’ll discuss next.

Better solutions

There is a famous book by business author Simon Sinek called *Start with Why* . In it, Sinek posits that the most successful organizations, Apple being the canonical example, do not start with what products they are selling, or how they make them, but with a fundamental purpose expressed in an answer to why the company exists in the first place.

While I won’t make any comments on the merits of this position relative to business in general, when it comes to information security, it is entirely correct. The reason why Compliant Is Secure is an anti-pattern is that it neglects the underlying “why” of information security.

The cure to this problem is, of course, to start from the answer to why keeping information secure not only is essential to avoid threats but actually enables the organization to remain resilient and grow in a safe manner. Then, you can move on to the how and what of compliance regimes and specific controls.

That is a big task for most Salesforce professionals to take on. Helpfully, there are smaller, more practical things we can also do to help:

- Continuously make other stakeholders aware that ticking the boxes does not mean the organization will remain secure.

- Train people in Salesforce security and the threats that can occur despite compliance.
- Find champions to take on security awareness on an ongoing basis.
- Advocate for ongoing monitoring of the Salesforce org on the security level by these trained and motivated people.
- On the technical level, regularly use Salesforce's Security Health Check or third-party tools to identify and address potential vulnerabilities in your org's security settings. Consider using Salesforce's Privacy Center to manage consent, track data subject requests, and maintain compliance with privacy regulations such as GDPR and CCPA. Tool support can sometimes help keep people engaged in ongoing activities.
- Spread knowledge and awareness throughout the organization by regularly bringing up the topic at meetings and dedicated sessions.

Of course, also remember that compliance is important, and you should do your part to stay compliant. For instance, Salesforce's compliance with ISO 27001 for information security and 27701 for privacy management demonstrates their commitment to data protection, but organizations must still ensure their own processes align with these standards. Having now considered a very general security anti-pattern, we will once again zoom in on a specific Salesforce security anti-pattern, Declarative Is Always Safe.

Declarative Is Always Safe

Falling into the Declarative Is Always Safe anti-pattern implies having an over-optimistic attitude toward the security of declarative customization features.

Example

Rakesh is the technical lead for Salesforce at BigCo. BigCo is a long-term user of Salesforce with a big footprint across half a dozen different clouds. They expanded usage massively during the COVID pandemic to facilitate a number of processes being carried out remotely.

Rakesh and his team worked with several smaller partners on the creation of these processes. Because of the time pressure involved, they built the processes using a combination of out-of-the-box features and flows, leaving out any complex requirements for a later time. The flow-based approach was recommended by the architects from both a speed of development and a security perspective.

With the cooperation of the business, this approach proved adequate as a stopgap measure and BigCo managed the transition to remote work relatively seamlessly, partly due to this work. After the initial release, Rakesh and the team kept working on improving and elaborating the new flows, and some have evolved into complex, multi-faceted business applications with powerful features to support the business.

This includes the ability to carry out mass updates, creations, and deletions; to call other systems via APIs; and even to grant new security features to employees when they take on new roles or responsibilities in the business. There are also administrative flows that superusers can use to manage the backend functions.

Functionally, this works well for the business, but on several occasions, mistakes by users lead to accidental record deletion. When a particularly unlucky superuser accidentally deletes a large number of opportunities using an administrative flow, the CTO decides to have a partner review the flows as they're worried about any potential wider issues.

The partner does a review of the flows and comes back with a long list of unsafe practices used in the flows. In particular, the flows give system administrator-level access to a very large number of users who do not have the requisite training to use them. That presents not only an issue of accidental deletion, as has already happened, but also a real security issue. That includes threats from both malicious employees and external threat agents as some of the flows are theoretically triggerable via API updates.

The harm from the incident is easily remedied as BigCo has an advanced backup solution in place, but the remediation of the flows takes much longer. For all future development, Rakesh works with the external partner to create a clear set of security guidelines to be followed when creating flows.

Problem

The problem that Declarative Is Always Safe seeks to solve is that of secure development or customization. That is to say, how do we keep our code and customizations from creating vulnerabilities that can be exploited to get at our data and systems?

It isn't surprising that there is a tendency to consider low-code or no-code approaches as a cure for security headaches. Many classic and well-advertised vulnerabilities have been down to obscure coding errors, so avoiding code seems like a good idea if you want to be more secure.

Of course, low-code/no-code approaches are a relatively recent phenomenon, so whether this will hold good in practice is still up in the air to some extent. The reasoning is generally sound, however, as vendors such as Salesforce will spend much more time securing the low-code components they make available than you can spend on your own code.

However, as we will see when we cover the details of the proposed solution next, this reasoning is not airtight.

Proposed solution

The proposed solution to the problem of secure development and configuration that Declarative Is Always Safe proposes is to circumvent the problem by only using declarative features, for example, custom objects, page layouts, flows, and even Omnistudio, which are assumed to be secure by default. That way, we do not have to spend much time thinking about security and we can get our features created quicker.

This is clearly an attractive proposition to many different stakeholders, for the following reasons:

- It is partially true. Low-code approaches are less likely than code-based ones to contain security gaps
- It simplifies development and configuration by putting clear bounds around the art of the possible
- It provides the technical team with a good way to push back on business requirements that are too complex
- It speeds up the delivery of features by leveraging pre-built functionality that can be adapted to your particular business needs

A focus on declarative features, then, brings many benefits with it and is usually considered a good practice within the Salesforce community. However, what can make this focus – if taken too far – a security anti-pattern is the blindness to the very real threats that can be present, even in declarative solutions.

Results

The key negative result of the Declarative Is Always Safe anti-pattern, then, is blindness to security threats that may be present in your solution and a lack of awareness that such threats might even be there. This can lead to serious negative consequences, as in our example. The problem, however, is only getting worse day by day as Salesforce brings out new declarative features.

It is clear from the way features are being released on the Salesforce platform, that Salesforce wants low-code tools such as flows and Omnistudio to be the go-to solutioning toolkits for customization on the platform. However, the more powerful these tools become, the more likely you are to be able to do something that will result in a security issue.

Equally, External Services, while declarative, can introduce security risks if not properly configured, especially when dealing with external endpoints and data. This also applies to the increasingly prominent AI solutions. For instance, when using Einstein Prediction Builder, you need to ensure that sensitive fields are not inadvertently included in prediction models and that model access is appropriately restricted. Custom metadata types, while declarative, can store sensitive configuration data and should be secured with appropriate access controls.

Flows are, however, the biggest potential risk area. Let us, therefore, consider some security problems that can happen with flows:

- General permissions such as Run Flows or the Flow User license can give a very large number of users permissions to run all flows in the org, leading to issues such as those in our example.

- If users have access to the All_Flows list view, they can then run any flow in the org from the UI. This includes external users.
- Flow execution contexts are complex to understand, and it is easy to end up running a flow in system mode by mistake, leading to a potential escalation of user rights.
- Client-side validation in flows can accidentally leak information in some cases by including data in the response.
- Component visibility can leak information as the component is still included in the HTTP response.
- Apex actions with wide-ranging effects may be called from flows without a clear understanding of the security implications.
- When using screen flows in Experience Cloud sites, ensure proper access controls are in place to prevent unauthorized access to sensitive data or operations.

I recommend the article *Understanding Salesforce Flows and Common Security Risks* , available at <https://appomni.com/resources/aolabs/understanding-salesforce-flows-and-common-security-risks/> , as a good introduction to some common flow issues.

However, the wider point is that there are real security issues that happen with flows, Omnistudio, and even with simpler declarative features. New flow features, such as Flow Orchestration, introduce additional complexity and potential security risks if not properly configured. Therefore, we cannot rely on them to be safe by default but have to take due precautions, which we'll discuss in our next section.

Better solutions

The first and most important piece of advice for avoiding this anti-pattern is to treat your declarative solutions every bit as seriously as you would your code-based ones. Tools such as flows and Omnistudio are increasingly as powerful as Apex and solve complex business requirements.

That means that for solutions based on these tools, you should follow a software development process that is fit for purpose in relation to the complexity and importance of the solution in question.

Specifically, when it comes to security, you should keep the following things in mind:

- Standards and guidelines for declarative solutions should be in place as they would be if you were writing code
- Make risk and threat assessments upfront for declarative solutions that deal with sensitive data or deploy a lot of power
- Train developers and configurators to understand the issues and create general awareness about what relying on declarative tools does and does not accomplish
- Keep an eye on the long-term trend toward declarative tools and update your guidelines as needed
- Monitor the external environment for issues or exploits related to declarative functionality on Salesforce

With that said, we will finish our discussion of security-related anti-patterns by taking a look at how you can really muck up your sharing model in Salesforce.

Sharing isn't always caring

Salesforce has one of the most powerful and complicated sharing architectures known to man. This is both a blessing and a curse. In this section, however, we will focus on showing how it can go wrong and what to do to prevent it.

Spaghetti Sharing Model

Spaghetti Sharing Model does to the security of a Salesforce org what spaghetti code does to the maintainability of an application.

Example

GlobalCo is a conglomerate with numerous different business lines selling products across a number of B2B verticals. Most of GlobalCo's organization is run within the business units, but IT has become a strong global function, as it has made it possible to collaborate effectively across business units using simple tools such as Slack for collaboration.

A few years back, GlobalCo adopted Salesforce Sales Cloud and Service Cloud as their corporate CRM. Huan was the lead architect on that project and now sits in the global Salesforce Center of Excellence. While some subsidiaries still use other CRMs operationally, all basic data about customer transactions must make its way to Salesforce. To encourage the standardization of processes and data, GlobalCo adopted a strong single-org strategy for its Salesforce environment.

Unfortunately, while GlobalCo would like to be able to share as much data as possible globally, a number of regulations prevent them from going as far as they would like. There are a number of problems:

- Some countries require their data to be isolated from others, creating country data silos
- Competition law in a variety of jurisdictions also prevents some subsidiaries from sharing data with specific other subsidiaries to prevent collusion
- Salespeople in some jurisdictions are somewhat paranoid about who can see their opportunities for fear of someone else stealing them, leading to a number of extra protections being put in place
- Finally, executives in certain business units are used to keeping sensitive opportunities and customer support cases secret and only sharing among a select few, which has also been at least partially accommodated

After go-live, there have been a lot of complaints that the structure GlobalCo put in place to meet these requirements was excessively rigid. There is a call for a number of exceptions from at least half a dozen countries.

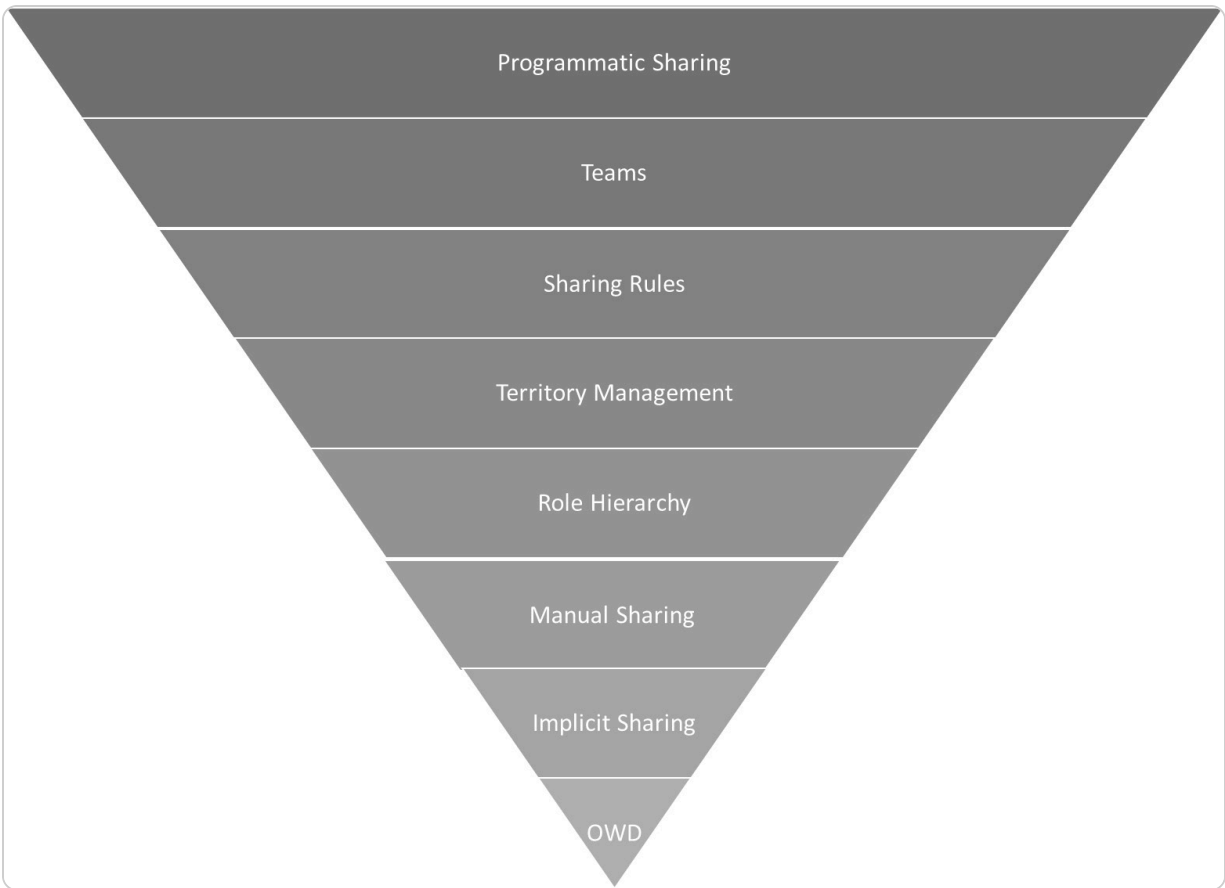


Figure 3.2 – Sharing mechanisms

Huan leads the team to investigate the requirements. Initially, he thinks the problem is solvable by making minor adjustments to the organizational hierarchy and enterprise territory management. This, however, proves too simplistic and so he devises a number of additional public groups to capture more refinements to the model.

Users, however, are still complaining, so Huan gets creative with the use of Account and Opportunity teams. However, because the process needed would be too onerous to manage manually, the team ends up implementing a number of automations to automatically add team members to Account and Opportunity teams based on certain criteria.

Even that, however, turns out to not be able to capture the full extent of the business requirements for when certain records should and shouldn't be shared between specified business units and countries. Huan specifies an Apex-based sharing mechanism and gives it to the development team to implement.

The complete adjustment to the sharing model goes live, globally, over a weekend. Smoke testing is fine, but everyone is holding their breath as the testing of the model is difficult and no one has been able to definitively say that all the important scenarios have been tested.

While there is no disaster, a steady trickle of access issues makes its way to the team over the week following go-live. There are a lot of cases, where either someone has access who shouldn't have or someone lacks access they should have.

What's much worse, however, is that the team is finding it nearly impossible to pinpoint the errors. There are just so many moving parts that figuring out what goes wrong in specific cases becomes too complex. Every time the team thinks they've fixed one issue, it seems another two appear as a consequence.

After several frustrating weeks of trying to resolve the problem, the change management board makes a decision to roll back the changes to the sharing model and find a set of manual workarounds until a more stable technical solution can be found.

Problem

In classical software applications, there is a well-known concept called **spaghetti code**. It happens when programmers work without design practices or the necessary experience to implement a system. Instead, they just do one thing at a time in a way that works for now with no thought for the larger picture.

Over time, this leads to code that resembles a portion of spaghetti on a plate. Everything is tangled up with everything else, changes to one part of the system impact other parts in seemingly random ways, and even the original writers of the code don't really understand what is going on.

Luckily, on Salesforce, we rarely see spaghetti code to this extent. It can occur, but in my experience, it is quite rare. What does occur frequently is the Spaghetti Sharing Model anti-pattern. That is, sharing models are so complicated that even their designers can't reliably tell in a specific case why someone has or hasn't got access to a given record.

The Spaghetti Sharing Model anti-pattern arises from an honest attempt to give the business what it wants in terms of record sharing. The problem is that for large, global organizations, the level of complexity experienced in this area is so large that it is nearly impossible to get right.

In our example, we saw some of the reasons why:

- Privacy laws differ between countries and determine what can and cannot be shared
- Competition law can restrict visibility between subsidiaries
- Concerns about bonuses and commissions are extremely important to sales teams and can lead to serious concerns about data sharing
- Executives often want to keep certain data a secret for a variety of business reasons, while sharing other data that is less sensitive of the same type

There are more scenarios than this concerning, for instance, the protection of sensitive health or financial data according to certain laws and standards or internal political factors within the organization that may impact sharing requirements.

Overall, if you find yourself in an organization with this kind of requirements setup for data sharing, you are in danger of ending up in the Spaghetti Sharing Model anti-pattern, simply by trying to accommodate the requirements you are given.

Proposed solution

The Spaghetti Sharing Model anti-pattern suggests that you address sharing requirements in an incremental, iterative way, seeking as closely as possible to meet the business needs.

This seems eminently sensible because of the following:

- It is exactly what we would do for most other types of requirements
- It fits with the dominant agile approaches to software development that are used by most Salesforce implementation teams
- You can nearly always achieve your goal in the short term
- It takes a long time before the complexity reaches a level to make the process go off the rails

Unfortunately, this process can break down in organizations of a certain size and complexity and results in the anti-pattern we are discussing, as we'll see in the next section.

Results

The results of the Spaghetti Sharing Model anti-pattern are a situation where some or more of the following characteristics are present:

- Records are shared with some users without anyone fully understanding why
- Record access is missing from some users without anyone fully understanding why
- Pinpointing access issues is extremely difficult
- Making changes to the sharing model is considered entirely off-limits due to complexity
- Changes to sharing often lead to new issues that are difficult to understand and fix

It is worth reflecting on why sharing is more prone to the Spaghetti Sharing Model anti-pattern than other configuration areas. The answer lies mostly in the vast number of mechanisms that are available to control sharing on Salesforce and their interactions. Depending on how you count, there are more than 20 different ways of extending the sharing of objects and records in Salesforce.

The following table shows a sampling of mechanisms that can be used to control sharing:

Mechanism	What it's used for
-----------	--------------------

Ownership	Access to the record owner, user, or queue
Profile	Access (or not) to an object at the basic level
Permission Set	Access (or not) to an object at the basic level
Org-Wide Default (Internal)	Default internal sharing level
Org-Wide Default (External)	Default external sharing level
Role Hierarchy	Access to subordinates' records
Grant Access Using Hierarchies	Share (or not) with role hierarchy
Public Groups	Share with a defined group of users
Ownership-Based Sharing Rule	Share based on a configured rule based on record ownership
Criteria-Based Sharing Rule	Share based on a configured rule based on record field values
Guest User Sharing Rule	Special sharing with the guest user
Account Teams	Share based on membership in an account team
Opportunity Teams	Share based on membership in an opportunity team
Case Teams	Share based on membership in a case team
Manual Sharing	Share records manually

Implicit Sharing	Automated sharing between parents and children for certain standard objects
Territory Hierarchy	Share based on configured territories and territory membership
External Account Hierarchy	Grant access based on account hierarchy
Sharing Group	Share records owned by portal users
Sharing Set	Share records with portal users based on matches between the Account or Contact field
Manager Groups	Share records with the management chain
Apex Sharing	Share programmatically
Restriction Rule	Restrict access to specific types of records

Table 3.1 – Salesforce sharing mechanisms

Ultimately, this vast number of mechanisms, if used injudiciously, can lead to the Spaghetti Sharing Model anti-pattern, simply because their interaction becomes opaque to a human-level intelligence.

Better solutions

As we have seen, the Spaghetti Sharing Model anti-pattern can occur just from people doing their jobs and incrementally seeking to create value for the business by meeting their requirements. That means this anti-pattern can occur without anyone doing anything wrong or making a specifically wrong decision. Someone needs to step outside the day-to-day work in order to spot this issue prior to it occurring.

At the risk of provoking severe retaliation from agile fundamentalists, I will suggest that if you find yourself in an organization where there are highly complex sharing requirements, you need to stop, take a step back, and put in place an upfront design for the sharing architecture along with governance mechanisms to enforce it over time. For instance, you might introduce a principle that it is best to be parsimonious in using sharing mechanisms and make the adoption of new types of sharing subject to approval by an architecture review board or design authority.

This concludes our presentation of security anti-patterns. We will now look at the key things to take away from the discussion.

Knowing the takeaways

In this section, we will abstract a bit from the specific patterns and instead try to pull out the wider learning points you can use in your day-to-day work as a Salesforce architect or in preparing for the CTA Review Board exam.

When architecting Salesforce solutions, you should be mindful of the following:

- Security is not just a technical issue; the way you frame the problems and create awareness is even more important.
- It is easy to assume away complex security issues in various ways. While you may not be able to singlehandedly change this as an architect, you can be a part of raising awareness.
- Instituting sharing sessions and sending out relevant material is one good way that you can help make things better.
- You can also ensure that security guidelines and practices on the technical level reflect the full threat picture.
- Do not assume that something is Salesforce's responsibility without good evidence.
- Do not assume that because you are using a declarative feature, you cannot have a security gap.
- Find ways of monitoring the security stance of your Salesforce orgs on a frequent basis and act on the intelligence.
- Make security matters a recurring discussion in governance forums where you can influence the agenda.
- Don't build sharing models incrementally unless your requirements are very simple.
- Instead, take the pain upfront and come up with a set of mechanisms that can work in your scenario.
- Regularly assess and update your understanding of the shared responsibility model as Salesforce introduces new features and services, especially AI-driven capabilities.
- Implement a continuous security monitoring strategy that includes automated tools and periodic manual reviews to stay ahead of evolving threats in your Salesforce environment.
- Develop a comprehensive security training program for your Salesforce team that covers both declarative and programmatic development, emphasizing the potential risks in both areas.
- Establish a security-first culture within your Salesforce team by incorporating security considerations into your development life cycle, from requirements gathering to deployment.
- Regularly audit and optimize your sharing model, using tools such as Salesforce Health Check and Optimizer, to prevent the gradual accumulation of unnecessary complexity.

In preparing for the CTA Review Board exam, you should be mindful of the following:

- Security is embedded in a lot of areas, so even though it is a distinct domain, you should still cover security matters concisely where relevant in your solution run-through.
- Security is a complex area, so be as clear and concise as possible. It is very easy to get drawn into long discussions on security matters and you don't have the time for that at a review board.
- Consider including some comments on security when reviewing your governance model and potential risks.

- Be parsimonious when picking security mechanisms. Having fewer mechanisms will make them easier to explain and make it less likely that you will have interactions that lead to unforeseen consequences.
- Utilize permission set groups to organize and manage complex permission structures more efficiently, reducing the risk of overly complicated security models.
- Of course, don't oversimplify or make assumptions that would point in the direction of not appreciating how things can go wrong in the security domain.
- Sharing is particularly easy to get wrong, so be quite careful about designing exotic sharing solutions.
- Don't invent additional security requirements where it isn't stated in the scenario. You can easily think yourself into additional implied requirements when reading many scenarios.
- The one exception to this rule is dealing with privacy law, such as GDPR, if you have an org in a jurisdiction where it applies.
- When integrating with external systems, ensure proper API security measures, such as OAuth 2.0, IP restrictions, and regular token rotation, are followed.
- Be prepared to discuss how you would approach security in a multi-cloud Salesforce implementation, considering the unique security challenges of each cloud.
- Practice explaining complex security concepts, such as the implications of AI on data security, in a clear and concise manner suitable for the time constraints of the review board.
- Be ready to explain how you would balance security requirements with other architectural concerns, such as performance and user experience, in a large-scale Salesforce implementation.

We have now covered the material for this chapter and are ready to proceed to the next one. First, however, we will summarize our learning.

Summary

In this chapter, we have seen how there are many ways to compromise your security by following seemingly innocuous courses of action or failing to realize subtle distinctions in responsibilities and approaches.

These include both general security anti-patterns that affect many different platforms, such as Shared Belongs to Salesforce and Compliant Is Secure, and unique-to-Salesforce anti-patterns such as Declarative Is Always Safe and Spaghetti Sharing Model. These underscore the particular complexity of the security domain.

When it comes to security, you have to attend both to high-level issues of organizational culture and to the minutiae of how mechanisms are designed specifically for the technology you are using. That is what makes it both frustrating and highly interesting at the same time.

Having covered the security domain, we will move on to have a look at data next.

4

What Not to Do about Data

This chapter is about the ways in which your data layer can be compromised on the Salesforce platform. Throughout this chapter, we cover four data domain anti-patterns that occur with some frequency in Salesforce environments and give you pointers on how to recognize and avoid them. In addition, we have added a new section on AI-related anti-patterns to help you navigate these new developments.

In this chapter, we're going to cover the following main topics:

- Why treating Salesforce as a relational database won't lead to acceptable results
- How failing to coordinate activities can lead to a disconnected data model with serious negative repercussions
- The negative consequences of failing to plan for growth in your database, especially when you should know better
- Why data synchronization can be a great solution on a small scale, but a nightmare on a larger scale
- How to approach AI architecture in the context of Salesforce

After completing this chapter, you will have a good understanding of how Salesforce is different from a traditional relational database and why that matters greatly. You will also know the importance of good governance, planning for data modeling and growth, and what can go wrong when you don't have governance in place. Finally, you will have a deeper insight into the complexities of data synchronization.

Mistaking Salesforce for a regular database

Many people come to the Salesforce ecosystem from other technology backgrounds, and they often come with preconceptions about how architecture should be done based on their past experiences. While that can be enriching to the platform, there are also cases where it can lead you astray, architecturally speaking. Perhaps the most frequent of these is the mistake of using Salesforce as though it were some other kind of database—generally a relational one.

Salesforce as a Relational Database

The Salesforce as a Relational Database anti-pattern consists of mistaking the Salesforce data layer for a relational database.

Example

UmbrellaCo is the largest global manufacturer of umbrellas for the tourism industry. They use Salesforce for managing their B2B sales channel, including opportunity management, quoting, and ordering.

However, in the overall systems landscape within UmbrellaCo, Salesforce is a relatively minor component. Overall, the landscape is dominated by a set of bespoke lines of business systems that have been developed and maintained in-house and an aging **Manufacturing Resource Planning (MRP)** system.

There is also a relatively modern middleware platform and UmbrellaCo's approach to enterprise IT relies heavily on integrating, translating, and combining data from various systems within the middleware. This allows them some additional agility that they otherwise would not be able to have, given their aging and bespoke systems portfolio.

Rishi is the responsible manager for Salesforce at UmbrellaCo. He is an external hire who only recently joined the company from a mid-size ecosystem partner. One of the first things he has to deal with in settling into his new role is a series of complaints from the middleware team about the Salesforce data model.

UmbrellaCo has longstanding practices for modeling certain kinds of data, such as different types of accounts, for example, SME accounts versus large corporate accounts, and strict standards for modeling addresses. The Salesforce model does not accommodate this model in the standard way, leading to substantial work on the middleware team in mapping the data to other systems.

John, an integration architect long employed by UmbrellaCo, is assigned to the Salesforce team in order to align the Salesforce data model. Rishi is immediately quite uncomfortable in the collaboration with John. John displays a high degree of distaste for the Salesforce data model and thinks it fundamentally fails to conform to good data modeling practices.

The standard within UmbrellaCo is to use relational databases, normalized to **Third Normal Form (3NF)**, and follow the corporate conventions for key data types. Salesforce does not meet these expectations and cannot easily be made to do so. Rishi tries to explain that the Salesforce data model shouldn't be thought of as a classical relational database, whatever superficial resemblance it might have to one, but his objections fall on deaf ears.

After a month of work, John puts forward his proposal for reworking the Salesforce data model. It involves replacing the standard Salesforce address field with a new set of custom objects incorporating the address and an arbitrary number of address lines linked to the master account object. Different types of accounts will get their own individual custom objects and refer back to the master account object using an inheritance-like interface.

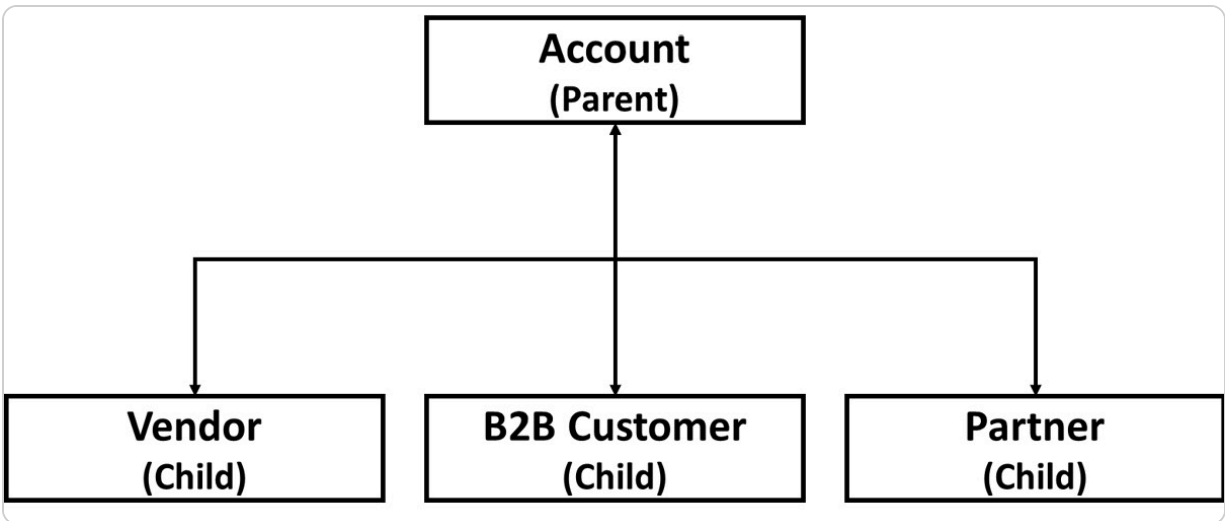


Figure 4.1 – Proposal for refactoring account model

This will require both rework on the user interface and an internal data migration exercise within Salesforce to accomplish, but John is adamant that this will bring the system into line with corporate standards and make life easier on the middleware team.

Rishi vehemently objects and even gets his Salesforce account executive to bring in a senior solution engineer from Salesforce, trying to explain why following this plan is a surefire way to cripple the Salesforce platform and make additional projects more complex and expensive. The objections make their way to the company's architecture forum but are rejected as conformance to corporate standards is considered a more important imperative.

Going forward, data modeling on Salesforce should follow the normal UmbrellaCo standards and deliver well-normalized data models that meet the specifications set out in the enterprise data model. Rishi draws a deep breath and consigns himself to having to do mostly bespoke solutions on Salesforce going forward.

Problem

The problem addressed by Salesforce as a Relational Database is the general one of modeling business entities in the context of a software system. In this case, use the most commonly used modeling paradigm in the software industry: relational database models.

In this approach, you map a logical model of a business domain into a set of database tables, using a process called **normalization**. The details of what normalization is do not need to concern us much, but it is fundamentally concerned with the elimination of redundancy within the database. That is to say, each piece of information should be held once and uniquely within the database and then referenced in all of the places that it's used.

In addition, there should not be repeating fields, such as multiple address lines, and all the fields in an entity should rely explicitly on the primary key: the ID field in Salesforce. You should always have one and only one unique key for the entity.

By applying these and a few more elaborate rules with greater or lesser stringency, you get to a certain normal form, the name for models of normalization. The strictest application is something called the **Sixth Normal Form (6NF)** where you end up with tables consisting of only a primary key and one other attribute. However, most enterprise applications strive for 3NF, which meets the criteria we outlined previously.

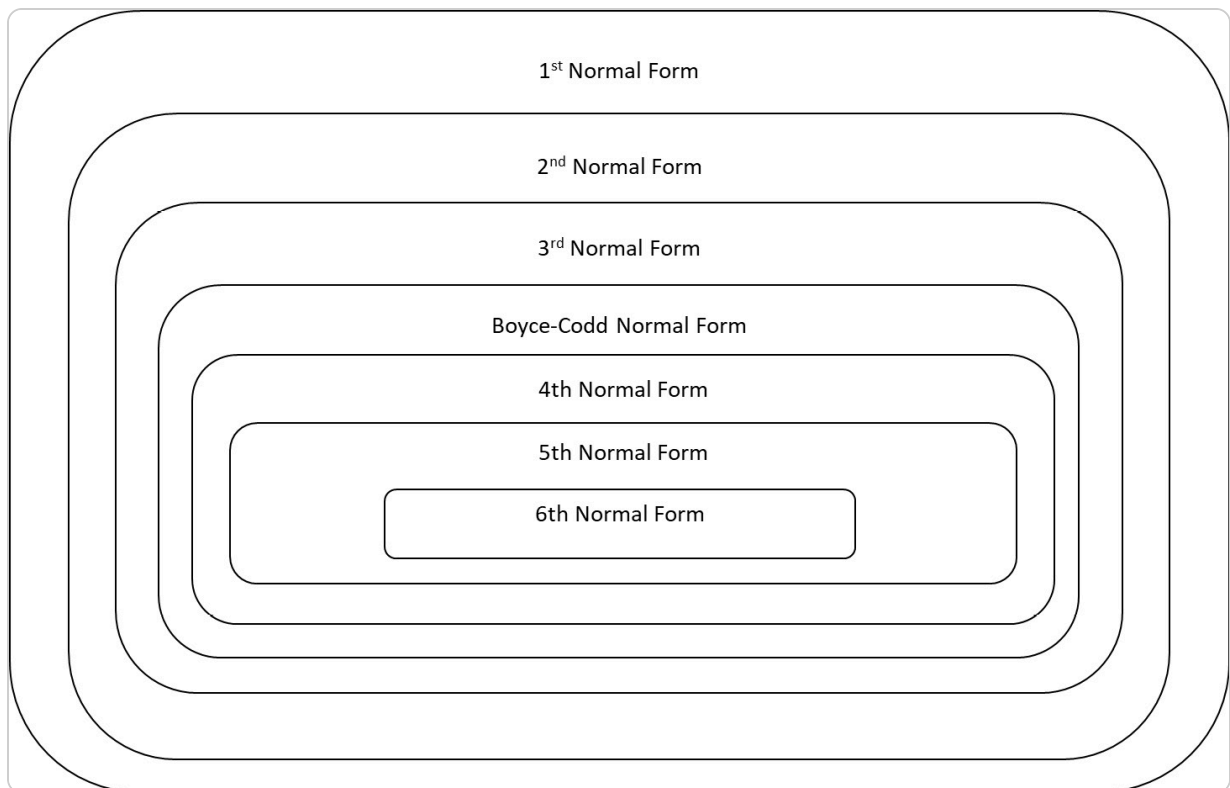


Figure 4.2 – Hierarchy of normal forms

This is all well and good and is the accepted way to do things for systems based on relational databases. However, Salesforce, while it has a number of surface similarities to a relational database (namely tables, called objects) and relationships (in the form of lookup and master/detail relationships that enable its use as a relational database), is not a relational database and should not be treated as such.

Proposed solution

Salesforce as a Relational Database ignores the distinctive nature of the data modeling used on the Salesforce platform and replaces it with a model that bases itself on relational databases. This means

using custom objects freely to serve as tables and relationships of either type to serve as foreign key relations.

It explicitly ignores Salesforce's standard ways of dealing with record differentiation, such as record types and page layouts, in favor of a relational model with a larger number of unique custom objects. In addition, this anti-pattern tends to make scant use of standard objects as these do not fit with a well-normalized data model.

This solution makes a lot of sense to many experienced architects coming to Salesforce from other platforms without a lot of understanding or training. It is, after all, the accepted way for things to be done in all traditional enterprise systems, and consistency with an accepted standard is extremely valuable in enterprise architecture.

However, bending a system so out of shape as this pattern does to Salesforce goes to show that every standard has exceptions.

Results

As we saw in the earlier example, the result of introducing this anti-pattern is fundamentally increasing the complexity of your Salesforce solution, resulting in additional costs both for solutioning and maintenance.

By relying on custom objects for nearly everything, you reduce your ability to leverage out-of-the-box features, because relational models tend to require a larger number of smaller objects, which makes some commonly used Salesforce features (for example, cross-object reporting or cross-object formula fields) nearly impossible to use.

That means you'll most likely need to write more custom code or construct complex custom flows, which is where you take a real hit in terms of both upfront implementation cost and time and the maintenance and extensibility of the solution afterward.

There is also the opportunity cost of not being able to use standard functionality to the same extent and not being able to adopt newly released features with the same frequency to consider. All in all, this is a high price to pay for consistency.

Better solutions

The data domain is an area where you should adopt the Salesforce way to get the best results. While sometimes there are exceptions to Salesforce's good practices, when it comes to data modeling, the guidance should be followed nearly without exceptions.

That means modeling broad-based business objects directly in the system rather than going through a normalization process and relying on the platform to manage and optimize the data or at least give you the tools to do so. It also means using standard objects whenever you can, avoiding custom objects whenever possible, and ensuring that the data model is well governed to avoid issues with data integrity and redundancy.

This is obvious to many people who have grown their careers in the Salesforce ecosystem, but it is counterintuitive to most traditional architects. This, then, is one of the few anti-patterns that is more likely to be initiated by a highly experienced architect than a brand-new one. It's also more common in AppExchange applications than in standard implementations as these are closer to the traditional development models found on other platforms.

Forgetting about governance

Governance is important in all architecture domains, but perhaps a few domains can go wrong where governance is missing as the data domain. In the following sections, we will explore two examples of this phenomenon, starting with what happens when you fail to coordinate data models on a common platform.

Disconnected Entities

Disconnected Entities is an anti-pattern characterized by unconnected data entities proliferating in a common database, often with multiple database representations of the same business-level entity.

Example

SmileCo is a major provider of dental supplies operating in several global cities. They have been using Salesforce for several years, using a departmental strategy, where each department is run independently on Salesforce without central oversight.

There are three departments that use Salesforce in a serious way, and they are as follows:

- **Finance** , which uses it for debt collection using a home-grown application
- **Logistics** , which uses it for vendor management using an application that has been developed by a small ecosystem partner
- **Sales** , which uses a standard Sales Cloud implementation

The information in these apps is spread out and there is no cross-visibility or collaboration between the teams. Kimmy is a consulting manager at a Salesforce Summit partner who is brought in with a small team to enable a comprehensive view of the underlying data.

She, however, soon discovers that the job is bigger than she bargained for. The finance application uses a flat, denormalized data model to model debt collection. All information is held in a giant debt custom object, including all contact and account information. This data is not validated and the data quality at a glance seems problematic.

The logistics application effectively mirrors the standard Salesforce data model using custom objects to represent vendors, vendor contacts, and information about quotes and purchases made from vendors. The fields differ substantially from the standard Salesforce data model but could be made to fit with standard objects such as the account and contact with a bit of creative data mapping.

The Sales Cloud implementation is relatively standard but uses quite a few additional custom fields that in no way match up with the way data is modeled in the other two platform apps. Kimmy reports her findings to the client, who after some hesitation requests that she continue with a plan for consolidation nonetheless.

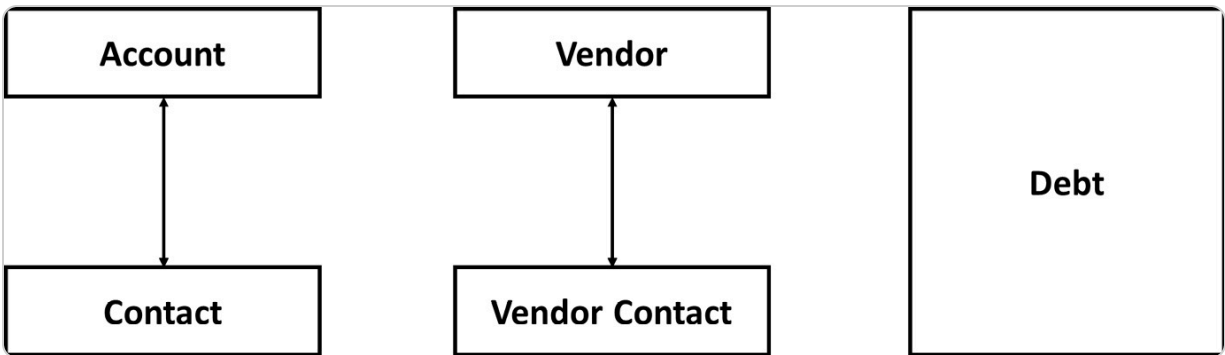


Figure 4.3 – Three different ways of representing an account and contact

The plan she proposes seeks to standardize the use of the Account and Contact objects, using the standard objects, across all three departments, and on that basis write a set of reports and dashboards giving the required overview. She gives up on trying to consolidate additional objects as they are too far away from any standard objects for the attempt to be likely to succeed.

For the finance application, she suggests renormalizing the account and contact parts of the gigantic debt object. This requires a data migration exercise as well as significant work to deduplicate and improve data quality. Rework on the application itself, thankfully, is relatively minor due to the simple declarative features used for its implementation.

The logistics application presents a larger problem. Most of the functionality is in custom code and requires substantial rework. The partner is reluctant to make the changes Kimmy suggested as they prefer working in their own space, but after some pushing, they relent. However, the changes are quite expensive and take several months to implement.

Overall, what was meant to be a small consulting engagement lasting a few weeks ended up taking more than three months, incurring substantial costs. Upon leaving the client, Kimmy sends a new proposal for introducing some basic Salesforce governance functions within SmileCo. They take the proposal under advisement and promise to get back to Kimmy in good time.

Problem

Disconnected Entities is another anti-pattern that happens when you fail to recognize that certain practices carried out without oversight or governance can result in serious issues at a higher level of operation. It is caused by a lack of awareness rather than by the attempt to achieve something specific.

It tends to occur in smaller or highly decentralized organizations where different groups may share the same infrastructure, in this case, a Salesforce org, but do not really coordinate activities beyond the absolute basics. It also requires an environment where IT maturity is low and governance of IT is minimal.

Proposed solution

So far, if we were to frame what the Disconnected Entities anti-pattern proposes as a solution, it would be something along the lines of “don’t worry about the data model; any design will be good enough.” At least that is how it may be interpreted because there will be no standards, practices, or guidelines to follow.

Lack of governance can have bad effects in many domains, but in the data domain, the Disconnected Entities anti-pattern is one of the worst. It means that teams build in isolation with no awareness of what else exists on the same platform or how other teams are using the same elements (even the same standard objects).

That means that you can have a cross-organizational impact without realizing it. Also, at the very least, the anti-pattern leads to the kind of scenario sketched in our earlier example where cross-departmental initiatives are hampered or blocked by the data model.

Results

For the Disconnected Entities anti-pattern, the results can be as chaotic as the process that produces them. For the small teams whose uncoordinated work eventually causes the anti-pattern, it usually works out just fine in the short term.

When you are working in isolation on your own thing, there really isn’t any short-term impact to doing things badly or idiosyncratically. The impact only comes with scale, coordination requirements, or a need to extend the application to new use cases.

The specific negative consequences can therefore vary considerably, but some common ones include the following:

- An inability to achieve 360 visibility of contacts or accounts, or any other relevant business entity for that matter
- An inability to do cross-departmental or cross-application reporting
- Increased cost of new development and maintenance due to variability and complexity in the data layer
- An inability to connect applications or create cross-departmental workflows without incurring substantial rework costs to bring different areas in line with each other
- Duplication of effort between different areas
- Less scope for adopting new standard features as they are released because of inconsistencies in the data models

Overall, then, this anti-pattern has many of the same characteristics in the data domain that Big Ball of Mud had in the system domain or the Spaghetti Sharing model had in the security domain.

Better solutions

Unsurprisingly, the key to avoiding an anti-pattern caused by missing governance and awareness is to institute the appropriate governance- and awareness-generating mechanisms. That means first and foremost having the appropriate governance forums in place, where different areas can coordinate their activities in a structured way.

This needs to be supported by good data governance standards and practices to support the work done in the relevant forums. These can take different forms depending on the organization and its culture, but to avoid Disconnected Entities, a list such as the following is a starting point:

- Rigidly enforce the use of standard objects and fields wherever possible
- Ensure that teams use standard objects and fields consistently
- Have an approval process for creating new custom objects that teams must follow
- Have a lighter approval process for creating custom fields
- Define ownership of all objects and ensure that object owners are consulted about all changes to their objects
- Have an architect or two take a global view of the data model and offer ongoing advice on its evolution

Having now covered what happens when you fail to coordinate your activities in the data domain, we will move on to look at what can happen if you fail to plan for growth.

Unplanned Growth

The Unplanned Growth anti-pattern happens when organizations fail to plan for growth in the data to be stored, although they could or should have known better.

Example

HappyWorkoutCo is a sportswear retailer with more than 200 shops globally. They have developed a customized retail management application based on Salesforce Industries but with a large amount of customization and some bespoke development. The application has been developed in an agile, iterative fashion and that is also the way they are proposing to go live.

Martin is in charge of the rollout to the stores. Initially, they will roll out the product for only three small product categories to a handful of stores in major markets. While the product tests well, Martin still has butterflies as the first stores go live.

But this time, the rollout is a great success. The product works flawlessly and the staff at the stores love the new capabilities the software gives them for greater individual engagement with customers across channels.

Buoyed by the initial success, Martin and his team start a program of incremental rollouts to both more stores and product categories. Everything initially continues to go well, but after a while, complaints start coming in about system performance. There is a noticeable slowdown on a number of screens in the system and some key reports take minutes to run.

A task force consisting of members of Martin's team as well as technical resources from the development partner is put together to have a look at the issue. However, the decision is made to continue the rollout of both new products and new stores.

The task force gets underway, but soon after it begins its work, a major incident occurs following a major go-live where 20 new shops and several thousands of products were being brought online. The system creeps to a halt and several screens stop working altogether.

Martin's team conducts an emergency rollback, which restores the system to an operational state, but now all eyes are on the performance task force. The team works quickly with renewed vigor and intensity and discovers what they believe to be the root cause.

Inside the package containing the core retail management application, there is a junction object that contains a record for each product configuration available at a given retail store. When the system went live, there were 5 stores, 100 products, and about 10 configurations each, for a total of 5,000 records.

After the big go-live, there were 150 stores with 17,000 products, each with an average of 35 configurations for a total of 89,250,000 records. What's worse is that the object didn't only contain current records but also historical ones that had been overwritten during the rollout process, leading to more than 117,000,000 records in the junction object.

The object is used in a number of automations, batch jobs, and reports and is also queried frequently by custom user interface components. The design seems to have been mainly a matter of development convenience and no one seems to have ever done any calculations around the amount of data to be stored in the object.

The task force presents its conclusion and recommends a number of stopgap measures, including removing the automations, archiving all of the historical records, removing rare product configurations, tuning some reports, and implementing some caching and optimizations in the user interface.

This program of work puts out the immediate fire, but the system still doesn't perform up to expectations. Martin starts negotiations with the development partner about how to redesign the system to eliminate the root cause.

Problem

This is another anti-pattern that tends to be instigated not by an active choice but by a deprioritization of certain practices in order to deliver on a specific goal. In the case of unplanned growth, the team deprioritizes the ability to scale in order to achieve greater speed in the initial delivery.

This can be appropriate in some cases, for example, if you are a start-up trying to prove product-market fit on a shoestring budget, and for this reason, it can be subtly seductive. Many large organizations these days want to see themselves in the mold of an agile start-up, adopting many practices from this world.

However, for large, well-established organizations that already have a certain scale, failing to acknowledge this fact upfront is a recipe for disaster.

Proposed solution

The Unplanned Growth anti-pattern proposes that a team should get on with the important business of delivering without paying much attention to the issues that may arise due to increasing scale down the road. Effectively, the proposal is to make small-scale progress and then adapt later, should the scale be required.

This anti-pattern is different from others such as Disconnected Entities as it is not an absence of awareness that causes it, but rather an active choice to deprioritize planning for scale. Again, it is worth noting that this is not always an anti-pattern.

In some cases, starting small and then taking the hit on rework later is a justifiable strategy. Typically, this is the case if you don't know upfront whether your idea will work or how many users it will need to accommodate when you are building the application partially to get more information.

But it is always an anti-pattern when done by a large organization that knows the application it is building will eventually need to reach a certain definable scale.

Results

The results of Unplanned Growth encompass a fairly large number of scaling issues, determined by the nature of the application in question. Most commonly, you will see **Large Data Volume (LDV)** type issues such as the following:

- Slowdowns in reports and dashboards
- Slowdowns in list views
- Locking issues on data loads or even in normal use
- Timeouts when loading custom components
- Errors cropping up about governor limits
- Third-party packages failing

You may also see the general user experience crawl to a halt, especially if it uses many custom pages and components. Integrations can also be affected by the same issues, leading to serious errors across systems. Even automations that were working perfectly may start to fail, although not necessarily in a systematic way.

All in all, if the unplanned growth is significant enough, it can bring your entire solution to its knees.

Better solutions

The number one thing you need to do to avoid this anti-pattern is to be clear from the start about what scale you are building for. If you are planning to build something that works on a small scale to test out an idea, then that's fine, but be explicit about it and have a plan to take a step back and redesign if it turns out you are successful and need to scale.

If you know for a fact that you need to hit a certain scale, it is nearly criminal not to design for it upfront. That means incorporating LDV mitigation tactics such as the following:

- Using selective queries
- Optimizing your queries with the platform query optimizer
- Avoiding synchronous automations on LDV objects
- Avoiding synchronous code running queries on LDV objects
- Using skinny tables to optimize specific queries and reports
- Using custom indices to optimize specific queries and reports
- Archiving non-current data continuously

- Using external objects to store data externally when possible

Designing for growth also means designing the data model in a way that reduces LDV issues in the first place. In our preceding example, there would most likely be many alternative data models that did not have to hold a complete record of all product configurations by store in a single junction object.

Having now considered what happens when you fail to consider scale in the data domain, we will move on to the last anti-pattern in this chapter, which is Unconstrained Data Synchronization.

Synchronizing excessively

Data synchronization is hard to get right but extremely useful when it works. It is, however, easy to get carried away when starting to put synchronization methods in place, which can cause a plethora of problems. We'll see how in the following sections.

Unconstrained Data Synchronization

Unconstrained Data Synchronization is an anti-pattern characterized by an overly ambitious approach to synchronizing data between systems without fully appreciating all the things that can go wrong.

Example

BusyCo, a large vendor of productivity-related products, has recently implemented a B2B omnichannel digital experience spanning web, social, call center, and partner channels. They have implemented the web application using a bespoke approach based on Microsoft Azure and use Salesforce to manage any manual processing steps for quotes and orders as well as opportunity management and any quotes or orders originating via calls or field sales.

All of the information eventually goes into the **Enterprise Resource Planning (ERP)** system, which handles the fulfillment and financial side of the ordering process as well as serving as the source of truth for financial forecasts. Because of its role in producing financial information, the information in the ERP must always be of the highest possible quality.

Oleg, the lead architect on the Salesforce side, recommends a structured approach to data governance with clearly defined roles and responsibilities for each system in relation to the core data model objects. Also, there should be clearly defined ownership in place for key fields and objects, so the source of truth is always known.

BusyCo, however, goes with the approach that any piece of information can be modified in any system at any point in time and all systems will be updated with the modified data in near-real time. To

achieve this goal, they have put in place bidirectional data synchronization between all three systems.

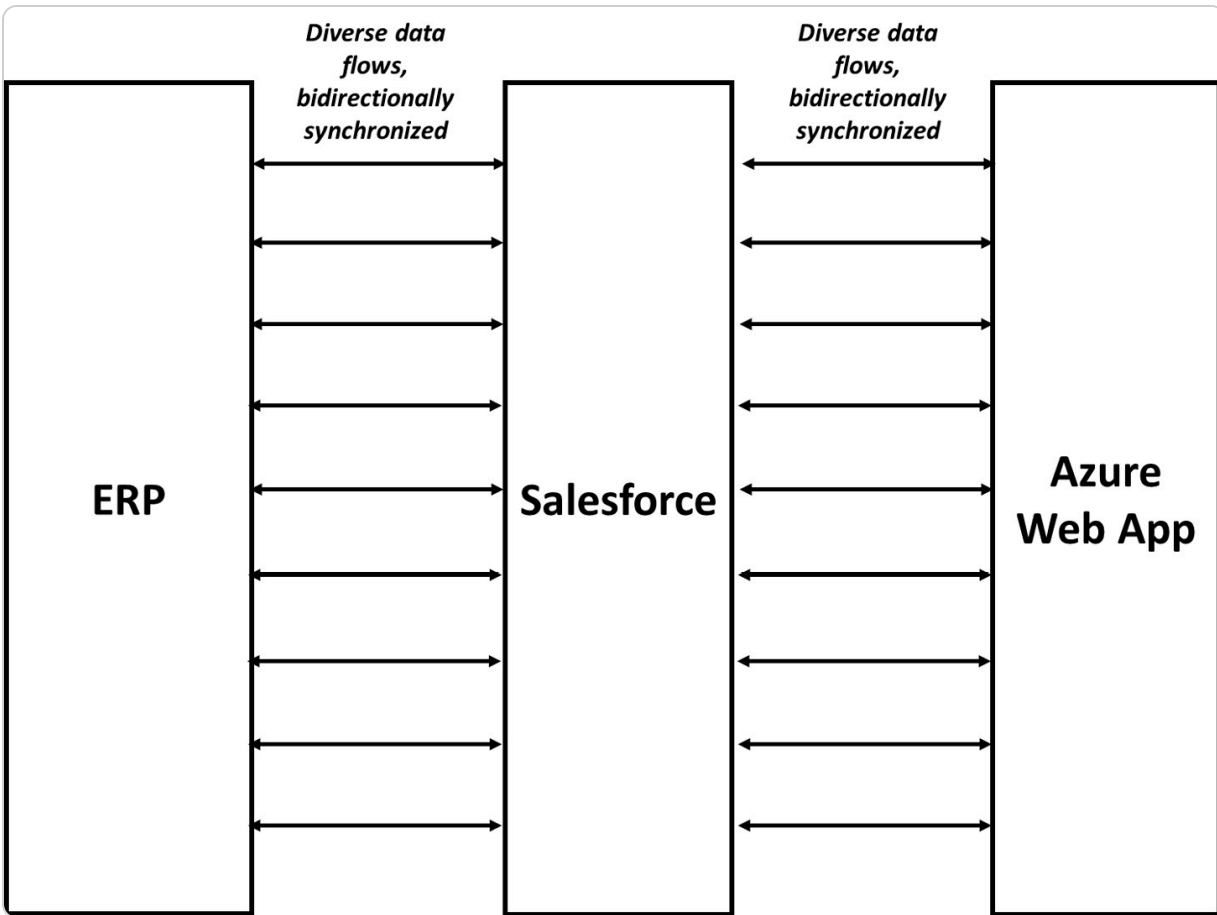


Figure 4.4 – BusyCo synchronization scenario

No problems are found during the acceptance testing phase, but Oleg is not particularly reassured. The testing scenarios do not begin to cover all of the things that can go wrong once the synchronization functions are out in the wild. The business users, however, love the flexibility of being able to change anything in whatever system context they happen to find themselves in.

After the go-live, things start to crop up. No showstoppers, but a number of annoying errors begin to emerge. One day, the link between the website and the other systems is down for a few hours and afterward a number of records fail to synchronize correctly. What's worse is that a few days later, it is discovered that some address updates were never synchronized to the ERP, so some customer orders were shipped to the wrong addresses.

There is also a recurring problem with quotes that seem to go back and forth between statuses in Salesforce and the ERP. The cause seems to be a batch job running in the ERP doing some routine maintenance. As the job can't be turned off, the only solution in the short term is a manual workaround where the support team takes a daily task to reclassify quotes with a wrong status.

Moreover, there are some fringe cases around the synchronization of manually inputted quotes and orders. They don't seem to reliably trigger the data synchronization in all cases and therefore some orders hang in Salesforce and never get shipped.

As the errors compound, business leadership starts to pay more attention to voices such as Oleg's that advocate simpler patterns for managing the data between the systems. However, as the system is already live and works at least partially, any refactoring will need to be done without requiring major downtime.

Oleg accepts the challenge of finding a way to refactor the system to use simpler data integration patterns while instituting better data governance. It won't be easy, but with time and effort, he believes BusyCo can succeed.

Problem

In contrast to the last couple of anti-patterns that happened due to a lack of awareness or a failure to act, the Unconstrained Data Synchronization anti-pattern happens because you're trying your best to give the business what it wants. Sometimes, you can have the best intentions and still end up causing a terrible mess.

The problem you'd be trying to solve in this case is how to have all the relevant business data available and editable in different systems at the same time. Users hate switching between contexts. If they are in System A, then they want to be able to edit all the relevant fields in System A. If they are in System B, then they want to be able to edit all the relevant fields in System B. If the fields they want to edit are the same from a master data perspective, say, an address or the name of a customer, that requirement will force you to synchronize data.

From a user experience perspective, this is perfectly rational. Switching between systems and screens in order to update data is a pain, as is not having the information you need available where you need it. Equally, you usually need this information to be available in near-real time across systems to avoid processing errors.

Near-real-time data synchronization can seem like a panacea, and it would be if it weren't so fiendishly complicated from a technical point of view.

Proposed solution

The solution proposed by Unconstrained Data Synchronization is to bidirectionally synchronize all relevant data points between all relevant systems in something near real time. As noted, this can be considered somewhat of a silver bullet and there are plenty of vendors willing to capitalize on this by promising you tools that will make synchronization easy.

The problem is that there are many failure modes in synchronization. These failures compound that the more data synchronization is used, no tool will reduce away the fundamental complexity. Small differences in processing on different systems or even within the same system can create unforeseen results, as can even minor outages or errors on some of the platforms involved. These errors compound the more systems and objects you are synchronizing.

Unless you can confidently say you have a complete grasp of all the ways your systems and processes can fail and you are certain you have full control of all your operational processes, then Unconstrained Data Synchronization will cause you grief.

Results

Data synchronization tends to work just fine in small-scale scenarios. Synchronizing a specific type of data between two systems is rarely a problem, and neither is synchronizing a relatively large amount of data unidirectionally.

The problems come when you have some combination of multiple systems, multiple objects, and bidirectional connectivity. Here, you will start to see scenarios such as those in our example where small errors compound over time to create a major problem.

Typically, what you'll get is a continuous stream of small-scale remediation activities that need to be handled by a technical support team coupled with a number of workarounds on the business level to avoid known synchronization issues. The more complex your scenario, the more of these kinds of issues you will have.

Occasionally, this anti-pattern can lead to outright disasters, such as when a synchronization link fails silently for an extended period of time, leading to data between systems becoming catastrophically out of sync and business errors proliferating without anyone knowing until the complaints start coming.

Mostly, however, you just see a trickle of small issues that never seem to stop.

Better solutions

You might be expecting us to say something to the effect that you should generally avoid data synchronization. We won't. That would be naïve and unnecessary.

What we are going to say is that you should be cautious about the kinds of data synchronization you do, particularly in the following cases:

- Synchronizing a data point bidirectionally between two systems is fine. Don't, however, add a third.
- If you need to synchronize to more than one system, make sure that you are doing it unidirectionally and that you have defined who owns the master data.
- Middleware and synchronization tools can help with implementation, but they do not solve the fundamental problem.

- Multiple synchronizations in the same org are fine as long as they are non-overlapping; be cautious about having multiple synchronizations running for the same object.
- Synchronizations that need to run in near-real time are much more complex than those that can be processed in a batch manner.

We will now proceed to cover some AI-specific anti-patterns that occur frequently when organizations start adopting AI.

Too much AI is not always a good thing

This section covers two AI-related anti-patterns that describe what happens when you don't think properly about how AI is different and when it makes sense to use it in your solution.

Let's Add AI to It

This is an anti-pattern that occurs when you use AI for its coolness factor rather than for good business reasons.

Example

MediCorp, a mid-sized life sciences manufacturing company, has been grappling with pricing consistency across its product lines. Their sales team, particularly the account managers handling incoming orders, often struggle to apply correct discounts and adhere to the company's pricing strategy. After much deliberation, leadership decides to implement a new pricing solution combining an off-platform pricing engine with Salesforce CPQ.

The project begins with great enthusiasm. The team meticulously maps existing pricing processes, identifies pain points, and designs a solution promising to streamline pricing decisions while maintaining flexibility for complex deals. But six months into the implementation, everything changes.

Sarah, the VP of sales, attends a high-profile tech conference where AI is the hot topic. She sits through a dazzling presentation about AI-powered pricing optimization that promises to boost revenue by 15% and reduce pricing errors to near zero. Excited by the possibilities, she returns to MediCorp determined to incorporate this cutting-edge technology into their ongoing project.

Despite concerns raised by the project manager and lead architect about scope creep and increased complexity, the executive team is won over by Sarah's enthusiasm. They green-light the addition of an off-platform AI pricing tool, convinced it will give them a competitive edge.

The project team now faces the challenge of integrating three separate systems: Salesforce CPQ, the original pricing engine, and the new AI tool. The AI is supposed to provide pricing guidance to CPQ

and feed into the pricing engine's calculations. What was once a relatively straightforward integration has become a complex web of data flows and decision points.

As the go-live date approaches, testing reveals numerous inconsistencies. Sometimes the AI suggests prices that contradict the rules in the pricing engine. Other times, discounts applied in CPQ aren't reflected in the final price. The team works overtime to iron out these issues, but the root cause is often difficult to pinpoint due to the black-box nature of the AI's decision-making process.

Despite the red flags, MediCorp pushes ahead with the launch, partly driven by the sunk cost fallacy and partly by the allure of being an "AI-driven company." The sales team receives extensive training on the new system, with particular emphasis on the AI-powered features that promise to make their jobs easier.

Initially, there is a buzz of excitement. Account managers marvel at the sleek interface and the purportedly intelligent pricing recommendations. However, as weeks pass, problems begin to surface. Sales reps find themselves second-guessing the AI's suggestions, often reverting to their old methods of calculating prices. Customers start questioning the inconsistencies in pricing across similar orders.

The finance team notices that while some deals are priced higher, overall margins are actually decreasing. They suspect that the AI is sometimes suggesting overly aggressive discounts to win deals, without fully accounting for MediCorp's cost structures.

Most worryingly, due to the complexity of the three-way integration and the opaqueness of the AI's decision-making, no one can confidently audit the pricing decisions. When discrepancies are found, it is nearly impossible to determine whether the issue stems from CPQ, the pricing engine, or the AI tool.

Six months after go-live, MediCorp's CFO calls for a full audit of the pricing system. The results are sobering. Not only has the promised 15% revenue boost failed to materialize, but pricing errors have actually increased. The complexity of the system has led to a lack of transparency that makes it difficult for sales reps to explain pricing decisions to customers, eroding trust.

In a difficult but necessary move, MediCorp's leadership decides to roll back the AI component of the system. They go back to the drawing board, planning a reimplementation that would focus on the core functionalities of CPQ and the pricing engine, with a more measured approach to incorporating AI in the future.

The "Let's Add AI to It" initiative, which seemed so promising, ultimately led to increased costs, decreased efficiency, and a temporary loss of pricing integrity. MediCorp learned the hard way that AI, despite its potential, is not a magic solution to be casually added to existing projects without careful consideration and planning.

Problem

The Let's Add AI to It anti-pattern stems from the widespread belief that AI can universally enhance and optimize existing processes or products. This belief is fueled by several factors, including the hype and media attention surrounding AI, misunderstanding of AI capabilities, fear of missing out, and the desire for innovation.

Many decision-makers have a superficial understanding of AI, leading them to overestimate its abilities or applicability to their specific use cases. There's often pressure from stakeholders to adopt AI without fully understanding the implications or necessary groundwork. The proliferation of AI platforms and tools makes it seemingly easy to incorporate AI into existing systems.

The core problem this anti-pattern attempts to address is how to improve existing products, services, or processes. AI is seen as a universal enhancer that can be applied to any situation to yield better results, higher efficiency, or improved user experience. However, this perspective often overlooks the complexity of AI implementation, the need for high-quality data, the potential for increased system complexity, and the importance of clear use cases and expected outcomes.

Proposed solution

The Let's Add AI to It anti-pattern proposes a seemingly straightforward solution to enhance existing systems or processes: integrate AI capabilities wherever possible. This typically involves identifying potential AI integration points, selecting AI tools or platforms, integrating AI components, training the AI models, and launching the AI-enhanced system.

This solution suggests that by following these steps, organizations can reap numerous benefits. It's believed that AI will improve decision-making, increase efficiency, enhance user experience, provide a competitive advantage, reduce costs, and offer scalability. The AI components are expected to improve their performance over time as they process more data, potentially uncovering new insights that humans might miss.

The proposed solution appears attractive because it promises to leverage cutting-edge technology to solve existing problems and create new opportunities. It seems to offer a way to quickly upgrade current systems without the need for a complete overhaul. Moreover, it often comes with the allure of being relatively easy to implement, especially with the rise of AI-as-a-service platforms and pre-trained models.

However, this solution often oversimplifies the challenges of AI implementation and integration. It assumes that AI can be easily plugged into existing systems and immediately provide value, without considering the potential disruptions to current processes or the need for significant changes in data management and decision-making frameworks.

Result

When organizations succumb to the Let's Add AI to It anti-pattern, the results often fall far short of expectations and can lead to a range of negative outcomes. The most immediate impact is usually an increase in overall system complexity. Adding AI components to existing systems typically makes the architecture more complicated and harder to maintain, debug, and update.

Data quality issues often come to the forefront. Many organizations discover too late that their existing data is insufficient, biased, or of poor quality, leading to unreliable AI outputs. Integration challenges are common, with the process of connecting AI components to legacy systems proving far more difficult than anticipated, leading to project delays and budget overruns.

The lack of explainability in many AI models, particularly deep learning ones, can be problematic. When AI operates as a "black box," it becomes difficult to understand or explain its decisions. This can be especially challenging in regulated industries or when transparency is crucial for user trust or legal compliance.

User experience often suffers. If the AI components don't perform as expected or make obvious mistakes, it can lead to frustration and rejection of the system. There's also a risk of overreliance on AI, where users may over-trust AI recommendations, leading to a decline in critical thinking or the ability to handle exceptions.

Ethical and privacy concerns may arise, especially if the AI implementation is hasty and doesn't adequately address these important considerations. This can potentially lead to reputational damage or legal problems.

From a resource perspective, AI projects can become a significant drain, consuming substantial computational and human resources. This can divert attention and funding from other important initiatives. Maintenance of AI systems often proves more challenging and resource-intensive than anticipated, with ongoing needs for monitoring, retraining, and updating.

In terms of performance and reliability, hastily implemented AI solutions may lead to inconsistent results across different scenarios or user groups. They may introduce new security vulnerabilities or fail to scale as expected. In some cases, the complexity introduced by AI can make it difficult to trace the logic behind certain outcomes, leading to a lack of accountability.

Perhaps most disappointingly, many organizations find it difficult to measure the return on investment for their AI initiatives. The complex nature of AI systems, combined with their impact on various aspects of the business, can make it challenging to accurately assess their value.

In essence, the Let's Add AI to It anti-pattern often results in a system that is more complex, less reliable, and harder to manage than its predecessor. Instead of enhancing the existing product or

process, it can introduce new problems and inefficiencies. The promise of easy improvements through AI integration proves to be an illusion, as organizations find themselves grappling with unexpected challenges and disappointing results.

Better solutions

To avoid falling into the Let's Add AI to It anti-pattern, organizations should adopt a more measured and strategic approach to AI implementation. The key is to start with a clear problem statement. Before considering AI, it's crucial to clearly define the problem you're trying to solve. This helps ensure that AI is genuinely the best solution, not just the trendiest one.

Conducting thorough due diligence is essential. Organizations should research and understand the capabilities and limitations of AI in their specific domain. This might involve consulting with experts and examining case studies of similar implementations. It's also important to assess data readiness. The quality, quantity, and relevance of available data should be evaluated, and investments in data infrastructure and governance may be necessary before implementing AI.

Starting small and iterating is a prudent approach. Beginning with a pilot project or proof of concept allows organizations to learn and adjust with minimal risk before scaling up. Throughout this process, it's crucial to focus on user needs. Conducting user research helps ensure that AI features truly enhance the user experience and solve real pain points.

Organizations should prioritize explainability in their AI implementations. Choosing AI models and approaches that offer transparency in decision-making is especially important in critical applications. This goes hand in hand with considering ethical implications. Developing an AI ethics framework can guide implementation and ensure responsible use of the technology.

Investing in AI literacy across the organization is valuable. Educating stakeholders, including executives and end users, about AI's capabilities, limitations, and implications can help manage expectations and facilitate smoother adoption.

It's important to plan for the long term. Organizations should consider the ongoing maintenance, updating, and scaling of AI systems in their planning and budgeting. This includes establishing clear metrics to assess the AI's performance and impact over time.

Sometimes, the best solution is to enhance existing processes before adding AI. Simple improvements to current systems and workflows can sometimes yield better results than complex AI solutions. When AI is implemented, maintaining human oversight is crucial. Systems should be designed to augment human decision-making rather than completely replacing it, especially in critical areas.

AI Architecture Is Like Normal Architecture

This is an anti-pattern that occurs when Salesforce architects treat an AI project like any other project.

Example

CloudForce Solutions, a successful Salesforce **Independent Software Vendor (ISV)**, had been developing apps for Salesforce AppExchange for years. Their portfolio includes various tools for sales analytics, customer management, and marketing automation. When Salesforce announces new AI capabilities, CloudForce sees an opportunity to create an AI-powered lead scoring and prioritization app.

The project, dubbed “LeadGenius,” is led by Sarah, CloudForce’s senior Salesforce architect. Sarah and her team have a track record of creating robust, successful Salesforce apps using traditional development approaches. They decide to apply the same methodologies to LeadGenius, viewing the AI components as just another set of features to implement.

They begin by gathering requirements from potential customers and creating detailed specifications. Sarah designs a familiar architecture: Apex classes for business logic, Lightning components for the user interface, and custom objects for data storage. The team plans to use Salesforce Einstein features for the AI functionality, treating them as black-box components that could be easily integrated into their existing patterns.

As development progresses, the team encounters its first challenge. Einstein Prediction Builder, which they are using to score leads, isn’t performing as consistently as they anticipated. Sometimes, it assigns unexpectedly low scores to leads that sales representatives know are promising. Sarah’s team treats this like a typical bug, attempting to add more fields to the prediction model and creating complex Apex logic to “correct” the AI’s output.

The next hurdle comes when they try to integrate LeadGenius with various **Customer Relationship Management (CRM)** systems for testing. The AI model struggles with the diverse, often messy data from different organizations, producing inconsistent results. The team’s solution is to implement more data cleaning and transformation steps in their Apex code, essentially trying to force the data to fit their predefined structure.

As the project nears its launch date, CloudForce runs into serious issues with the app’s lead prioritization logic. In complex sales scenarios, the AI sometimes makes recommendations that seem counterintuitive to experienced sales managers. The team’s response is to add more **if-then** statements and decision trees in their Apex code, attempting to account for every possible scenario.

Despite these challenges, CloudForce pushes forward with the AppExchange launch, confident that their thorough architecture and rigorous testing would ensure success. Initially, LeadGenius receives positive feedback from early adopters who are excited about the prospect of AI-powered lead scoring.

However, as weeks pass, problems begin to surface. The app's performance degrades over time, struggling with new types of leads and industries that hadn't been in the initial training data. The team finds it difficult to diagnose issues, as the AI's decision-making process isn't as transparent as their traditional Apex code.

Customers report frustration with the app's inability to adapt to their specific sales processes or handle nuanced lead qualification criteria. Some notice that LeadGenius is sometimes giving higher scores to leads from larger companies, regardless of actual fit or interest, reflecting biases present in the training data that the team hadn't anticipated or accounted for.

Most alarmingly, there are instances where the app makes ethically questionable suggestions, such as prioritizing leads based on factors that could be seen as discriminatory.

Six months after the launch, CloudForce faces a growing number of customer complaints and cancellations. They request a comprehensive review of LeadGenius. The results are disappointing. Sales teams using the app aren't seeing the promised increase in conversion rates and, in some cases, are missing out on valuable opportunities due to misclassified leads.

In a difficult meeting with key customers, Sarah and her team have to admit that their traditional approach to Salesforce app development has failed to address the unique challenges of AI systems. They realize too late that AI-powered apps require a fundamentally different mindset, focusing on data quality, model behavior, ethical considerations, and continuous learning—aspects they had overlooked in their conventional Salesforce development approach.

CloudForce decides to pause new sales of LeadGenius and initiate a complete redesign of the app. They are forced to go back to the drawing board, this time with a much better understanding of the unique requirements of AI architecture within the Salesforce ecosystem.

Problem

The AI Architecture Is Like Normal Architecture anti-pattern stems from the misconception that AI-powered Salesforce solutions can be designed and implemented using the same principles and methodologies as traditional Salesforce development. This belief is often rooted in familiarity with Salesforce development practices and an underestimation of AI complexity. Many teams feel more comfortable applying techniques they've used successfully in past Salesforce projects, without fully grasping the fundamental differences between deterministic Apex code and probabilistic AI systems such as Einstein.

There's also often pressure within organizations to rapidly implement AI features to stay competitive and maximize their Salesforce investment. This urgency, combined with a tendency to view Einstein

features as simple plug-and-play components, can lead to an oversimplification of AI capabilities within the Salesforce ecosystem.

The core problem this anti-pattern attempts to address is how to efficiently integrate AI capabilities into Salesforce implementations. The assumption is that by treating AI architecture like normal Salesforce architecture, teams can leverage their existing skills and processes to deliver AI solutions effectively on the platform. However, this approach fails to account for the distinct characteristics of AI systems within Salesforce, such as their probabilistic nature, dependence on data quality and quantity, the potential for autonomous behavior, the opacity of decision-making processes, and ethical implications.

Result

When organizations succumb to the AI Architecture Is Like Normal Architecture anti-pattern in their Salesforce implementations, the results often fall far short of expectations and can lead to a range of negative outcomes.

The most immediate impact is usually poor AI performance. The AI models may not perform as expected in real-world scenarios, leading to inaccurate predictions or recommendations. This is often due to insufficient consideration of data quality and quantity, or a lack of understanding of the model's limitations within the Salesforce ecosystem.

Adaptability also becomes a significant issue. The AI components may fail to adapt to new data or changing business conditions, as the traditional architecture doesn't account for the need for continuous learning and model updates in Salesforce. As the Salesforce org grows and data volumes increase, the AI solutions may struggle to handle diverse datasets and use cases, leading to performance degradation.

Ethical concerns can arise when proper consideration isn't given to AI ethics. Salesforce AI implementations may make biased or unfair decisions, potentially exposing the organization to reputational or legal risks. This is compounded by the difficulty in troubleshooting AI issues. When problems occur, Salesforce administrators and developers may struggle to diagnose the root cause due to the opacity of AI decision-making processes, which aren't as transparent as traditional Salesforce configurations or Apex code.

User satisfaction often suffers as a result. Salesforce users may become frustrated with the AI features' inability to handle nuanced scenarios or adapt to specific business processes, leading to poor adoption rates. Maintenance becomes increasingly challenging as the Salesforce org evolves, with teams finding it difficult to update AI components without unintended consequences.

Resource drain is another common outcome. Attempts to “fix” AI issues using traditional Salesforce customization approaches can lead to increasingly complex configurations and code, consuming more compute resources and potentially hitting Salesforce governor limits.

Perhaps most disappointing is the missed opportunity for innovation. By treating AI as just another feature, organizations may fail to leverage the full potential of Salesforce’s AI capabilities, missing out on chances to truly transform their business processes.

Integration difficulties often arise, with AI solutions struggling to work smoothly with other Salesforce features or third-party tools. This is typically due to the unique requirements of AI systems not being properly addressed in the initial architecture.

In essence, the AI Architecture Is Like Normal Architecture anti-pattern often results in Salesforce AI implementations that fail to deliver on their promise. Instead of enhancing business processes and providing valuable insights, they can introduce new problems and inefficiencies. The initial allure of easy AI integration proves to be an illusion, as organizations find themselves grappling with unexpected challenges and disappointing AI performance within their Salesforce environment.

Better solutions

To avoid falling into the AI Architecture Is Like Normal Architecture anti-pattern, organizations should adopt a more nuanced and AI-aware approach to Salesforce development and implementation.

First and foremost, investing in AI education is crucial. The entire Salesforce team, including admins, developers, and architects, should have a solid understanding of AI concepts, particularly in the context of Salesforce Einstein’s capabilities. This knowledge foundation is essential for making informed decisions about AI integration.

Embracing a data-centric design approach is key. Organizations should start with a thorough analysis of available data within their Salesforce org. AI solutions should be designed around data quality, quantity, and relevance for AI models, taking into account how Salesforce data structures and relationships impact AI performance.

Planning for model monitoring and retraining is another critical aspect. Mechanisms should be implemented to track model performance over time and retrain models as needed. This might involve creating custom objects in Salesforce to store model metadata and performance metrics.

Adopting an iterative development approach allows for the continuous refinement of AI models based on real-world performance and feedback within the Salesforce environment. This agile methodology is better suited to the evolving nature of AI systems than traditional waterfall approaches.

Robust data preprocessing is essential. Organizations should develop Apex classes or leverage Salesforce Flow specifically for cleaning and preparing data for AI models, considering the unique requirements of machine learning algorithms.

Designing for explainability is crucial for user trust and compliance. Where possible, Einstein features that provide insight into model decisions should be used. For custom models, additional logging or visualization features can help users understand AI outputs within their Salesforce workflows.

Ethical considerations should be prioritized. Guidelines for ethical AI use within Salesforce should be developed, including mechanisms to detect and mitigate bias. This might involve creating custom validation rules or Apex triggers to flag potentially problematic decisions.

Organizations should make full use of Salesforce's AI infrastructure, such as Einstein Platform Services, which is designed to integrate smoothly with other Salesforce features and respect platform governance. However, it's also important to plan for graceful degradation, ensuring that the Salesforce architecture can fall back to rule-based decision-making if AI components fail or produce low-confidence results.

Comprehensive testing strategies should be implemented, going beyond traditional unit tests to include techniques such as A/B testing of model performance and scenario-based testing with diverse datasets, all within the Salesforce testing framework.

User education is vital for successful AI adoption. In-app guidance and external documentation should be developed to help Salesforce users understand the capabilities and limitations of AI features, setting realistic expectations.

Finally, maintaining human oversight is crucial. Salesforce workflows should be designed to allow for human review and override of AI decisions where appropriate, especially for high-stakes scenarios.

By following these best practices, organizations can create AI-powered Salesforce solutions that truly leverage the potential of AI while addressing the unique challenges of AI systems within the Salesforce ecosystem. This approach helps ensure that AI truly adds value to Salesforce users, rather than just adding complexity or following a trend.

We have now covered anti-patterns and are ready to move on to looking at our key takeaways for the data domain.

Knowing the takeaways

In this section, we will abstract a bit from the specific patterns and instead try to pull out the wider learning points you can use in your day-to-day work as a Salesforce architect or in preparing for the

CTA Review Board.

When architecting Salesforce solutions, you should be mindful of the following:

- Salesforce has a unique approach to data modeling that does not easily map to traditional methods, such as relational database design.
- Trying to shoehorn Salesforce into such approaches will only end in tragedy.
- Instead, you should follow the good practices established by Salesforce themselves and exemplified by the data model that comes with the platform out of the box.
- This includes modeling using broad business objects rather than a normalized model, relying heavily on standard objects, and limiting the amount of custom work you put into the data layer.
- Failing to coordinate activities between teams can cause serious issues at the data model layer and really reduce your ability to get things done in future projects.
- Therefore, always promote appropriate coordination or governance forums that ensure teams understand what each one is doing.
- If possible, add to this coordination forum other data governance mechanisms such as data ownership, standards and guidance for data modeling, and approval processes for decisions that have serious data-layer consequences, such as adding a new custom object.
- It is a cardinal sin to know that you need to scale to a certain level and not do the necessary calculations that tell you whether that scale should be actively mitigated in your project or not.
- If you decide not to take active steps toward mitigating potential consequences of scale, you need to do so with your eyes fully open and with a plan to refactor and redesign later if needed.
- In general, failing to take scale into account can be appropriate for start-ups and experimental projects, but it is rarely appropriate in an enterprise systems context.
- Business users often love the idea of data synchronization because it enables a much superior user experience. However, they rarely appreciate the technical complexities involved.
- Small-scale data synchronization is often a good way to solve a problem, but beware of scaling them too much in both size and complexity.
- Avoid synchronizing the same data points bidirectionally between more than two systems and be cautious about synchronizing large amounts of objects and fields unless it's for strictly one-way use cases, such as data going to a data warehouse.
- Resist the urge to add AI features without a clear business case and understanding of data requirements.
- Treat AI components as fundamentally different from traditional Salesforce customizations, requiring unique architectural considerations.
- Prioritize data quality and quantity when designing AI-powered solutions in Salesforce.
- Implement robust monitoring and retraining mechanisms for AI models within your Salesforce org.
- Design Salesforce processes that allow for graceful degradation if AI components fail or produce low-confidence results.

In preparing for the CTA review board, you should be mindful of the following:

- Understand and follow good modeling practices when creating your data model for your scenario.
- That means modeling things the “Salesforce way” and not taking too many liberties that you’ll have to explain and defend.

- Use standard objects first, and only if you are entirely convinced that no standard object will suffice should you turn to using a custom object in your solution.
- Be clear about what objects are used for what purpose and avoid having the same data in multiple places.
- Incorporate data ownership for all non-detail objects at the role level in your data model overview.
- You may want to put in a couple of words about good data governance when describing your governance model.
- Do the math on data volumes for the key objects in the scenario and clearly identify any that may have LDV requirements.
- Often, these are junction objects between key entities in the system, but there are exceptions to this rule.
- Create a clear and well-thought-out LDV mitigation approach for the objects in question. Simply reciting a list of common LDV mitigation techniques is not sufficient in and of itself.
- Be careful when specifying data flows between systems. You need to be realistic in what data you propose to move back and forth and how.
- Prefer one-directional synchronizations and integrations to bidirectional ones.
- Prefer data virtualization to data synchronization whenever possible in the scenario at hand.
- Demonstrate an understanding of the fundamental differences between traditional Salesforce architecture and AI-driven solutions.
- Articulate strategies for ensuring data quality and relevance for AI models within Salesforce orgs of varying sizes and complexities.
- Explain approaches to monitoring and maintaining AI model performance over time in a Salesforce context.
- Discuss methods for integrating AI components with existing Salesforce features and third-party systems.
- Outline architectural considerations for scaling AI solutions as Salesforce org data and the user base grow.

We have now covered the material for this chapter and are ready to proceed with the solution architecture domain. First, however, we will summarize our learning.

Summary

In this chapter, we have covered a lot of things that can go wrong in the data domain if you aren't careful. The things that go wrong in the data layer tend to affect everything in the layers above, so if you fall into any of these anti-patterns, consequences will be serious for all aspects of your future configuration, integration, and development work.

It is, therefore, especially important to learn the lessons of good structure, good practice, and sound governance when it comes to the data domain. Not that they are unimportant elsewhere, but if you get your data layer wrong, then it is very hard to get everything else right.

This applies both in real life and in the CTA exam. As many aspiring or current CTAs will tell you, if you get any element of your data model for a scenario substantially wrong, that tends to ripple through the other domains, making it very hard to pass the overall scenario.

Having now covered the data domain, we will proceed to solution architecture—an area so rife with anti-patterns that we'll have to be quite selective in which we choose to include.

Part 2: Solution Anti-Patterns

This part will teach you how to identify and mitigate anti-patterns in the functional domain of solution architecture as well as integration architecture.

This part has the following chapters:

- [Chapter 5](#), *Unpicking Solution Architecture Troubles*
- [Chapter 6](#), *Keeping Integration Straight*

5

Unpicking Solution Architecture Troubles

This comprehensive chapter will cover anti-patterns related to your solution architecture. We'll start by looking at anti-patterns related to your choice of solutions. Then, we'll examine things that can go wrong when you do functional design. Third, we'll explore some particular anti-patterns that affect customizations, first at the conceptual level and then at the code level. We'll end this chapter by summarizing the key takeaways.

In this chapter, we're going to cover the following main topics:

- How to avoid picking the wrong solutioning approach by basing it on bad research and analysis
- Why your assumptions can lead you astray and make your architecture and design go off the rails
- What can go wrong when you apply good features in a bad way and how to use good governance to overcome this problem
- How classic code anti-patterns also find their way into Salesforce implementations and how you can design your code with better structure and patterns

After completing this chapter, you'll have a good idea of how to avoid common anti-patterns when designing solutions on Salesforce and what steps you can take to use the right solution for the right problem.

Picking the wrong thing

This section will explore how seemingly small decisions in solution selection can have far-reaching consequences for your Salesforce implementation. We'll look at two anti-patterns related to picking the wrong solution based on considerations that aren't technically relevant. We'll start by examining what goes wrong when you ignore the wider ecosystem.

Ignoring the Ecosystem

Willfully ignoring third-party options in favor of in-house solutions can become an anti-pattern.

Example

SafeCo is a major insurance company that provides a variety of policies to the mass consumer market. It has a large customer service organization for which it's in the last stages of rolling out Salesforce Service Cloud to achieve a 360-degree view of customer inquiries.

Royce is a manager within SafeCo's IT department and has been tangentially related to the Service Cloud project. However, now, he has been tasked with leading a team that has been asked to find good answers to several new generic requirements that have come up during the project but were left out of the initial scope, as outlined here:

- First, SafeCo, on learning more about Salesforce, has decided that it needs a backup solution to be in place as it doesn't want to rely on the rather limited out-of-the-box offerings
- Second, while policy documents are generated by a **line-of-business (LOB)** system, stakeholders have identified a need for printing several standard letters for customers, which should be done directly from the Salesforce UI
- Finally, within the IT department, several people have asked whether they could have a project management tool on Salesforce as they think it would be a good platform for collaborating with business users

Royce starts by addressing the most pressing issue of backup. He's generally aware that there are several solutions to this problem on AppExchange, but after having a brief look, he finds them too expensive and complicated. Besides, SafeCo already has a backup infrastructure in place for other systems. Royce gets a developer in the IT department to write a script to copy Salesforce objects one to one to a **Structured Query Language (SQL)** database by exporting them to CSV files and then using a standard batch process for the import. From there, the data can be backed up using the standard SafeCo backup solution.

Document generation is another area where Royce is aware that several third-party solutions exist. However, there's also some internal document generation capability in the corporate **document management system (DMS)**, and that's where he starts looking. While the solution isn't perfect, the DMS team can provide the capability by accepting the request for document generation via a web service, which puts the request in a queue that's regularly emptied by a batch job that generates documents and puts them in a file share. From there, another job picks them up and uploads them back into Salesforce. It's a little slow and clunky, but it should be OK.

For the project management module, Royce has some good luck. Over a weekend, one of his team puts together a small application that seems to demonstrate a lot of the functionality SafeCo will need. He reckons that in a few weeks, he can develop something that will be good enough to roll out to the business. Royce takes the win and decides to go with it.

Royce now has his solutions, and he pushes his team hard to get them ready on time for the global service rollout. However, while nothing catastrophically fails, there are quite a few problems to contend with. The backup script works well, but when they test full and partial restores in both new and partial environments, they run into a massive and unexpected level of complexity. They abandon the **business continuity** (**BC**) test and make a plan to develop some more elaborate restore scripts at a later date.

As expected, there are some complaints about the slow speed of document generation, but the worst problem turns out to be template management, which Royce hadn't considered in detail during his analysis. It turns out that templates change frequently, and with the new solution, all template changes have to be done by someone in the DMS team. That makes the customer service team quite irritated.

The project management application is rolled out without major problems, but the response is underwhelming. There are several bugs and misunderstandings, and in general, the team was expecting to see several features that haven't been included. As a result, adoption across teams is lackluster.

Royce takes a breath. While everything sort of works, there are a lot of issues. Royce wonders whether he might have missed something in the process, but it's time to close the project and move on.

Problem

The problem that was addressed in the *Ignoring the ecosystem* section is how to handle common capabilities related to the main requirements of implementation, but not strictly part of the core requirements. These include common backup and restore, document management and generation, automating the deployment pipeline, dispatching logs to a central log server, and a range of similar requirements.

For this type of requirement (and many others to boot), a large number of third-party solutions exist on AppExchange that solve this particular need generically, as depicted in the following screenshot. In addition, many Salesforce consulting partners have standard approaches that involve using accelerators or internal assets, which can also help with this sort of requirement:

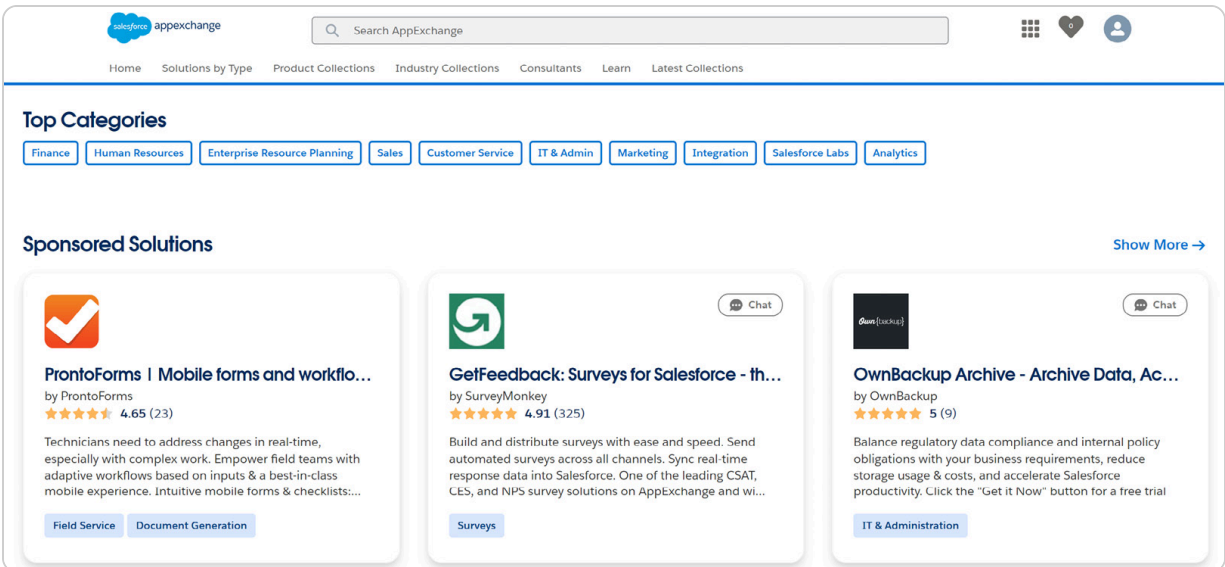


Figure 5.1 – AppExchange’s Top Categories section

With this anti-pattern, you can ignore these options to the point of not even looking at them seriously.

Proposed solution

When you ignore the ecosystem, you actively disregard third-party options for generic requirements and instead move ahead with bespoke or in-house solutions to common capabilities. While there can be legitimate reasons for a bespoke answer to a generic requirement – for instance, if you have a highly developed infrastructure for some capability that you want to leverage across all enterprise systems – not making a reasoned comparison with the available commercial options is an anti-pattern and goes against the core philosophy of Salesforce.

That’s to say, an anti-pattern arises when you don’t make a considered build-versus-buy decision for important but generic capabilities and instead reflexively embrace an in-house approach, whether because of the “not invented here” syndrome or because it’s cognitively easier to handle for your organization’s key stakeholders. This also often happens when users know other platforms and want you to rebuild their functionality on Salesforce.

Again, that isn’t to say that a build approach is never valid for these capabilities. But in my experience, you’re highly likely to underestimate the complexity and ongoing costs related to these kinds of builds and will probably end up with less functionality for more money than your business case said.

Results

The results of ignoring the ecosystem are generally quite predictable. However, at the detailed level, they’ll vary by the capability under consideration. At a higher level of abstraction, however, they tend

to be as follows:

- A less capable solution that meets fewer requirements than what you could have obtained for a similar cost in the ecosystem
- Fewer new features are added to the capability over time, often at higher maintenance costs than the run-rate cost of the third-party solution
- The need to maintain an additional skillset and knowledge base for the development and maintenance of the in-house capability
- A higher degree of coupling between the new capability and the existing system than if you had used a third-party solution
- The cost picture can be complicated, but building a generic capability yourself rarely results in a significant saving

All in all, the results are rarely catastrophic, but commonly, the resulting solutions are worse than third-party equivalents and achieved with greater pain and risk.

Better solutions

There's just one general piece of advice to keep in mind to avoid this anti-pattern, although implementing it can be a challenge, and that's to make a considered and well-reasoned build-versus-buy decision whenever you encounter relatively commonplace capability requests.

A more thorough evaluation of AppExchange solutions is crucial for organizations such as SafeCo. By carefully assessing various options, companies can uncover tools that not only align with their unique requirements but also offer additional features and ongoing support. This proactive approach can lead to significant time and resource savings in the long run, ensuring that the selected solution fosters business growth and operational efficiency effectively.

To do this, you should do the following:

- Carefully explore ecosystem alternatives, both on AppExchange and things that are available directly from your Salesforce partners.
- Engage actively with vendors; often, they're more willing to consider your unique concerns than you might think.
- Outline the potential options in enough detail to make a reasoned decision, remembering to include architectural, operational, and commercial considerations.
- Count the **total cost of ownership (TCO)** when outlining the internal options. Often, operational, maintenance, and further development costs are undercounted in these types of comparisons.
- Drive a hard commercial bargain with third-party vendors – you can often get quite significant discounts. Conduct regular **ecosystem audits** to stay informed about new AppExchange offerings that could replace or enhance custom solutions, potentially reducing technical debt. Additionally, regularly review existing solutions to identify opportunities for optimization and to be able to integrate new tools that align with business needs.

Having looked at the consequences of ignoring the ecosystem, let's consider the consequences of fitting your solution to your licenses, rather than your licenses to your solution.

License-Based Solutioning

Creating a solution based on a license set that you can afford rather than the license set you really need without making the necessary compromises.

Example

Alexis is the lead architect for a major Salesforce partner working directly with Salesforce on a deal for the Department of Social Welfare in the mid-sized European country Ruritania. The department services millions of residents and has a staff of several thousand caseworkers spread approximately 50/50 between a central HQ and several hundred local offices. They're adopting Salesforce as a case management platform for a new high-profile initiative to help families hurt by the increasing cost of living.

As part of the deal, the Salesforce **account executive** (**AE**) pushed very hard for a licensing model that gave the HQ users a full Salesforce license, but only a Customer Community Plus license for the users in the local offices. He argued that as the local officers were technically members of the local municipalities, this would be acceptable under the Salesforce licensing terms and would give a more acceptable license cost for the client.

The implication, as Alexis is quick to point out, is that caseworkers will be working in two different interfaces, one internal and one external, based on a community. However, the AE argues that the differences are minor and that most of the work that's been done for the internal interface can be replicated directly in the community.

While that's generally true, as work continues, many subtle differences start to have an impact. There are differences in what Customer Community Plus users can have access to within the system and also differences in which things can be exposed on a Lightning page versus a Community page and how this can be done.

This situation forces increasing amounts of functionality into custom code, especially into a collection of small **Lightning Web Components** (**LWC**) to accommodate the two interfaces. Furthermore, all test cases need to be written and executed twice, once for each interface, adding considerable overhead.

When it comes to rolling out the application, they also need to create two versions of the documentation, the training materials, and the actual user training. Alexis also flags that her company will need to increase the application maintenance fee as there's a lot more customization to maintain and many future changes will need to be done in two versions.

The users are generally happy with the new application, but a lot of communication errors and mistakes are caused by the differences in the interface between the local offices and the HQ. The financial director within the Department of Social Welfare is also unhappy with the increase in

implementation cost and application maintenance he has seen, something he doesn't feel is fully compensated by the lower license cost.

Problem

The problem that's addressed by license-based solutioning is an old chestnut. How do we deliver all the required functionality within the cost envelope we've been given? If you've never had to struggle with fitting user demands to a budget that's just too small for your level of ambition, I want to know where you work so I can apply.

It's nearly universally true that stakeholders would like more features than they have money to buy. One way to try to get around this is to try to carefully optimize the licenses you buy to get the best value for money. This is called license optimization and is an essential skill set on the Salesforce platform. An overview of the Salesforce license model is shown in the following diagram:

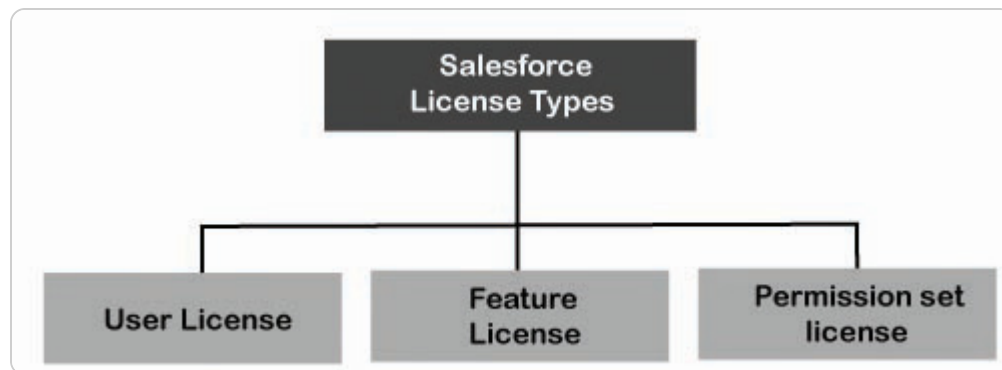


Figure 5.2 – Salesforce license model

License-based solutioning turns this commendable practice into an anti-pattern by specifying which licenses can be afforded and then shoehorning the rest of the solution to make it fit.

Proposed solution

License-based solutioning proposes to make an affordable license mix work for a given set of requirements by using workarounds and custom functionality to fill the gap between the required license type and the license type you can afford. This means that you engage in this anti-pattern when there's a standard license type that would meet the user requirements, but you instead choose to go with a bespoke or partially bespoke solution that somehow bridges the functionality that's available in the license you have and the license you want.

Here are a few common examples:

- Using community licenses for internal users – although just external enough for there not to be a blatant breach of terms – who ideally would use the internal UI if it were not too expensive

- Using shadow objects to replicate standard objects that aren't covered by your license type – for instance, having a shadow case object bi-directionally replicating the information from Case to allow internal caseworkers to have a cheaper platform license
- Replicating key features from industry clouds – for example, Action Plans – because of an unwillingness to adopt additional licenses
- License sharing between different users that have the same organizational role, circumventing terms and security
- Using cheaper solutions for some features – for example, using a WordPress site instead of a community to save license costs and then syncing the data between the site and Salesforce

While there can be a commercial rationale for these practices, from a technical perspective, they're undoubtedly maladaptive.

Results

The results can vary depending on the specific use case, but often, they include things such as the following:

- Increased need for custom work to compensate for the lack of license features
- Overall increased complexity of the solution to make the different parts fit together reasonably
- More maintenance going forward as a result of the increased custom work and increased overall complexity
- Reduction in value from future feature upgrades as you'll need to consider the added customization work
- Reduced security from license sharing or reusing a single license for many functions

Increased technical debt may arise from the implementation of workarounds and custom solutions designed to circumvent license limitations. These practices can lead to significant long-term maintenance costs as they often require additional resources to manage and support, further complicating the technical landscape.

This can leave you in a gray area vis-à-vis Salesforce as the licensing agreement isn't easy for laypeople to interpret and Salesforce isn't transparent about how it enforces the terms in cases where there's considerable ambiguity.

That being said, from a TCO perspective, it's sometimes true that you might be better off not buying the licenses and doing custom work instead. Salesforce licenses are expensive, and if you only need a little bit of what's included in a feature license, you may be able to rebuild it for a lower price.

However, my advice would be to avoid this if at all possible and instead seek an alternative option, as we'll discuss in the next section.

Better solutions

In general, you're better off trying to negotiate a better deal with Salesforce, looking at third-party options for the specific functionality you need, or reengineering the solution to avoid the specific

features you need either by changing the scope or redefining who needs which feature license.

Let's look at these in turn:

- Your first port of call, when faced with unaffordable licenses, should be to push your Salesforce AE harder. They often have considerable leeway to offer discounts, especially if we're talking about a new license type or you're willing to commit to a longer period for the deal. It also helps to push shortly before the end of Salesforce's financial quarters.
- Second, you should look at third-party options. You can look at AppExchange to see if a less expensive product might meet your needs, or you can ask your implementation partners if they have assets you might be able to leverage.
- Third, you can go back to the business and ask them to reprioritize features so that you can either cut something else to make space for the licensing cost or remove features that require the extra license.
- Fourth, you can go back and do another round of license optimization where you go through the details of the proposed licensing and see where you can cut specific licenses for specific users. Often, some licenses are must-haves for some users, but only nice to have for others.

If you can't find a good solution and have to go ahead with this anti-pattern, be sure to do so with an awareness of the real cost and risk involved. That being said, let's proceed to our next anti-pattern: *assumption-driven customization*.

Applying the thing wrong

In this section, we'll consider two anti-patterns where we get unsatisfactory solutions based on a failure of the assumptions we bring to the table. These assumptions can be either about user needs or the capabilities of our technology – both can lead to trouble.

Assumption-Driven Customization

Letting the assumptions of a small, unrepresentative group stand in for the needs of the wider user base.

Example

Helene is the product owner for a new initiative to roll out Health Cloud within LifeCo, a major provider of medical devices for monitoring certain chronic diseases. In addition to the Health Cloud rollout, LifeCo is also developing a custom care application that will be used by **healthcare professionals (HCPs)**, principally nurses, working in clinics associated with LifeCo's care programs.

Helene was a nurse for several years before entering the IT field after working as a **subject matter expert (SME)** on a few major IT products. Therefore, she feels that she has a strong grasp of the requirements and also has several friends, still working as nurses, who she uses as sounding boards.

After some internal discussions, Helene has decided to deliver the application as an application optimized for the kinds of tablet computers she knows are common within the clinics. This will allow HCPs to note down information relevant to the care programs as they interact with patients.

The team develops an attractive and highly functional tablet application that meets all of the key requirements Helene has identified. In initial user testing with a friendly audience, the application receives great praise, and everyone is looking forward to the production rollout.

However, as they start to plan the rollout, many clinics push back and refuse to implement the new application. After digging into the issue, it turns out that staff in many clinics, while they have tablet devices available, prefer not to use them directly with patients as they feel it hampers the interaction.

For other HCPs, the idea of capturing the data directly while with the patient goes against a longstanding practice of collecting information in paper notes during the day and then typing it into the system after hours, using a PC with an interface highly optimized for rapid data entry. The HCPs feel that switching to the new system would force them to spend more time on data entry and decrease the impact of patient contact.

Helene has a long list of counterarguments to address the concerns raised by the HCPs and strongly feels that with a modicum of training and process change, the end users will come to see how the new application can significantly improve their working practices. However, she's met with the unfortunate fact that none of the clinics refusing to adopt the new application is owned directly by LifeCo, so there's no way to force them to change.

This presents a major issue to LifeCo as this means they have to keep the old system for managing care programs live and that they'll have two separate databases of care information being used at the same

time, causing serious consolidation issues. Helene is pushed to find a quick solution that will satisfy the hold-out clinics.

This misalignment between the solution and actual user needs not only affected adoption rates but also led to significant declines in productivity. Users, struggling to navigate an unsuitable system, resorted to finding workarounds that enabled them to perform their tasks, but these alternative methods introduced inconsistencies and errors in data management. Consequently, the reliance on informal processes diminished data integrity and reliability, which further complicated decision-making and impaired overall organizational effectiveness.

In the end, she has to relent. She sits down with her team to see what can be done to create a version of the application that can be used from a PC. They quickly port the key functionality and release a barebones version to the clinics that have declined to accept the tablet application. While it gets a lukewarm response, at least no one refuses to use it this time.

Problem

The thing with assumption-driven customization is that it's less than the problem it's trying to solve – it could theoretically be anything – and more about the way you're trying to solve it. With assumption-driven customization, you're trying to solve a problem as defined by your product management, product owners, or developers.

However, it turns out that the actual problem you need to solve is something else entirely. In other words, the problem with assumption-driven customization is that you're unwittingly trying to solve the wrong problem.

This is different from just getting the requirements wrong because it can happen with good teams following a good process. All it takes is for the sample that you're using to represent your user base to be in some way systematically biased, as we saw in our example.

Proposed solution

The proposed solution of assumption-driven customization is to let a group of experts stand in for the wider user base to facilitate rapid delivery of the solution being built. This is a perfectly reasonable approach. We let experts guide us in most things, and generally, it works quite well.

This only goes wrong if, for some reason, your representative experts turn out to have systematic biases relative to your real user base – for instance, someone who used to work in a function but now is out of touch with day-to-day reality or someone who has made the transition to an IT career some time ago and has now adopted an IT perspective on things.

Note that sometimes, you'll want to roll something out that isn't what your user base wanted or expected to drive transformational change. That's a different ballgame and is all about the way you do change management. However, if you do it unintentionally and unknowingly, you have an anti-pattern.

Results

The results of assumption-driven customization depend on the degree of misfit with user needs and expectations:

- At one end of the scale, you have a set of basically benign misunderstandings that can be fixed with goodwill, training, and a set of workarounds.
- If that's insufficient, you may have to add or rework a limited number of features to get the users to a place where they're happy to work with the new system. This is still relatively manageable.
- However, sometimes, you'll get to a point where users flat-out refuse to adopt a new system, even with substantial modifications.
- At this point, you have two options: either you force it through and deal with the backlash from the user base or you scrap the tool and go back to the old way of doing things.

The following diagram provides a visual representation of the scale of results:

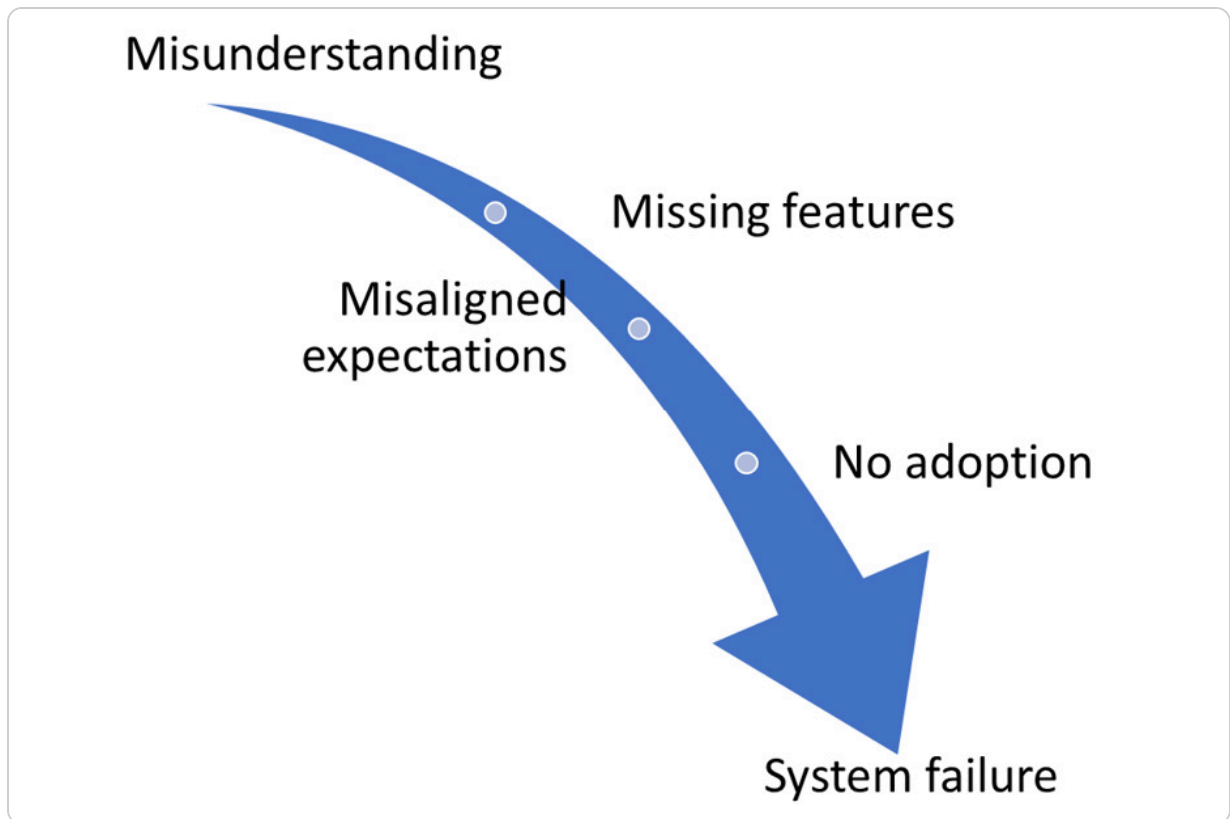


Figure 5.3 – Scale of deteriorating results for assumption-driven customization

So, the consequences of this anti-pattern, while often minor, can be disastrous, and you should spend time being mindful about avoiding it.

Better solutions

This is a difficult pattern to guard against because it occurs when the people we trust to be the voice of our user base fail in some significant ways to represent those users. It can be deeply uncomfortable having to raise these kinds of issues with product managers or product owners, who are convinced about their position, and in fairness, most of the time, they'll be right.

However, some mechanisms can help with this:

- Broader consultation with the user base – for instance, via larger events or a common online forum.
- Earlier feedback from the broader user base. Roadshows, demo videos, and webinars are good options here.
- Sprint-by-sprint acceptance testing from a larger user group.
- If you're looking to create deep change, plan for it, and include the necessary change management. Never try to sneak in a major change.

That concludes our discussion of this anti-pattern. Now, let's look at a genuine classic: the Golden Hammer.

Golden Hammer

The Golden Hammer makes the fundamental error of mistaking a tool that's a good solution for some use cases for a tool that's a good solution for all use cases.

Example

WheelCo is a large manufacturer of tires, especially for the transportation sector. It's rolling out Salesforce with Manufacturing Cloud at its core to closely connect its manufacturing and sales processes.

As part of the deal that's been made with Salesforce, WheelCo has received unlimited use of OmniStudio and has been party to several pieces of training and accelerators related to the capabilities of the product. As a result, the architects, developers, and business users within WheelCo are extremely excited about what OmniStudio can do for their business.

During initial architecture discussions, the decision is made to base the entire solution on the OmniStudio toolset, including all UI processes (both internal and external), integrations, and any subsidiary processes where OmniStudio could conceivably be used. Only batch and asynchronous processes are left out of scope. That way, WheelCo believes it will maximize the value it can generate from the Salesforce purchase.

The project starts, and in many areas, rapid progress is made. To stay consistent, most parts of the standard UI are ignored and FlexCards and OmniScripts make up nearly the entire **user experience (UX)**. Integrations are developed using integration procedures, except for a few that run in the background.

The volume of development is much larger than originally anticipated, but by adding a sufficient amount of resources, WheelCo still manages to get its implementation done on time. It has some trouble finding enough qualified resources that have the required level of OmniStudio experience, but by paying top rates, it manages to secure enough people for the project to succeed.

The go-live is, broadly speaking, successful, although there's some grumbling that not everything that was promised has made it into the release. Several key features had to be descoped during development to make it on time.

Over the following year, some problems with the solution start to become evident. The integrations aren't as flexible as was expected and can't accommodate all the different use cases the business wants to add. There's also a problem with new feature releases on the Salesforce platform. Every time something new is added, it takes a change to OmniScripts, FlexCards, and sometimes supporting code to make it available to users.

The cost of supporting the solution is also high. Any change needs intervention from a senior technical person, and that reduces the flexibility of the platform. Within WheelCo, many started to question whether OmniStudio was the right solution to all the requirements it was used for in the project.

This is especially true as some of the native Manufacturing Cloud features that were excluded from the OmniStudio work seem to be rapidly improving without anyone needing to lift a finger to make it work.

After about a year, WheelCo's CTO appoints a new Chief Architect, who has to look at the architecture of the Salesforce platform as one of his early priorities. He identifies high complexity caused by large amounts of customization as a major driver of cost and functional issues. To address the issue, he recommends refactoring the solution so that it uses a much more standardized UI, retaining OmniStudio for complex multi-step processes and areas where particular views of the data and associated actions generate real value and eliminate it elsewhere.

Problem

The Golden Hammer solves any problems – that's the nature of the Golden Hammer. Once you've found the Golden Hammer, every problem you encounter is a fit. There's no evaluation of fitness for purpose because the Golden Hammer is the right solution.

The tech industry, in general, is very prone to Golden Hammers. We're always looking for the next cool technology to solve our problems. Salesforce, with its aggressive sales approach and Kool-Aid drinking community, is perhaps even more prone to this anti-pattern than most platforms in the technology space.

Note that Golden Hammers are often technically excellent solutions within their own solution space, but they're stretched well beyond that space because of enthusiasm, hype, and excitement.

Proposed solution

The solution that's proposed by the Golden Hammer is the Golden Hammer – that is, whatever the problem, whatever the parameters of the problem, they'll somehow be twisted into something the Golden Hammer could conceivably deliver, or alternately, the Golden Hammer will be twisted into something that delivers a solution to the problem.

If you have to pick, it's better to mold the problem to a bad-fitting tool than to use a tool for a problem that it's unsuitable for. However, neither of these options is particularly good.

A good way to check if you're under the influence of a Golden Hammer is to check whether your discussions about new technology features start with looking for problems your tech stack could

deliver or whether you start with problems and then try to find a suitable technology stack. If it's the former, you need to be careful as you could have a Golden Hammer on your hands.

Results

The results of a Golden Hammer depend on the degree of technical badness of fit that you've engaged in:

- At a minimum, you'll end up with a gap versus the requirements that you were meant to fulfill as there will be areas the Golden Hammer didn't manage to solve because it's not the thing it was meant to solve
- Higher cost and more customization work as you increasingly stretch the Golden Hammer beyond its capabilities
- Additional maintenance work and resulting cost as a result of the overstretched solution that has been created
- Reduced faith in the real capabilities of the Golden Hammer within the organization, even where it's genuinely a good fit
- A need to maintain a specialist skillset across different areas in your business that will persist even when the hammer is no longer popular or trending

All in all, the results will depend on how much you're stretching the tool's capabilities, but choosing the tool before you know the problem is invariably an anti-pattern.

Better solutions

Architects differ in their willingness to adopt new tools and technologies. Salesforce architects, in my experience, are fairly aggressive in adopting new technologies, so long as they're on the platform. Therefore, I'm probably going to come off as a bit of a curmudgeon when I say that my general recommendation is to be quite conservative about adopting the newest toolsets, especially on Salesforce.

In the Salesforce ecosystem, we're constantly bombarded with new features, new toolsets, and new ways of doing things. While that's great, changing an approach that has worked for you in the past and that continues to deliver value to your organization should only be done after careful consideration, not because Salesforce has just launched a shiny new toy.

Implement a regular review process for your technology stack. This helps ensure that you're using the most appropriate tools for each task and allows you to phase out outdated or inefficient solutions. Approach new toolsets on a trial basis with limited business use cases to gather insights and evaluate their value. If a tool delivers genuine value, consider scaling it across your organization; if it doesn't provide value, discontinue its use, regardless of its appeal.

Engineering overflow

The two anti-patterns in this section both represent failures of technical governance, resulting from too much engineering work going into a single component without appropriate design patterns and practices applied to mitigate it.

Automation Bonanza

Automation Bonanza is what happens when you fail to govern how automations are used on your key objects.

Example

DrinksCo is a long-time user of the Salesforce platform, using it for sales and customer service as well as having a large and elaborate footprint on Marketing Cloud. While there are multiple orgs in use across the estate, there's one main org that manages the core processes for which Salesforce is in use.

While this org has been creaking slightly for a while, the situation has now gotten to a point where DrinksCo users are making serious complaints. The time to save records on key objects such as Opportunity and Account can sometimes be measured in minutes. Occasionally, records will simply fail to save, giving an obscure technical error message. Worse still, there are synchronization jobs containing important information about new leads and opportunities that randomly fail without giving a clear explanation as to why.

Francesca is a Salesforce consultant with a boutique Salesforce partner. She's called in to analyze the situation and propose improvements to the org that ideally fix the root cause of the issues, but at a minimum reduce the symptoms quickly.

After a week of examining the org, she's no longer in doubt: the problems are caused by interaction effects between a large number of complex automations on key objects. On Opportunity, for instance, there are deep layers of automation that have evolved over many years that combine to create serious issues.

This includes a large and complex Apex trigger, two additional triggers from managed packages, a handful of workflow rules, three process builders that are yet to be migrated to flows, a before-record-triggered flow, an after-record-triggered flow, several batch jobs that operate on the object, and multiple integrations directly using both the native API and a custom web service to manipulate records in the Opportunity object.

Francesca takes a deep breath and starts methodically mapping out the various sequences of events that can occur, depending on the Opportunity records and their updates. She does this by

systematically following the order of execution and looking at where there might be potential undesirable interaction effects between the different automations.

After some time, she concludes that the number of ways in which these sequences can go wrong is too high for the current model to stand. It's unfixable without seriously reducing the complexity of the automations.

Therefore, she devises a refactoring plan that she believes will be successful in alleviating the most serious symptoms of the Opportunity object, although probably not eliminating the problem entirely. If it's successful for the Opportunity object, similar initiatives will be attempted for other key objects in the core org.

The plan includes moving the process builders and several of the workflow rules into the two flows, consolidating the integrations so that there's only one that regularly handles imports into Opportunity, simplifying the Apex trigger, and getting rid of one of the managed packages that aren't strictly needed anymore.

The plan takes much longer than initially expected to implement – there are just too many things to consider for everything to go to plan. But eventually, the users start reporting an improved experience when using Opportunity. The old random errors no longer occur. And although saving is still slower than they would like, it's no longer so slow as to be a serious issue.

Having momentarily enjoyed her success, Francesca takes another deep breath and starts the process for the other key objects in the org.

Problem

The problem that automation bonanza tries to resolve is how to accommodate different architectural styles or preferences when creating automations on Salesforce. This occurs almost inevitably when you have multiple teams working on the same objects as part of different projects and often coming from different implementation partners.

As organizations grow, the complexity of interacting automations can increase exponentially. Different teams may have varying approaches to automating their specific requirements, but this can lead to confusion and inefficiencies. Therefore, it's crucial to establish a clear automation strategy from the outset to manage these complexities effectively.

That's when you need a way to guide the evolution of your architecture over time to keep things consistent. If you fail to do so, you may end up with automation bonanza.

Proposed solution

Automation bonanza responds to the multiplicity of automation options, some of which are shown in the following screenshot, and the commensurate multiplicity of automation approaches by allowing for diversity. This means that teams are allowed to choose a best-fit approach in isolation:



Figure 5.4 – Some Salesforce automation options

This reduces coordination requirements between the teams and usually doesn't result in short-term issues. Therefore, it can be quite difficult to spot that you have a problem or are generating a problem before things have already gone wrong.

Results

The negative consequences of automation bonanza can be rich and varied. Some of the things that frequently occur are shown here:

- Slowing performance on key record pages – sometimes, saves can take a long time to complete
- Failures in integration call-ins, especially where these update records in batches or a bulk fashion
- Failures in batch jobs or import jobs
- Strange, unpredictable errors occurring in the UI, typically associated with a save or update event
- Difficulties in debugging issues occurring on the platform as interaction effects from automations overshadow the general picture

Overall, if you get into a serious case of automation bonanza, you can have a real mess on your hands. What's worse is that this tends to happen to your most treasured objects first as these are the ones used the most across projects.

Better solutions

The key to avoiding the automation bonanza anti-pattern is good architecture governance around your core objects and their automations. Some things to keep in mind to do this are shown here:

- You should start by defining which types of automation you want to use in your org and for which use cases.
- Ideally, you want a number as close to one as possible for each object. Choose either flows or triggers as your go-to automation type and stick to it.
- It's possible to mix Apex and flows safely, but it requires careful planning and a good framework.
- If you're using flows as your principal mechanism, make callouts to Apex via Actions rather than by creating triggers unless you're forced to do so for performance reasons.
- Ensure you have a framework that helps the architect use flows. This should include the use of sub-flows to retain flexibility. Have decoupled automation processes that are easy to troubleshoot in case of problems instead of having a large monolithic flow containing all the business logic in a tightly coupled design that's complex and expensive to maintain in the long term.
- If you're using triggers as your principal mechanism, use a trigger framework such as Apex Trigger Actions (<https://github.com/mitchspano/apex-trigger-actions-framework>), which enables you to call out to flows as part of the trigger execution.
- Avoid Process Builder and workflow rules as these are being retired.
- Batch processes and incoming integrations need to be designed carefully and be mindful of existing automations. Remember mechanisms that enable you to selectively bypass automations for some processes, such as bypass flags.
- Watch managed package triggers and flows carefully. You don't have control over them, but they may still cause trouble by interacting with your functionality.

In general, trying to keep your automation approach simple and uniform will generate substantial long-term benefits for your Salesforce orgs. That being said, let's look at the next anti-pattern: overweight component.

Overweight Component

An Overweight Component is a single component that's grown into an application of its own without having the architecture and design to sustain it.

Example

HealthyMindCo is a broker for mental health services across the United Kingdom. It's implementing a new case management solution on Salesforce. In this solution, caseworkers collect relevant information about new clients and determine which actual local provider is best placed to help them. Additionally, they track the consultations their clients go through to determine the effectiveness of the intervention.

The company has very special requirements around the recommendations the system should make for potential treatment options and referral possibilities, which can only really be accommodated by using custom code. Because HealthyMindCo prefers a simple and highly optimized UX, Amir, the project lead for the partner implementing the solution, decides to go with a Lightning component that can incorporate the necessary logic for creating new cases.

Initially, the thought is that the component will only be used for new cases. However, after initial user testing, it's clear that update functionality will also need to be added. There are too many instances where initial assumptions need to be checked, which can lead to different outcomes in terms of the treatment plan.

After the update functionality has been added, the component is tested with users again, and what appears to be evident is that the component will also need to support the management of activities and activity templates as that's a crucial part of the initial setup of a treatment plan. The team works hard to add this to the component, which is starting to be a substantial piece of engineering.

Once the activities and activity templates have been added to the component, several users point out that it doesn't make any sense to have the activity management part of the equation in the new component, which is now coming to be known simply as the case-handling component, and also to not be able to track SLAs against those activities in the component. This implies also adding the visit-tracking capability as that's required to manage the SLAs.

At this point, the Lightning component is the main interface that's used by HealthyMindCo staffers. Due to this, several integration callouts that get information from other systems are added directly to the code in the component, simply because this is where it will be used anyway.

The system goes live in an unspectacular manner. There are a lot of bugs, in particular in the case-handling component. But with a lot of goodwill and hard work, the team comes to a level where the quality is just acceptable enough for the client to use it in anger.

After the go-live, HealthyMindCo keeps finding new bugs in the case-handling interface. It's generally quite unstable. In addition, it still wants to add large amounts of new functionality to it. However, the company isn't happy with the quotes it's receiving for its change requests.

Amir consults with some of the senior architects within his company. They assess the situation and tell him that the component is too big, too monolithic, and too difficult to maintain. He needs to refactor the code base and find the funding to do so either from the client or internally. He tries, but no one seems particularly inclined to pick up the tab.

Problem

Generally, heavyweight components are built to accommodate a real or perceived need for a custom UX that goes well beyond what can be delivered with Salesforce's out-of-the-box tools. Note that in some cases, the perception that you need such a specialized UX can be stronger than reality.

Users who come from old systems that have been used to working in a specialized application or who had their old system customized heavily to their needs might assume that such customization will also be necessary on a platform such as Salesforce. There can be a marked degree of skepticism in some users toward the idea of using standard functionality. We all like to think that what we do is somehow special, after all.

Not to say that there aren't cases where there's a genuine need to have a unique and highly customized UX but, in my experience, they're less common than many would think. In any case, the way a heavyweight component attempts to deliver such an experience is less than ideal.

Proposed solution

A heavyweight component proposes to deliver a unique UX within Salesforce, using a development framework such as LWC to create what's effectively an embedded monolith application residing on the platform. This can be attractive for several reasons:

- You have full control over the UX and UI within a single component, allowing you to respond to almost any user request
- All functions are integrated, and the architecture is simple to understand
- There's one place for everything, and all changes can be made there, reducing the need for tooling and coordination
- The component is still embedded within the standard UX and with judicious use of styling, it may even look like something that came as standard
- You can accommodate changes to the UX within the component, giving users more of what they need

This solution also tends to not seem dangerous at the outset, even to experienced architects and developers, because all we're talking about is "just" another LWC component. It's only when the true scope of that component is realized that it becomes a problem.

Results

While the component remains relatively small, you won't see any negative results, and, of course, lightweight components are useful and architecturally appropriate elements of most Salesforce architectures. However, as a component continues to grow, it starts to resemble a standalone application – a code-based monolith that lives within the larger application but still has a life of its own.

Often, this happens unintentionally when a component that was meant to do something relatively simple evolves into a catch-all location for functionality. This often happens because the initial component was very successful, and users want it to do more things.

The consequences of this evolution are familiar:

- Quality starts to decrease and the error rate increases as the new component becomes too big and complex for developers to handle
- Maintenance slows down, and bug fixes and feature requests take longer to come through the pipeline
- Only certain developers can work on the heavyweight component because it gets too complex for others to handle
- The overall UX begins to deteriorate due to the reduced quality and crowded functionality
- The component is increasingly hard to integrate with other parts of the platform and is increasingly treated as a standalone application

As components grow in size and complexity, testing becomes increasingly difficult and time-consuming. This can lead to potential quality issues that may compromise the functionality of the system and result in longer release cycles, ultimately impacting delivery timelines and resource allocation.

Occasionally, this pattern can deteriorate to such an extent that you end up with a Big Ball of Mud – see [Chapter 2](#). However, even when it doesn't go that far, you'll eventually need a major refactoring effort to address the issues.

Better solutions

Given that the issue with a heavyweight component is that you effectively end up with a monolith application without a well-defined architecture, it shouldn't be a surprise that the way you must address the anti-pattern is to improve the architecture of the custom functionality you're going to deliver.

Generally, this is done by decomposing the larger component into a set of smaller, more manageable pieces while leveraging as much standard functionality as you can. Using an **event-driven architecture** (**EDA**) that allows the smaller components to communicate by sending and responding to events can also be helpful.

Additionally, architecture governance has a key role to play in this anti-pattern. You should have an assessment and approval process in place for new components, and you should also have someone periodically check in on what happens with the custom components that get approved as they can tend to balloon.

That doesn't mean you need a formal code audit to check through what everyone is doing unless your project is very large, but you should have architects who are part of your governance approach and who know the details of what's going on with any custom development being done in your org. That will help you spot evolving issues early.

Next, we'll look at our two code-focused anti-patterns, starting with the God Class.

Making a mess of code

The last two anti-patterns we'll cover in this chapter focus on two anti-patterns that happen in code. While these aren't the most common in a Salesforce world, we, as architects, should still be aware that our code-based solutions must adhere to good practices or fall into common anti-patterns.

God Class

The God Class is a single class that contains all or most of the business logic that's been defined for an application.

Example

ShoeCo, a maker of some of the world's most popular sneaker brands, has a complex, multi-cloud Salesforce org that it uses to run key parts of its business. As it has many custom requirements as well as a substantial amount of legacy on the platform, it has a large number of Apex classes to handle both business logic and integrations.

It's recently started to make the switch to flows, and in practice, most functionality is a combination of code and declarative functionality. Over time, ShoeCo would like to simplify its Salesforce functionality and avoid some of the problems it's currently facing.

In particular, many of the architects at ShoeCo are starting to feel like the org is out of control and that they don't have any clear overview of the logic that's activated at different parts of the system. Pranit, one of the Salesforce architects, proposes to simplify the current system architecture by creating a `CentralDispatch` Apex class that will take on the role of dispatching calls to other code or flows.

With all business logic funneled through the `CentralDispatch` class, Pranit believes that it will be much easier to identify which code is called in which circumstances, allowing for greater transparency, lower maintenance costs, and a greater potential for code reuse.

A key philosophical requirement for this approach is that the `CentralDispatch` class should contain no logic of its own. It's just there to forward requests to the right handler. However, this philosophical principle is soon abrogated in practice.

To make the class work as intended, more and more of the business logic that determines what gets called in which contexts has to move into the `CentralDispatch` class itself. That means more and more of the actual functionality lives in a class that's growing ever larger by the day.

While this causes some concern initially, Pranit and his fellow architects accept the development as they still have better visibility than before the introduction of the `CentralDispatch` class. It's only when developers start to complain that the class is too big and can't be changed without there being unknown side effects in various parts of the system that they start to take serious notice.

The class has grown to more than 5,000 lines of Apex code, and it effectively orchestrates all logic that's called from other places within the ShoeCo org. To some extent, it has been a victim of its own success, but at this point, most of the developers consider it more of a hindrance than a boon.

Pranit decides to create a new plan to refactor the `CentralDispatch` class and get it back to its original skinny self. He wants to move out all the complex logic and keep it as a dispatch hub that determines

which other Apex code or flows get called in particular contexts.

However, on inspecting the code, he finds that in many cases, it's impossible to do that kind of request forwarding without also including substantial amounts of logic. Disheartened, he considers whether the approach needs to be fundamentally changed.

Problem

The God Class anti-pattern is deployed to centralize technical problem-solving with an application. It can be much easier, conceptually speaking, to use a single pattern deployed in a single class to manage all the technical complexity that arises during custom development than to spend a lot of time doing detailed architecture and design.

You can see the God Class code structure in the following diagram:

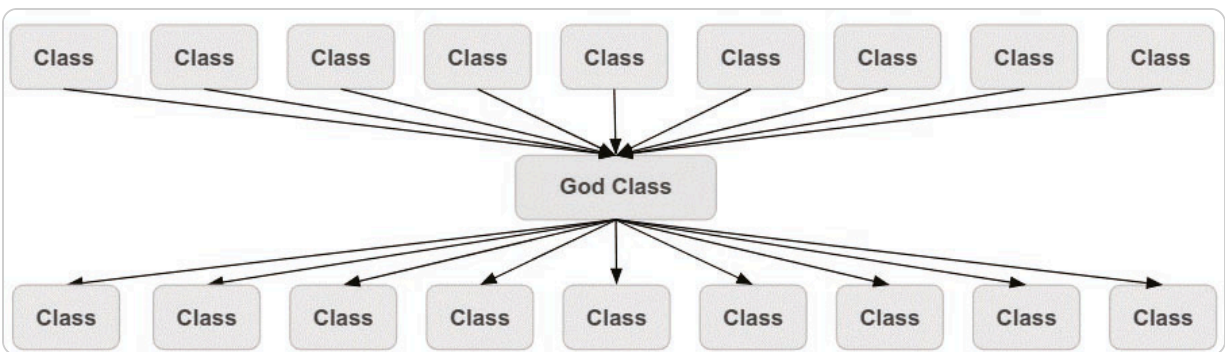


Figure 5.5 – God Class code structure

You don't see this anti-pattern frequently in Salesforce implementations, but it's a real danger to ISVs building for AppExchange or any organization building bespoke applications on the Salesforce platform.

Proposed solution

The God Class is a single file – generally an Apex class when we're talking about Salesforce – that hosts all or at least the majority of the critical business logic of a complex application. Centralizing functionality in this way can seem like a good design decision as it makes the overall architecture and design quite manageable, and there are even some patterns, such as the Dispatcher pattern, which we used in our preceding example, that can be quite used effectively in such a centralized way.

Often, as an application evolves, you can start to lose the big picture. This is especially true if you have a small application that's growing into a big application over time. You probably didn't have a strong architecture in mind at the outset as it was only meant to be a small application that could be built quickly.

In this case, resorting to centralizing the code base can be a way of managing the application and getting back in control. Alternatively, you might simply have started to put all the logic in one place for convenience and you didn't notice that the class had grown to several thousand lines of code.

However, as with all cases of poorly managed evolution of functionality, you'll end up paying the costs eventually.

Results

At this point in this book, you should be able to predict the negative outcomes from an anti-pattern that results from badly managed application growth. We've seen several such patterns, from heavyweight component to Big Ball of Mud, and the God Class shares many of the negative consequences we've come to expect:

- The code becomes hard to understand, resulting in longer maintenance cycles, slower new development, and increased error rates
- Only certain programmers may be able to work on the God Class to avoid unfortunate side effects
- All changes in the code base, anywhere, also require changes to be made to the God Class
- There are often strange interaction effects between what happens in the God Class and what happens in the rest of the system, leading to hard-to-debug errors
- The class presents a **single point of failure** (**SPOF**) within the application, and if it fails, none of the additional logic can be processed

Overall, you end up with an application that's fragile, brittle, and a bit of a mess.

Better solutions

When building custom solutions on Salesforce, you'll generally be using Apex, an **object-oriented** (**OO**) language closely related to Java. This means you should be following good **OO analysis and design** (**OOAD**) practices if you're proposing to build anything non-trivial on the platform. While teaching OOAD is way beyond the scope of this book, a good place to start is by studying the SOLID principles.

The **SOLID** principles are a set of guidelines that you can use when you're designing software to make it more understandable, maintainable, and extensible. The acronym stands for **Single responsibility** , **Open-closed** , **Liskov substitution** , **Interface segregation** , and **Dependency injection** (**DI**). Let's take a brief look at them:

- The **single responsibility principle** (**SRP**) is the principle that a class or module should have one, and only one, reason to change. This principle is often used in conjunction with the **open-closed principle** (**OCP**).
- The OCP states that software should be open for extension but closed for modification. This means that new functionality should be added by extending the existing code, rather than modifying it.

- Liskov substitution is a principle in **object-oriented programming** (**OOP**) that states that objects of a superclass should be able to be replaced with objects of its subclasses without the need to change the functionality of the program.
- Interface segregation is the process of separating a single interface into multiple, smaller interfaces. This can be done for a variety of reasons, such as increasing the modularity of a system or reducing the complexity of an interface.
- DI is a technique for achieving loose coupling between objects and their dependencies. It involves providing a component with its dependencies, rather than allowing it to create or obtain them itself. This can be done either manually by the programmer or automatically by a tool.

Having now covered the God Class, let's move on to our last pattern: error hiding.

Error Hiding

Error Hiding is what happens when you hide technical errors from end users without adopting a structured error-handling approach.

Example

SailCo, a provider of custom-designed sailboats, has a complex configuration application for configuring the boats it sells, which is built on top of the Salesforce platform. The application is built in a fairly traditional way using a combination of LWC and Apex code.

SailCo users are in general extremely non-technical, and one of the key requirements that the custom application had to satisfy was to never show a technical error message to an end user. Therefore, the developers of the original application devised a way to catch all exceptions and wrap them in a generic error message before they were shown to the user.

This general message of telling users that an error has occurred in a friendly way is the main error message that's seen in the system since the developers were unable or didn't have sufficient time to differentiate the errors that might occur reasonably. Users, however, seem comfortable enough with the solution and don't enquire much about the exact nature of the errors that occasionally occur.

However, after a while, it becomes clear that there are some systematic ways in which the configuration application can fail. There are complaints from several customers that their desired configurations haven't been captured successfully as part of the order they've made and that had been registered in Salesforce using the custom application.

No one can pinpoint the error exactly, and it falls to Sarah, a relatively junior Salesforce engineer, to try to debug the problem. She starts by reviewing the logs that she can generate from the production system but finds that the only errors she can see are the generic error messages that are thrown by the wrapping mechanism that hides the technical information from the users. In many cases, exceptions are simply caught but never handled.

This means the production logs are, effectively, of no use to her debugging efforts. She tries to reproduce the errors in lower environments after making some changes to the code to get the raw stack trace instead of the wrapped error message. However, she's unable to reproduce the errors in the lower environments.

She seeks and eventually gets permission for a special production release to add improved logging to the system. However, she needs to do a major refactoring of the existing code base as the quick hack she used initially wasn't deemed fit for production use.

After doing the major code refactor with the help of a few other SailCo developers, she manages to get the new logging functionality into production. As luck would have it, the problem turns out to be a

trivial coding error.

Sarah fixes the issue and releases the new code to production, confident that with the new error-handling approach, she'll be able to find issues even if the current fix doesn't address all the problems that may be found in the code base.

Problem

Error hiding is done for an excellent reason: to not show end users ugly technical error messages that are going to confuse them and make them lose faith in our application. In an ideal world, we want to recover gracefully from all errors or at least show a friendly, easy-to-understand error message.

You can see a simple example of error hiding here:

```
public class OpportunityAddController {  
    public Opportunity newOpp {get; set;}  
  
    public OpportunityAddController(){  
        newOpp = new Opportunity();  
    }  
  
    public void addOpp(){  
        try{  
            insert newOpp;  
        }  
        catch(Exception ex){ }  
    }  
}
```

Figure 5.6 – Simple example of error hiding

In practice, we mostly only have time to analyze the most common failure modes during analysis and design, leaving a large number of potential failures that we haven't prepared for. With the error hiding

anti-pattern, we end up hiding all of these errors not only from the end user but also from ourselves.

Proposed solution

The error-hiding anti-pattern solves the problem of not showing end users technical error messages by simply hiding away errors or replacing any error with a generic innocuous error message. This is done technically in a way where an exception is caught and the information contained in that exception is simply discarded, whether or not a message is shown to the end user.

That means we have no logging or way to capture error information systematically. Consequently, we have no way of knowing what, if any, systematic issues are happening in our application that we might not have thought of and no way of debugging issues that occur in production environments that aren't immediately replicable in lower environments.

This anti-pattern saves time for developers because there's no need for them to apply a structured error-handling approach during development, and it meets the users' need to not see confusing messages. However, it comes with many negative side effects.

Results

The most obvious result of error hiding is that you can't find the error. Errors are systematically hidden, and you have to rely on user reports and your ability to reproduce those to address any issues.

Debugging errors in Salesforce without good log information can be extremely hard, especially when the errors may be related to user context, data volumes, or similar variables that aren't easy to control.

This means your maintenance costs will be substantially increased and your cycle time for bug resolution will lengthen. Users are also likely to start losing faith in the system as they find that there may be issues that are swept under the carpet, which can lead to a vicious cycle of users starting to blame all sorts of ailments on hidden system issues.

Eventually, you'll need to refactor the application and introduce a systematic approach to logging and error handling. This will generally turn out to be much more expensive than having done things properly the first time.

Better solutions

The solution to error hiding is to adopt a structured approach to error handling, starting at the analysis phase of the project. You should analyze common failure modes and prepare for them proactively rather than simply focusing on the happy path.

You should also have a standard for error handling that includes how to handle exceptions, how to log errors, and how to show error messages to end users in different contexts. You should also adopt a

good logging framework. I tend to recommend Nebula (<https://github.com/jongpie/NebulaLogger>) as a framework, but there are many good options available from the community.

When looking for one, keep the following in mind. A good logging framework should be easy to use and configure, enabling logging from multiple sources (for example, Apex, flows, integrations, and so on) without impacting application performance. Additionally, implement a tiered logging system that captures different levels of detail (for example, `INFO` , `WARN` , and `ERROR`) to balance between user-friendly messages and detailed diagnostic information. Ideally, you also want to be able to easily visualize errors and follow them across process boundaries.

In all of this, of course, you'll want to insulate your end users from technical error messages. But you'll want to do it without resorting to an anti-pattern.

Now that we've covered our final anti-pattern, we're ready to move on to our key takeaways.

Knowing the takeaways

In this section, we'll abstract a bit from the specific patterns and instead try to pull out the wider learning points you can use in your day-to-day work as a Salesforce architect or in preparing for the CTA Review Board.

When architecting Salesforce solutions, you should be mindful of the following:

- Don't rush into a solution based on what your organization usually does. Carefully consider your options.
- When making a buy-versus-build decision, be careful to count the real TCO of a custom solution, including long-term run and maintenance costs.
- Don't design solutions for a specific license set if you can avoid it.
- Instead, push for a better commercial deal, different scope, or different license mix.
- You must make assumptions about your user base, but be careful about systematic bias.
- To help mitigate bias, use various consultation methods during development to derisk.
- Don't become too invested in a new technology. No matter how cool it seems, it's not the answer to everything.
- Automations is a particular pain point in many Salesforce solutions. It should be a particular focus of technical governance.
- Prefer a simpler automation setup to a more complex one.
- If you need a complex custom UX, decompose the functionality in a meaningful way rather than putting everything in a single component.
- When doing substantial development on Salesforce, you should apply all the same good practices that apply to other OO development projects in similar languages. This includes how you structure your classes and how you handle exceptions and errors.
- Regularly review and refactor your solution architecture to ensure it remains aligned with evolving business needs and Salesforce platform capabilities.

In preparing for the CTA Review Board, you should be mindful of the following:

- Using AppExchange products in your solution is entirely appropriate and usually preferable to custom solutions
- Recommend products you know and can justify or are well-known in the market to avoid unnecessary questions
- Assume that budget isn't a constraint and recommend the right licenses for the job
- However, don't go overboard and recommend lots of additional feature licenses if you don't need them
- Make assumptions freely, but state them as you go along
- Be prepared for judges to ask you to consider what would happen if you changed a key assumption on the fly
- Recommend the right tool for the job and avoid relying on too one-sided of a solutioning approach
- Don't go overboard with automations, and always be clear about what approach you're recommending and why
- Prefer a simpler automation setup to a more complex one
- Recommending custom components is often appropriate, but be careful not to overdo its functionality
- Do remember that when recommending code-based solutions, you should also know good practices for OO development, although it might mainly be relevant during Q&A
- Have a structured error-handling approach in mind that you can tell the judges should they ask
- Be prepared to discuss strategies for managing technical debt in long-term Salesforce implementations, including refactoring approaches and when to consider re-architecting

With that, we've covered the material for this chapter and are ready to proceed. But first, let's summarize what we've learned.

Summary

In this chapter, we covered eight anti-patterns, more than any chapter so far. That isn't accidental. Solution architecture is probably the most time-consuming activity for most architects. Even if we aren't directly responsible for all parts of the solution, we get pulled in to consult on features all the time.

Solution architecture is the most visible part of our efforts. While integration architecture, data architecture, and security architecture set the baseline on which we create our solutions, ultimately, it's the solution that we devise for the users that's the basis of their judgment.

If we get the core architecture right but fail to deliver a good solution, in the eyes of most observers, we'll have failed. Unfortunately, it's also a really hard area to get right.

As these anti-patterns show, keeping the right balance between flexibility and discipline and between giving the users what they want and adhering to good technical practice is something that requires experience and good judgment. Hopefully, you now have an even deeper appreciation of that fact.

Having covered solution architecture, let's move on to our next domain: integration architecture.

6

Keeping Integration Straight

This chapter examines anti-patterns related to integrations with other systems, a critical aspect of modern Salesforce implementations. The first part of the chapter explores anti-patterns in the integration landscape and integration patterns, the second part addresses potential pitfalls in the design and use of interfaces, and the third focuses on issues with integration styles. As always, we conclude the chapter by distilling the key takeaways.

In this chapter, we're going to cover the following main topics:

- How to not misuse technology to avoid overcomplicating your systems landscape
- When it is and isn't appropriate to create custom services for integrations
- How to avoid common failures in designing interfaces
- How to use event-based integrations well
- How to use integration patterns appropriately
- How AI will impact integration patterns

After completing this chapter, you will have a good idea about how to better select integration patterns and structure your integration architecture by learning about various ways in which it can go terribly wrong. As Salesforce continues to evolve, understanding these integration anti-patterns becomes increasingly crucial for architects and developers working with the platform.

Messing up the integration landscape

This section covers two anti-patterns that in different ways can create havoc in your integration landscape. We will start by looking at how not to use middleware.

Middleware in name only (MINO)

Using middleware to create point-to-point connections rather than using its extended capabilities

Example

PumpCo is a large B2B company that specializes in the production of pumps for industrial production. It operates in more than 30 markets and has historically underinvested in IT systems and used largely manual processes for sales that have varied substantially between countries and product lines.

Over the past year, they have started to implement Salesforce Sales Cloud and CPQ to drive standardization of the sales process globally. Their one major IT platform that has received substantial investment in the past is SAP and, fundamentally, SAP runs all key parts of the business today.

Michelle is brought in as an integration architect on the Salesforce implementation at an early stage. The integration roadmap is very ambitious as the business wants to see all relevant data and processes seamlessly operate across the Salesforce/SAP boundary. Fundamentally, they want to be able to access all the relevant back-office data and processes directly in Salesforce without the need for a context switch.

Once the initial mapping has been completed, there are 75 discrete integration points covering everything from pricing and logistics to HR that would need to be implemented to give the full experience that the business is looking for. The good news, however, is that many of these integration points can be grouped into similar areas, such as a *customer* or *order interface*.

The architects from the Salesforce and SAP sides discuss a number of alternative architectures and implementation approaches for creating a small number of stable interfaces that would cater to the majority of use cases, but unfortunately, progress is slow and there is no real agreement between stakeholders and architects. There is a large number of potential solutions in play as well as several technologies that could potentially do the job, and the willingness to compromise is low.

The two sides instead agree to let the middleware team manage the process. PumpCo has just bought a new middleware platform, and the team there is looking for opportunities to get started.

The middleware team will expose services to Salesforce and translate the calls to SAP. Any modifications to the APIs will also be its responsibility. Thus, the two platform teams don't have to agree on an approach and can work independently.

As the project progresses, Michelle makes a count of the interfaces Salesforce is calling on the middleware. She counts 45. Not quite the original 75, but then the scope has also been somewhat reduced as they have gone along.

Here's a diagram of PumpCo's integration architecture:

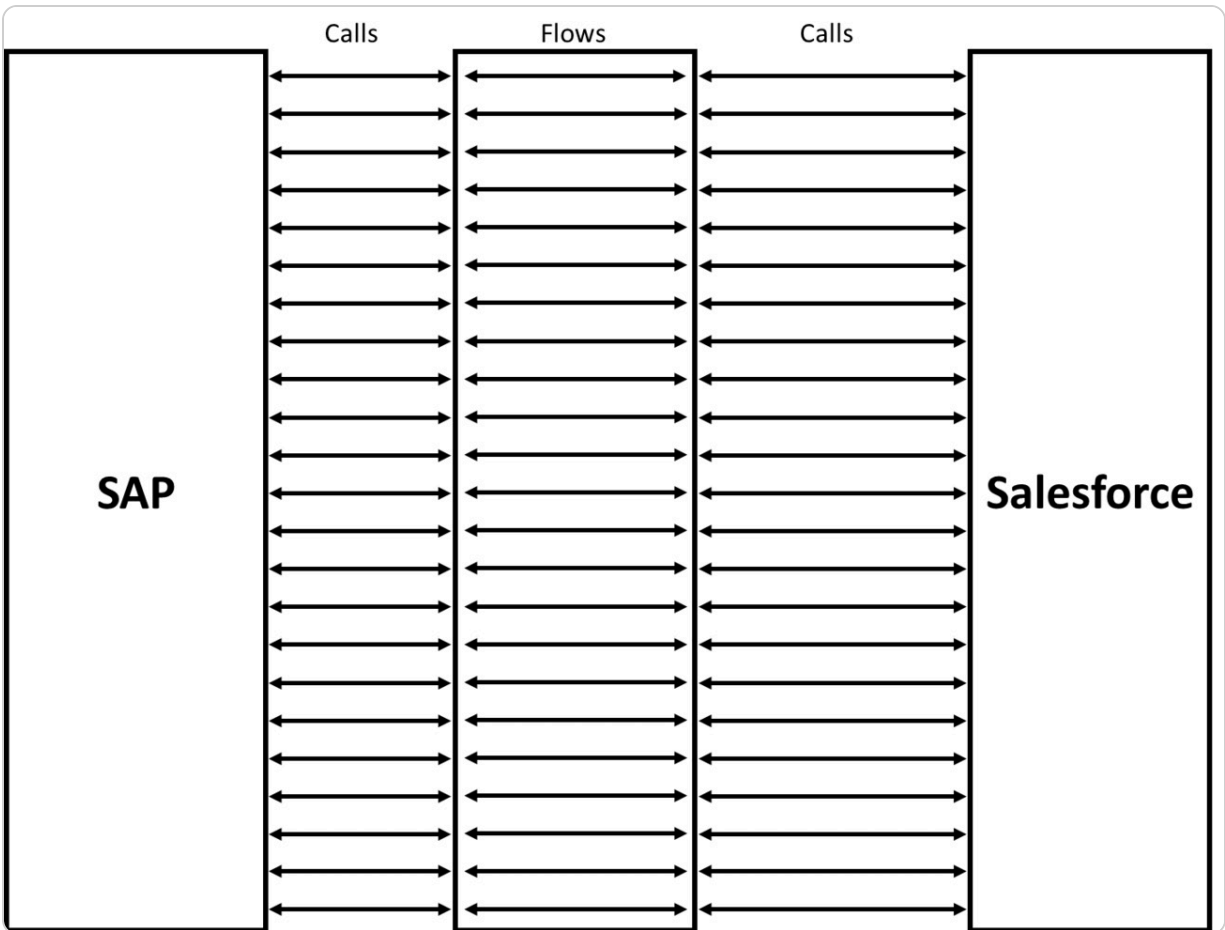


Figure 6.1 – PumpCo integration architecture

From what she can see, most of these middleware interfaces do little other than simply forward a call from Salesforce to SAP and back again. She wonders if that is really the best approach, but it's not her problem anymore.

However, over time, as Salesforce starts seeing more use, issues start occurring with the integrations. The error rate is high, there are performance issues, and maintenance is getting increasingly complex. Integrations, overall, are causing the majority of technical issues on the platform.

A consultant is brought in to assess the situation, and they recommend rationalizing the integration architecture as the current setup effectively consists of 45 increasingly customized point-to-point connections via the middleware. This creates a lot of potential for failure and is generally hard to understand.

Michelle is asked to participate in the redesign process, and after several weeks of design work, they end up with a proposal not too far away from one of the iterations that were done during the initial project implementation. When a senior stakeholder pointedly asks why this wasn't done in the first place, no one is really able to give a convincing answer.

Problem

The MINO anti-pattern tries to reduce the complexity of creating a good integration architecture by introducing middleware. However, it does so in a way that fails to leverage the capabilities of a middleware platform, instead simply recreating flows via the middleware that might have been found in a point-to-point scenario.

It tends to occur in organizations that have complex system landscapes with many interfaces and touchpoints between systems. However, in these organizations, there is often little technical governance coordinating between different silos, leading to a messy integration architecture.

Other characteristics that are often found with the MINO anti-pattern include the presence of dated and inflexible system APIs on key platforms and difficulty in agreeing on standard representations of core business entities across those key platforms, making it impossible to settle on common interfaces or APIs.

Proposed solution

The solution proposed by MINO is simply to introduce a middleware platform without paying too close attention to how it is used. By introducing a modern middleware platform, the complexities and inflexibilities of legacy systems can at least be partially hidden, which leads to good initial results.

That is to say, MINO often seems right on the surface, but if the middleware implementation only replicates the existing mess in a new format, relatively little is gained. Not nothing, mind you; you may still get some basic middleware capabilities such as better error logging, a retry mechanism, or some easier-to-use protocol conversion.

Another, and often more influential, reason to go down the road of this anti-pattern is that it decouples teams on core platforms from having to deal directly with each other. You can often have very different views of the world if you're working on the CRM side than if you are working on the ERP side of a key integration.

MINO allows different teams to only have to deal with the common middleware team, who are then given the responsibility for managing the rest. That, unfortunately, tends to lead to architectures that don't really move much beyond the basics.

Results

The result of MINO is often turning your system landscape into an even greater spaghetti-like mess than it was before. Now, after all, you have middleware in the center, so you can pay less attention to the details of the integration.

This lack of attention usually has the following consequences:

- Hard-to-understand integration architecture with too many individual interfaces and point-to-point connections, albeit mediated by the middleware
- Increased maintenance costs as the complexity is still high and there are now more teams involved
- Lack of technical governance and a potential lack of awareness that such governance is even needed
- Increasing error rates on integrations and a commensurately increased fix time as teams try to track down errors across platforms
- Decreasing performance as there is a more complex flow in place that spans more platforms

Overall, if all you are going to do with your middleware is proxy point-to-point connections, you are probably better off not using it at all.

Better solutions

What the MINO anti-pattern teaches is that there is no shortcut you can take to get your integration architecture right. You have to carefully consider the linkages and dependencies between systems, the business requirements both today and going forward, how your master data is distributed, which core capabilities your key platforms have, and how you can structure interfaces and patterns to best support all these elements.

Some common middleware capabilities are shown in the following diagram:

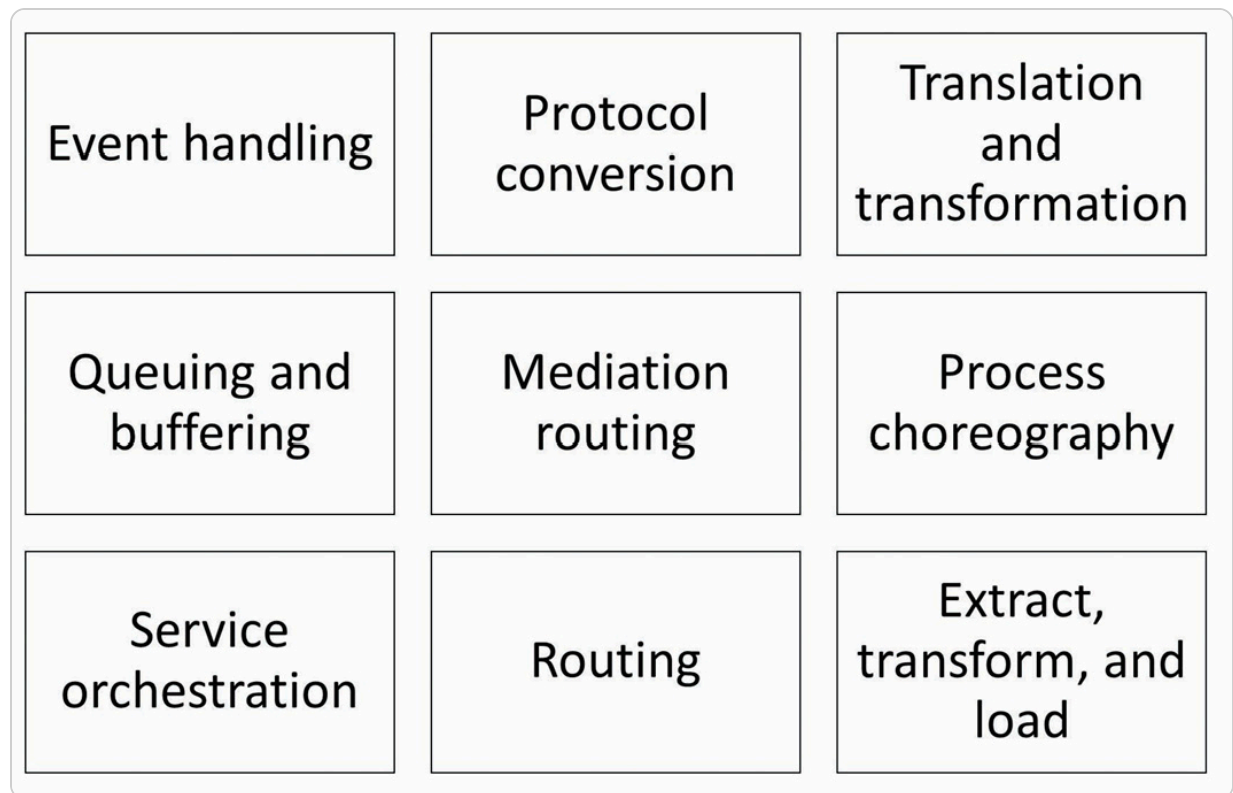


Figure 6.2 – Common middleware capabilities

With that in mind, you can select the right tools for the job, which may very well be a middleware tool. However, before you have thought about which interfaces you will need to support across the business and which integration patterns you can and must support for different scenarios, and determined how you are going to do technical governance across different teams, you shouldn't be jumping at any tool, no matter how cool it looks. We will now move on to the next anti-pattern, service proliferation syndrome.

Service proliferation syndrome

Service proliferation syndrome is an anti-pattern where an excessive number of custom services are created, often without proper justification or consideration of alternatives.

Example

OmniCo has consistently positioned itself as a technology leader and deploys many cutting-edge software platforms across its many diversified service lines. The company was an early adopter of **service-oriented architecture (SOA)**, which they still use to great advantage combined with an **event-driven architecture (EDA)** for high-velocity data and processes.

OmniCo is now implementing Salesforce as a replacement for its old Siebel CRM system, which served as an integration hub for many other systems. The Siebel CRM was heavily customized to OmniCo processes, and the company is expecting that the new Salesforce system will be as well. While their implementation partner has made a reasoned argument for staying with standard capabilities, this goes against the grain of how OmniCo has historically done things, and they are not really looking to change their approach as part of the CRM project.

Here's what the old Siebel setup looked like:

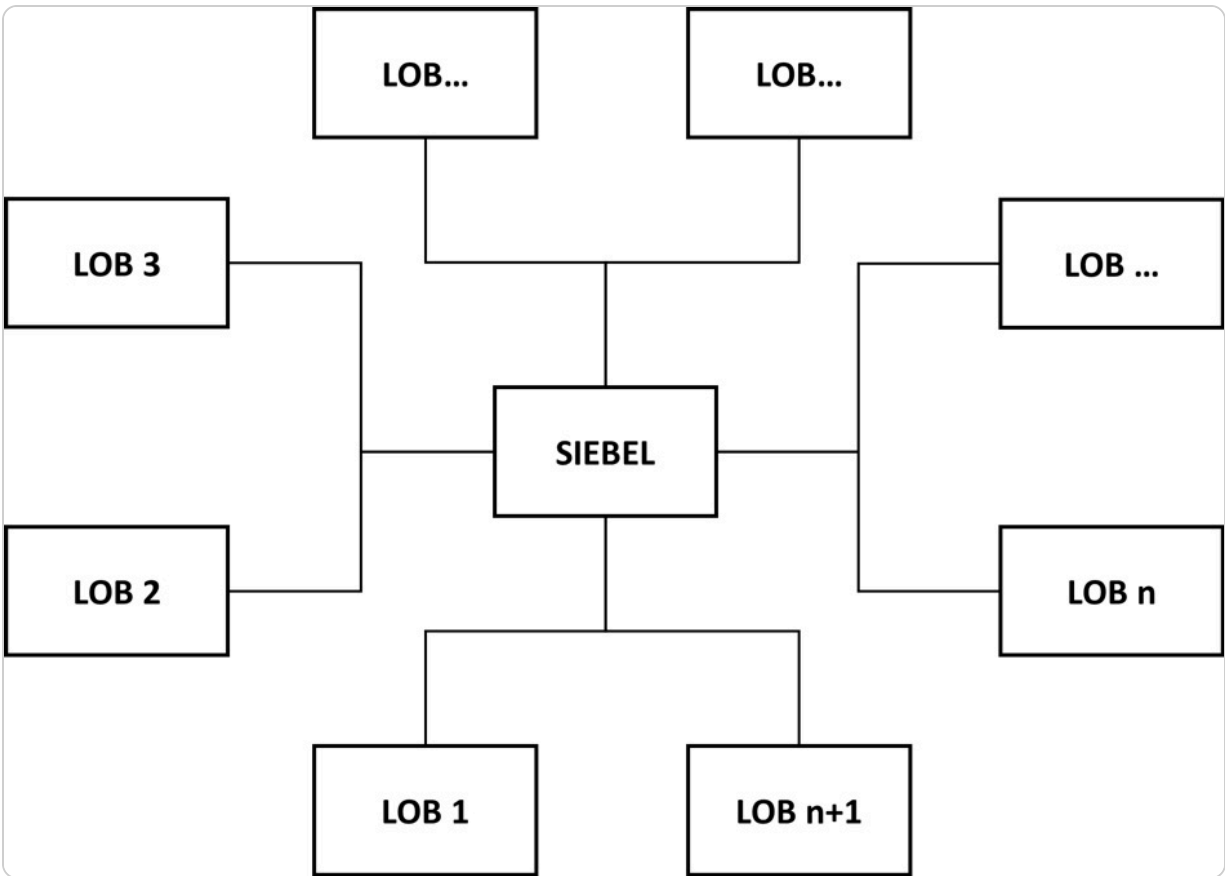


Figure 6.3 – Old Siebel setup at OmniCo

For Erhat, the consulting manager in charge of the integration part of the implementation project, this is causing a lot of anxiety. He is being pushed by a number of senior stakeholders from OmniCo toward building a range of special business services that would fit into the OmniCo SOA.

Erhat has tried building customized APIs on vanilla Salesforce in the past, and the experience, while ultimately successful, was neither straightforward nor fast. In fact, given the number of custom services that are being mentioned, he is in doubt about the basic feasibility, given constraints on time, budget, and people with the necessary skillsets.

After some pushback, he agrees with stakeholders to deliver two crucial services using interfaces similar to the ones exposed by the old Siebel system that are used in order management, and that would take a lot of time to re-engineer on a different pattern.

These are delivered but prove difficult to get through testing, partially because the testing protocols for the services are incomplete and partially because the complexity is extraordinarily high.

As the first two services are near completion, a crisis meeting is called by OmniCo's enterprise architecture board. They have just realized that not all the business services provided by Siebel will be

available in the new Salesforce setup. In the view of several members of the board, this will fundamentally undermine a range of business processes as other systems would need to change their integration approach substantially or switch to manual processing of certain steps.

Erhat, not knowing the processes at OmniCo very well, finds himself unable to argue on the merits of the case. The members of the board present him with definitions for an additional 13 services that cover different parts of the lead-to-cash process and that were available in the previous Siebel setup. Here's a schematic view of the integrations:

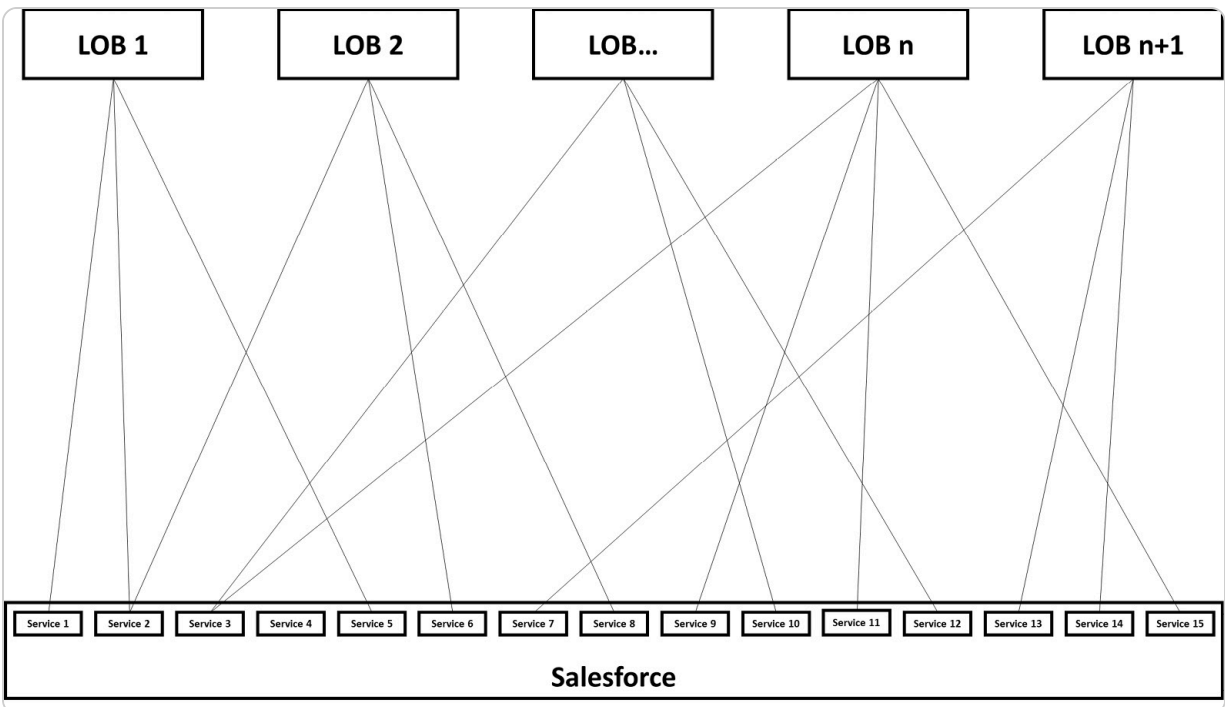


Figure 6.4 – Schematic view of OmniCo Salesforce integrations, not including ESB

Erhat can only really push back on a practical level, which he does, arguing that the services are outside the scope and are not included in pricing and that he doesn't have the team to deliver the solution. OmniCo grumbles a bit and there are a few escalation meetings. However, Erhat is eventually told not to worry—they will solve it some other way.

That way turns out to be bringing in a team of disparate contractors to quickly build the services alongside the main consulting partner's team. Erhat can only wait and observe as the contractors run into the inevitable technical complexities. He leaves the project prior to going live with something of a bad taste in his mouth.

Twelve months later, he is back at OmniCo. He has been brought in as an expert on the company's Salesforce APIs, which have proved very error-prone and expensive to maintain.

OmniCo are looking for the rationale for why it was built like this in the first place and what to do to fix it. They are also considering different approaches such as changing integration patterns or bringing in some middleware. Erhat takes a deep breath and starts planning the analysis.

Problem

The problem that service proliferation syndrome constitutes is how to fit Salesforce into an existing enterprise architecture that will have preexisting expectations of the capabilities delivered by its key systems. That may be an SOA, as in our example, or a different organizing principle, but typically, one that requires very specific things from key platforms.

This anti-pattern is common across a variety of platforms as it will potentially affect any newcomer to an integration landscape. In the past, it would have been more common as organizations were busily building SOAs, often without a lot of thought as to the organization of specific services.

This anti-pattern often arises from a misunderstanding of microservices architecture or a desire to solve every integration challenge with a custom service.

Today, it is perhaps most commonly encountered when old systems are replaced and interfaces are required that don't quite fit with the standard capabilities of built-in system APIs. While Salesforce has extensive APIs, they are very data model-centric, and many integration architectures are built along different principles, such as coarse-grained business services, which aren't aligned at all.

Proposed solution

Service proliferation syndrome addresses the issue of navigating the integration landscape by advocating for the unnecessary creation of new custom services for each integration need. This approach overlooks the potential of leveraging existing services or standard APIs, which could adequately meet these requirements without adding unnecessary complexity.

This is another anti-pattern that can seem deceptively rational as you are, after all, directly delivering business value and sometimes in line with expectations from cross-company technical governance forums. Therefore, you can be under a lot of pressure to go down this route, knowing full well the damage it is going to do to your platform in the long run.

Because the capabilities are there on the platform and the need is there in the enterprise environment, fighting this anti-pattern can sometimes be impossible. This is even more true because the costs only really accumulate over time.

Results

The first issue you are likely to see with service proliferation syndrome is increased complexity and build costs. Custom services on Salesforce, while feasible and sometimes the right choice, are complex to get right. Salesforce isn't inherently an API platform, which is a large reason why they acquired MuleSoft a few years back.

The increased complexity will over time lead to increased maintenance costs and not just on Salesforce. The various teams using the proliferation of services may also face ongoing costs to rework and upgrade their connections.

In a DevOps context, service proliferation poses significant challenges to CI/CD pipelines by introducing increased complexity that makes automation and orchestration more difficult. This complexity can lead to more frequent deployment failures, as the interactions between numerous services can be unpredictable. Furthermore, the expanded surface area for potential issues extends recovery times, as teams must navigate a greater number of components to diagnose and resolve failures. As a result, organizations may find it harder to achieve the speed and reliability that DevOps aims to deliver, ultimately impacting the overall efficiency of software delivery.

Sometimes, a custom service can be the right choice, and the benefits can outstrip the costs both for the initial build and for the maintenance. That is rarely—if ever—the case for 15 custom services as in our example. If you think you need that, you probably need to rethink your approach.

Better solutions

The first piece of advice is to start by looking at standard integration patterns to see whether you can find a standard piece of technology that fits the bill. Maybe there is a way of using standard APIs. Maybe you can use a batch process. Maybe you can emit events and have other systems subscribe to them.

You should look broadly and not immediately jump to a custom interface, even if that seems like a good initial fit. The danger is that you go for the gold-plated solution, not realizing the real long-run costs of the decision.

If you do decide that a large number of custom APIs are required—and there are situations where that can be justified—you should use a platform that is built for this purpose to implement. MuleSoft would be the canonical choice for Salesforce, but there are others in the marketplace that can fill this niche.

To enhance our service offerings, we can take a step-by-step approach:

1. Identify core business capabilities.
2. Map existing services and APIs.

3. Identify gaps in our services.
4. Consider standard solutions before opting for custom services.

Additionally, we can consider implementing a robust API management strategy to promote the reuse of services and discourage unnecessary service creation.

In a DevOps context, service proliferation can lead to increased complexity in CI/CD pipelines and more challenging release management. Therefore, adopting a more disciplined approach to service creation is essential, as it aligns better with the DevOps principles of simplicity and automation. This ensures streamlined processes and enhances overall operational efficiency.

Overall, you run the risk of overcomplicating both your Salesforce interface and your integration landscape by indiscriminately building services. As always, consider the hard trade-offs and make a decision based on a real view of the pros and cons. We will now move on to the next section, where we discuss anti-patterns that relate specifically to the design of interfaces.

Interfacing indecently

In this section, we will look at two common anti-patterns that affect the design of integrations at a concrete level. The first, fat interface, looks at a problem with interface design, while the second, chatty integration, looks at a problem with how interfaces are used.

Fat interface

When a single interface starts doing too many things, you run into a host of problems.

Example

Joe works as an integration architect for a small Salesforce partner that specializes in complex implementations often requiring substantial amounts of code or other kinds of customization. He has just started a new contract with RealCo, a major real estate investment company that uses Salesforce as the key system of engagement across the entire business.

The work he is initially contracted to do includes building a custom interface for accessing information about valuations that is used by RealCo's partners as part of making deals. The information is held in RealCo's Salesforce org and needs to be available in real time.

Joe sets up the API constituting the interface on RealCo's middleware platform and orchestrates calls to a few different Salesforce standard APIs and a single custom one as part of the implementation. However, close to the go-live of the new interface, a bunch of new use cases drops on his desk.

These include not just extensions to the valuation API, such as getting historical valuations and trend data, but also entirely different categories of property data, including real-time updates through platform events, changes in data streams using **Change Data Capture (CDC)**, information about the structure of the building, access to key metrics from various assessments, and insights from feedback collected during viewings conducted by RealCo partners.

Joe pushes back a bit on the customer, saying that if he'd known that this much new functionality would be coming, he might have structured the API differently and might have done some things in a suboptimal way to include it with the current go-live.

RealCo's manager listens attentively to Joe but says that ultimately, the decision is to go ahead with the additional functionality. RealCo is aware of the potential downsides, and it can live with them.

The day before the go-live, the same thing happens again. This time, the information includes more property-related information but also entirely different classes of information, such as demographics and segmentation data related to the area in which the property is located, as illustrated in the following diagram:

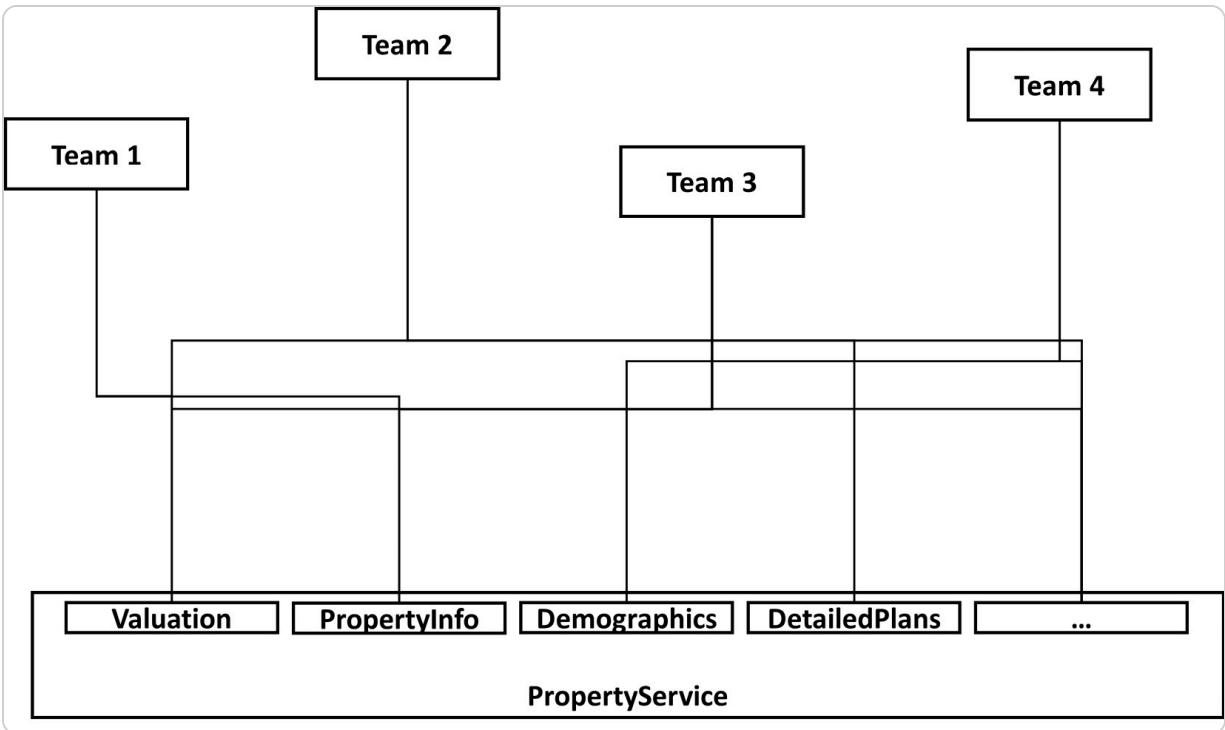


Figure 6.5 – View of RealCo PropertyService

Nothing can be done to include this in a day, which RealCo begrudgingly accepts. However, it wants a plan from Joe to include it posthaste. Given the immense pressure, Joe simply extends the current interface, which is now becoming enormous with calls to dozens of Salesforce services, both standard and custom.

Fortunately, both the initial go-live and the subsequent update go relatively well. The teams at RealCo's partner organizations grumble a bit about the complexity of the interface and log a number of bugs. But it works, and they can get on with the various applications that the interface needs to drive.

The barrage of change requests keeps happening. And somehow, the changes are always exceptionally urgent and needed for critical use cases in one partner or another. What's worse is that an increasing number of teams are relying on the interface, which makes upgrades increasingly hard.

Even with good API versioning, the changes between versions are proving quite strenuous for the partner teams who are complaining quite loudly about both the frequent version changes and the high error rate that has crept into the API implementation.

The final straw comes when a critical bug is found just prior to a new release. Joe, having been told to prioritize getting the new functionality out, includes a hotfix with the new version, which effectively forces an upgrade on all the major partners.

However, it turns out that there are additional breaking bugs in the new API version, which means the key partners lose access to key functionality for over a week until Joe can get the situation under control.

An escalation meeting is called where the partners can voice their various complaints. Joe tries as best he can to explain the history and the reasons for the current situation, but unfortunately, the complexities get lost in the general din.

Joe is told that his contract won't be renewed and that another consultant will be brought in to refactor and restructure the interface. He breathes a sigh of relief and starts looking forward to his next engagement.

Problem

The problem posed by the fat interface is fundamentally one of the interface structure—that is to say, where to place functionality so that it can be called by clients that have a need for the services the interface provides.

NOTE

*Originally, this anti-pattern was applied more to interfaces in the sense of the word used in **object-oriented programming (OOP)**. However, it applies equally in an integration setting.*

There are several different philosophies on interface design. These days, most people have an instinctual preference for microservices, which are smallish services that carry out a single well-defined and coherent set of functionalities. An example could be a notification service that does nothing but send notifications.

A few years back, however, the preference was for coarse-grained business services that provided an entry point to a business process—for example, processing an order. This was the foundational style associated with SOA, which we have mentioned previously.

With a fat interface, however, you are violating a fundamental tenet of interface design that is common to most—if not all—of these philosophies. That tenet is called interface segregation and holds that clients should never be forced to depend on methods that it doesn't need.

Proposed solution

Fat interface proposes to simply continue to add logic to an existing interface because it is the easiest thing to do. Designing well-segregated interfaces can take work and careful thought, and putting all your functionality into a single basket simplifies the problem.

In addition, you can sometimes convince yourself that all the functionality really does belong together because there are some tangential commonalities between them, and this can be especially true if you have consumers that use substantial parts of the functionality you expose.

Often, this anti-pattern is simply the consequence of drift over time. The code starts to do one thing, then it does another, and another, and at the end of the day, it does everything and walks the dog.

That would be all well and good if it weren't for the fact that it comes with a number of hidden costs that have to be taken into account. We'll explore these next.

Results

The results of the fat interface anti-pattern will be familiar if you have been reading this book straight through. It resembles the consequences of poor structure that can occur in a multitude of domains.

When your interface has grown to the extent that it becomes a fat interface—that is to say, it now includes so much diverse functionality that it has effectively become unmanageable—you are likely to see some or all of the following consequences:

- Increased complexity, leading to increased cost of change, cycle times, and error rates
- Increased maintenance costs as errors have additional repercussions, clients depend on a variety of existing parts of the interface, and the code base is large and hard to understand
- Only certain developers can make changes to the fat interface because the interdependencies and consequences for users of the interface of making changes require in-depth knowledge of the entire code base
- Bugs in the interface can affect clients that do not even use the kind of functionality exposed in the failing method
- A large number of client dependencies on the interface make disentangling the status quo difficult

Overall, while this pattern can seem like a minor code smell at a glance, in fact, it can create serious issues for your companywide integration landscape if you have a failing fat interface in a central position.

Better solutions

This anti-pattern is one of the few that can consistently be avoided by applying good practice and discipline to your development processes. If you diligently follow your interface standards and apply the interface segregation principle whenever you are adding new functionality, this anti-pattern will never occur.

While it may be tempting to take shortcuts, and they might not have serious consequences in the short term, you should know the long-term consequences and apply sound design and programming practice. This is also an area where an architect or developer may have a lot to say as it is too technical an area for most business users to really take a position.

The difficulty is, of course, in maintaining the required level of discipline when you are under serious pressure to deliver. However, hopefully, making the kinds of points raised in this description will help you push back on quick-fix thinking.

Chatty integration

Chatty integration is an anti-pattern where integration of arbitrary patterns makes an unduly large number of calls to one or more interfaces, often resulting in performance degradation.

Example

WoodCo is a furniture maker with a long legacy of making top-tier bespoke furniture for well-off clients. They have been growing rapidly over the past years due to the launch of an e-commerce platform, built on Salesforce Experience Cloud with B2B commerce, that connects their customer community directly with furniture makers assigned to their projects.

That way, customers and makers can connect directly and discuss requirements for the bespoke builds. Customers can also follow the progress of their furniture throughout its life cycle.

WoodCo project manages the builds, including the budgets, and ensures that any conflicts are resolved amicably wherever possible. They track these projects in an old project control module that they also use to manage their own business.

This project module has recently been extended with a custom-built REST API that sits on top of the legacy application. The vendor has built this API specifically for WoodCo at great expense, but it's considered a no-brainer as it will allow the direct integration of the portal into the project control module, replacing the current manual process where everything is rekeyed into the project control module by data entry professionals.

Lina is hired by WoodCo to head up digital projects, the first of which is to connect the e-commerce platform to the project control module. She commissions a specialist Salesforce partner to lead the work.

They put in place a lightweight middleware platform that subscribes to events from Salesforce and translates them into REST calls in the format of the new REST API. The REST API doesn't support any aggregation, so it's strictly one event to one call.

On the Salesforce side, events are initially only fired on status updates or when key pieces of standing information, such as a project title, are entered or changed. However, over time, this should be extended to tracking the status of activities within the project and the communication between maker and customer.

You can see an overview of the WoodCo integration architecture [here](#):

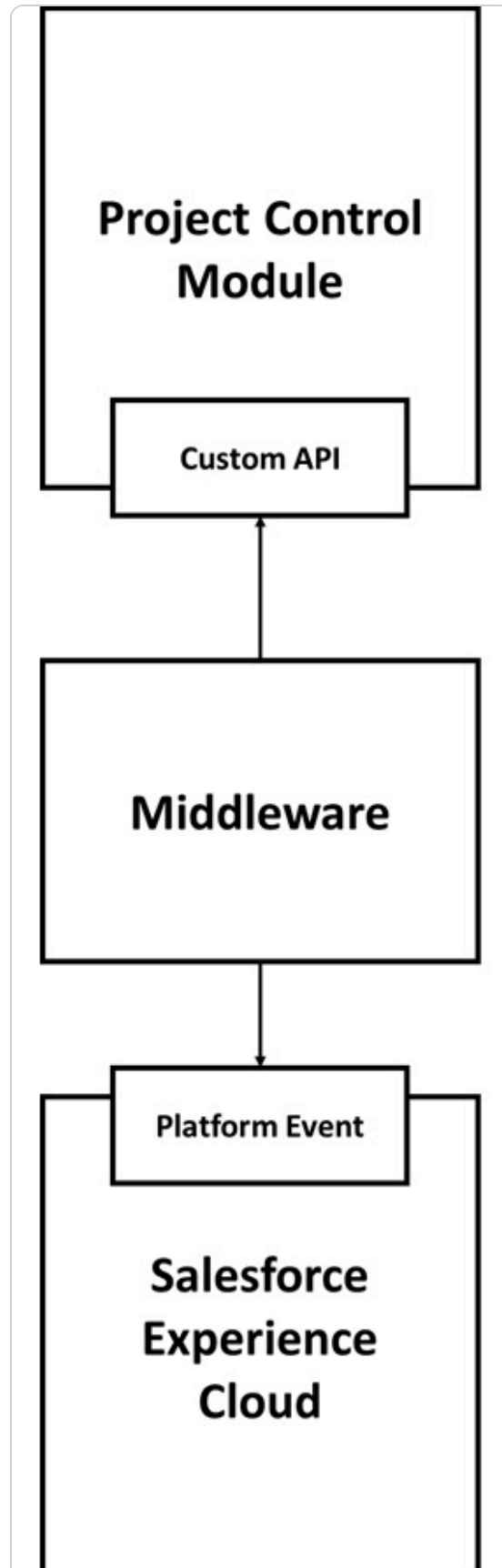


Figure 6.6 – WoodCo integration architecture

The integration launches successfully, and everyone in WoodCo cheers. However, the full benefit is not yet realized as the activities and customer communication are still only in the customer portal, and WoodCo staff still use a combination of rekeying and working in multiple systems to get around the constraint.

Finally, the day of the update, which will include the full synchronization, arrives and everybody waits excitedly for the go-live. However, within minutes of the launch, the project module crashes.

The vendor investigates and attributes the error to scalability issues related to the initial synchronization. The rollout is deferred to the weekend, and after many restarts, the initial data is indeed synchronized.

Monday comes, and people start using the system in anger. There are some complaints about slowness in the project control module, but at least the integration seems to be working and bringing the critical data across.

Then, on Tuesday morning, an administrator is running some routine batch jobs on the customer portal. Near instantly, the project control module comes crashing down again and the new integration is taken offline in order to get it back up.

A crisis meeting is called for that evening by WoodCo's CIO. This has reached the level of CEO attention, and something needs to be done. Lina entrusts Aki, her most technically gifted subordinate, to figure out the root cause.

At this point, both vendors are blaming each other for the failure and are digging their heels in, refusing to take unilateral action to fix the problem. Aki, therefore, dives right into the middle of the technical setup.

He examines log files and code from Salesforce, the middleware, and the project control module, and after a few hours, he is confident that he has found the root cause. The problem is caused by a recursive update rule that applies certain updates to a parent project, such as a change of billing code, to all activities in that project's scope.

Before the new update, this didn't matter as changes on the activity level weren't synchronized. However, with the update, each such change triggers hundreds of update events on Salesforce, each of which triggers a callout to the project control module.

You can see an overview of the WoodCo project structure here:

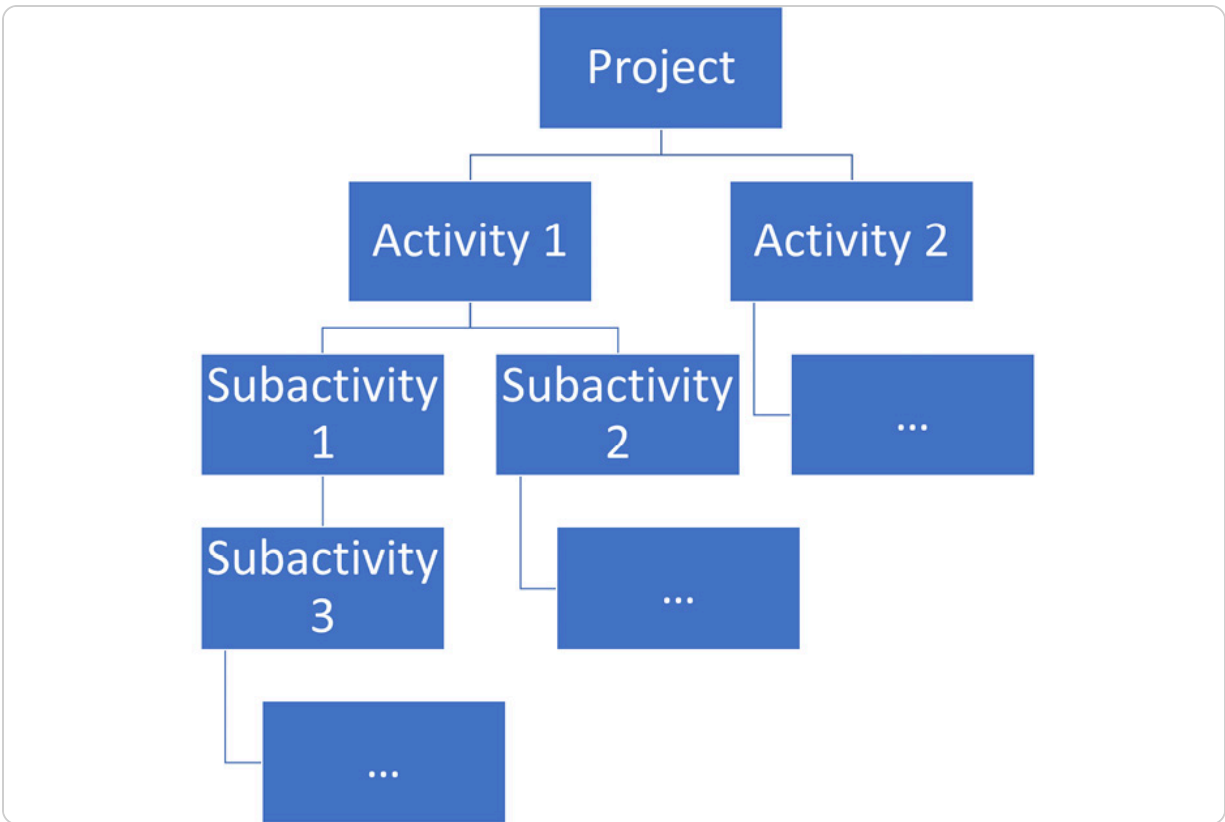


Figure 6.7 – WoodCo project structure

This legacy system can handle maybe one or two such updates at the same time while also serving users normally, but anything more than that causes issues—first, performance degradation, and eventually, a crash. When the administrator launched a batch job to reassign a number of **project identifiers (PIDs)**, this triggered updates for dozens of projects and their activity trees, crashing the project control module in the process.

At the evening's meeting, the mood is somber. However, as Aki explains the facts of the matter, no one can really disagree. The decision is taken to temporarily disable the activity level and for Aki to lead a team to redesign the integration so that it avoids overloading the project control module.

Problem

The chatty integration anti-pattern is a byproduct of the solution to some other integration problem that for some reason requires very frequent communication between systems. Often, as in our example, that problem involves the transfer of state between two or more systems.

It is an anti-pattern that can be hard to spot during development as it tends to only become problematic with scale. After all, in most test scenarios, except explicit performance tests, we don't update enough data to really reach a problematic number of integration calls.

The problematic implementation can be due to bad practices, such as firing separate callouts for every record in a loop, or it can be more insidious, as in our example, where subscribed events are mapped 1:1 to REST calls in the middleware. Whichever way it occurs, it is at best wasteful and at worst catastrophic.

Proposed solution

As noted, chatty integration tends to be a byproduct of the solution to another problem; therefore, it doesn't quite fit our schema. However, in so far as we can say it proposes a solution, that solution is to make as many calls across system boundaries as necessary to support the business use case without any concern for system limitations.

This is usually done for reasons of simplicity. Once you start introducing bulkification, queuing, systematic delays, aggregation of calls, throttling, or any other mechanism you might consider to limit the rate of calling other interfaces, you also introduce complexity in the implementation.

You will have noted from other anti-patterns that complexity is often a driver of serious negative consequences, so avoiding it will usually seem like a good thing. This, however, is a case where the adage *make the solution as simple as possible but no simpler* applies.

With chatty integration, you are actually making the solution too simple as it doesn't meet the basic functional requirement without the additional complexity. That may mean you need more time and additional tools to get your solution to work, but there really is no way around it in this kind of scenario.

This is still true even when using low-code integration tools, sold to make your life easy. If you get the integration strategy wrong, the integration won't work as intended. Architecture is about trade-offs, after all.

Results

The results of chatty integration form a spectrum:

- Often, there are no immediate consequences if the target systems are able to cope with the extraordinary volume of calls and you stay within Salesforce limits as well.
- Sometimes, you see performance degradation. That can happen on the Salesforce side if you make too many async callouts over a time period, and it can obviously happen on the target side if you start overloading its capacity.
- Performance degradation can turn into periodic errors if you start experiencing timeouts or the target servers get temporarily overloaded.
- Finally, you can sometimes crash the target system altogether, leading to a critical error.

You can refer to the following diagram for a visual demonstration:

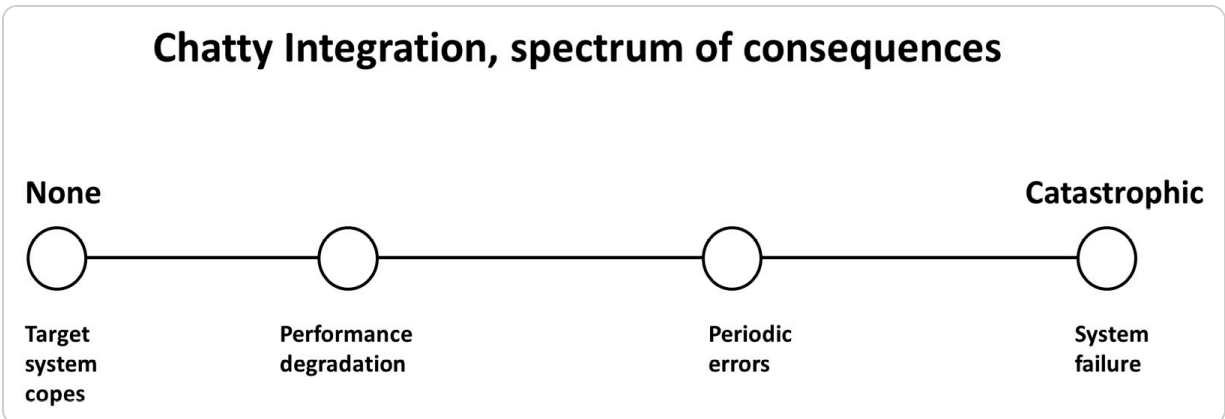


Figure 6.8 – Chatty integration spectrum

That doesn't mean this isn't an anti-pattern if you happen to be lucky enough to be on the left side of the spectrum. It just means that for now, you have gotten away with it.

Better solutions

The general advice to follow in order to avoid the chatty integration anti-pattern is to understand and design with system limits in mind. We are not creating theoretical architectures for theoretical systems. If we were, we would be in academia.

When you design an integration as an architect, you are generally doing so with a well-defined target environment in mind. That means you should be mindful of the following:

- Understand the hard constraints, such as system limits—for example, the maximum number of calls supported over a time period
- Get information on the actual performance of target systems under whatever level of duress they currently experience
- Don't just fire off calls without consideration of the performance implications of doing so

To mitigate this anti-pattern, organizations can leverage Salesforce's bulk API to efficiently handle large volumes of records and the composite API to make multiple related calls in a single request. This approach minimizes the number of separate transactions and improves overall performance while ensuring data integrity and reducing potential errors.

Instead, use one or more of the following strategies to ensure that the target system can cope:

- **Bulkification** : Send multiple logical calls in the same message.
- **Consolidation** : Combine multiple updates in a single call.
- **Aggregation** : Combine changes to multiple areas into one call covering several areas.
- **Delay** : Introduce a delay in sending off a call when performance is spiky.
- **Reduce frequency** : Send updates less often.
- **Buffer** : Add your calls to a queue that is gradually drained as the target system has capacity.

- **Event** : Consider using Salesforce's Streaming API to reduce the number of callouts and implement a more event-driven architecture. By leveraging the Streaming API, you can listen for changes to Salesforce records in real time, allowing your applications to react swiftly to data changes without the need for frequent polling. This approach enhances efficiency and responsiveness in data handling, making it a valuable integration for modern applications.

Overall, a chatty integration has the potential to level your integration landscape if you are unlucky. Therefore, it should be avoided even when you don't believe you will run into trouble in the short term.

Getting the patterns wrong

In this section, we look at how becoming too obsessed with a single integration style can cause serious problems by looking at the integration pattern monomania anti-pattern.

Integration pattern monomania

Integration pattern monomania happens when a single integration pattern is used to the exclusion of all others, regardless of whether it is a good fit for the requirements under consideration.

Example

WineCo has been a heavy user of Salesforce technology for more than a decade. It has a large estate of many orgs across core clouds: Marketing Cloud and Commerce Cloud.

They have developed a custom approach to building on Salesforce, including a range of frameworks and libraries that they use consistently across projects. Many of these could use an update, but they are still preferred for the sake of consistency.

Clare is brought in from a leading consultancy to lead the build of a new app that WineCo is building for its distributors. The app will include communications between WineCo managers and the distributors, e-commerce for standard items, rebate management, custom pricing logic, joint opportunity management, and the ability to request quotes for special requirements.

To deliver the required level of functionality and realize the business value attributed to the distributor portal, Clare will need to ensure that integrations are in place to many systems.

She must funnel all communications through the central notification service that ensures all communications are appropriately logged and have the right footers in place. Then, she must integrate with the existing rebate management system that calculates the rebates due to distributors based on their segment and historical orders.

Pricing will come from Salesforce CPQ but in a different org, requiring a Salesforce-to-Salesforce integration. Content for the portal web pages will come from the corporate CMS. However, there are also services for user and behavior tracking that must be integrated from the CMS into the portal website.

Finally, the ERP system needs to be updated with any custom quotes and standard orders that are made through the distributor portal. The distributor portal will also need to get a substantial amount of data about distributors, their existing orders, and standing data about them from the ERP system. All in all, while the basic build on Salesforce is complex, the integration landscape is even more complex.

The proposed architecture is represented in the following diagram:

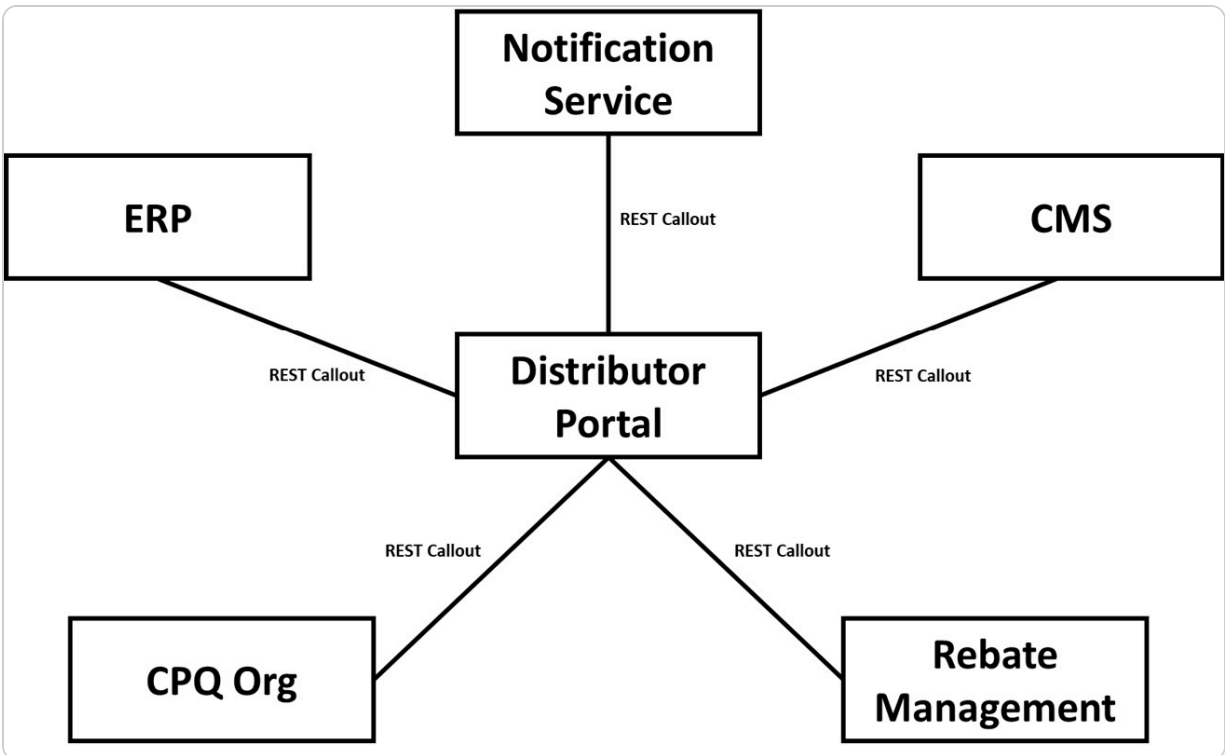


Figure 6.9 – Proposed integration architecture for WineCo

Further complexity is added by WineCo's corporate integration standard. It states that all integrations from Salesforce must be implemented using the company's integration library. That would be fine; however, the only patterns supported by this library are synchronous or asynchronous REST callouts, limiting the options for the integration design considerably.

Clare and her team get on with it, regardless. They complete much of the core Salesforce build ahead of schedule, but the integrations are lagging behind. They run into several different problems.

First, the rebate system doesn't support any kind of bulkification, leading to large numbers of calls to its APIs. The system owner questions whether the current setup will scale in a go-live scenario, and eventually, four new servers need to be procured to give the business comfort that the system can handle the load.

Second, there are too many queries to the ERP system, leading to slow response time and long wait times in the UI. This proves unsolvable under the current architecture, and a decision is made to handle it as a training issue.

Third, the CMS services prove hard to call from the server side, and a compromise is made to include the tracking code via JavaScript, although this requires a formal exception and causes substantial delay.

Finally, the notification service is a bad fit for many of the messaging requirements that the distributor portal has and results in weirdly formatted messages that are hard to reply to in many cases. That is also accepted and consigned to be handled as a training issue.

The project is completed and goes live without much fanfare. The internal users dislike the new interface and distributor adoption is lukewarm. Clare makes a final argument for redesigning the basic integration architecture to something more suitable, before moving on to the next project.

Problem

Integration pattern monomania seeks to address the problem that designing a good integration architecture is fundamentally hard, as is determining the right patterns, design, and implementation characteristics of concrete integrations.

It is, therefore, attempting to simplify the problem by focusing on a single approach that works well in many cases. That way, you can define a standard way of doing things without having to grapple with the exact details of each individual case.

This is attractive for several reasons:

- First, standards often work. In many cases, having a standard way of doing things is the right way to go. However, an entire integration architecture is too broad a target for such an approach.
- Second, once you have chosen an approach, you can create supporting assets, such as code libraries and frameworks for logging and error handling, that will work across integrations, which is generally beneficial.
- Third, you only really need developers to know how to do a single thing. That reduces development complexity, training, and onboarding needs.

All of that would be great if it weren't for the inconvenient fact that no single integration pattern is universally applicable.

Proposed solution

Integration pattern monomania proposes to use a single integration pattern for all—or at least, nearly all—concrete integrations needed in an integration architecture. It can either be explicitly set via a corporate standard or be implicit in the way the architects and developers work and think. In either case, when such a preference becomes excessively strong, you have an anti-pattern.

The aims of integration pattern monomania are usually quite sensible:

- Reducing the complexity of integrations by limiting the choices architects and developers need to make
- Enforcing consistency in the enterprise architecture to avoid unwanted side effects
- Make developers' lives easier by giving them a clear way forward and supporting them with relevant tools and frameworks

The issue is that not all concrete integrations will fit a given pattern. Characteristics such as the velocity of updates, acceptable latency, data volumes, and the needs of the **user experience (UX)**, among other concerns, mean that overreliance on a single integration pattern is detrimental in the long term. We will explore this next.

Results

The problems caused by integration pattern monomania boil down to technical misfits. This is something we have seen in other anti-patterns—for instance, Golden Hammer. While some positives may apply in specific cases, overall you are likely to see some of the following consequences:

- Instead of decreasing complexity as intended, your integration landscape becomes more complex due to a variety of workarounds needed to accommodate the limitations of the integration pattern that is being used exclusively.
- Some integrations may not function within their intended quality parameters because of poor technical fit. That may mean periodic errors, poor performance, or similar.
- These factors generally imply a burden of higher maintenance and support on an ongoing basis to correct the issues.
- Finally, the UX in some areas will be underwhelming to end users as the pattern can't meet expectations.

Overall, you don't get the anticipated benefits and instead end up with a bit of a mess.

Better solutions

The solution to integration pattern monomania is simple. Don't get overly fixated on a single integration pattern, whether that is RESTful callouts, EDA, or batch transfers.

Instead, you can give good guidance to application developers and architects about which patterns are appropriate in which circumstances. It is fine to have a preference where all else is equal, but in practice, things rarely are.

Here's an overview of Salesforce integration patterns:

Integration from Salesforce		
Type	Timing	Preferred Pattern
Process Integration	Synchronous	Remote Process Invocation—Request and Reply
Process Integration	Asynchronous	Remote Process Invocation—Fire and Forget
Data Integration	Synchronous	Remote Process Invocation—Request and Reply
Data Integration	Asynchronous	UI Update Based on Data Changes
Virtual Integration	Synchronous	Data Virtualization

Integration to Salesforce		
Type	Timing	Preferred
Process Integration	Synchronous	Remote Call-In
Process Integration	Asynchronous	Remote Call-In
Data Integration	Synchronous	Remote Call-In
Data Integration	Asynchronous	Batch Data Synchronization

Figure 6.10 – Overview of Salesforce integration patterns

A certain level of conservatism toward choosing integration patterns can be warranted. That can give you some of the benefits that come from having a consistent approach, such as leveraging standard frameworks and libraries.

You can also use middleware to create a certain commonality of integration stance between systems, although that just moves the complexity onto another platform. However, when push comes to shove, if a certain approach is the right one for your integrations, then you should use it.

Event hell

When event based architecture goes bad, it results in this anti-pattern, involving failing scalability and control.

Example

SalesCo, a mid-sized company specializing in custom manufacturing equipment, has been using Salesforce as their CRM for several years. Their implementation has grown organically, starting with basic lead and opportunity management and gradually expanding to include custom objects for managing complex product configurations, quotes, and order fulfillment.

As the business grows, SalesCo's leadership decides to modernize their approach to system integration. They have heard about the benefits of EDA and want to leverage Salesforce's event capabilities to create a more responsive and integrated system landscape.

Sarah, a senior Salesforce developer at SalesCo, is tasked with implementing this new event-driven approach. She is excited about the potential of using platform events to decouple various processes and create a more flexible system.

Sarah starts by identifying key business processes that could benefit from an event-driven approach. She creates platform events for various stages of the sales process, product configuration updates, quote generation, and order fulfillment. The idea is to have different parts of the system react to these events, rather than having tightly coupled, synchronous processes.

For example, when a sales representative marks an opportunity as `Closed Won`, an **OpportunityClosedEvent** would be fired. This event would trigger several actions:

1. The order fulfillment team would receive a notification to start processing the order.
2. The finance system would be notified to generate an invoice.
3. The inventory management system would reserve the required products.
4. The customer success team would be alerted to initiate onboarding.

Similarly, when a product configuration is updated, a **ProductConfigChangedEvent** would be emitted, triggering recalculations of quotes, updates to related opportunities, and notifications to relevant sales representatives.

Initially, this approach seems to work well. The system becomes more responsive, and different teams can work more independently. Encouraged by this success, Sarah and her team continue to add more events and event-driven processes.

However, as the number of events and event handlers grows, problems begin to emerge:

- Debugging becomes increasingly difficult. When issues arise, it is challenging to trace the exact sequence of events and handlers that led to the problem.
- Performance starts to degrade. With multiple handlers firing for each event, and some handlers triggering additional events, the system begins to slow down, especially during peak usage times.
- Data inconsistencies start to appear. With multiple processes updating related records based on events, it becomes hard to maintain data integrity.
- Development velocity decreases. As the event-driven system grows more complex, even small changes require careful consideration of potential ripple effects across the entire system.
- Onboarding new team members becomes challenging, as understanding the full system behavior requires knowledge of numerous events and their handlers spread across different parts of the code base.

Despite these issues, the SalesCo team is reluctant to move away from their event-driven approach. They have invested significant time and resources into this architecture, and many of their processes now rely on it. Sarah finds herself in a difficult position, trying to maintain and extend a system that has become unwieldy and hard to manage.

Problem

The event hell anti-pattern occurs when a system becomes overly reliant on events for communication between components, leading to a complex and difficult-to-maintain code base. This anti-pattern is particularly prevalent in systems that embrace EDA without fully understanding its implications or establishing proper guidelines for its use.

In the context of Salesforce, event hell often manifests through the overuse of platform events, the Streaming API, and trigger-based automations. While these technologies offer powerful capabilities for building responsive and decoupled systems, their misuse can lead to a tangled web of event emissions and handlers that obscures the system's behavior and makes it challenging to maintain.

Key characteristics of event hell include the following:

- **Excessive event usage** : Events are used for nearly all inter-component communication, even for simple, synchronous operations that don't require the complexity of event-driven design.
- **Complex event flows** : As the number of events and handlers increases, the flow of control becomes increasingly difficult to follow. A single user action might trigger a cascade of events, making it hard to predict or understand the full impact of any given operation.
- **Lack of clear ownership** : In an event hell scenario, it often becomes unclear which component or process is responsible for specific data or functionality. Multiple components may emit or react to the same events, leading to confusion about where certain operations actually occur.
- **Debugging difficulties** : Tracing the execution path in an event hell scenario is extremely challenging. When an issue arises, developers may need to sift through numerous event handlers across multiple components to understand what's happening.
- **Performance issues** : The constant emission and handling of events can lead to performance degradation, especially in systems with high transaction volumes.
- **Data inconsistencies** : With multiple processes updating related records based on events, maintaining data integrity becomes increasingly difficult.
- **Testing challenges** : Event hell makes unit testing particularly difficult, as components often rely on complex event interactions that are hard to mock or simulate in isolation.

In the Salesforce ecosystem, event hell can be particularly problematic due to the platform's governor limits. Excessive event publishing and handling can quickly consume CPU time, SOQL query limits, and DML operations, leading to system timeouts and failures.

Proposed solution

The **event hell anti-pattern** often emerges as a proposed solution to legitimate architectural challenges. It promises to deliver a highly decoupled, responsive, and scalable system by leveraging the power of EDA. The allure of this approach is strong, particularly in the Salesforce ecosystem, where platform events and the Streaming API offer powerful capabilities for building event-driven systems.

Proponents of this approach argue that by using events for most inter-component communication, you can achieve the following:

- **Loose coupling** : Components only need to know about the events they publish or consume, not about other components in the system
- **Scalability** : Event-driven systems can theoretically handle increased load by adding more event consumers
- **Adaptability** : New functionality can be added by creating new event consumers without modifying existing code
- **Real-time responsiveness** : Events allow for immediate reactions to changes in the system state
- **Integration simplification** : Events can serve as a universal language for integrating diverse systems and services

In the context of Salesforce, the proposed solution often involves the following:

- Using platform events for most inter-object and inter-process communication
- Implementing numerous Apex triggers to publish events in response to record changes
- Creating multiple Apex classes to subscribe to and handle various events
- Utilizing Process Builder or Flow to react to events and update records
- Integrating external systems primarily through event publication and consumption

This approach seems to solve many common architectural challenges. It appears to reduce direct dependencies between different parts of the system, allow for easy extension of functionality, and provide a clear mechanism for real-time updates and integrations.

However, as we'll see in the next section, this solution often creates more problems than it solves when applied indiscriminately.

Results

When the event hell anti-pattern takes hold, the result is a system that, despite its promise of flexibility and decoupling, becomes increasingly brittle, hard to understand, and difficult to maintain. The consequences of event hell in a Salesforce implementation can be severe:

- **Reduced system stability** : As the web of events and handlers grows, the system becomes more prone to unexpected behavior. A small change in one part of the system can have far-reaching and unforeseen consequences, leading to increased bug rates and system instability.
- **Performance degradation** : The constant publishing and handling of events can put a significant strain on system resources. In Salesforce, this can quickly lead to governor limit exceptions, especially in high-volume scenarios. Users may experience slower response times and more frequent timeout errors.
- **Debugging nightmares** : When issues occur, tracking down the root cause becomes extremely challenging. Developers must trace through complex chains of event publications and handlers, often spread across multiple objects and classes. This significantly increases the time and effort required to resolve problems.
- **Data integrity issues** : With multiple processes updating related records based on events, maintaining data consistency becomes a major challenge. Race conditions can occur when multiple event handlers attempt to update the same records simultaneously,

leading to data conflicts and integrity problems.

- **Reduced developer productivity** : As the system complexity increases, even small changes or feature additions require careful consideration of potential ripple effects across the entire event-driven landscape. This slows down development and increases the risk associated with each change.
- **Onboarding difficulties** : New team members struggle to understand the system's behavior, as the logic is distributed across numerous event handlers rather than being centralized in more traditional service layers or controllers.
- **Testing challenges** : Writing comprehensive unit tests becomes extremely difficult, as individual components are tightly coupled to the event system. Integration tests become more critical but are also more complex to set up and maintain.
- **Scalability issues** : Ironically, while EDAs are often adopted for their perceived scalability benefits, event hell can lead to the opposite result. As the volume of events increases, the system may struggle to keep up, leading to event backlogs and processing delays.
- **Increased operational costs** : The complexity introduced by event hell often requires more powerful (and expensive) computing resources to maintain acceptable performance levels. It may also necessitate more sophisticated monitoring and alerting systems to track the flow of events and identify issues.
- **Limited visibility** : Understanding the current state of the system becomes challenging, as important business logic and data transformations happen as side effects of events rather than through more visible and traceable direct operations.
- **Integration complexity** : While events can simplify some aspects of integration, event hell can make it harder to maintain a clear contract between systems. Changes to event structures or handling logic can have unexpected impacts on integrated systems.
- **Governance challenges** : In regulated industries, event hell can make it difficult to provide clear audit trails and demonstrate compliance, as the sequence of operations that led to a particular system state can be hard to reconstruct.

In the Salesforce context, these issues are often exacerbated by platform-specific limitations. For example, the inability to control the order of trigger execution can lead to race conditions between event publishers and handlers. Similarly, the platform's governor limits can be quickly exceeded in complex event-driven scenarios, leading to runtime errors and incomplete transaction processing.

Better solutions

To avoid falling into the event hell anti-pattern while still leveraging the benefits of EDA in Salesforce, consider the following strategies:

- Reserve events for genuinely asynchronous or decoupled operations. Not every state change or business process needs to be event-driven. Use direct method calls or synchronous operations for simple, linear processes.
- Establish well-defined domains within your Salesforce org and use events primarily for cross-domain communication. Within a domain, prefer more direct forms of interaction.
- Put thought into your event structures. Use versioned events and include only the necessary information in each event. This can help manage system evolution and reduce the coupling between event producers and consumers.
- For complex domains where you need a complete history of state changes, consider implementing event sourcing. This pattern can provide better traceability and make it easier to reason about the system's state.
- Use platform event triggers to centralize event-handling logic, making it easier to manage and debug compared to having multiple separate handler classes.

- Use circuit breaker patterns to prevent event handling from overwhelming the system during high-load situations or when experiencing downstream system failures.
- For complex domains, consider separating read and write models using **Command Query Responsibility Segregation (CQRS)**. This can simplify your event-handling logic and improve performance.
- Accept that in some cases, eventual consistency is sufficient. This can help reduce the complexity of your event-handling logic and improve system performance.
- Implement consistent error handling and dead-letter queues for event processing. This can help identify and resolve issues more quickly.
- Keep clear, up-to-date documentation of your event schemas, producers, and consumers. Consider using tools to generate visual representations of your event flows.
- Use Salesforce's event monitoring features and consider implementing additional logging to track event publication and consumption. This can help identify performance issues and aid in debugging.
- Use event-storming sessions with your team to model complex business processes before implementing them. This can help identify appropriate boundaries and event usage.
- For some scenarios, other integration patterns, such as request-response or publish-subscribe, might be more appropriate than a purely event-driven approach.
- For complex event handling that may approach governor limits, consider using Queueable Apex to break the processing into manageable chunks.
- Use Salesforce's platform cache to store frequently accessed data that's derived from events, reducing the need for repeated complex event processing.
- Use feature flags to gradually roll out event-driven features and to quickly disable problematic event flows if issues arise in production.

By applying these strategies, you can create a more maintainable and performant event-driven system in Salesforce, avoiding the pitfalls of event hell while still reaping the benefits of reactive, decoupled architectures. Remember, the goal is to use events as a powerful tool in your architectural toolkit, not as a one-size-fits-all solution to every integration and communication challenge.

We will now take a slight detour to look at the general impact of new AI technologies on integration anti-patterns.

AI's impact on integrations

As AI continues to evolve and permeate various aspects of software development and system integrations, it's crucial to consider both the potential benefits and risks it introduces. This section explores how AI might influence integration practices, potentially giving rise to new anti-patterns while also offering opportunities to avoid existing ones.

The integration landscape is on the cusp of a significant transformation as AI technologies become more prevalent. While the promise of AI-driven integrations is enticing, it's important to approach

this new frontier with a critical eye. As with any technological advancement, the introduction of AI into integration practices may give rise to new anti-patterns that could potentially undermine the very efficiencies they aim to create.

One potential anti-pattern that may emerge is what we might call *AI overreliance syndrome*. This occurs when organizations become excessively dependent on AI-powered integration tools, neglecting human oversight and domain expertise. For instance, consider a scenario where a company implements an AI-powered integration platform that promises to automatically map data fields and optimize data flows between Salesforce and their legacy ERP system. Impressed by the initial results, they might reduce their integration team and rely entirely on the AI system. However, the AI could fail to understand nuanced business rules and complex data relationships, leading to critical errors in inventory management and order processing. This overreliance could result in a loss of human expertise and context, potential for systemic errors due to misunderstood business logic, and difficulty in troubleshooting and resolving complex integration issues.

Another potential anti-pattern we might see is *black-box integration*. This could emerge when AI models used in integrations become so complex that they're incomprehensible to human developers and architects. Imagine a company employing a sophisticated machine learning model to manage real-time data synchronization between Salesforce and multiple external systems. The model continuously evolves based on data patterns. Over time, the integration behavior might become unpredictable, and the team could struggle to explain or control certain data transformations. This lack of transparency and auditability could lead to difficulties in maintaining compliance with data regulations and increase the risk of unexpected behavior in critical business processes.

We might also encounter what we could call *training data bias amplification*. This anti-pattern could occur when AI models used in integrations are trained on biased or incomplete datasets, leading to skewed results and potentially discriminatory outcomes. For example, a financial services company might use an AI model to integrate customer data from Salesforce with their credit scoring system. If the model is trained primarily on historical data from urban areas, it might fail to accurately assess creditworthiness for rural customers, leading to unfair loan rejections. This could result in the perpetuation and amplification of existing biases, potential legal and ethical issues, and erosion of trust in AI-driven integration systems.

While these potential anti-patterns paint a cautionary picture, it's important to note that AI also offers significant opportunities to mitigate existing integration anti-patterns. For instance, AI could help prevent the *fat interface* anti-pattern by analyzing usage patterns and suggesting optimal interface structures. It could also address *integration pattern monomania* by suggesting the most appropriate integration patterns based on real-time system behavior and requirements. Moreover, AI models

could help prevent *chatty integration* by predicting potential performance bottlenecks and suggesting optimizations.

To harness the benefits of AI in integrations while avoiding potential pitfalls, a balanced approach is crucial. This might involve combining AI recommendations with human expertise for critical integration decisions, prioritizing AI models and tools that provide clear explanations for their decisions, implementing robust monitoring systems to track AI-driven integration performance, ensuring AI models are trained on diverse, representative datasets to minimize bias, and conducting periodic audits of AI-driven integration systems.

As AI continues to reshape the integration landscape, it's crucial for Salesforce architects and developers to stay informed about both the opportunities and risks it presents. The journey of AI in the realm of integrations is just beginning, and it's likely that new anti-patterns and solutions will emerge as the technology evolves. Staying adaptable, critical, and informed will be key to navigating this exciting yet challenging landscape.

We have now completed our coverage of the patterns in this chapter and will continue to the key takeaways.

Knowing the takeaways

In this section, we will abstract a bit from the specific patterns and instead try to pull out the wider learning points you can use in your day-to-day work as a Salesforce architect or in preparing for the CTA Review Board.

When architecting Salesforce solutions, you should be mindful of the following:

- Middleware can be a great way to create order and improve the structure of your system landscape. However, you can also use it in ways that do more harm than good.
- If all you are doing is replacing point-to-point connections with equivalent one-to-one flows through the middleware, you are probably in anti-pattern territory.
- Custom services can be a great addition to your Salesforce org in certain cases. However, they come with considerable added complexity.
- You should never reflexively add lots of custom services because they seem to be what the business is calling for. Instead, take a step back and look at the big picture of requirements to see what options you have for realizing the specific integration flows required.
- If you are building custom interfaces, whether on Salesforce or on your middleware platform, have appropriate governance in place to avoid ending up with a fat interface that does everything.
- Avoid excessively frequent integration calls if possible.
- If you do need high-frequency state transfer, consider using a dedicated technology such as CDC.
- In general, always design integrations with system limitations explicitly considered. Don't assume an old ERP can handle unlimited calls, for example. Salesforce, of course, also has defined limits.

- Keep a flexible approach to integration architecture. Don't fall in love with a particular style and use it for everything. Just because event-driven microservices are hot, doesn't mean they are right for every scenario. Consider leveraging Salesforce's *Integration Patterns* guide for guidance on selecting appropriate integration patterns.
- Do give good guidance on how to select integration patterns for both members of your team and external partners. Don't assume that they will make the right decisions by themselves.

In preparing for the CTA Review Board, you should be mindful of the following:

- Almost every scenario will have a requirement for middleware. You should know the key capabilities of common platforms such as MuleSoft and be able to talk intelligently about how they fit into the system landscape.
- While you may not see enough integrations to create a problem along the lines of the MINO pattern in the scenario, it is still worth thinking about the specific value the middleware is adding to each of the integration flows you are funneling through it.
- Don't reflexively just funnel every integration through the middleware—there are often exceptions that may require different treatment.
- Suggesting a custom web service is a major piece of customization and would need a strong justification to include in a review board architecture.
- If you need a custom interface, it is more likely that you should be building and exposing via the middleware in most scenarios.
- While you are not likely to face a situation with a fat interface in a review board situation, it is worth considering how you are structuring your integration interfaces and whether it is balanced.
- You need to consider the potential performance implications of the designs you suggest. Often, scenarios will have high-volume requirements that, if specified in an obvious way, will lead to performance issues similar to the chatty integration pattern.
- Be prepared to discuss how DevOps practices can be applied to manage and deploy complex integrations in a Salesforce context.
- Review the key objectives of the CTA Review Board and be ready to articulate how your experience aligns with those objectives.
- Familiarize yourself with the latest Salesforce features and best practices relevant to your project.
- Ensure you have all supporting documentation and artifacts ready for presentation.
- You may get quizzed on integration limits for the Salesforce platform, so it's worth having these memorized for the board.

You should know all the common integration patterns on the Salesforce platform inside out. This includes the decision guidance on when to choose what pattern. You are likely to need several at the board, and you should be able to clearly articulate why you have chosen as you have.

We have now covered the material for this chapter and are ready to proceed. First, however, we will summarize our learning.

Summary

In this chapter, we have reviewed five different anti-patterns that in very different ways can contribute to failing integration architecture. It is worth keeping these in mind.

The integration domain is exceedingly complex and there is scope for getting things wrong at multiple levels. From choosing the wrong integration patterns or misusing your middleware to the technical

details of your concrete implementation, there are anti-patterns that can potentially cause serious problems not only to your project but also at an enterprise architecture level.

This complexity underscores why seasoned architects approach integrations with caution. They are one of the most common causes of project failures, both on Salesforce and in general.

It's worth reiterating that this chapter is the only one where all the anti-patterns apply not just to Salesforce but to all enterprise software systems. Having covered this material, you are hopefully slightly better prepared to tackle the challenges ahead of you in the integration domain.

Having now covered the integration domain, we will continue to talk about anti-patterns that apply to your deployment processes and governance.

Part 3: Process and Communication Anti-Patterns

This part will teach you how to identify and mitigate anti-patterns around process, governance, and communication.

This part has the following chapters:

- [Chapter 7](#), *Keeping the Development Life Cycle from Going Off Track*
- [Chapter 8](#), *Communication without Mistranslation*
- [Chapter 9](#), *Conclusion*

7

Keeping the Development Life Cycle from Going Off Track

In this chapter, we will examine how modern DevOps practices, particularly **continuous integration and continuous deployment (CI/CD)**, relate to various anti-patterns in software development. We will begin by identifying significant process-level anti-patterns, and then discuss how CI/CD influences software packaging. Finally, we will examine a common anti-pattern associated with testing. At the end of the chapter, we will summarize the key takeaways for real-life applications and the CTA Review Board exam.

In this chapter, we're going to cover the following main topics:

- How you can avoid big-picture mistakes in the delivery of your project, such as how you structure releases and deal with key trade-offs
- How to avoid structuring your packages in a way that may be organizationally convenient but architecturally problematic
- How you can avoid falling into the trap of compromising on code and deployment quality when under pressure to deliver

After completing this chapter, you will have understood the ways in which common mistakes affect the development life cycle and related activities and mastered some tools to help you keep that from happening.

Misaligning the process

In this section, we will look at two anti-patterns that in different ways make the development life cycle go off the rails. First, we will look at how big-bang releases can lead to disastrous outcomes in many cases. Second, we will look at project pieism—the disastrous belief that you can avoid making important trade-offs.

Big-Bang Release

The Big-Bang Release anti-pattern places the release of all functionalities in a single event at the end of a long-running project.

Example

RollerCo, a major manufacturer of roller skates and skateboards, is going through a major business transformation centered on creating a digitally enabled business that better meets the needs of today's

buyers. While not the be-all and end-all of the transformation, several system replacement projects are planned as part of the journey to provide more agility on the IT side.

Anandh is leading the project that aims to replace the legacy, home-grown CRM that RollerCo has been using for the past 15 years. The system will be replaced with a combination of Salesforce Sales Cloud and Service Cloud, but due to the high level of customization in the legacy system, it has proven impossible to keep the Salesforce design close to standard.

Another consequence of the high-level customization required is that the entire system will need to go live as a single unit. It won't be possible to release smaller **minimum viable product (MVP)** increments prior to full go-live as that would make life too complex for the customer support staff.

Shortly after the project kicks off, Anandh is informed that the board of directors has OKed an **enterprise resource planning (ERP)** upgrade to SAP S/4HANA. As the CRM and ERP are closely coupled for several processes, that means that not only will Anandh have to contend with an increased integration backlog, but the rollout plans for the two systems will also need to be coordinated so that both can go live at the same time.

It quickly turns out that the added complexity from the ERP upgrade means the original Salesforce implementation plans will have to slip. In particular, the integrations are proving to be more complex than anyone anticipated.

The CIO, after meeting a trusted vendor representative, announces that to address the issues regarding the CRM/ERP integration, RollerCo will invest in a new strategic middleware platform. This will replace the legacy middleware and go-live with the overall CRM/ERP timeline.

While Anandh and his team are busy redesigning all the integrations to fit with the new middleware platform, yet another discovery is made. The team responsible for RollerCo's web shop determines that it will be next to impossible to get it to work with the new CRM/ERP setup.

It would require changing the fundamental architecture of the application's data layer, and the developer who built that years ago is no longer with the company. No one else is able to figure out how to do it, so a new e-commerce application is also added to the overall program backlog. For convenience, the timeline is aligned with the overall CRM/ERP rollout:

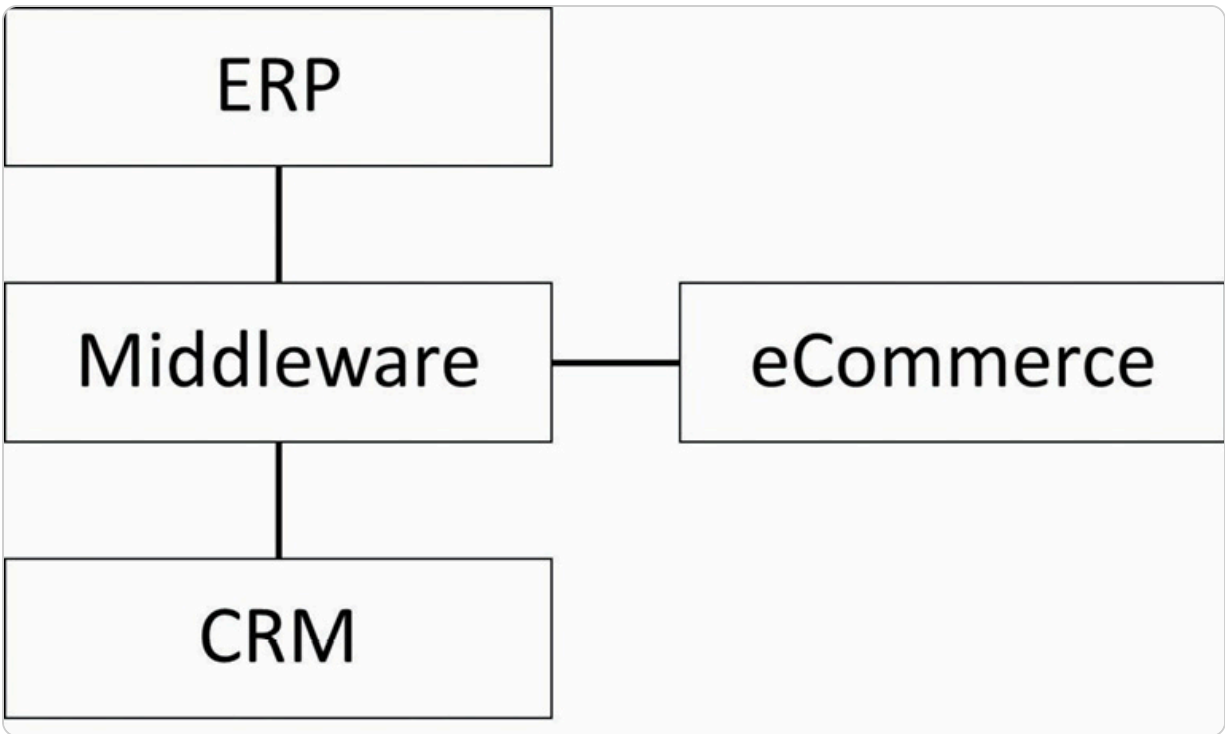


Figure 7.1 – RollerCo's new platform

The program suffers never-ending delays. While some areas suffer more than others, none of the major platforms come out of the implementation phase looking good. Three years go by, and finally, an initial release is nearing completion.

RollerCo has built an elaborate deployment pipeline and cutover plan to be able to handle the multi-system go-live, all slated to happen during a weekend in the company's low season. The testing prior to go-live takes more than three months to complete and requires a number of scope reductions and hotfixes.

Even after that, no one working on the project dares say unequivocally that the system will work as expected. The number of moving parts is simply too large.

The go-live weekend starts promisingly, with the successful completion of required data migration activities. However, as soon as real business users are brought into the systems, issues start cropping up.

By Sunday night, the number of issues has reached a volume that makes going forward seem excessively risky. While no individual issue has so far been a showstopper, the total disruption caused by small issues is significant. On a late-night call, the steering committee makes the decision to roll back and postpone going live.

The fallout is significant. After all, the program has already had three years to complete its work. However, it quickly becomes clear that the issues cannot be fixed in a matter of weeks. There is still much to be done.

The teams come together to find a solution over the next six months. Throughout this period, they focus on reducing the scope and simplifying features to make the rollout process easier to manage.

Eventually, the combined platform with CRM, ERP, middleware, and e-commerce goes live. However, it does so with a large list of known issues and workarounds and a much-reduced scope from the one originally designed.

Problem

Big-Bang Release is an anti-pattern that usually tries to solve the problem of how to handle complex dependencies in the release process. That can be dependencies between systems or inside them. It most frequently occurs where there isn't strong leadership of a digital transformation, and the leadership that there is is highly risk-averse.

Dependencies are usually linked to certain business processes that have a legacy of being carried out in a certain way, which means that a large amount of functionality has to be deployed as a unit. Sometimes, as in our example, this can lead to massive dependencies even at the system level.

Disaggregating existing business processes and coming up with transition architectures (that accommodate the need to get work done while a partial solution is in place) can be quite difficult, as can MVP subsets of functionality that will deliver value on their own. Big-Bang Release avoids those issues by simply pushing everything to one big event in the future.

Proposed solution

The Big-Bang Release anti-pattern proposes to just do one big release for all linked functionality at the end of a long delivery process. That can mean several years of work go live at the same time without any intermediate feedback.

This solution can be highly attractive to both the delivery team and the customer for a variety of reasons:

- It reduces the entire system to a single deployment quantum that can be managed as a unit.
- It simplifies the planning process as you don't have to plan for multiple releases, partial data migrations, and co-existence scenarios. That means the overall architecture is also simplified as there is no need to come up with transition architectures for the scope of the project.
- You also avoid tough decisions about what to include when and which business needs you can accommodate, and at what times.
- It's easy to understand both for the delivery team and the customer.

- You successfully push the problem down the line, making it something to deal with later. In some cases, there may even be other people doing it by then.

You can, therefore, understand why—even at a time when small releases and DevOps thinking have become the leading paradigm—many projects still end up being deployed with a big bang.

Unfortunately, it doesn't tend to go well.

Results

The fundamental problem with the Big-Bang Release anti-pattern is that you—often inadvertently—take on a massive amount of risk. Technical risk doesn't grow linearly with the number of components to deploy. Rather, it grows much faster due to the superlinear growth of interconnections and dependencies that come with a larger number of components.

Put simply, if you deploy one component, you only have the risk of that component failing to deal with. If you have just two interconnected components, you now have up to four failure modes to contend with:

- Component 1 failing independently
- Component 2 failing independently
- Component 1 failing and triggering a subsidiary failure in component 2
- Component 2 failing and triggering a subsidiary failure in component 1

All four may present unique symptoms and the root cause may not be obvious. Consider how many potential failure modes you'd have if you deployed 20 components together with a large number of interconnections between them.

A pipeline in software development is like a factory assembly line for code, where each step automatically flows into the next. Just as a car moves through different stations for assembly, testing, and quality control, code moves through various stages of building, testing, and deployment. DevOps, which combines “development” and “operations,” is a set of practices that brings together software development and IT operations teams to enable faster, more reliable software delivery. Think of it as breaking down the traditional walls between the people who write code and those who maintain the systems where that code runs.

Modern DevOps practices heavily emphasize automated testing and deployment pipelines, which act as safeguards against the risks of “big-bang” deployments (releasing many changes at once). When developers make changes to the code, these changes automatically trigger a series of tests—unit tests check individual components, integration tests verify different parts work together, and system tests ensure the entire application functions correctly. This automation, known as continuous integration, helps teams catch and fix problems early, before they can affect the entire system. Continuous delivery

then takes these verified changes and automatically prepares them for release, ensuring every change is deployment-ready.

The implementation of CI/CD frameworks transforms the traditional approach of infrequent, large releases into a stream of smaller, more manageable updates. Instead of saving up dozens of changes for a major release (which can be risky, like trying to change multiple parts of an engine at once), teams can deploy individual updates as soon as they're ready and verified. This incremental approach makes it easier to identify and fix problems—if something goes wrong, teams can quickly pinpoint which specific change caused the issue and either fix it or roll back that single change, rather than having to undo multiple updates at once. The result is more stable software systems, as teams can monitor each small change's impact and address any issues before they become major problems.

Because the number of failure modes on a large deployment is so large, that means that testing becomes an enormous task, and often, you can test for weeks and still not be sure whether you have really tested all the important cases. Debugging errors is also much harder because tracing the potential knock-on effects between components is much harder than simply finding an error in a single component.

The same difficulties also apply to rolling out and rolling back any other cutover tasks that need to be done. Even training and change management become harder in a big-bang scenario.

Overall, the bigger the release, the more risk there is of it becoming a serious failure. Unless you are a gambler, you shouldn't go down that route.

Better solutions

If big releases are the problem, it stands to reason that small releases are the solution. From a risk management perspective, the ideal amount of functionality to release at a time is a single feature or a single user story. That is the premise of **continuous delivery** (**CD**), as practiced by leading DevOps-focused organizations.

However, we must acknowledge that not all organizations have the scale or technological sophistication to adopt the pure DevOps setup that would allow CD of a single feature at a time. However, if you aspire to the ideal of small releases, at least you will start to mitigate the problems.

Smaller releases have the following attributes:

- Less risky
- Easier to test
- Easier to debug
- Easier to fix errors once found

- Easier to roll out and roll back
- Deliver value quicker to business users
- Facilitate and increase system adoption
- Make change management easier to control

Overall, once you do the necessary technical and business process work to enable smaller releases, there are few—if any—downsides.

In general, aiming for a greater degree of resilience will be helpful in avoiding the problems sketched in this chapter. A good place to start is with the well-architected framework:

<https://architect.salesforce.com/well-architected/adaptable/resilient> .

Project Pieism

Project Pieism fails to contend with key architectural trade-offs, insisting instead that you can have your pie and eat it too.

Example

ConglomoCo is a large, diversified conglomerate that counts many **business units** (**BUs**)—some related to business lines, others to geography. Most BUs operate quite independently both operationally and with respect to IT infrastructure. In addition, BU heads have a lot of power relative to **headquarters** (**HQ**) staff as they are effectively the masters of their own businesses.

When ConglomoCo's CIO decides to push for a global rollout of Salesforce, they are therefore met by staunch resistance on the part of several division heads who have their own CRM strategies and don't want HQ to get involved. However, as part of a new initiative, the CEO and CFO have requested a consolidated view of the global pipeline, which the CIO is leveraging for their Salesforce strategy.

Kim, a senior IT project manager with ConglomoCo's HQ staff, is therefore given the responsibility to drive the global rollout of Salesforce Sales Cloud and CPQ. The CIO wants to use this as an opportunity to standardize the core sales process across BUs and simplify the reporting of sales data.

After corresponding with his architects, Kim, therefore, proposes a plan based on a single org with a standard process. There will be local variations within the BUs, but these are to be kept limited in scope.

As Kim starts presenting this plan to stakeholders, he finds that it is clearly not what they were expecting. The BUs have separate processes, UX expectations, local integrations, and reporting needs that they are expecting a new system to cater to.

At a senior leaders workshop, many of these issues come to light, and a compromise is reached at the highest level. The new system must be able to cater to separate sales processes, automations, and local integrations and accommodate some level of customization of the UX for each BU:

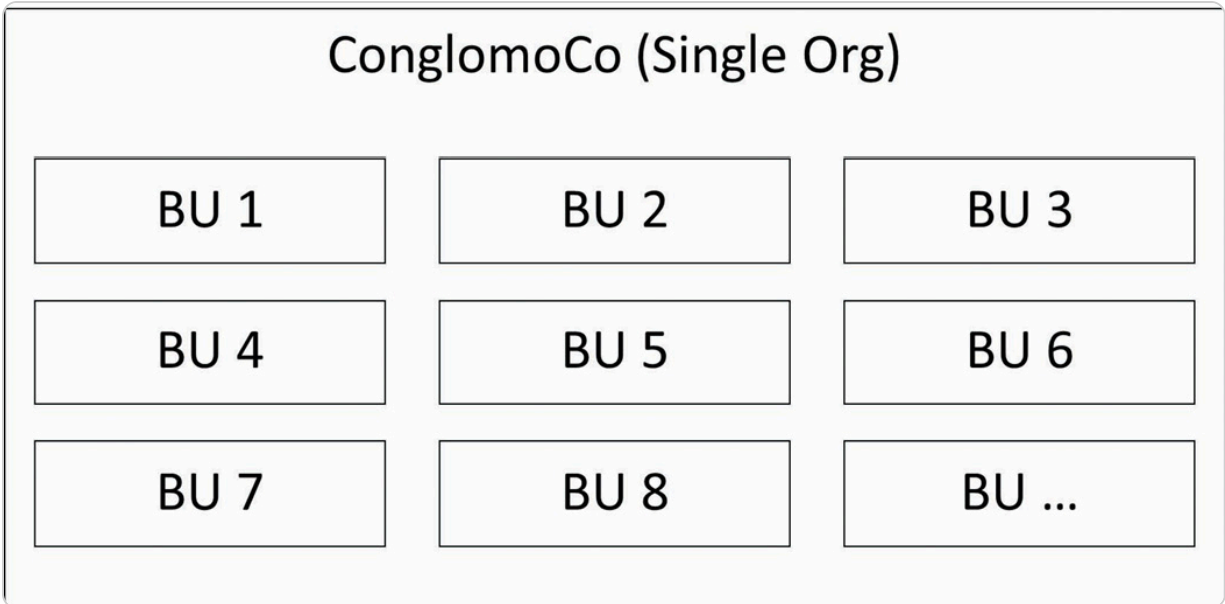


Figure 7.2 – ConglomoCo org

However, all of this must still reside within the same Salesforce org and leverage the same data model to make reporting across BUs simpler. It is understood that this will lead to a more customized implementation and have a cost implication, but the CIO would still like the implementation to be as close to standard as possible.

The implementation is long and slow. By necessity, it includes many separate streams related to the different BUs. Every stream seems to have its own complexities, and coordinating between streams is an ongoing and uphill battle. It is fair to say that as the implementation goes on, a level of attrition sets in, and everybody lowers their expectations for the future system, seeing it more as a necessary evil than a driver of positive change.

Kim negotiates with the business and manages to get two of the smaller areas to go live in a pilot fashion. They go live without much fanfare and with a lot of bugs. Worse than the bugs, however, is the low adoption. The salespeople in the BUs seem to do everything possible to avoid putting data in the system, and when they do, it is of low quality.

Given the unpromising early results, the rest of the rollout is put on hold. Instead, a management consultancy is brought in to investigate why adoption is so poor and what could be done better going forward.

Problem

Project Pieism is an anti-pattern that has its root cause in an aspiration to avoid having to make difficult trade-offs when implementing a system. That may be because of political difficulties or weak project leadership, but it always involves a failure to acknowledge a key trade-off that needs a decision.

In a sense, most engineering problems in software architecture and in general are about balancing a set of trade-offs. However, those trade-offs are often uncomfortable and will involve not pleasing all stakeholders.

Therefore, it's perfectly understandable that many projects proceed without making the necessary trade-offs in the hope that a solution may be found later or simply by closing their eyes to the facts. In any case, it is an anti-pattern.

Proposed solution

Project Pieism suggests as a solution to one or more important trade-offs in your solution that the trade-off is not real, that you don't really have to compromise, and that you can in fact have your pie and eat it too. Sometimes this is done by ignoring the trade-off or sweeping it under the rug; sometimes it is acknowledged but special pleading is deployed to suggest that this case is somehow special and therefore the trade-off does not apply.

This position is attractive for obvious reasons. When there are key trade-offs to be made in an implementation project, you will probably have to disappoint one or more groups of stakeholders. If those groups are powerful within your organization, that may not be an easy or pleasant thing to do. You can even find yourself in situations where you are forced into Project Pieism because of corporate politics.

However, not even the most powerful executive in your organization can change the basic facts of software architecture. There are always trade-offs. Whenever you have a situation on a technical project where there doesn't seem to be a trade-off in the decision you are making, you should be worried.

Results

The result of Project Pieism is the return of the trade-off you've tried to ignore. Murphy's law being what it is, that will probably happen at the most inconvenient point in your project life cycle and cause major damage to your project.

Of course, the specific damage will depend on the trade-off you have been ignoring. In our preceding example, we ignored trade-offs between the priority of global and local processes and the level of standard functionality versus customization.

ConglomoCo attempted to have a standard system with a global process that also accommodated all relevant local variations. That led to a system that no one wanted to use because it met no one's needs well enough.

Other trade-offs will have different consequences, but you can count on adverse consequences if you fail to make important decisions about the key trade-offs that drive your architecture.

Better solutions

The way to a less pieist future lies in openly acknowledging the trade-offs to be made and engaging constructively with stakeholders about the options. Often, there are ways of giving stakeholders more of what they all want, but it may come at an additional cost or a longer timeline.

One way to formalize this engagement is to set up good architectural governance early in a project and discuss the key architectural trade-offs in both architecture forums and business forums to ensure that all perspectives are taken into account. In our example, a competent architecture board might well have pointed out that there was no way of making a single org strategy work well in the ConglomoCo scenario.

As architects, we need to be honest with the business about what is and isn't possible. We are the people who know the art of the possible on the technical side, and we shouldn't pretend that our favorite technologies somehow transcend the need to make hard choices. Stand firm, be constructive, give options, and don't ignore important trade-offs.

Infrequent Check-In

Infrequent Check-In is an anti-pattern where developers work in isolation for extended periods without integrating their changes into the main code base, leading to significant integration challenges, reduced visibility, and potential project delays.

Example

CloudTech, a rapidly growing technology company, has been developing a custom Salesforce solution to manage their complex partner ecosystem. The project involves multiple teams working on different aspects of the solution, including partner onboarding, certification management, and revenue-sharing calculations.

Sarah, a senior developer, is assigned to build the revenue-sharing calculation engine, a critical component that needs to interface with multiple parts of the system. Given the complexity of the calculations and the numerous business rules involved, she decides to work on the complete feature before integrating it with the main code base.

The project manager, Tom, is comfortable with this approach as he believes it will allow Sarah to focus entirely on getting the complex calculations right without being distracted by integration issues. The team's technical lead, Marcus, expresses some concerns about the lengthy isolation period but ultimately agrees, given Sarah's experience and the critical nature of the component.

Sarah spends six weeks working on the calculation engine. During this time, she makes significant progress and regularly updates the team during stand-ups about her work. However, she keeps all changes in her local development environment, believing this is the safest approach until the feature is complete and thoroughly tested.

Meanwhile, other team members continue their work. The partner onboarding team implements several changes to the data model. The certification team adds new validation rules that affect partner status. The integration team modifies several APIs that the calculation engine needs to interact with.

When Sarah finally completes the calculation engine and prepares to integrate it with the main code base, several issues emerge. The data model changes made by the partner onboarding team conflict with Sarah's assumptions about the schema structure. The new validation rules affect how partner status is determined, which is a key input for the calculation engine. The modified APIs require different parameters and return structures than what Sarah's code expects.

The team now faces a crisis. The sprint deadline is approaching, but integrating Sarah's code will require significant refactoring. The code review process becomes overwhelming as team members struggle to understand and validate the massive changeset. Several critical bugs are discovered during testing, but fixing them is complicated by the interconnected nature of the changes.

After three additional weeks of intensive effort to resolve the integration issues, the feature is finally merged into the main code base. However, the delay has impacted other dependent features, and the project timeline needs to be adjusted. The team also discovers that some of Sarah's work duplicated solutions already implemented by other team members during her isolation period.

Problem

The Infrequent Check-In anti-pattern emerges when developers work in isolation for extended periods without integrating their changes into the main code base. This pattern often appears tempting in scenarios involving complex features or critical components, where teams believe isolation will lead to better focus and fewer disruptions.

The anti-pattern manifests itself through developers working for weeks or months without merging code, resulting in large, monolithic changesets that are difficult to review and test. It creates limited visibility into development progress and isolation from evolving code base changes and team decisions.

While this approach might seem logical for complex features, it fundamentally misunderstands the collaborative nature of software development and the importance of CI.

Proposed solution

The Infrequent Check-In approach suggests that complex features are best developed in isolation. Proponents argue that developers can focus entirely on their tasks without worrying about integration issues, keeping their code “clean” and unaffected by ongoing changes until it’s ready. They believe testing can be more thorough when the feature is complete, and code reviews will be more meaningful when examining the complete feature. Project management appears simpler by treating the feature as a single unit of work.

This solution appears attractive because it promises to reduce complexity by avoiding the overhead of CI and dealing with ongoing changes in the code base. It suggests that quality will be higher because developers can focus solely on their specific features without external distractions.

Result

The Infrequent Check-In anti-pattern leads to several significant problems. Integration becomes a major challenge, with massive merge conflicts when finally integrating with the main code base. Significant refactoring is often required to accommodate changes made during the isolation period, and there’s an increased risk of integration errors and bugs.

Visibility suffers throughout the development process. Teams have limited insight into actual development progress, making it difficult to identify potential issues early and reducing the ability to make informed project decisions.

Collaboration issues arise as team members work in isolation. Knowledge sharing and peer learning become limited, and feedback on design decisions and implementation approaches is delayed. This often results in the duplication of effort as different team members solve similar problems without awareness of each other’s work.

Quality impact becomes evident through overwhelming code reviews that may miss critical issues. Comprehensive testing becomes difficult due to large changesets, and there’s an increased likelihood of bugs slipping through due to complex integration scenarios.

Project delays are common, with extended integration periods not accounted for in initial timelines. This creates cascading delays affecting dependent features and additional time needed for resolving conflicts and issues.

Technical debt accumulates through architectural misalignments due to divergent development. Teams often find duplicated solutions and inconsistent approaches, along with difficulty in

maintaining code history and understanding change rationale.

Better solutions

To avoid the Infrequent Check-In anti-pattern, teams should implement true CI. This means establishing automated build and test processes, requiring developers to integrate changes at least daily, setting up monitoring and alerts for integration issues, and using feature toggles to manage incomplete features in the main code base.

Complex features need to be broken down into smaller, more manageable chunks. Teams should plan incremental development with clear integration points, identify dependencies early, and manage them proactively. Clear acceptance criteria for each component help maintain focus while allowing for regular integration.

Clear integration policies are essential. Teams should define the maximum acceptable time between integrations and guidelines for changeset sizes. Code review processes should encourage frequent, smaller reviews, and standards should exist for feature toggle usage.

Communication and visibility must be prioritized through regular design reviews during development and pair programming for complex features. Teams should maintain up-to-date documentation of architectural decisions and hold frequent team sync-ups to discuss technical challenges.

Modern development practices play a crucial role in preventing this anti-pattern. Trunk-based development minimizes long-lived branches, while comprehensive automated testing helps catch issues early. Feature flags manage incomplete features effectively, and proper monitoring and observability ensure teams can quickly identify and resolve issues.

Team culture is fundamental to success. Organizations should encourage knowledge-sharing and collaboration while promoting collective code ownership. Continuous learning and improvement should be supported, and teams should create a safe environment for early feedback.

The key to avoiding this anti-pattern is recognizing that software development is inherently collaborative and iterative. Success comes not from avoiding integration challenges but from making them small enough to manage effectively through frequent integration and clear communication. By implementing these solutions, teams can maintain high quality while avoiding the pitfalls of isolated development.

Unpacking the pipeline

This section will introduce a key DevOps anti-pattern related to how you should structure your packages for package-based development.

Using Packages to Create Silos

Using Packages to Create Silos divides up the packages for your Salesforce project based on team structure without considering architectural concerns.

Example

MillCo, a producer of CNC milling machines with subsidiaries across Europe and North America, is implementing Salesforce for its B2B sales. This includes Sales Cloud, CPQ, and B2B Commerce.

Abigail is the CRM tech lead from MillCo, a role she has recently taken up after leading a CRM implementation project in another B2B manufacturing company. Her principal job is to ensure that the technical delivery from the three different vendors that are implementing the different elements of the new system is consistent and of high quality.

At project kick-off, Abigail invites all three vendor teams along with internal stakeholders and emphasizes many times that while people are working for different companies, everyone should think of themselves as being on the same team. MillCo wants everyone on the same page and working together toward a common goal.

That turns out to have been all for naught. Once work commences for real, the three suppliers are quickly at each other's throats. Abigail finds it impossible to get them to agree to any common approach and standard.

While she can mandate certain things, she has no team behind her, and there is only so much she can do if the vendors don't cooperate. She escalates the issue but is told to find a solution that allows work to continue as planned.

The solution she decides to go with is to isolate each vendor in a separate package. That way, they can build in isolation, except for cases where their work directly clashes with one another on key elements of the platform. Abigail takes it upon herself to monitor and mediate these disputes:

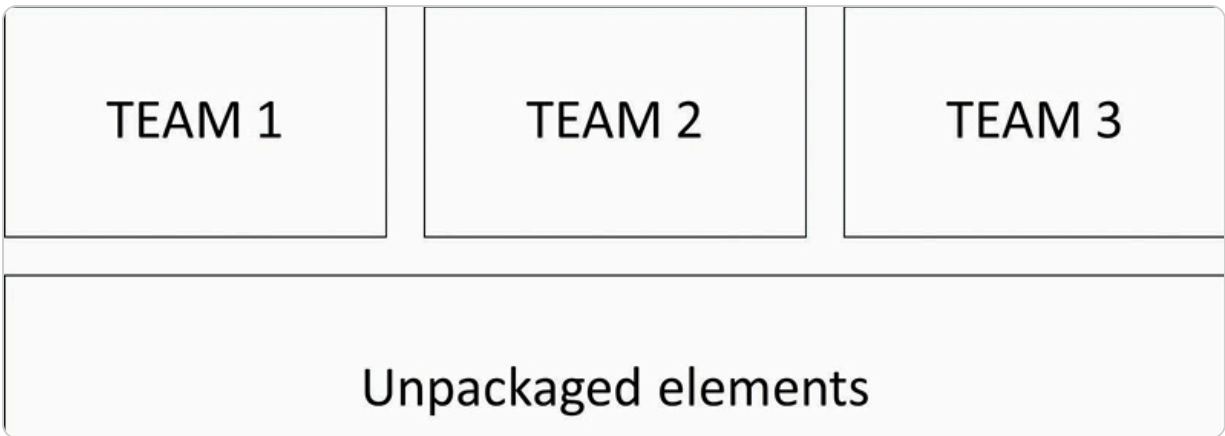


Figure 7.3 – Package structure

She ends up spending most of her time on the project mediating these kinds of inter-vendor conflicts. That also means she has very little time to investigate the general approaches used by the vendors in areas that aren't subject to acrimony.

However, in the end, the system is built. As each vendor has been working in their own environment with only occasional deploys to a thinly tested integration environment, the first step to prepare for **user acceptance testing (UAT)** is to deploy all the packages to a common environment and test the basic processes. Each package has been thoroughly tested in its own environment, but no systematic testing has been done across packages.

Even before the test results start coming in, the situation starts to deteriorate. It turns out that there are incompatible system and package settings between the vendors—for instance, on the CPQ-managed package. This means the deployment stalls until a workaround can be found.

When the testers actually get started, things go from bad to worse. There are many subtle incompatibilities between the packages. For instance, there are different interpretations and uses of common fields, duplicate fields used by different packages for the same purpose, and custom objects that have a partially overlapping scope and the creation of redundant data. There is also a wide range of different automation patterns in use between the three vendors, despite this being an area where Abigail specified the standard upfront.

All in all, the differences are too large to reconcile quickly, and the project has to back off. Abigail's boss negotiates an extension to the timeline in order to refactor the packages and bring them in line. While she is happy about the extension, Abigail does not relish the wrangling it will take to get the vendors to actually do this.

Problem

Sometimes, you want to allow teams to work independently without having to depend on or coordinate with other teams. In a Salesforce context, that may be because the teams are working on different clouds, come from different vendors, or represent different departments or BUs within your organization.

This is very understandable as coordination problems can be hard to resolve. However, when that becomes the basis for dividing up your Salesforce platform into packages, it becomes a DevOps anti-pattern.

Proposed solution

The proposed solution of Using Packages to Create Silos is to give each team its own package to work in, reflecting what it is going to be working on. That avoids coordination issues but creates a host of other problems.

That means you structure the package setup for your project based on organizational needs rather than architecture. That can seem like a good idea at the outset and obviously works for the teams in the short term, but unfortunately means that hidden conflicts are likely and can remain hidden for a long time.

Results

The likely result of Using Packages to Create Silos is that you will have hidden conflicts that only become apparent on the integration of the packages.

These conflicts include cases such as the following:

- Different use of data models—for instance, fields and objects
- Issues in the security model where conflicting configurations are used
- Conflicting flows, validation rules, or other business logic
- Deviation from best practice, for instance, multiple triggers on the same object
- Different automation patterns used on the same object
- Conflicting assumptions about system settings
- Replicated functionality in different packages that is often only partially overlapping

This can lead to serious issues and can be a real mess to refactor, which is both costly and time-consuming.

Better solutions

In general, package design should be taken quite seriously. Package-based development is the way to go for most large-scale Salesforce projects, not least because it scales better than the alternatives.

To avoid falling into the Using Packages to Create Silos anti-pattern, follow these guidelines:

- Design your package structure with your architecture in mind. For instance, in a typical architecture that's split by service lines, you could place common functionality in a "Common" package that all other packages depend on and then have separate packages for each service line containing relevant components. You might also have higher-level packages cutting across service lines for things such as common integrations and you could have separate packages split out for cross-cutting functionality, although that can be a bit tricky to manage. The point is that you should ensure that you manage dependencies cleanly and don't have potentially conflicting overlaps.
- Be careful with dependencies; a layered approach can work well to only have dependencies go in one direction.
- Have teams work across packages when needed.
- Consider having certain teams own certain packages and others request work from them if they need anything.
- Coordinate via technical governance forums—for example, an architecture board.

In modern DevOps workflows, leveraging tools such as the Salesforce CLI is essential for effective source tracking and version control. The Salesforce CLI streamlines the integration process, allowing developers to easily manage and deploy changes across environments. By utilizing source-tracking capabilities, teams can ensure consistency and accuracy in their development processes, enhancing collaboration and reducing errors. This approach enables the efficient management of metadata and code changes, aligning with best practices in agile development.

Salesforce DX unlocked packages are a crucial component of modern development workflows in the Salesforce ecosystem. They enable developers to create modular and reusable components, facilitating code organization and improving team collaboration. With the ability to manage package versions and dependencies effectively, teams can streamline their CI/CD processes. Unlocked packages also support a more granular approach to source tracking, allowing teams to work on different features without affecting the entire code base. By utilizing unlocked packages, organizations can enhance their development practices, making them more agile and responsive to changes in business requirements.

In general, you don't want to mistake package design for something else. Build your package structure so that it makes sense for your architecture and is workable for your developers.

Testing without testing

Testing is a critical activity within any large software project. In this section, we will see how trying to avoid writing the necessary unit tests can cause major issues in the mid-to-long term.

Dummy Unit Tests

The Dummy Unit Tests anti-pattern gets around the built-in requirement for unit testing in Salesforce by writing dummy tests that optimize for coverage but do nothing .

Example

TruckCo is a start-up automotive company that focuses on building next-generation EV trucks. It sees itself as more of a technology company than a manufacturing company and therefore invests heavily in top-range solutions to keep its technological edge.

Generally, TruckCo tends toward heavy customization and intense agile projects with long hours and short timelines. The supply chain application TruckCo is currently building on Salesforce is no exception to this rule.

The application will integrate their B2B commerce portal with order management and their supply chain backend systems. It will enable **just-in-time (JIT)** ordering from spare parts vendors and in general significantly reduce the **quote-to-cash (QTC)** process for spare part orders.

In order to achieve these aims, TruckCo plans to integrate half a dozen key backend systems with Salesforce, build a custom **user interface (UI)** optimized for their special processes, and build a set of cross-platform, AI-enabled automations that drive efficiencies in the ordering process.

Because the application is expected to be code-heavy and development-focused, TruckCo repurposes a team of Java developers from its permanent staff to work on it, figuring that Apex is close enough for them to pick it up as they go along. Yana is the only traditional Salesforce person on the project, and her role is to guide the rest of the team in the ways of Salesforce.

Matt, the team lead, is a bit of a maverick, and the same is true of the rest of the team. While they use unit tests, they do not do so consistently, often preferring to have bugs reported by users and then fix them afterward. Inside TruckCo, there is an unstated but strong preference for releasing products on time, even if that means releasing something buggy and incomplete.

Coming from the Java world, the developers quickly start to get annoyed with the mandated 75% unit test coverage enforced by Salesforce and built into the deployment tooling for the platform. Yana accidentally mentions some ways to fool the Salesforce unit test calculation by creating dummy tests while having lunch with the rest of the team.

Although she mentioned it jokingly, much to her chagrin, she finds that a few days after, dummy unit tests are starting to appear in the code base. When she calls out Matt on this, he simply shrugs and says that they're behind schedule, so they don't have time to spend all their time coding tests.

Yana, however, is not deterred, and at the next project board, she raises the issue formally. She refers to the guidance provided by Salesforce and supported by the company's own Salesforce architect.

The project sponsor promises to have someone investigate the issue. Unfortunately, that's the last Yana hears of it. It seems that Matt had indicated that major delays would happen to the project if they had to retroactively write a complete set of unit tests. In keeping with TruckCo's normal way of operating, this is seen as unacceptable, and the dummy unit test issue is ignored.

Therefore, the practice keeps accelerating as the project falls more behind, and toward go-live, almost every new unit test written is a dummy. While the automated scanner lists an impressive 90% coverage, Yana doubts whether the real number is any higher than 10%.

A few weeks before go-live, a list of last-minute changes comes in from business stakeholders. Yana can't see any way that they can be incorporated, but Matt agrees to deliver them.

The last period is a flurry of changes back and forth with lots of bugs discovered by testers, followed by fixes that often lead to further regression issues. This continues unabated into the actual UAT period, which finds another large number of issues. However, no one makes the decision to halt the go-live, and on the appointed day, the supply chain application goes live.

As users start onboarding on the application, several breaking bugs are discovered in the main flows. These occur in areas that had been tested and found working just weeks before go-live and had also been part of user training.

The application must be taken down, and an escalation meeting is called. Here, the fundamental soundness and quality of the application are put into serious question and a new initiative is agreed upon to look into the quality issues and devise a way of fixing them before the application can be brought back online.

Problem

Dummy unit tests seek to address a very simple question: how do you get around the hard 75% unit test limit for production deployment in Salesforce? It's common for long-running Salesforce projects to find that they are under the limit either for a specific deployment or in general.

Many teams find this limit annoying. This is especially true when developers are coming to the platform from other technologies where no such limit existed, when under pressure, or when they are doing something "simple."

Proposed solution

The solution proposed is to create dummy unit tests that just optimize for code coverage and don't really test anything. This is another of the few anti-patterns that edge close to just being bad practice. You can see an example of a dummy test in the following code snippet:

```

@Test
class dummyTest{
    static testMethod void notRealTest(){
        //assume A is a class with two methods
        A aInstance = new A();
        aInstance.methodA();
        aInstance.methodB();
        //nothing is done to test anything
    }
}

```

However, it is something that can be tempting and that otherwise serious Salesforce practitioners sometimes may engage in. When you are under pressure to meet a sprint goal or deployment deadline, it can seem like a waste of time to write a bunch of unit tests just to get the coverage up.

But obviously, this attitude fails to acknowledge the real value provided by unit tests and the reason why the coverage is mandatory: code without good unit tests is much more brittle and prone to regression errors than code that has them. Therefore, skipping them, even when you are under pressure, is a genuinely bad idea.

Results

This pattern tends to lead to a death by a thousand cuts. While you probably won't experience ill effects in the short term, after a while, you won't have any real unit tests to rely on.

Even worse, you may still psychologically project a false sense of security, if you still think you've only cut the corners in a few places and generally can rely on your testing. Basically, you are cheating the system and probably releasing poor-quality code. Also, you are creating a major amount of technical debt that the organization will at some point have to pay off.

Better solutions

The solution to this anti-pattern is simple: build good unit tests and always meet the minimum thresholds. You should never compromise on the minimum bar as it will lead to deteriorating code quality and hence delivery quality over time.

There is a lot of guidance available on building good unit tests, so let us just repeat a few basic points:

- A unit test should test a single functionality in the code
- The test should be written before the code it is testing
- The test should be written in such a way that it can be easily repeated
- The test should be independent of other tests
- The test should not rely on org data but should create its own test data, ideally using a test data factory
- The test should be concise
- Assertions should be used to check that the expected results are obtained

- The test should test both positive and negative cases
- The test should be run often, preferably automatically
- The test should be easy to understand and maintain

In addition to traditional testing methods, consider incorporating modern testing frameworks such as ApexMocks, Jest, and Mocha. These frameworks offer powerful features for unit testing and mocking in various programming environments, facilitating more efficient and effective testing processes. Familiarizing yourself with these tools can significantly enhance your testing strategy and improve code quality.

It is a rare anti-pattern that can be avoided just by following good practice, so be sure to avoid this one.

The challenge of DevOps integration

As Salesforce platforms grow more complex and organizations push for faster delivery cycles, many teams find themselves grappling with the integration of modern DevOps practices into their architectural approach. While the promise of DevOps is compelling—faster releases, higher quality, and more reliable deployments—the reality often falls short of these aspirations, particularly in the Salesforce ecosystem where traditional development patterns can clash with DevOps principles.

Consider FinTechCo, a rapidly growing financial technology company implementing Salesforce as their new customer engagement platform. Like many organizations, they begin their DevOps journey with great enthusiasm, implementing all the right tools—Git repositories, CI/CD pipelines, automated deployments, and Salesforce DX for development. Their lead architect, Sarah, meticulously sets up the technical infrastructure, confident that this will transform their delivery process.

However, as the project progresses, the team discovers that having DevOps tools in place isn't enough. Developers continue making changes directly in sandboxes, only committing to Git when reminded. The automated test suite remains sparse, with most testing still performed manually. The CI/CD pipeline frequently breaks because different team members follow different processes. Integration between environments becomes increasingly problematic—what works in one developer's scratch org fails in another's, and changes that pass all automated tests in the pipeline sometimes break functionality in integration environments.

This scenario plays out frequently across the Salesforce ecosystem, where organizations implement DevOps tooling without fully embracing the cultural and architectural changes required. The challenge lies not in the tools themselves but in the fundamental shifts needed in how teams approach

development and deployment. Teams often find themselves caught between the immediate pressure to deliver features and the longer-term need to build sustainable delivery practices.

The situation typically deteriorates gradually. Teams start bypassing DevOps processes for emergency fixes, making direct changes in higher environments. Environment states become inconsistent, and tracking changes becomes increasingly difficult. What starts as the occasional deviation from the process becomes common practice, undermining the very benefits DevOps was meant to deliver.

At FinTechCo, this comes to a head during a critical deployment that fails repeatedly during a planned release window. The team discovers various components have different versions across environments, and nobody can confidently say which version should be considered correct. The release has to be postponed while the team manually reconciles the differences—exactly the sort of situation DevOps is supposed to prevent.

The root cause often lies in approaching DevOps as a technical transformation rather than an architectural and organizational one. Teams implement tools without redesigning their development processes and solution architecture to support modern delivery practices. This creates a disconnect between the ideal process and the practical reality teams face daily.

Success with DevOps in Salesforce requires rethinking architectural approaches from the ground up. This means considering how your metadata will be packaged and deployed, how environments will be managed, how testing will be automated, and how changes will be tracked and versioned. It means making architectural decisions that support continuous delivery—for instance, designing features to be independently deployable and building automated testing into the development process from the start.

Organizations need to recognize that DevOps isn't just about faster deployments—it's about building quality and reliability into the development process itself. This might mean making architectural trade-offs, such as sacrificing some flexibility in how changes can be made in exchange for more reliable deployments. It might mean redesigning features to support partial deployments rather than requiring all-or-nothing releases.

The path forward typically involves starting small and scaling gradually. Rather than attempting a complete transformation overnight, successful organizations often begin with a single team or project, allowing practices to mature before expanding. This approach allows teams to learn and adapt while managing risk appropriately. It also provides opportunities to demonstrate success, building organizational support for broader adoption.

Most importantly, organizations need to recognize that DevOps transformation is a journey rather than a destination. It requires ongoing commitment to improvement, willingness to learn from

failures, and patience as teams develop new skills and habits. The goal isn't perfect DevOps implementation, but rather building a sustainable approach to delivery that balances speed and reliability.

Success comes from aligning architectural decisions with delivery practices, building robust automated testing approaches, establishing clear governance around development processes, and investing in team training and cultural change. This means making hard choices about standardization versus flexibility, where to enforce processes and where to allow deviation, and how to balance immediate delivery needs with long-term sustainability.

By approaching DevOps as an architectural challenge rather than just a tooling one, organizations can avoid the common pitfalls of superficial adoption and build delivery practices that truly support their business needs. This requires careful thought about how architectural decisions impact delivery capabilities and how delivery practices influence architectural choices. Only by addressing both aspects together can organizations achieve the true benefits of modern DevOps practices in their Salesforce implementations.

Knowing the takeaways

In this section, we will abstract a bit from the specific patterns and instead try to pull out the wider learning points you can use in your day-to-day work as a Salesforce architect or in preparing for the CTA Review Board exam.

When architecting Salesforce solutions, you should be mindful of the following:

- Don't put all your eggs in one basket. Plan smaller releases wherever possible to de-risk and get feedback.
- Confront the tough decisions that can come from having to break down functionality into smaller buckets. Don't just accept a statement that everything must be there from day one.
- Always confront the key architectural trade-offs early on in your project timeline. They usually don't get easier to manage as time goes by.
- Communicate clearly and openly about the trade-offs that need to be made and the options for doing so. Trying to please everybody and sweep things under the rug is a recipe for disaster.
- Don't allow team rivalries or organizational silos to dictate your development model, whether for package-based development or otherwise.

Instead, ensure that your development model is consistent with the architecture you are pursuing and that components and packages are structured accordingly.

- Integrate modern DevOps practices including CI/CD pipelines, automated testing, and version control to improve delivery quality.
- Unit testing is a requirement in Salesforce for a reason. Don't compromise just because you are under pressure.
- Writing good-quality unit tests will ensure that you have higher code quality and fewer regression issues.

In preparing for the CTA Review Board exam, you should be mindful of the following:

- Prefer an agile, multi-release process when faced with a big, complex scenario, rather than a single big-bang release.
- Some scenarios, however, do seem written more for a waterfall/big-bang approach. This may be an area where best practices for the board and reality can diverge. In today's rapidly evolving tech landscape, it is increasingly crucial for CTAs to understand DevOps and modern development practices. This knowledge not only enhances collaboration between development and operations teams but also enables CTAs to better support organizations in implementing agile methodologies, CI, and continuous delivery, ultimately driving innovation and efficiency.
- You should be very clear about the trade-offs you are making and include them in your justification.
- If you fail to make an appropriate choice, it will with near certainty make your solution unravel during the Q&A.
- Package-based development is the up-and-coming development model for Salesforce projects, but that doesn't mean it will be right for your scenario. If it is, having some words to say about a good package structure will potentially give you some extra kudos.
- Know the minimum limits for unit testing and when they are applied so that you can answer if asked.
- Also, be prepared to answer general questions about what makes good unit tests and how to use them to improve code quality.

We have now covered the material for this chapter and are ready to proceed to the next one. First, however, we will summarize our learning.

Summary

In this chapter, we have seen examples of how the development life cycle can be impacted by anti-patterns at different levels of abstraction. At the highest level, how you deal with key trade-offs and how you structure your releases have a huge impact on the potential success of your project.

However, we also saw that technical elements such as how you structure your packages and whether you write good unit tests can be major contributors to project success or failure. That means you must master all of these levels to be a successful architect in this domain.

For many architects, this domain can be a bit of a lower priority relative to solution architecture or the hardcore technical domains. However, paying insufficient attention to these issues can lead to serious project failures just as easily as a badly designed solution.

With that being said, we will now continue to our last domain—namely, communication—and see how we can mess up our Salesforce projects by communicating in the wrong way.

8

Communication without Mistranslation

This final chapter before the conclusion will deal with anti-patterns centered around the way you communicate architecture to different audiences, incorporating insights from the new change management section. The first part describes an anti-pattern related to information control. Then, we look at a few anti-patterns that relate to the clarity of your communication. Third, we examine an anti-pattern specifically concerning the way you create architectural artifacts. Finally, we will look at how to avoid underestimating the challenge posed by change management. As we've been doing all along, we will end the chapter by summarizing our key takeaways.

In this chapter, we're going to cover the following main topics:

- How overcommunication can, in some cases, be worse than under-communication, especially when communicating with a generalist audience
- How keeping things ambiguous in order to avoid making a decision can result in serious damage to your project
- How people's seeming inability to understand what you are saying is often a symptom of underlying issues you need to address
- Why straying from standards when documenting technical specifications is almost always a bad idea
- How effective change management practices are essential for successful architectural implementations

After completing this chapter, you will have a better understanding of how technical communication can go wrong. More importantly, you will have picked up a number of concrete approaches you can apply to help you avoid common technical communication pitfalls.

Communicating too much

This section will explore what happens when you communicate too much in a technical context. To do this, we will start by exploring an anti-pattern taken straight out of cognitive psychology: cognitive overload.

Cognitive overload

Cognitive overload happens when the amount of information presented becomes overwhelming to the recipient to the extent that it impairs action.

Example

LilacCo, the world's leading producer of scented candles, is having major issues with its integration landscape. It has a relatively recently purchased MuleSoft platform, but it has seen little use, and most integrations still run through legacy middleware or point-to-point.

Martin is a technical architect who has been given the task of coming up with a solution to the various issues LilacCo is experiencing. The issues include high latency and slow performance on many integrations, poor logging and monitoring, long and error-prone development cycles, and the inability to create new integrations for several key legacy systems.

Martin has thought long and hard about the issue and has concluded that moving more systematically to MuleSoft, implementing more event-driven and asynchronous integrations, writing some key adaptor APIs in front of older legacy systems, and providing a clear pattern for error handling and logging to be used by all teams will solve the problem.

It is not an easy fix. Martin has created a slide deck and accompanying report numbering hundreds of pages. The material contains many technical diagrams, and although he has tried to create appropriate summaries, he feels that there is only so far this material can be simplified without losing the essence of what is being said.

Martin pitches his new model to the executive board. Unfortunately, the questions asked reveal that no one has really understood the message he has tried to convey. They get hung up on relatively insignificant details and fail to grasp the overall plan.

He is given feedback that his proposed model is too complex and that he needs to find a simpler approach. Martin is exasperated. He already felt like the plan he had come up with was the simplest possible. The elements all rely on each other, and removing any would compromise the approach.

Martin tries to schedule one-to-one conversations with the people he knows best on the executive board in order to rerun his key points by them. Even in this more intimate context, Martin finds it difficult to bring his points across. The key points simply seem to get lost in translation.

Then, Martin is told that a new integration platform vendor has had a pitch in front of the executive board and that everyone is very excited about the new approach they are bringing. Martin is asked to give feedback, which he does in the form of a detailed and nuanced rebuttal of the vendor pitch. He concludes that while the new platform indeed has some exciting features, it does not solve the fundamental problems LilacCo is facing.

Martin's feedback, however, accomplishes very little. The vendor proposes to do a free **proof of concept (POC)** to create more comfort in their software's capabilities, which the executive board accepts. Martin is not asked to be part of the POC team.

Six weeks later, the POC concludes with roaring success. The team has been able to demonstrate all the capabilities required during this period. Of course, it's all on a pilot basis and much complexity remains, but the executive board is confident based on the presentation it receives from the team that the new software will deliver what it needs.

Martin asks whether he can look into the details of the POC and what has actually been demonstrated, but his request is silently ignored. Instead, the vendor sends a formal proposal, which is accepted, and a new project to implement the new integration platform is underway.

Problem

Cognitive overload is a general term from psychology for situations where our brains are overloaded with information or options to the point where we are unable to take the required actions that we would normally be able to in the presence of less information. In technical communication, creating cognitive overload by including too much information when presenting architectural options is a very common anti-pattern.

This especially happens when we convey information to non-technical audiences. For technical audiences, we can, to a large extent, rely on an understanding of diagram formats, standard ways of doing things, good practice, architecture patterns, and similar.

That reduces the cognitive complexity for such an audience, although complex architectures can still be hard to follow even for experienced practitioners. However, when you are trying to convey this information to a general audience, you cannot rely on these supports, and therefore the task of conveying key trade-offs and many moving parts can be extremely difficult.

Proposed solution

Cognitive overload isn't a solution per se. What leads to cognitive overload is the belief that you, somehow, need to include a certain amount of detail in your presentation in order to be fair to your audience or complete in relation to your subject.

My personal belief is that a large number of talented architects fall into this anti-pattern because of a sense of intellectual honesty. They have considered many options and are aware of many arguments and counterarguments for and against the solution they are proposing.

Therefore, they feel dishonest in simply doubling down on a highly simplified version of the option they have chosen to recommend. In scenarios where business stakeholders are the ultimate decision-makers, which is often the case even for seemingly architectural decisions, that almost inevitably leads to cognitive overload.

Results

The obvious result is a failure of communication. Your main recommendation and supporting points are lost in a quagmire of information that your audience is unable to process.

That means you lose your audience and their attention and, ultimately, your ability to influence them to make the decision you want them to make. Cognitive overload leads to inaction, so the most likely result is that nothing will come out of what you are proposing because it has not been assimilated.

That also leaves the field open for other players to sell a different message, as we saw in our example. It is an unfortunate fact in our industry that it is often the most well-placed messaging that leads to technology adoption rather than the optimal choice from a technical point of view.

Better solutions

Effective communication is not something that can be taught in a few sentences. However, your takeaway should be that it is important to simplify a complex message sufficiently for it to be understandable to your audience.

In general, do the following:

- Structure the message for the audience and their level of understanding
- Don't assume the audience knows even the basics of what you are proposing unless you know for a fact that they do
- Focus on a single main message; don't include lots of subsidiary messages alongside the main one
- Don't include all additional considerations and options you have considered along the way
- Use images, graphics, tables, and so on to help get information across
- Have backup material ready if anyone wants to dive deeper into an area, but don't include it upfront

Overall, you should be cautious about making demands of your audience. They have a lot on their plate and limited time to engage with what you have to say. Make sure that time counts.

Being unclear in several ways

In this section, we explore two anti-patterns that, in different ways, explore intentional and unintentional lack of clarity in technical communication. We start by exploring an anti-pattern that seeks to deploy a lack of clarity for a tactical purpose but ends up causing adverse effects.

Ambiguous solution

Ambiguous solution proposes to hide away a conflicted or uncertain decision behind ambiguous language and representations to postpone the need to make it.

Example

GrillCo, a major online retailer focused on outdoor cooking equipment, is facing major issues in its order management process. There are frequent unanticipated stock-outs, wrong delivery calculations both for duration and costs, and taxes are not consistently applied correctly.

This is not entirely an IT problem, and in fact, many areas of process improvement have been identified. However, as part of the wider order management transformation, GrillCo management has also decided that it needs a new software platform for the order management process.

There are, however, several opinions as to which software to get and how to go about implementing it. Some support putting it in the Salesforce CRM, using the out-of-the-box order management module. Others would prefer dealing with a dedicated vendor with a more retail-focused attitude. Finally, IT would prefer implementing the functionality themselves, using the microservices architecture they have been building up.

Rainer has the technical project management responsibility for the order management implementation. He tries to facilitate the discussion between the different proposals, but the problem is that nobody seems to know who should be making the decision.

In the meantime, there are many other smaller technical tasks to get on with, so Rainer focuses on these, leaving the new order management system as a generic capability on diagrams, presentations, and documentation. He figures that a solution will be arrived at eventually, so there is no need to rock the boat.

However, as the transformation proceeds on other fronts, the question becomes more urgent in the minds of the senior stakeholders. As he has been unable to gain consensus by talking to people, he proposes to do a formal vendor selection with well-defined criteria and let that be the decision-making process.

However, as he starts to prepare the groundwork for the selection process, he makes a startling discovery. Two teams have separately gone ahead with completely different pilot implementations of the order management software based on their own interpretation of requirements:

- The first is within the IT department, where a few developers have started prototyping new software to show how flexible and powerful their microservices platform is. While it has some good functionality, it does not, in Rainer's assessment, begin to meet the real needs of the business.
- The second pilot is even further along, already working with mock ordering scenarios based on data taken from the existing e-commerce solution. It is based on a piece of standard software from a small retail-focused software vendor that is investing heavily in making the necessary adaptations to meet GrillCo's needs. However, Rainer again assesses that there is much to be done before this solution might be fit for purpose.

A dramatic meeting ensues with the CIO and the IT team on one side and the COO and the supply chain team on the other, both championing one of the solutions. The meeting ends as a stalemate, and a decision is made to bring in an independent consultant to assess the two solutions and make a recommendation.

GrillCo brings in a high-profile and well-respected expert in order management solutions who spends a few weeks looking at the pilot software. His assessment is damning.

Neither pilot really has a good way of delivering the level of capability GrillCo needs. He recommends proceeding with neither solution and instead going back to the market to find a better fit. Rainer polishes his presentation on the technology vendor selection process and gets to work.

Problem

Some decisions are hard to make. This can be because of politics—that is to say, not wanting to annoy powerful stakeholders or groups with a decision they won't like. It can also be due to missing information that will only appear at a later stage in the process and therefore complicates making a decision now.

The art of avoiding a controversial decision by postponing it is the core of the problem addressed by the ambiguous solution anti-pattern. While it can be reasonable to take some time to collect more specific information that will significantly increase decision quality, taking this too far becomes an anti-pattern, no matter whether it is done due to politics or a general fear of getting the decision wrong.

Proposed solution

The ambiguous solution anti-pattern proposes to handle difficult decisions by hiding them behind ambiguity. This can easily be done through things such as capability models, by deferring to a separate decision process, or by simply listing multiple alternatives—for instance, in diagrams and documentation.

This can be a highly attractive option in a number of cases; for instance, you may know that certain very senior stakeholders favor a certain approach that you have determined to be unviable. By hiding behind ambiguity, you may hope to postpone making this clear until the project is too far along to change things.

You may also, secretly, be hoping that as the project progresses and more information becomes available, the decision will become easier; either it will be too late to activate some options or the answer will become obvious.

That is to say, you are hoping to not have to handle a situation that has a high degree of uncertainty or conflict attached to it by postponing it indefinitely. This is very human behavior, but, unfortunately, is a bad architectural practice.

Results

Ultimately, of course, you will need to deal with the issue you have been hiding away. If you are lucky, the decision will now be easy as most avenues will be foreclosed. This is not a good thing, however.

It will rarely be the case that the option that can be implemented most swiftly is also the optimal solution. Therefore, your procrastination in dealing with the issue will nearly always result in a worse solution than if you had made the decision sooner.

There is something to be said for keeping options open when the decision isn't necessary, but if you do it for too long, it will adversely affect your solution. Also, keeping options open for too long when it comes to key architectural decisions is generally problematic.

If you aren't lucky, the decision still won't be obvious; you still must deal with uncertainty and conflict, but now it will be compounded by the additional time pressure you have added by waiting. This is altogether a worse outcome both for you personally and for the project.

Better solutions

First, don't postpone a decision beyond its last sell-by date. If you don't have the necessary information, make a plan with a clear deadline for getting it together. Then, plan a decision-making process that will take place at that time whatever else happens.

If the decision is politically unpalatable, try to get your supporters within the organization to stand by you as you make and communicate it. There's always a risk of the wrong decision being made for political reasons.

However, even the wrong decision is better than no decision after a point. At least you will be making progress and you will be able to start mitigating the consequences of a wrong decision sooner.

When decisions are needed, flag them clearly in documentation and presentations so that stakeholders are aware the decision is coming. Don't hide behind vagueness, even when it's easy to do so.

If you can't decide by consensus, see whether you can get stakeholders to agree on a formal decision-making process instead. Often, people find it easier to live with the outcome of a formal process than with something that appears as just some guy's opinion.

Groundhog Day

The **Groundhog Day anti-pattern** happens when people endlessly ask the same questions, even after multiple repeated explanations.

Example

Rolf is leading a local Salesforce implementation for MediCo, a medical device manufacturer with a presence in most European markets. The Salesforce implementation has been defined by the HQ staff in Paris and is meant to be rolled out as is to the various subsidiaries in other countries.

Rolf has been given the responsibility to roll out the system to the German subsidiary. The German operation is new to MediCo, having only been acquired the previous year. It, therefore, is not tightly integrated into MediCo's standard processes or IT infrastructure. The Salesforce implementation is seen as a way to improve upon this state of affairs.

When he delivers the initial presentation on the Salesforce implementation as it exists in France and other countries to the German team, he is met with a barrage of questions. The team wants to drill into very specific process questions that Rolf has no way of answering, as he doesn't know the details of the German operation.

He takes away a ton of questions, which he tries his best to answer in written format. However, he quickly starts feeling like he is playing a game of whack-a-mole. Every time he seemingly answers a question, another similar but subtly different question pops up from another member of staff.

Rolf starts consolidating all the questions and prepares clear and concise answers to all of them. Given the high degree of confusion, he has not been able to make any real progress with the implementation, so he decides to give it one more go.

He gathers the German team together one more time to give the team members the highlights of the new Salesforce system and as systematically as possible to address all the concerns that have been raised so far. To no avail, he is once again met with more uncomprehending glances and questions that he thought he had already answered more than once.

Frustrated and almost ready to give up, Rolf decides to call one of his old mentors in the company, who he knows is aware of the situation in the German subsidiary. She tells Rolf that the problem is probably not with his communication skills at all.

The staff in the German subsidiary prefer to do things their own way and resent the imposition from HQ. Salesforce is just a pretense for voicing that opposition. The system could be the most perfect thing since sliced bread, but the German team would still oppose it out of principle.

With his new knowledge, Rolf decides on a new plan. He takes the bold decision to put things out in the open. He holds another presentation, where he makes it clear that he is aware of the real reason for the opposition, and rather than trying to address all the concerns, he invites the team into the process to see how it might find a way of making the new system its own.

While the system is what it is and HQ is not about to approve a lot of local changes, how the German operation decides to adopt it and integrate it into its own processes is up for grabs. Rolf opens up the implementation to the German team and welcomes its suggestions for how it might best use the new CRM. Slowly, some people start to engage, and while the timeline has to be moved a couple of times, eventually, adoption is successful.

Problem

Sometimes, people just don't seem to understand what you're saying. No matter how hard you try to explain a point or how many times you explain it, there doesn't seem to be any way to get your point across.

The Groundhog Day anti-pattern, named after the movie of the same name, describes this situation where you can't seemingly communicate something enough times for it to make any difference.

That means you feel like you are banging your head against a wall without making the slightest mark. Often, this is because you aren't really engaging with the underlying issues that are in play.

Proposed solution

When you engage in the Groundhog Day anti-pattern, you respond to the situation of having seemingly never-ending queries about the same issues by repeating yourself a lot. You may repeat yourself in different languages or different modalities, but you are reiterating the same message.

Sometimes, it can make life feel like an endless repetition and like you are making no progress at all. You question both your own sanity and the sanity of the people doing the continuous questioning.

That's generally because you are not really engaging the underlying reason for why the questions are asked. Generally, when you are asked the same thing more than a couple of times by the same people, some kind of resistance is at play.

Maybe, the people feel like a bad decision has been made over their heads and they are trying to pick it apart. Maybe they are anxious that the direction you are proposing will lead to bad personal consequences for them.

There are many reasons people might have to resist your message that have nothing to do with how you present or communicate that message. In this case, the problem becomes more about change

management than about communication, and being aware enough to catch this phenomenon is a key soft skill to acquire if you are a practicing architect.

Results

The results of the Groundhog Day anti-pattern are substantially negative both on a personal level and for your project:

- First of all, you get nowhere with the message you are trying to communicate, which usually needs to be accepted by the audience to which you are communicating
- That leaves you frustrated as you ponder how to better communicate your message to the audience and continuously fail
- It leaves the people you are communicating with frustrated as they don't feel you are listening to their real concerns
- The project, in the meanwhile, usually stalls and progress on key areas can be blocked due to the communication impasse

Overall, this anti-pattern can be a serious strain on your project and you.

Better solutions

The first step is to recognize that the problem exists and is not about you finding a better way of saying the same thing.

In general, you should do the following:

- Once you begin hearing the same questions over and over again in slightly different variations, pause and take a step back
- Engage with stakeholders and allies to figure out what are the real reasons behind the resistance that you are facing
- If you find that there are reasons that lie behind the consistent questioning that go beyond just comprehension, work actively to find an alternative approach without compromising your core goals
- Then, apply good change management practices to help drive the needed transformation

You can't always give people what they want, but you can find a way to acknowledge their real concerns and be open to their feedback.

Drawing diagrams with excess creativity

This section explores what happens when you try to communicate technical information with too much creativity rather than relying on established practice within your community. We do this by looking at the non-standard documentation anti-pattern.

Non-standard documentation

Non-standard documentation refers to an anti-pattern where teams produce technical documentation in an idiosyncratic format rather than relying on standard practice.

Example

SafeCo, the leading provider of security solutions for facility management, is in the visioning phase of a new Salesforce implementation. It is trying to come up with a strategy for completely transforming its business using technology and has bought into Salesforce as part of the solution.

Qin, a strategy manager with SafeCo, is leading the visioning exercise and is supported by an external consultancy with particular strength in design thinking. They run a series of highly engaging workshops that result in a vision and key requirements for the new strategic technology platform.

The outputs of the workshop are documented in a fairly large number of rich pictures and ad hoc diagrams, many combining elements of process design, data models, and UX prototypes. The senior managers who have been part of the workshop like the outputs so much that they decide to include them in the **request for proposal (RFP)** material that will be sent to potential vendors.

SafeCo shortlists three vendors that all make explicit reference to the workshop outputs in their RFP responses. After final presentations, SafeCo selects one of the largest global Salesforce partners to carry out its implementation in line with requirements as principally stated by the workshop outputs.

However, once the vendor has mobilized and the project enters a formal discovery phase, it becomes clear that there are problems. Many features and requirements that SafeCo thought were obvious from the provided documents have not been included in the vendor's estimates.

SafeCo pushes hard on the vendor, pointing to the documentation provided with the RFP. However, the vendor disputes the interpretation of the documentation, and it proves impossible to demonstrate clear intent for many of the points under dispute.

For a moment, SafeCo considers going back to the market, but the vendor was selected as the only one that had the necessary mix of competencies available on the timeline it needed. Senior management is not willing to compromise on the project deadline so redoing the procurement doesn't seem feasible.

Therefore, the only thing to do is to negotiate an agreement for additional features that the vendor had not included. After some amount of wrangling, a compromise is reached that leaves neither party fully satisfied.

The price of the project goes up but not by as much as the vendor wanted. In addition, the vendor insists on more formal documentation of the requirements to be implemented during the project, taking away some of the flexibility that was part of the initial plan. With the compromise in place, the project can finally commence.

Problem

The problem addressed by the non-standard documentation anti-pattern is how to best communicate complex requirements, architectures, designs, and general expectations of technical systems. This touches on the differences between communicating general concepts to a generalist audience versus communicating technical concepts to a technical audience.

Generally, highly visual and engaging material tends to play well when in a workshop setting, such as defining the vision for a new system or how to use an enterprise software platform such as Salesforce for strategic advantage. In this case, you want to use formats that resonate with your audience and build a common understanding.

However, this approach can lead you into trouble when communication needs to be precise and technically correct.

Proposed solution

Non-standard documentation proposes that the best documentation is the one clearest to the key stakeholders involved in producing it, whatever that might be. This is an extremely seductive position because it is true in many cases.

For instance, the preceding statement is true in the following cases:

- If you are trying to gain alignment about a vision or project
- If you are trying to foster mutual understanding within an organization about a certain topic
- If you are trying to create general awareness of an issue

In all these cases, the perfect documentation is the one that works well for the audience. However, when it comes to technical specifications, architectures, and designs, this ceases to be the case.

Results

Normal language tolerates a lot of ambiguity. The same is true of most visual representations of concepts. However, ambiguity is the mortal enemy of technical specification.

To reduce the amount of ambiguity when writing technical specifications, whether of requirements, architectures, designs, or in general, we use a number of tools. These include the following:

- Language that is stripped of everyday niceties and strives consistently for clarity
- Referring to technical standards that have a well-defined common understanding
- Referring to patterns that are common within our technical community and likely to be readily understood by the recipients of our communication
- Referring to conventions or common ways of doing things that are common within our technical community and likely to be readily understood by the recipients of our communication

- Using standard formats for documentation that have a structure sure to address the common concerns expected by members of our technical community
- Using standard formats for diagrams that are well understood by members of our technical community

All these elements help to reduce the ambiguity of technical specifications, although even with all of these in play, there is still plenty of scope for misunderstanding, as any experienced architect will tell you.

That is why there are standard ways, such as user stories or a traditional requirements specification, to convey requirements, and standard diagram types, such as system landscapes, process diagrams, and data models, to convey technical design.

If you stray from these standards, you add to the total ambiguity, and that will cause communication trouble, which can be expensive as your vendors or staff will not understand precisely what is needed. There is enough trouble with communication even with precisely defined requirements and design—don't add to it.

Better solutions

Given the description in the previous section, it's probably not difficult to surmise what you should do instead. Let us list them for good measure:

- Write your technical specifications with a view to reducing ambiguity. Strive for clarity above all else.
- Rely as much as possible on standard diagram types, ways of presenting material, naming, and so on whenever you are creating a technical specification that needs to be shared.
- Follow the patterns and conventions common within your technical community. For Salesforce professionals, this generally means following the approach that Salesforce itself uses.

That way, you can communicate clearly with vendors and new technical people that you hire, avoiding the parts of ambiguity that are avoidable.

Remember—in contrast with most communication where you are trying to make a general point, with technical communication, you are trying to make an exact one. That is a very hard thing to do, as it turns out.

For Salesforce, specifically, the *Salesforce Architects* website (<https://architect.salesforce.com/diagrams>), will help you get a good overview of resources shared by Salesforce itself now available to all architects to avoid re-inventing the wheel when documenting and presenting Salesforce solutions. For readers interested in preparing for the CTA board, this is a great place to keep in mind.

Mismanaging change

In this section, we talk about mismanaging change. Mismanaging change can lead to significant setbacks for organizations, often resulting from a lack of communication, insufficient training, or failure to involve stakeholders in the process. Common pitfalls include underestimating the resistance to change, neglecting to clearly define goals and expectations, and overlooking the importance of a structured change management plan. These mistakes can create confusion among employees, decrease morale, and ultimately hinder the organization's ability to adapt, leading to wasted resources and missed opportunities for growth. It is essential for leaders to be aware of these challenges and to implement strategies that promote a smooth transition while fostering a culture that embraces change.

Ignoring stakeholder concerns

Ignoring stakeholder concerns is an anti-pattern where implementation teams push forward with Salesforce development and customization while disregarding or minimizing critical input from key stakeholders. While often rooted in valid technical considerations, this approach ultimately leads to solutions that fail to meet business needs.

Example

MedSupply, a growing medical supplies distributor, decided to modernize their sales operations by implementing Salesforce. The company's legacy CRM system was showing its age, and users had been complaining about its clunky interface and limited functionality for years. The IT team, led by senior developer Marcus, was excited about the opportunity to build something modern and technically sophisticated.

The sales team had a complex quote-to-order process that involved multiple approvals and specific regulatory compliance checks. Sarah, the sales operations manager, tried repeatedly to schedule meetings with Marcus's team to walk through their current process and explain their requirements in detail. However, Marcus had valid concerns about replicating overly complex legacy processes in Salesforce. From his perspective, many of the existing process steps seemed to be artifacts of the old system's limitations rather than genuine business requirements.

During the initial requirements gathering, Sarah expressed concerns about certain compliance checks that needed to happen before quotes could be sent to customers. She explained that their current process required cross-referencing multiple databases to ensure they weren't selling restricted medical supplies to unlicensed facilities. Marcus believed that Salesforce's validation rules and approval processes could streamline these checks significantly, eliminating what he saw as redundant steps in the current process.

The development team created what they considered an elegant solution using Salesforce's standard quote functionality with some custom approval processes. They were proud of how they leveraged platform features and kept customization to a minimum, which would make the system easier to maintain and upgrade. When Sarah pointed out that the solution didn't account for their need to maintain detailed audit trails of all compliance checks, Marcus insisted that Salesforce's standard history tracking would be sufficient, arguing against building custom audit functionality that would require significant maintenance.

During UAT, the inherent tension between technical elegance and business requirements became apparent. Sales reps found the new process cumbersome because it didn't match their workflow, though some of their complaints stemmed from unfamiliarity with Salesforce's interface paradigms. The compliance team discovered they couldn't perform all necessary checks within the system. Sarah raised these issues in project meetings, but the IT team, seeing the elegance of their solution, characterized many of these concerns as "resistance to change" and "user training issues."

The system went live as scheduled, despite unresolved concerns from the sales and compliance teams. Within the first week, two major problems emerged. First, sales reps were taking twice as long to process quotes because the system forced them into a workflow that didn't match their needs. Second, and more seriously, the compliance team discovered that some quotes had been sent to customers without complete regulatory checks because the system hadn't enforced all required validations.

After a near miss where they almost shipped restricted supplies to an unlicensed facility, senior management intervened. The company had to roll back to their legacy system while they reassessed their approach. The project had cost over \$1,500,000 and resulted in zero improvement to their sales operations.

The problem

Ignoring stakeholder concerns occurs when implementation teams dismiss or minimize valid input from key stakeholders during Salesforce projects. However, the root of this anti-pattern is more complex than simple dismissal—it often stems from genuine technical considerations and the challenge of balancing competing project objectives.

Technical teams typically have valid reasons for wanting to stick close to standard Salesforce functionality. They understand that heavily customized solutions are harder to maintain, more prone to breaking during updates, and more expensive in the long run. They've often seen the problems that arise when organizations customize themselves into a corner, creating systems that are effectively unmaintainable.

At the same time, stakeholders often have legitimate requirements that push against platform standards. These might stem from regulatory requirements, established business processes, or genuine efficiency concerns. The challenge lies in distinguishing between stakeholder preferences that could be addressed through training and change management, and fundamental requirements that genuinely require technical accommodation.

This anti-pattern is particularly dangerous because both sides often believe they are making the right long-term decision for the organization. Technical teams push for standardization and maintainability, while business stakeholders advocate for functionality and familiarity. Without proper mediation, this tension often resolves in favor of technical considerations simply because the technical team has more direct control over the implementation.

Proposed solution

The anti-pattern presents itself as an efficient approach to Salesforce implementation that promises to deliver results quickly by relying on the platform's standard functionality and the implementation team's expertise. This solution appears compelling because it addresses very real technical concerns about maintainability and sustainability.

The proposed solution typically emphasizes platform best practices and standard functionality, positioning stakeholder concerns about process fit as change management issues rather than technical requirements. This approach isn't entirely wrong—many legacy business processes can and should be simplified when moving to a new platform. However, it tends to swing too far in the direction of platform purity, dismissing valid business requirements in the process.

This approach can seem attractive because it promises significant benefits: faster implementation times, lower development costs, and easier maintenance. These benefits are real, but they need to be weighed against the costs of forcing business processes to conform to platform standards in cases where those standards don't fully meet business needs.

Results

When stakeholder concerns are ignored, the results are usually negative for both the technical implementation and business operations. User adoption suffers not just because of change resistance, but because the system genuinely fails to support critical business processes. While some user complaints might indeed reflect change management issues, others highlight real gaps in functionality that impede business operations.

The technical team's vision of a clean, maintainable implementation often proves illusory. As business needs assert themselves, the system typically accrues a layer of hasty customizations and workarounds that are far messier than a properly planned custom solution would have been. These emergency fixes often violate platform best practices more egregiously than a deliberately designed custom solution would have.

The financial impact extends beyond the immediate project costs. While custom development is expensive, the cost of business disruption from an inadequate system can be far higher. However, it's equally true that over-customized systems can become expensive maintenance burdens that need to be entirely replaced after a few years.

The organizational impact often results in a false dichotomy between "technical best practices" and "business requirements" that can poison future projects. Technical teams become more convinced that business users make unreasonable demands, while business users become convinced that technical teams don't understand or care about their needs.

Better solutions

Avoiding this anti-pattern requires replacing the false dichotomy between technical excellence and business requirements with a more nuanced approach that recognizes both technical and business constraints as essential considerations in system design.

Effective stakeholder engagement shouldn't mean uncritically accepting all stakeholder requests, but rather creating a genuine dialog about business needs and technical constraints. Technical teams should explain the long-term maintenance implications of various approaches, while business stakeholders should clarify which requirements are genuine business needs versus preferences based on current processes.

The technical approach needs to balance platform best practices with business requirements. Rather than starting from either platform features or business processes, teams should begin by understanding the core business objectives and constraints. Solutions should thoughtfully blend platform capabilities with business requirements, using custom development judiciously where it genuinely adds value.

Requirements gathering needs to go beyond documenting current processes or platform capabilities. Teams need to understand why processes exist in their current form, distinguishing between historical artifacts and genuine business needs. This understanding helps identify opportunities where process changes might be better than technical customization, while also highlighting areas where customization is truly necessary.

Implementation should be guided by business value and technical sustainability rather than either platform purity or process preservation. Teams should be willing to customize when standard functionality doesn't meet critical needs, but should also be willing to challenge stakeholders on process changes when standard functionality offers a simpler solution.

Effective governance means creating a balanced decision-making process that gives appropriate weight to both technical and business concerns. This requires technical leaders who understand business impacts and business leaders who understand technical implications. Regular checkpoints should evaluate both technical quality and business fit, with clear escalation paths when these goals conflict.

By following these better solutions, organizations can create Salesforce implementations that balance technical excellence with business requirements. The key is recognizing that both technical best practices and business requirements represent real constraints that must be thoughtfully balanced, rather than competing priorities where one must triumph over the other.

Underestimating the impact of change

Underestimating the impact of change is an anti-pattern where implementation teams fail to fully appreciate and plan for the cascading effects that modifications to a Salesforce org can have across the entire ecosystem, leading to widespread disruption and technical debt.

Example

Global Manufacturing Corp (GMC) has been using Salesforce for five years as their primary CRM platform. Their implementation has grown organically to include Sales Cloud, Service Cloud, and multiple custom objects supporting various business processes. The system serves over 2,000 users across sales, customer service, and operations teams.

Sarah, a senior Salesforce administrator, receives what seems like a straightforward request from the VP of sales. The sales team wants to modify the opportunity stage field to better align with their evolved sales process. The change involves adding two new stages and splitting an existing stage into two more granular ones.

The request appears simple enough—it's just updating a picklist field that's been stable for years. Sarah's initial assessment suggests this is a minor change that could be implemented within a day. She plans to make the change directly in production during a quiet period, seeing no need for extensive testing or communication given the apparently limited scope.

What Sarah doesn't fully consider is how deeply embedded the opportunity stage field is within GMC's Salesforce ecosystem. The field drives numerous automated processes, including multiple approval workflows that trigger based on stage changes, complex forecasting rules used by finance, integration with their ERP system that initiates order processing, dozens of reports and dashboards used across the organization, several third-party apps that rely on standard opportunity stages, and custom commission calculations based on stage progression.

Sarah implements the change on a Tuesday morning. By Wednesday, the problems start surfacing. The finance team discovers their forecasting reports are showing incorrect numbers because the new stages don't map to their established forecasting categories. The VP of finance is furious because they're in the middle of quarterly planning.

The sales operations team finds that their commission calculations are failing because the logic doesn't account for the new stages. Several sales reps are concerned about how this will affect their commission payments. The integration with their ERP system starts throwing errors because it hasn't been updated to handle the new stage values, causing orders to get stuck in processing. Multiple dashboards across the organization show blank values or incorrect data, prompting a flood of help desk tickets. The third-party sales analytics tool they use stops syncing data correctly, breaking several executive dashboards.

What started as a seemingly simple change has cascaded into a crisis affecting multiple departments. Sarah and her team spend the next two weeks scrambling to fix these issues, requiring emergency meetings with finance to redesign forecasting rules, urgent updates to commission calculation logic, coordination with their integration team to modify the ERP interface, recreation of numerous reports and dashboards, and multiple communications to users explaining the issues and timelines for resolution. The incident prompts a formal review by IT governance, and GMC realizes they need a more robust change management process for their Salesforce org.

The problem

The underestimating the impact of change anti-pattern occurs when teams fail to recognize the interconnected nature of modern Salesforce implementations. This anti-pattern is particularly dangerous because it often starts with changes that appear simple and localized but have far-reaching consequences.

The problem manifests through failure to map dependencies across the complex ecosystem of modern Salesforce organizations, where seemingly isolated components are often deeply interconnected through automation, integrations, and reporting relationships. Teams frequently conduct only surface-level reviews of proposed changes, missing critical dependencies and downstream effects.

Testing often focuses only on the immediate functionality being changed, rather than the entire chain of affected systems and processes. Teams may fail to identify all groups affected by a change, leading to inadequate communication and preparation.

This anti-pattern is especially common in mature Salesforce implementations where the original architects and developers may have moved on, taking with them crucial knowledge about system dependencies and design decisions. The complexity of modern Salesforce implementations means that even small changes can have ripple effects throughout the organization, affecting processes, integrations, and reporting in ways that might not be immediately obvious.

Proposed solution

The apparent solution that leads to this anti-pattern is to treat Salesforce changes as isolated, technical modifications that can be implemented quickly and directly. This approach seems attractive because it promises faster delivery of requested changes, reduces overhead and bureaucracy, minimizes resource requirements, appears to simplify the change process, and reduces initial planning effort.

Teams often justify this approach by pointing to Salesforce's robust platform capabilities and built-in safeguards. They might argue that since Salesforce handles many dependencies automatically, extensive impact analysis isn't necessary. This mindset is particularly tempting in organizations where there's pressure to deliver changes quickly, resources are constrained, documentation is incomplete, the system has grown organically without strict governance, or the team has successfully made similar changes in the past without issues.

The seduction of this approach lies in its apparent efficiency and the immediate gratification it provides to stakeholders requesting changes. It can seem like a pragmatic response to business demands, especially when teams are under pressure to deliver quickly or when resources are limited.

Result

When teams fall into the underestimating the impact of change anti-pattern, the consequences can be severe and far-reaching. The immediate technical issues often manifest as breaking changes in integrations and automated processes, data integrity problems, performance degradation, failed batch jobs and automation, and incorrect calculations and workflows.

The business impact can be devastating, with disruption to critical business processes, inaccurate reporting and forecasting, financial implications from incorrect calculations, lost productivity across multiple departments, and decreased user trust in the system. These immediate problems often cascade into long-term consequences including accumulated technical debt from rushed fixes,

degraded system documentation, increased resistance to future changes, higher maintenance costs, and reduced system reliability.

The anti-pattern often creates a vicious cycle where emergency fixes introduce additional unplanned changes, potentially causing further issues that require more rushed solutions. This can lead to a deteriorating situation where each fix introduces new problems, and the overall stability and reliability of the system gradually degrades.

Better solutions

To avoid the underestimating the impact of change anti-pattern, organizations should implement a comprehensive change management approach that fundamentally shifts how changes are evaluated and implemented. The key is to move from a reactive to a proactive stance on change management.

A robust impact analysis process forms the foundation of better practice. Organizations need to maintain detailed system documentation and dependency maps, using automated tools to identify field and object dependencies where possible. Regular system audits help understand integration points, while a comprehensive test inventory ensures nothing is overlooked during change implementation.

The change process itself needs careful structuring. This means implementing a formal change advisory board for significant modifications, establishing clear criteria for what constitutes a major versus minor change, and requiring documented impact assessments before approving changes. A standardized change request template should prompt consideration of dependencies and potential impacts across the system.

Testing strategy becomes particularly crucial in avoiding this anti-pattern. Organizations need to maintain multiple sandbox environments for different testing purposes and implement automated regression testing where possible. End-to-end testing of all affected processes, including integration testing, should become standard procedure for any significant change.

Stakeholder management plays a vital role in successful change implementation. Organizations should maintain a clear understanding of which groups are affected by different system components and establish clear communication protocols for system changes. Feedback channels for users to report issues and a visible change calendar help maintain transparency and trust.

Risk mitigation strategies need to be built into the change process. This includes developing rollback plans for all significant changes, implementing changes in phases when possible, scheduling changes during appropriate business windows, and maintaining emergency response procedures for critical issues.

Knowledge management becomes a crucial element in preventing this anti-pattern. Organizations need to document all system dependencies and integration points, maintain updated process flows and system architecture diagrams, and create a knowledge base of previous changes and their impacts. Regular training for team members on system architecture and dependencies helps maintain awareness of the system's complexity.

The key principle to remember is that the cost of proper planning and impact analysis is almost always lower than the cost of recovering from a poorly implemented change. In the Salesforce ecosystem, where changes can have widespread effects, it's essential to maintain a healthy respect for the complexity of the system and the potential impact of even seemingly minor modifications. Success in avoiding this anti-pattern comes not from avoiding changes, but from approaching them with the appropriate level of diligence and respect for their potential impact.

Now, having covered the patterns for this chapter, let us proceed to discuss the key takeaways.

Knowing the takeaways

In this section, we will abstract a bit from the specific patterns and instead try to pull out the wider learning points you can use in your day-to-day work as a Salesforce architect or in preparing for the CTA review board.

When architecting Salesforce solutions, you should be mindful of the following:

- Don't include too much information when presenting your architecture to business stakeholders.
- Instead, focus on your main message and include only essential supporting information.
- Don't feel bad about not including all the additional considerations you had in coming up with your solution. If anybody wants more details, they can ask for them afterward.
- Don't postpone decisions beyond the latest point it makes sense to take them, even if making a decision is hard.
- If a decision is controversial or politically sensitive, deferring to a formal decision process can provide a way forward.
- If you have a situation where it seems like you can't get your audience to understand you, you should start suspecting that there are some underlying sources of resistance.
- In that case, you will most likely need to focus on the change management aspects of the situation before you can make real progress.
- When it comes time to prepare documentation for a technical audience, whether an external audience or your internal technical teams, you should be rigorous about sticking to accepted practice within your technical community.
- For Salesforce professionals, that generally means adopting the practices disseminated by Salesforce itself.

In preparing for the CTA review board, you should be mindful of the following:

- Be short and crisp when presenting your solutions. The judges are experienced architects, so you can rely on a lot of background knowledge.

- Include a clear justification of your recommendations but do not dwell on alternatives.
- Don't include additional considerations that are not essential to your main point. If anybody wants to test your thinking, they will do so in the Q&A.
- Ambiguity is nobody's friend. Don't present solutions that aren't clear or fail to make important architectural decisions.
- The judges are highly experienced in ferreting out areas where you are missing clarity, so if you aren't sure what the right decision is, you should make one anyway.
- If the judges ask the same or nearly the same question multiple times, they are probably hinting at an area of weakness in your solution that you may want to reconsider on the fly.
- Use the standard diagrams and presentation formats that have been successful for previous CTA candidates.
- Do things the same way repeatedly in your mock exams to develop facility and speed.

We have now covered the material for this chapter and are ready to proceed. First, however, we will summarize our learning.

Summary

In this chapter, we saw how communication can seriously affect the outcomes of our projects. You can have the best architecture in the world and it can still fail to make any headway if you fail to communicate it properly or fail to deal with the resistance it engenders in your target audience.

This can be dispiriting to some architects of a more rationalistic bend. Surely, the facts and substance should be the determining factors. Unfortunately, in most organizations, better communication skills will beat stronger technical architecture skills in terms of getting things done.

The good thing is that communication skills, as with all skills, can be learned. There isn't anything particularly hard about the ways in which you need to communicate to have greater success as an architect—it is just something that takes some practice.

We have now covered all the subject matter of the book and are ready to proceed to the conclusion, where we will summarize the journey we've been on together and look at where to go from here.

9

Conclusion

This conclusion gives a broad overview of the content of the book, including the overarching takeaways from the preceding chapters. It then explains how you can use the anti-pattern method and perspective in your own work to improve your architecture skills. Finally, it gives you pointers on how you can progress further in this direction.

In this chapter, we're going to cover the following main topics:

- The key messages from the book – specifically, we will look at some underlying root causes of anti-patterns
- How to use anti-patterns in your own architectural practice to improve the quality of the architectures that you create
- How you can further your understanding of anti-patterns and develop your own architecture skills

After completing this chapter, you will have a good understanding of how to further your own architectural development and know how to integrate anti-pattern-based thinking in your day-to-day practice as an architect.

Summarizing the key messages

In this book, we have covered dozens of anti-patterns across seven different architectural domains. However, it won't have escaped the perceptive reader that many of the problems and suggested solutions are similar across several different patterns in seemingly different domains.

This indicates that, at a deeper level, anti-patterns have similar root causes that are often due to larger organizational or even psychological factors. Let's look at some of these recurring causes:

- **Wishful thinking** : Several anti-patterns are based on some form of wishful thinking – that is to say, not wanting to confront the technical reality in front of you. In Project Pieism, it is an inability to confront trade-offs and uncertainty; in Golden Hammer, it's an overoptimistic belief in a single tool; and in Unplanned Growth, it's the belief that a solution can be found later. This extends to new patterns such as *AI architecture is like normal architecture* where teams fail to acknowledge AI's unique challenges. Always be wary of optimism in technical architecture.
- **Systemic blindness** : Many anti-patterns originate from an inability to see the entire system and focusing on a local solution to the detriment of the overall architecture. This includes anti-patterns such as *Disconnected Entities* , *Ignoring the Ecosystem* , *Using Packages to Create Silos* , and *Event Hell* . The growing complexity of modern Salesforce implementations, especially with AI integration and DevOps practices, makes this even more critical. Forgetting about the overall sociotechnical system that you are working inside is a sure way to end up with anti-patterns.
- **Replacing a simpler problem with the real problem** : Quite a few anti-patterns originate in a wish to make things simple. Many architectural problems are wicked and allow only for solutions that are imperfect and highly complex. Often, that is not entirely satisfactory to stakeholders who cry out for an easy-to-understand answer, the giving of which often leads to an anti-pattern. *Compliant Means Secure* and *Declarative Means Safe* are two examples in the security domain, where we substitute a simple answer

for a complex reality. This extends to new patterns such as *Let's add AI to it* where complex AI implementation challenges are oversimplified.

- **Organizational constraints** : Another very common underlying cause of anti-patterns is working to the constraints given by the current organizational setup you find yourself in. Whether it is because of silos within the organization, bad relations with vendors, or political games, the place in which you find yourself organizationally can sometimes lead or even force you down the path of an anti-pattern. This is particularly evident in new patterns such as *Ignoring stakeholder concerns* and *Underestimating the impact of change* .
- **Weak technical governance** : A lack of effective technical governance is another common cause of anti-patterns. When you don't have any architectural authority or clear standards or conventions, you often get drift that results in anti-patterns. This includes many patterns, such as *Intense Coupling* , *System Proliferation* , and *Unconstrained Data Synchronization* . The increasing adoption of DevOps practices and AI capabilities makes strong governance even more critical.
- **Lack of discipline** : Finally, a number of anti-patterns boil down to a lack of discipline in the software development life cycle. This includes patterns such as *Big Ball of Mud* , *Spaghetti Sharing Model* , *Fat Interface* , and *Dummy Unit Tests* , among others. New patterns such as *Infrequent check-in* also fall into this category. These are, to some extent, the simplest to deal with as the solutions are obvious, but the underlying pressures that drive teams toward taking the easy road can be much harder to deal with.

Explaining how to use anti-patterns in practice

If you've gone through this book cover to cover, you will have encountered a large number of patterns for how things can go wrong on a Salesforce project. We have structured this by architecture domain to facilitate your understanding and give you more context.

However, you may still be left with some questions about how to best go about applying this new storehouse of knowledge in your day-to-day work. We propose that there are two main ways in which you can proactively use your knowledge of anti-patterns to improve your architecture practice and hone your architecture skills:

1. First, you can use it constructively as part of your architecture and design process
2. Second, you can use it diagnostically as part of your approach to understanding architectures designed by others

Using anti-patterns constructively

Using anti-patterns constructively means that you incorporate anti-pattern-based thinking in your normal architectural process. There are several ways you can do this, from simply having a mental checklist of common anti-patterns that you validate your work against to a systematic check against all the anti-patterns that apply to the current stage of your architecture process.

We recommend a halfway house between these two extremes where you do the following:

1. At the beginning of your architecture process, make an assessment against the six factors mentioned in the previous section. For each factor, rate your environment for how much it is likely to impact your work on a standard 1–5 scale.
2. For all the factors you've rated 4 or 5 on the assessment, compile a list of the various anti-patterns that could potentially happen in your environment. Pay special attention to new challenges around AI integration, DevOps practices, and change management.

3. For those anti-patterns you have selected, make a check both during the initial design and regularly throughout the project to see whether they are starting to occur.
4. If you start to see a drift toward a certain anti-pattern, take proactive steps, as indicated under the *Better Solutions* part of the pattern description.

Using anti-patterns diagnostically

When you come into a brownfield environment as an architect, doing an analysis based on anti-patterns to diagnose the health of the existing architecture can be a very fruitful endeavor that we highly recommend. This analysis can inform both your understanding of the strengths and weaknesses of the current environment and also indicate ways in which you may start to refactor it.

We recommend you do the following:

1. Gather information from existing users and team members familiar with the existing environment, particularly the non-functional characteristics that are likely to indicate the presence of an anti-pattern.
2. Based on the information you gather, make a shortlist of potential anti-patterns that may be found in this architecture, including newer patterns related to AI, DevOps, and change management.
3. Perform a thorough check of the relevant technical artifacts to confirm or refute the presence of each anti-pattern.
4. Based on your findings, start looking at potential mitigations and assess the impact of the work that you have been brought in to do.

Going further with anti-patterns

While there is much information in this book, obviously, your architectural learning journey does not end here. We suggest that there are three pathways you may want to consider going forward:

- Diving deeper into the anti-pattern literature
- Extending your knowledge of Salesforce architecture
- Extending your general architecture knowledge

For each of these areas, we have selected a range of helpful resources that we will cover in the following sections.

Diving deeper into the anti-pattern literature

There are three resources we particularly recommend for getting even deeper into the concept of anti-patterns:

- The original book, *Anti-patterns: Refactoring Software, Architectures, and Projects in Crisis* by Raphael C. Malveau, William J. Brown, Hays W. “Skip” McCormick, and Thomas J. Mowbray. This book, while highly dated, was extremely important in formalizing and popularizing the concept of anti-patterns. Several of the anti-patterns found in this book can be traced back to this one.

- The best overall discussion of anti-patterns, both in general and of specific anti-patterns, can be found on WikiWikiWeb, at <https://wiki.c2.com/?Anti-pattern>.
- DevIQ has an excellent repository of anti-patterns that discusses many of the common ones that can be found across platforms, at <https://deviq.com/anti-patterns/anti-patterns-overview>.

We will now proceed to look at how you might fruitfully combine knowledge of anti-patterns with deeper knowledge of Salesforce architecture.

Extending your knowledge of Salesforce architecture

One of the best ways to apply your newfound knowledge of Salesforce anti-patterns is to combine it with a deeper knowledge of the key Salesforce architecture domains. We recommend the following books for that:

- First, we recommend *Becoming a Salesforce Certified Technical Architect* by Tameem Bahri for anyone who is on the CTA track or might consider it in the future. Like this book, it is structured by the CTA architecture domains and is therefore highly complementary.
- Another very good Salesforce architecture book is the *Salesforce B2C Solution Architect's Handbook* by Mike King.
- For people with a special interest in the data domain, *Salesforce Data Architecture and Management* by Ahsan Zafar is highly recommended.

However, Salesforce architecture is only one aspect to consider. You should also consider extending your general architecture knowledge.

Extending your general architecture knowledge

While anti-patterns are important, we should also strive to improve our knowledge of architecture in general. Here are some resources to do that:

- First, we recommend the *Solutions Architect's Handbook* by Saurabh Shrivastava and Neelanjali Srivastav, which gives a good introduction to many general architecture subjects
- For a general introduction to CRM and its architecture, we recommend *The Art of CRM* by Max Fatouretchi
- Finally, we recommend *Multi-Cloud Architecture and Governance* by Jeroen Mulder to get a good overview of the increasingly common multi-cloud architecture scenario

We have now covered the content of this chapter and the book and will finish with a final summary.

Summary

Well done, you made it! This was the last chapter of the book and you are now an official anti-pattern aficionado.

In this chapter, we have summarized the main points of the book. In particular, we looked at six underlying factors that are common to many anti-patterns and are part of their root causes. These provide signposts that architects can use to create an awareness of what anti-patterns are likely to occur in a given architectural context and what measures may be taken to mitigate them.

In order to use this knowledge in practice, we outlined two different approaches to incorporating anti-patterns into your architecture process. First, we looked at how to use it constructively for your own architectures and then how to use it diagnostically for examining existing architectures.

Finally, we looked at where you might go from here in order to expand your knowledge further. We outlined three pathways you might take and gave some suggestions for books and other resources.

Anti-patterns are one of the most interesting areas of general architecture because they tell you how things go wrong in repeatable ways. Learning from the mistakes of others is usually a better idea than making all the mistakes yourself, and I sincerely hope that reading this book will make you much less likely to repeat these anti-patterns yourself as well as know how to spot them when others in your context are about to make them.

Software systems are important. Organizations rely on them. Users spend much of their working life interacting with them. As architects, we have a responsibility to help avoid the common errors that keep them working less well than they could have done. This responsibility has only grown with the increasing complexity brought by AI, DevOps practices, and the need for effective change management in our architectural solutions.

Index

As this ebook edition doesn't have fixed pagination, the page numbers below are hyperlinked for reference only, based on the printed edition of this book.

A

account executive (AE) [110](#)

AI Architecture anti-pattern [94](#)

example [94](#) , [95](#)

problem [96](#)

result [96](#) , [97](#)

solutions [97](#) , [98](#)

AI integration

impact [164](#) , [165](#)

AI overreliance syndrome [164](#)

AI-specific anti-pattern [90](#)

AI Architecture anti-pattern [94](#)

example [90](#) , [91](#)

problem [92](#)

proposed solution [92](#)

result [93](#)

solutions [94](#)

ambiguous solution [199](#)

example [199](#) , [200](#)

problem [200](#)

proposed solution [200](#)

results [201](#)

- solutions [201](#)
- anti-patterns [4](#) , [5](#) , [113](#)
 - architectural skills [8](#) , [9](#)
 - architecture process, analysing [219](#)
 - architecture process, using [219](#)
 - architecture resources [221](#)
 - assumption-driven customization [113](#)
 - failure [9](#)
 - Golden Hammer [116](#)
 - history [7](#)
 - pathways [220](#)
 - pattern to [5](#) - [7](#)
 - recurring causes [217](#) , [218](#)
 - reference link [220](#)
 - resources [220](#)
 - Salesforce architecture [220](#)
 - template [8](#)
 - types [6](#)
 - usage [218](#)
- anti-patterns, assumption-driven customization
 - example [113](#) , [114](#)
 - problem [114](#)
 - proposed solution [115](#)
 - results [115](#)
 - solutions [116](#)
- anti-patterns, Golden Hammer
 - example [116](#) , [117](#)

problem [117](#)

proposed solution [118](#)

results [118](#)

solutions [118](#) , [119](#)

Apex Trigger Actions

reference link [122](#)

Architecture as Requirements anti-pattern [28](#)

better solutions [33](#) , [34](#)

example [28](#) - [31](#)

problem [31](#)

proposed solution [32](#)

results [32](#) , [33](#)

Architecture Governance Forum [49](#)

artificial intelligence (AI) [7](#) , [19](#)

B

Big Ball of Mud

better solutions [38](#)

example [35](#) , [36](#)

problem [37](#)

proposed solution [37](#)

results [37](#)

Big-Bang Release anti-pattern [172](#)

example [172](#) , [173](#)

problem [174](#)

proposed solution [174](#)

results [175](#) , [176](#)

solutions [176](#)

black-box integration [164](#)

business continuity (BC) [107](#)

business units (BUs) [177](#)

C

Center of Excellence (CoE) [47](#)

Certified Technical Architect (CTA) [8](#)

Change Data Capture (CDC) [145](#)

chatty integration [148](#)

 example [148](#) - [151](#)

 problem [151](#)

 proposed solution [151](#)

 results [152](#)

 solutions [152](#) , [153](#)

code-based solutions [126](#)

 error-handling approach [129](#)

 God Class [126](#)

code-based solutions, error-handling approach

 example [130](#)

 problem [130](#)

 proposed solutions [131](#)

 results [131](#)

 solutions [132](#)

code-based solutions, God Class

 example [126](#) , [127](#)

 problem [127](#) , [128](#)

 proposed solution [128](#)

 results [128](#)

solutions [129](#)

cognitive overload [196](#)

example [196](#) , [197](#)

problem [197](#)

proposed solution [197](#)

results [198](#)

solutions [198](#)

Command Query Responsibility Segregation (CQRS) [163](#)

Compliant Is Secure anti-pattern

example [59](#)

problem [60](#)

proposed solution [60](#) , [61](#)

results [61](#) , [62](#)

solution [62](#)

continuous delivery (CD) [176](#)

continuous integration and continuous deployment (CI/CD) [171](#)

coupling [38](#)

CTA Review Board

considerations [49](#) , [100](#)

preparing [73](#) , [133](#) , [166](#) , [167](#)

D

data synchronization [86](#)

Declarative Is Always Safe anti-pattern

example [63](#)

problem [64](#)

proposed solution [64](#)

results [65](#) , [66](#)

solution [66](#)

DevOps integration

challenges [189](#) - [191](#)

Disconnected Entities anti-pattern [80](#)

example [80](#) , [81](#)

problem [81](#)

proposed solution [82](#)

results [82](#)

solutions [83](#)

document management system (DMS) [106](#)

Dummy Unit Tests [186](#)

example [186](#) , [187](#)

problem [188](#)

proposed solution [188](#)

results [189](#)

solutions [189](#)

E

ecosystem audits [109](#)

engineering work [119](#)

Automation Bonanza [119](#)

Overweight Component [123](#)

engineering work, Automation Bonanza

example [119](#) , [120](#)

problem [120](#)

proposed solution [121](#)

results [121](#)

solutions [122](#)

engineering work, Overweight Component

example [123](#) , [124](#)

problem [124](#)

proposed solution [124](#)

results [125](#)

solutions [125](#)

enterprise resource planning (ERP) [87](#) , [172](#)

event-driven architecture (EDA) [125](#) , [140](#)

event hell [158](#)

characteristics [160](#)

example [158](#) , [159](#)

problem [160](#)

proposed solution [160](#) , [161](#)

results [161](#) , [162](#)

solutions [163](#) , [164](#)

F

fat interface [144](#)

example [144](#) - [146](#)

problem [146](#) , [147](#)

proposed solution [147](#)

results [147](#)

solutions [148](#)

fire-and-forget pattern [4](#)

G

Global Manufacturing Corp (GMC) [212](#)

example [212](#)

problem [213](#)

proposed solution [213](#)

results [213](#)

solutions [214](#) , [215](#)

governance [80](#)

Groundhog Day anti-pattern [202](#)

example [202](#) , [203](#)

problem [203](#)

proposed solution [203](#)

results [204](#)

solutions [204](#)

H

healthcare professionals (HCPs) [113](#)

Hero anti-pattern

deployment planning [10](#) - [12](#)

development lifecycle [10](#) - [12](#)

example [10](#)

example, analyzing [15](#) , [16](#)

issues [12](#)

proposed solution [12](#) , [13](#)

results [13](#) , [14](#)

solutions [14](#) , [15](#)

I

Independent Software Vendor (ISV) [94](#)

Infrequent Check-In [180](#)

example [180](#)

problem [181](#)

proposed solution [181](#)

results [181](#)

solutions [182](#)

integration pattern monomania [153](#)

example [154](#) , [155](#)

problem [156](#)

proposed solution [156](#)

results [157](#)

solutions [157](#) , [158](#)

intense coupling

better solutions [42](#)

example [38](#) - [40](#)

problem [40](#)

proposed solutions [41](#)

results [41](#)

J

just-in-time (JIT) [186](#)

L

Large Data Volume (LDV) [85](#)

Lightning Web Components (LWC) [110](#)

line-of-business (LOB) [106](#)

M

machine learning (ML) [47](#)

Manufacturing Resource Planning (MRP) [76](#)

middleware in name only (MINO) [136](#)

example [136](#) , [137](#)

problem [138](#)

proposed solution [138](#)

results [138](#)

solutions [139](#)

minimum viable product (MVP) [172](#)

mismanaging change [208](#)

example [208](#) , [209](#)

problem [209](#)

proposed solution [210](#)

results [210](#)

solutions [211](#)

stakeholder concerns, ignoring [208](#)

Model-View-Controller (MVC) [4](#)

N

non-standard documentation [205](#)

example [205](#)

problem [206](#)

proposed solution [206](#)

results [206](#) , [207](#)

solutions [207](#)

normalization [77](#)

O

object-oriented (OO) [129](#)

object-oriented programming (OOP) [129](#) , [146](#)

OO analysis and design (OOAD) [129](#)

open-closed principle (OCP) [129](#)

Org complications [42](#)

P

Packages to Create Silos

example [183](#) , [184](#)

problem [184](#)

proposed solution [184](#)

results [185](#)

solutions [185](#) , [186](#)

using [183](#)

project identifiers (PIDs) [151](#)

project management office (PMO) [16](#) , [49](#)

Project Pieism [177](#)

example [177](#) , [178](#)

problem [178](#)

proposed solution [178](#)

results [179](#)

solutions [179](#)

proof of concept (POC) [197](#)

Q

quote-to-cash (QTC) [186](#)

R

request for proposal (RFP) [205](#)

S

Salesforce architect

considerations [99](#) , [100](#)

Salesforce as Relational Database anti-pattern [76](#)

example [76](#) , [77](#)

problem [77](#) , [78](#)

proposed solution [78](#) , [79](#)

results [79](#)

solutions [79](#) , [80](#)

Salesforce implementation [105](#)

license-based solution [109](#)

third-party options, ignoring [106](#)

Salesforce implementation, license-based solution

example [109](#) , [110](#)

problem [110](#) , [111](#)

proposed solution [111](#)

results [112](#)

solutions [112](#)

Salesforce implementation, third-party options

example [106](#)

problem [107](#)

proposed solution [108](#)

results [108](#)

solutions [109](#)

Salesforce Operating, Governance, and Architecture (SOGAF) framework [26](#)

elements [27](#)

Salesforce solutions

architecting [132](#) - [166](#)

architecting, considerations [48](#) , [72](#) - [74](#)

security [58](#)

Compliant Is Secure anti-pattern [59](#)

Declarative Is Always Safe anti-pattern [63](#)

separation of concerns (SoC) [31](#)

service-oriented architecture (SOA) [140](#)

service proliferation syndrome [140](#)

example [140](#) - [142](#)

problem [142](#)

proposed solution [142](#) , [143](#)

results [143](#)

solutions [143](#)

Shared Belongs to Salesforce anti-pattern

better solutions [58](#)

example [54](#) , [55](#)

problem [55](#) , [56](#)

proposed solution [56](#) , [57](#)

results [57](#)

solutions [57](#)

shared responsibility [53](#)

Shared Belongs to Salesforce anti-pattern [54](#)

single point of failure (SPOF) [128](#)

single responsibility principle (SRP) [129](#)

singleton design pattern [4](#)

Sixth Normal Form (6NF) [78](#)

SOLID principles [129](#)

spaghetti code [69](#)

Spaghetti Sharing Model

better solutions [72](#)

example [67](#) , [68](#)

problem [69](#)

proposed solution [70](#)

results [70](#) , [71](#)

Stovepipe anti-pattern [6](#) , [21](#)

better solutions [25](#) - [27](#)

example [21](#) - [23](#)

problem [23](#)

proposed solution [24](#)

result [24](#) , [25](#)

Stovepipe Enterprise [27](#)

Structured Query Language (SQL) [106](#)

subject matter expert (SME) [113](#)

systems landscape

Architecture as Requirements anti-pattern [28](#)

Big Ball of Mud anti-pattern [35](#)

coupling anti-pattern [38](#)

intense coupling anti-pattern [38](#)

muddling up [20](#)

Stovepipe anti-pattern [21](#) - [23](#)

T

Third Normal Form (3NF) [76](#)

total cost of ownership (TCO) [109](#)

U

Unconstrained Data Synchronization anti-pattern [86](#)

example [87](#) , [88](#)

problem [88](#)

proposed solution [89](#)

results [89](#)

solutions [90](#)

Ungoverned Org Proliferation [43](#)

better solutions [46](#) , [47](#)

example [43](#) , [44](#)

opposite [47](#)

problem [44](#) , [45](#)

proposed solution [45](#)

results [45](#) , [46](#)

Unplanned Growth anti-pattern [83](#)

example [83](#) , [84](#)

problem [84](#)

proposed solution [85](#)

results [85](#)

solutions [86](#)

User Acceptance Testing (UAT) [11](#) , [184](#)

user experience (UX) [117](#) , [157](#)

user interface (UI) [187](#)



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

<packt>



1ST EDITION

Salesforce DevOps for Architects

Discover tools and techniques to optimize
the delivery of your Salesforce projects



ROB COWELL | LARS MALMQVIST

Salesforce DevOps for Architects

Cowell, Lars Malmqvist

ISBN: 978-1-83763-605-1

- Grasp the fundamentals of integrating a DevOps process into Salesforce project delivery
- Master the skill of communicating the benefits of Salesforce DevOps to stakeholders
- Recognize the significance of fostering a DevOps culture and its impact on Salesforce projects
- Understand the role of metrics in DevOps architecture within Salesforce environments
- Gain insights into the components comprising a Salesforce DevOps toolchain
- Discover strategies for maintaining a healthy Salesforce org with a variety of supporting DevOps tools



1ST EDITION

ChatGPT for Accelerating Salesforce Development

Achieve faster, smarter, and more cost-effective
Salesforce Delivery with ChatGPT

ANDY FORBES | PHILIP SAFIR
JOSEPH KUBON | FRANCISCO FÁLDER

ChatGPT for Accelerating Salesforce Development

Andy Forbes, Philip Safir, Joseph Kubon, Francisco Fálder

ISBN: 978-1-83508-407-6

- Masterfully craft detailed and engaging user stories tailored for Salesforce projects
- Leverage ChatGPT to design cutting-edge features within the Salesforce ecosystem, transforming ideas into functional and intuitive solutions
- Explore the integration of ChatGPT for configuring Salesforce environments
- Write Salesforce flows with ChatGPT, enhancing workflow automation and efficiency
- Develop custom LWCs with ChatGPT's assistance
- Discover effective testing techniques using ChatGPT for optimized performance and reliability

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *Salesforce Anti-Patterns*, we'd love to hear your thoughts! If you purchased the book from Amazon, [please click here to go straight to the Amazon review page for this book and share your feedback or leave a review](#) on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/978-1-83546-784-8>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Contents

1. [Salesforce Anti-Patterns](#)
2. [Contributors](#)
3. [About the author](#)
4. [About the reviewers](#)
5. [Preface](#)
 1. [Who this book is for](#)
 2. [What this book covers](#)
 3. [To get the most out of this book](#)
 4. [Conventions used](#)
 5. [Get in touch](#)
 6. [Share Your Thoughts](#)
 7. [Download a free PDF copy of this book](#)
6. [Part 1: Technical Anti-Patterns](#)
7. [Chapter 1: Why Do We Need to Understand Salesforce Anti-Patterns?](#)
 1. [Understanding anti-patterns](#)
 1. [From pattern to anti-pattern](#)
 2. [Where do anti-patterns come from?](#)
 3. [An anti-pattern template](#)
 2. [How can anti-patterns deepen your architectural skills?](#)
 1. [How great architects learn from mistakes](#)
 2. [An example – the Hero anti-pattern](#)
 3. [Hero \(development life cycle and deployment planning\)](#)
 4. [Analyzing the example](#)
 3. [Summary](#)
8. [Chapter 2: How to Keep Your System Architecture Clean](#)
 1. [Muddling up the systems landscape](#)
 1. [Stovepipe](#)
 2. [Architecture as Requirements](#)
 3. [A Big Ball of Mud](#)
 4. [Too much coupling](#)
 5. [Intense coupling](#)
 2. [Org complications](#)
 1. [Ungoverned Org Proliferation](#)
 3. [Knowing the takeaways](#)

4. [Summary](#)
9. [Chapter 3: How Not to Get Confused about Security](#)
 1. [Getting confused about shared responsibility](#)
 1. [Shared Belongs to Salesforce](#)
 2. [Mistaking other things for security](#)
 1. [Compliant Is Secure](#)
 2. [Declarative Is Always Safe](#)
 3. [Sharing isn't always caring](#)
 1. [Spaghetti Sharing Model](#)
 4. [Knowing the takeaways](#)
 5. [Summary](#)
10. [Chapter 4: What Not to Do about Data](#)
 1. [Mistaking Salesforce for a regular database](#)
 1. [Salesforce as a Relational Database](#)
 2. [Forgetting about governance](#)
 1. [Disconnected Entities](#)
 2. [Unplanned Growth](#)
 3. [Synchronizing excessively](#)
 1. [Unconstrained Data Synchronization](#)
 4. [Too much AI is not always a good thing](#)
 1. [Let's Add AI to It](#)
 2. [AI Architecture Is Like Normal Architecture](#)
 5. [Knowing the takeaways](#)
 6. [Summary](#)
11. [Part 2: Solution Anti-Patterns](#)
12. [Chapter 5: Unpicking Solution Architecture Troubles](#)
 1. [Picking the wrong thing](#)
 1. [Ignoring the Ecosystem](#)
 2. [License-Based Solutioning](#)
 2. [Applying the thing wrong](#)
 1. [Assumption-Driven Customization](#)
 2. [Golden Hammer](#)
 3. [Engineering overflow](#)
 1. [Automation Bonanza](#)
 2. [Overweight Component](#)
 4. [Making a mess of code](#)
 1. [God Class](#)

- 2. [Error Hiding](#)
 - 5. [Knowing the takeaways](#)
 - 6. [Summary](#)
- 13. [Chapter 6: Keeping Integration Straight](#)
 - 1. [Messing up the integration landscape](#)
 - 1. [Middleware in name only_\(MINO\)](#)
 - 2. [Service proliferation syndrome](#)
 - 2. [Interfacing indecently](#)
 - 1. [Fat interface](#)
 - 2. [Chatty integration](#)
 - 3. [Getting the patterns wrong](#)
 - 1. [Integration pattern monomania](#)
 - 4. [Event hell](#)
 - 1. [Example](#)
 - 2. [Problem](#)
 - 3. [Proposed solution](#)
 - 4. [Results](#)
 - 5. [Better solutions](#)
 - 5. [AI's impact on integrations](#)
 - 6. [Knowing the takeaways](#)
 - 7. [Summary](#)
- 14. [Part 3: Process and Communication Anti-Patterns](#)
- 15. [Chapter 7: Keeping the Development Life Cycle from Going Off Track](#)
 - 1. [Misaligning the process](#)
 - 1. [Big-Bang Release](#)
 - 2. [Project Pieism](#)
 - 3. [Infrequent Check-In](#)
 - 2. [Unpacking the pipeline](#)
 - 1. [Using Packages to Create Silos](#)
 - 3. [Testing without testing](#)
 - 1. [Dummy Unit Tests](#)
 - 4. [The challenge of DevOps integration](#)
 - 5. [Knowing the takeaways](#)
 - 6. [Summary](#)
- 16. [Chapter 8: Communication without Mistranslation](#)
 - 1. [Communicating too much](#)
 - 1. [Cognitive overload](#)

2. [Being unclear in several ways](#)
 1. [Ambiguous solution](#)
 2. [Groundhog Day](#)
3. [Drawing diagrams with excess creativity](#)
 1. [Non-standard documentation](#)
4. [Mismanaging change](#)
 1. [Ignoring stakeholder concerns](#)
 2. [Example](#)
 3. [The problem](#)
 4. [Proposed solution](#)
 5. [Results](#)
 6. [Better solutions](#)
5. [Underestimating the impact of change](#)
 1. [Example](#)
 2. [The problem](#)
 3. [Proposed solution](#)
 4. [Result](#)
 5. [Better solutions](#)
6. [Knowing the takeaways](#)
7. [Summary](#)
17. [Chapter 9: Conclusion](#)
 1. [Summarizing the key messages](#)
 2. [Explaining how to use anti-patterns in practice](#)
 1. [Using anti-patterns constructively](#)
 3. [Going further with anti-patterns](#)
 1. [Diving deeper into the anti-pattern literature](#)
 2. [Extending your knowledge of Salesforce architecture](#)
 3. [Extending your general architecture knowledge](#)
 4. [Summary](#)
18. [Index](#)
 1. [Why subscribe?](#)
19. [Other Books You May Enjoy](#)
 1. [Packt is searching for authors like you](#)
 2. [Share Your Thoughts](#)
 3. [Download a free PDF copy of this book](#)

Landmarks

1. [Cover](#)
2. [Table of Contents](#)
3. [Index](#)